USB协议简介

USB是一种协议总线，即主机与设备之间的通信需要遵循一系列约定。协议内容较多，这里仅作一些简单介绍，深入学习，可参看USB规范(WWW．usb．org)。

为了理解协议中的名称，先看图10．32。该图突出了主机上的客户软件和USB逻辑设备(编程涉及的设备)之间的通信流(Communication Flow)，该通信流跨越了USB驱动程序USBD、主控制器驱动程序UHCD、主控制器等硬件接口及其连接。端点(Endpoints)是USB设备的惟一可识别的部分，是主机和设备之间通信流的终点。每一个逻辑设备有若干个独立端点，每一个端点在设计时被分配一个惟一的由设备确定的标识符，称之为端点号。

如图10．32所示，将用于通信流流动的通道称为管道(Pipe)，这是忽略了许多中间环节的很形象的称呼，对于理解USB系统中的信息传输很有帮助。图中把3个端点看成了一个接口，关于接口的说明安排在后面。



USB2.0协议

USB2.0

(一)USB2.0的背景



- COMPAQ、Hewlett Packard、Intel、Lucent、Microsoft、NEC和PHILIPS这7家厂商联合制定了USB 2.0接口标准
- USB2.0规范是由USB1.1规范演变而来的 ,USB 2.0将设备之间的数据传输速度增加到480Mbps，折算为MB为60MB/S,比USB 1.1标准快40倍左右.
- USB2.0与USB1.1可以互相兼容， USB2.0设备可以工作在USB1.1接口上 ， USB1.0设备也可以工作在USB2.0接口 ，USB1.1设备的速度不会因为安装在USB2.0接口上而有任何提高，同样安装在USB1.1接口上的USB2.0设备的速度也会被限制在12mbps（1.5MB/S）以下. USB2.0和USB1.1使用的连接电缆及端口均相同 .

www.cpit.com.cn

(二)USB2.0协议规范目的

- USB2.0规范描述了总线属性,协议定义,处理类型,总线管理和编程接口,以满足设计和构建符合该标准的系统和外设的要求.
- USB2.0规范的主要目标用户是外设开发者和系统OEM.
- USB2.0规范的目标是使不同厂商的器件可以在开放的结构中相互操作





(三)USB应用分类

PERFORMANCE	APPLICATIONS	ATTRIBUTES
LOW-SPEED • Interactive Devices • 10 – 100 kb/s	Keyboard, Mouse Stylus Game Peripherals Virtual Reality Peripherals	Lowest Cost Ease-of-Use Dynamic Attach-Detach Multiple Peripherals
FULL-SPEED • Phone, Audio, Compressed Video • 500 kb/s – 10 Mb/s	POTS Broadband Audio Microphone	Lower Cost Ease-of-Use Dynamic Attach-Detach Multiple Peripherals Guaranteed Bandwidth Guaranteed Latency
HIGH-SPEED • Video, Storage • 25 – 400 Mb/s	Video Storage Imaging Broadband	Low Cost Ease-of-Use Dynamic Attach-Detach Multiple Peripherals Guaranteed Bandwidth Guaranteed Latency High Bandwidth

符合USB 1.1规范的标志　　符合USB 2.0规范的标

www.cpit.com.cn

(四)USB2.0的结构体系

- USB系统
- 物理层接口
- 电源
- 总线协议
- 健壮性
- 系统配置
- 数据流类型
- USB设备
- USB主机------硬件和软件
- 体系结构的扩展

USB2.0系统

- USB2.0系统主要被定义为三个部分：
  1. USB的互连；
  2. USB的设备；
  3. USB的主机

USB的互连是指USB设备与主机之间进行连接和通信的操作，主要包括以下几方面：

1. 总线的拓扑结构：USB设备与主机之间的各种连接方式；
2. 内部层次关系：根据性能叠置，USB的任务被分配到系统的每一个层次；
3. 数据流模式：描述了数据在系统中通过USB从产生方到使用方的流动方式；
4. USB的调度：

USB提供了一个共享的连接。对可以使用的连接进行了调度以支持同步数据传输，并且避免的优先级判别的开销 .

1. USB2.0的主机

在任何USB系统中，只有一个主机。USB和主机系统的接口称作主机控制器，主机控制器可由硬件、固件和软件综合实现。根集线器是由主机系统整合的，用以提供更多的连接点.

2. USB2.0的设备

USB的设备如下所示:

1. 网络集线器，向USB提供了更多的连接点;

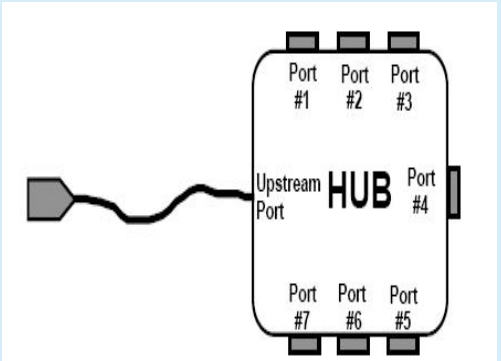
2. 功能器件,为系统提供具体功能，如ISDN的连接，数字的游戏杆或扬声器;

USB设备提供的USB标准接口的主要依据:

1. 对USB协议的运用;

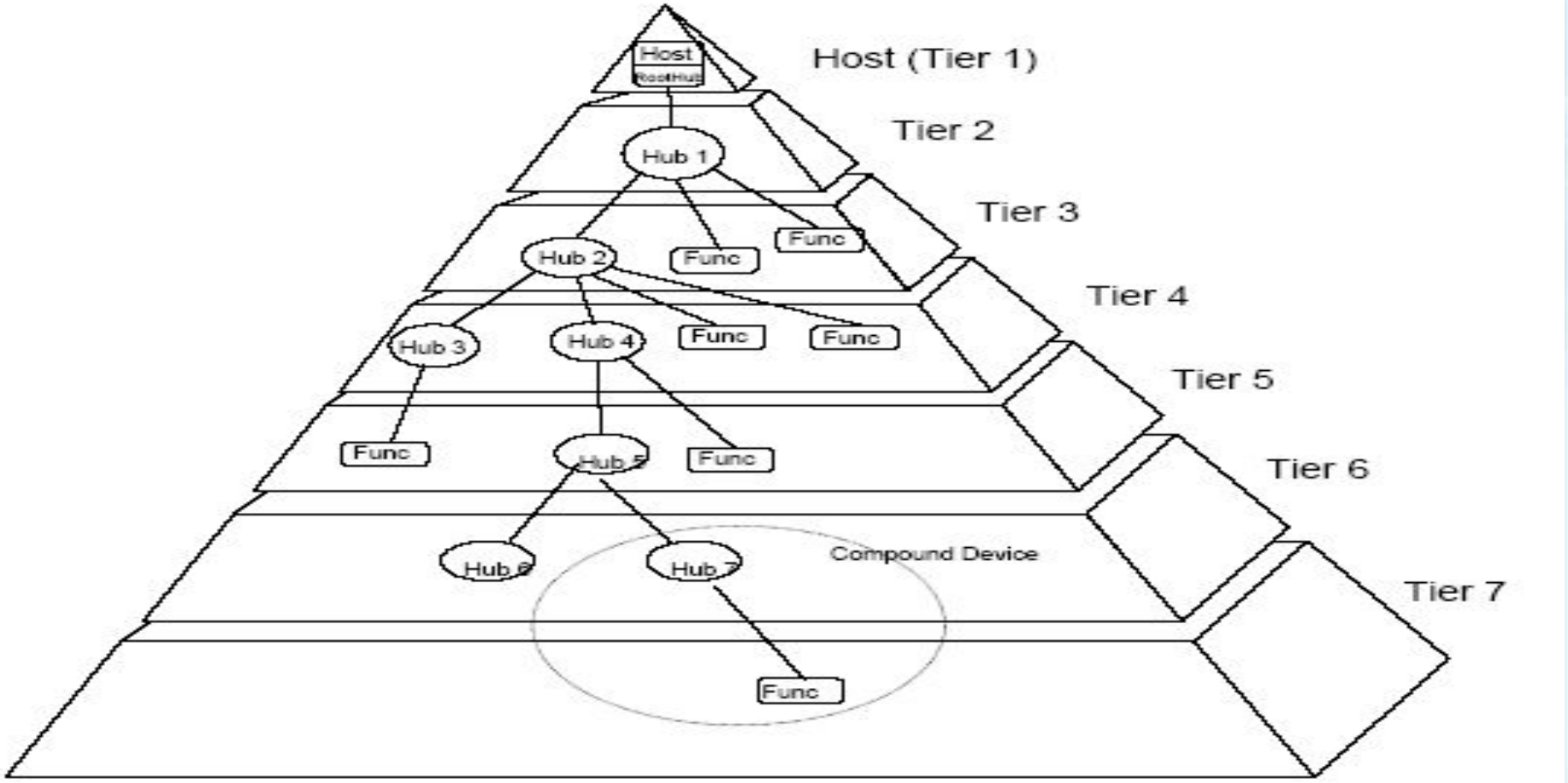
2. 对标准USB操作的反馈，如设置和复位;

3. 标准性能的描述性信息;



- **USB2.0总线布局技术**

USB连接了USB设备和USB主机，USB的物理连接是有层次性的星型结构。每个网络集线器是在星型的中心，每条线段是点点连接。从主机到集线器或其功能部件，或从集线器到集线器或其功能部件.



USB2.0的物理接口

USB的物理接口分为总线的电气规范和机械特性.

1. 电气特性

USB在一条4线的电缆上传输信号和电源. 信号在两条点对点连接线上传输.

USB2.0支持3种数据数率:

- USB高速信号位速率是480Mb/s
- USB全速信号位速率是12Mb/s
- USB低速信号位速率是1.5Mb/s

USB2.0主机控制器和集线器提供了一种性能, 使全速和低速数据可以在主机控制器和HUB之间高速传输, 而HUB和设备之间以全速或低速传输, 这种性能将全速和低速设备对高速设备可用带宽的影响减到最小.

低速模式用于支持例如鼠标等数量有限的低宽带设备, 因为使用太多这种设备将降低总线的利用率.

时钟与差动数据一起编码传送, 时钟是以带位填充的NRZI方式编码, 以保证有足够的跳变沿. 接收器用每个包前面的SYNC场同步它们的位恢复时钟.

每段电缆还有向设备提供电源的V_{BUS}和GND线. V_{BUS}在源极的标称值是+5V. 通过选择适当的导线规格来匹配指定的IR drop和其它一些特性, 如设备能源预算和电缆适应度. 为了保证足够的输入电压和终端阻抗。重要的终端设备应位于电缆的尾部。在每个端口都可检测终端是否连接或分离，并区分出高速，或低速设备.

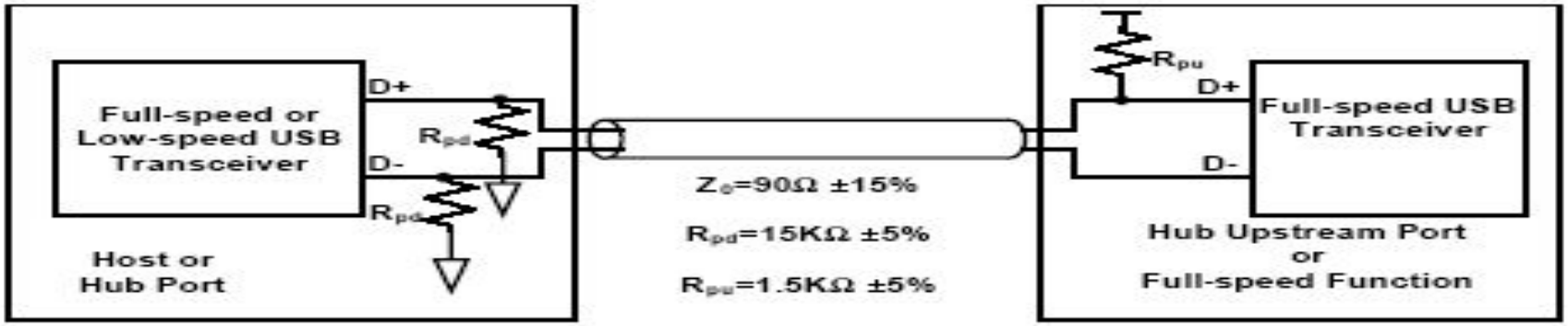


Figure 7-20. Full-speed Device Cable and Resistor Connections

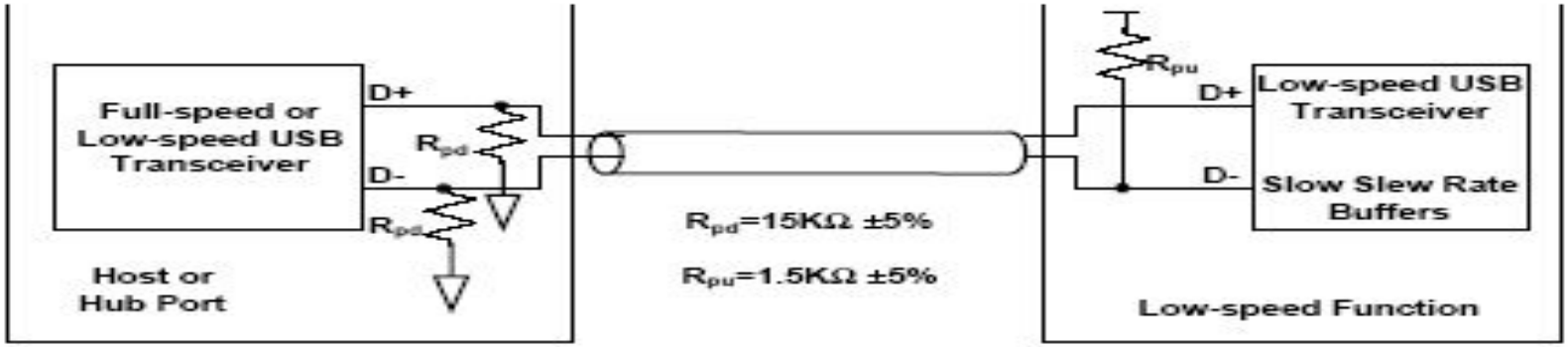
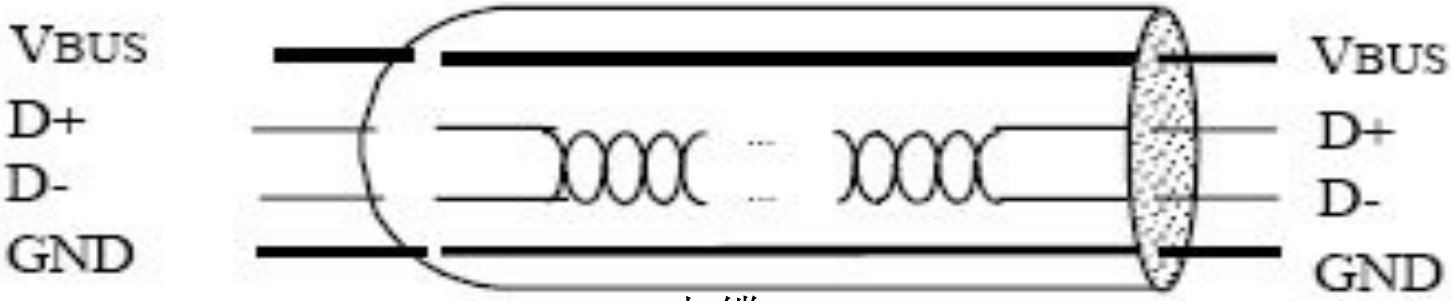


Figure 7-21. Low-speed Device Cable and Resistor Connections



USB电缆

www.cpit.com.cn

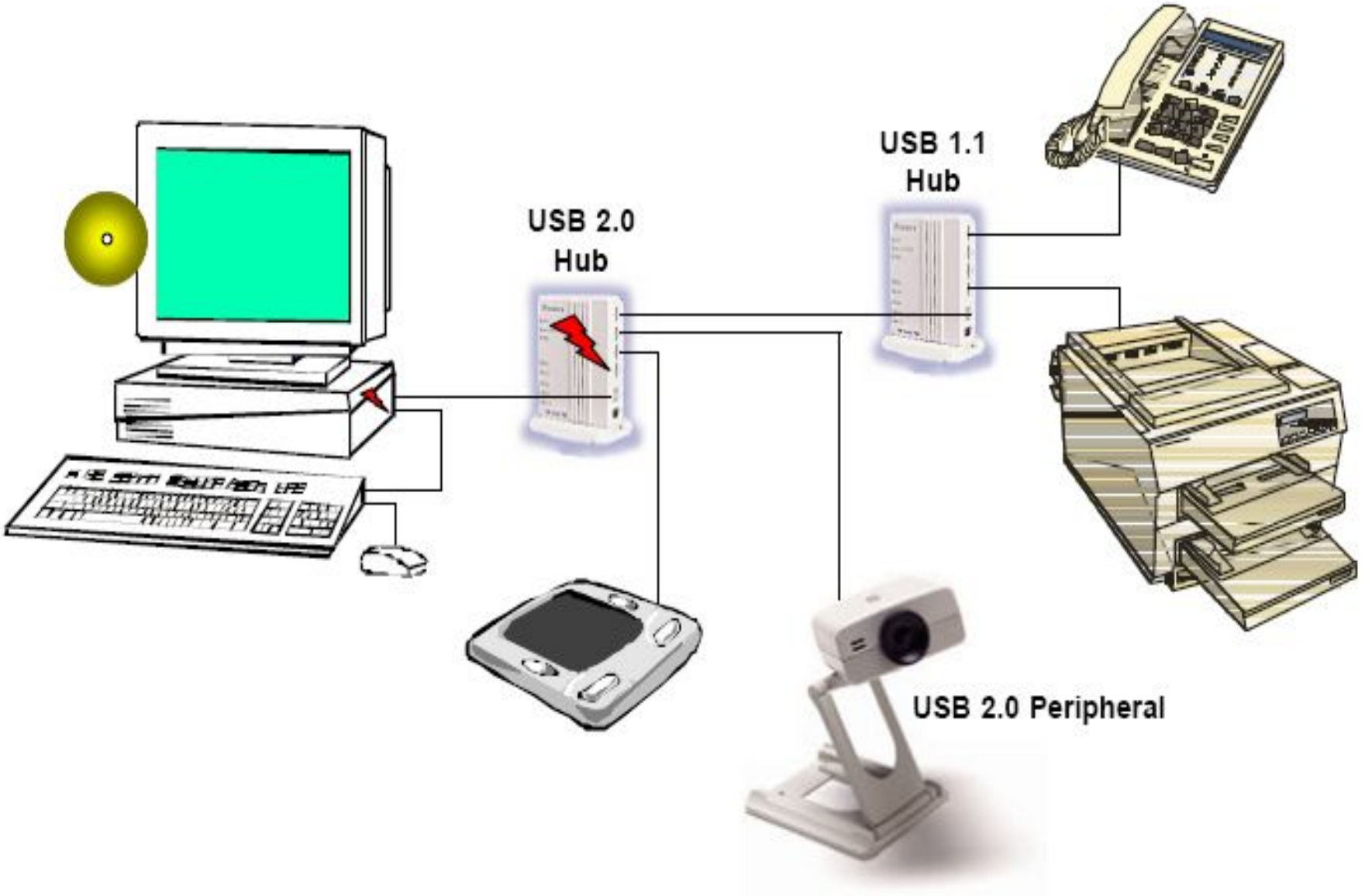
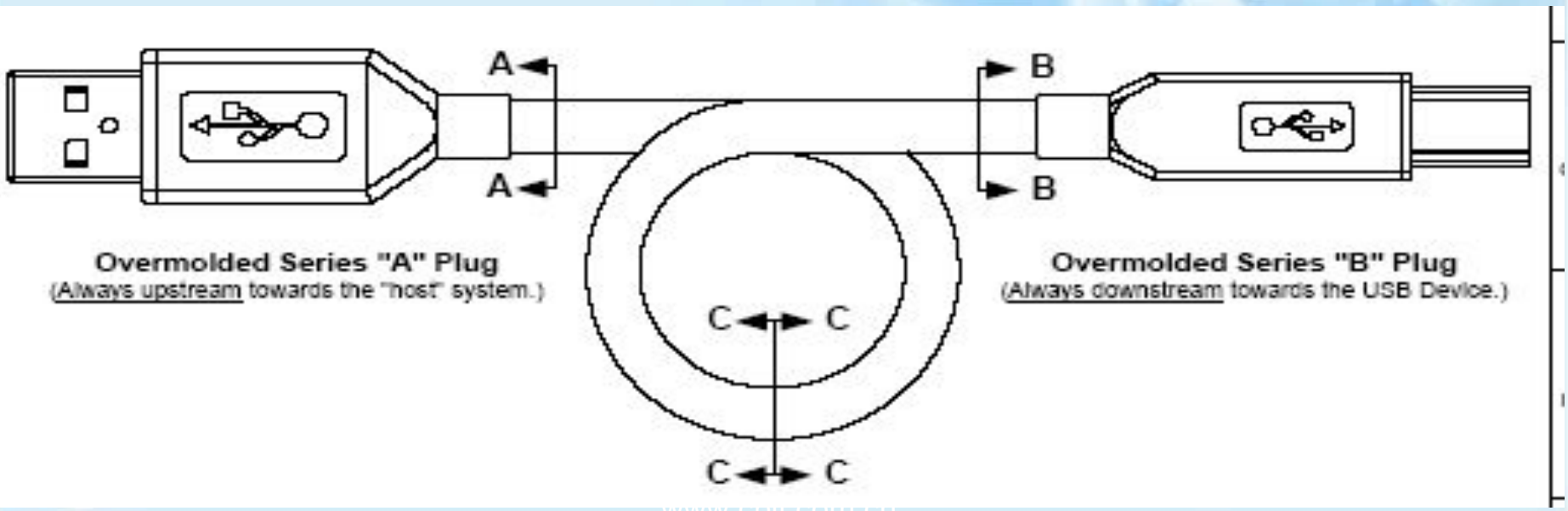


Figure 2. Example Future USB 2.0 System Configuration

2. 机械特性

电缆和连接器的机械特性将在机械特性章节详述。所有设备都有一个上行的连接。上行连接器和下行连接器是不可简单的互换，这样就避免了集线器间的非法的循环往复的连接，电缆中有四根导线：一对互相缠绕的标准规格线，一对符合标准的电源线，连接器有四个方向，具有屏蔽层，以避免外界干扰，并有易拆装的特性。



USB2.0电源

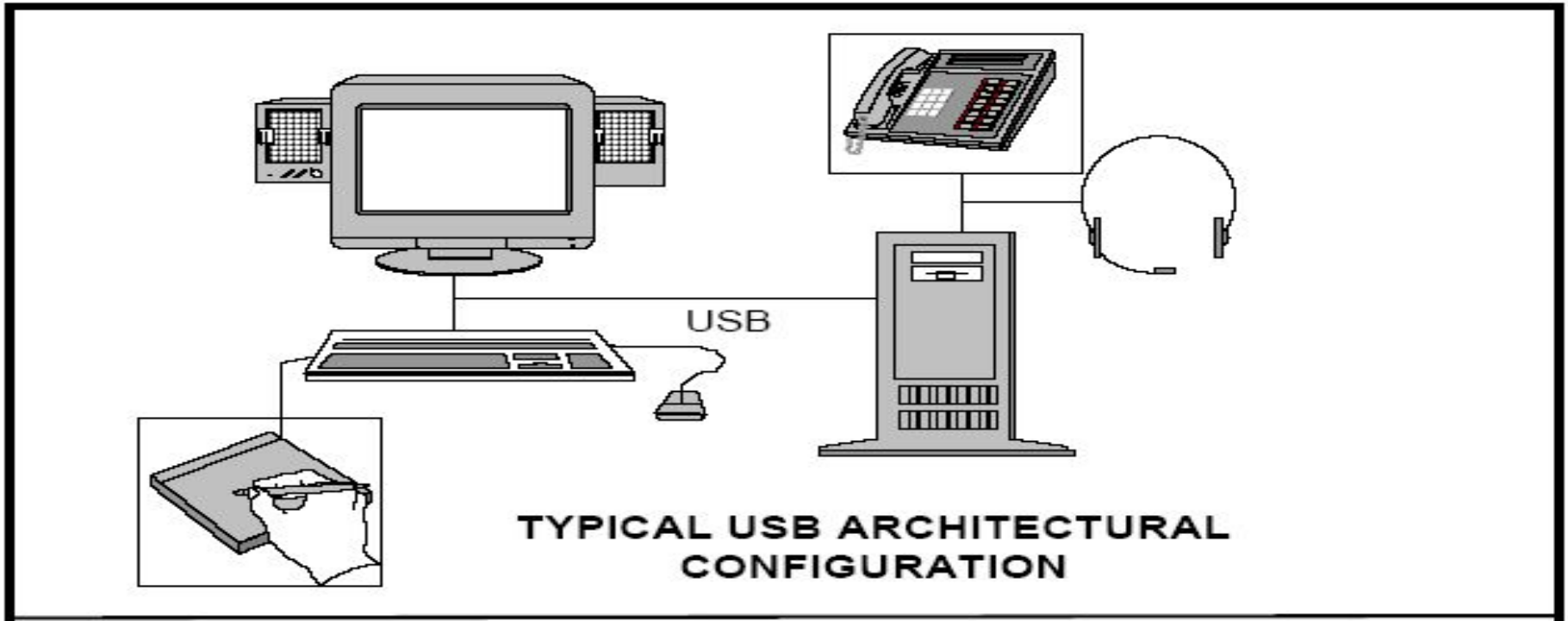
主要包括两方面：

·电源分配：即USB的设备如何通过USB分配得到由主计算机提供的能源；

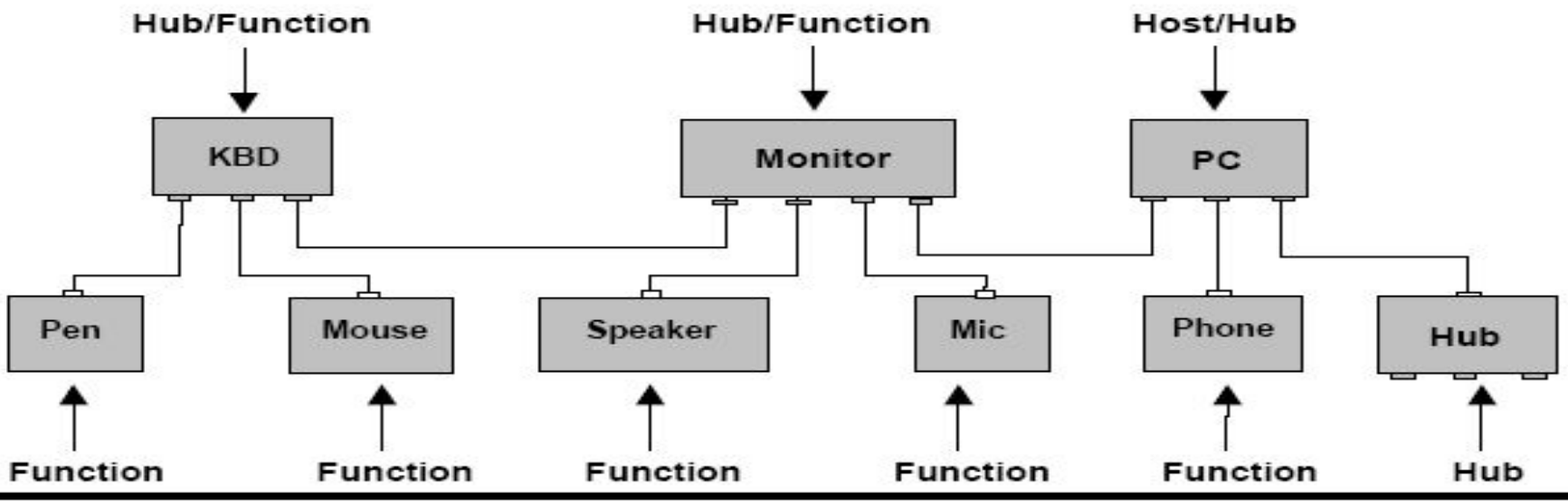
·电源管理：即通过电源管理系统，USB的系统软件和设备如何与主机协调工作.

1. 电源分配

每个USB单元通过电缆只能提供有限的能源。主机对那种直接相连的USB设备提供电源供其使用。并且每个USB设备都可能有自己的电源。那些完全依靠电缆提供能源的设备称作“总线供能”设备.相反，那些可选择能源来源的设备称作“自供电”设备.而且，集线器也可由与之相连的USB设备提供电源。受一定布局限制的带有“总线供能”集线器的体系结构将在集线器部分讨论。在图中，键盘，输入笔和鼠标均为“总线供能”设备.



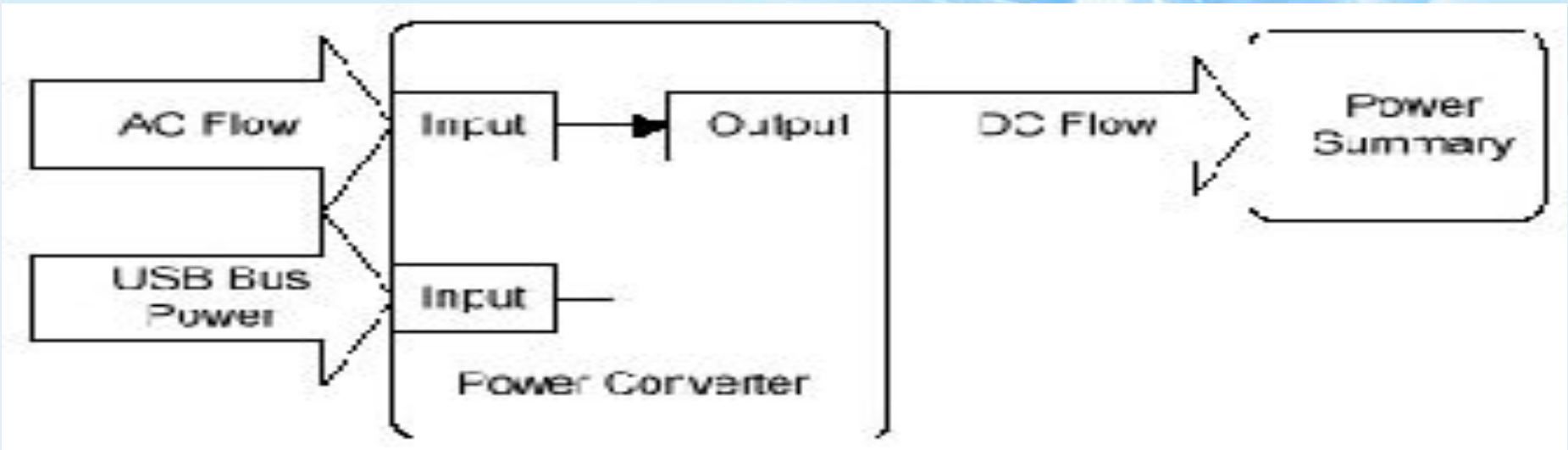
TYPICAL USB ARCHITECTURAL CONFIGURATION



2. 电源管理

USB主机与USB系统有相互独立的电源管理系统。USB的系统软件可以与主机的能源管理系统结合共同处理各种电源子件如挂起、唤醒，并且有特色的是，USB设备应用特有的电源管理特性，可让系统软件和控制其电源管理.

USB的电源分配和电源管理特性使之可以被设计在电源传感系统中，如采用电池的笔记本电脑.



典型USB外设的电源供应

USB2.0总线协议

USB总线属一种轮讯方式的总线，主机控制端口初始化所有的数据传输.每一总线执行动作最多传送三个数据包。按照传输前制定好的原则，在每次传送开始时，主机控制器发送一个描述传输运作的种类、方向，USB设备地址和终端号的USB数据包，这个数据包通常称为标志包(token packet)。USB设备从解码后的数据包的适当位置取出属于自己的数据。数据传输方向不是从主机到设备就是从设备到主机。在传输开始时，由标志包来标志数据的传输方向，然后发送端开始发送包含信息的数据包或表明没有数据传送。接收端也要相应发送一个握手的数据包表明是否传送成功。发送端和接收端之间的USB数据传输，在主机和设备的端口之间，可视为一个通道。存在两种类型的通道：流和消息。流的数据不像消息的数据，它没有USB所定义的结构，而且通道与数据带宽、传送服务类型，端口特性（如方向和缓冲区大小）有关。多数通道在USB设备设置完成后即存在。USB中有一个特殊的通道——缺省控制通道，它属于消息通道，当设备一启动即存在，从而为设备的设置、查询状况和输入控制信息提供一个入口。

事务预处理允许对一些数据流的通道进行控制，从而在硬件级上防止了对缓冲区的高估或低估，通过发送不确认握手信号从而阻塞了数据的传输速度。当不确认信号发过后，若总线有空闲，数据传输将再做一次。这种流控制机制允许灵活的任务安排，可使不同性质的流通道同时正常工作，这样多种流通常可在不同间隔进行工作，传送不同大小的数据包。

USB2.0健壮性

USB健壮性的特征在于：

·使用差分的驱动接收和防护，以保证信号完整性；
·在数据和控制信息上加了循环冗余码(CRC)；
·对装卸的检测和系统级资源的设置；
·对丢失或损坏的数据包暂停传输、利用协议自我恢复；
·对流数据进行控制，以保证同步信号和硬件缓冲管理的安全；
·数据和控制通道的建立，使功能部件的相互不利的影响独立开，消除了负作用.

1. 错误检测

USB传输介质产生的错误率是与自然界的异常现象的概率相吻合，是瞬时一现的，因此就要在每个数据包中加入检测位来发现这些瞬时的错误，并且提供了一系列硬件和软件设施来保证数据的正确性。

协议中对每个包中的控制和数据位都提供了循环冗余码校验，若出现了循环冗余码的错误则被认为是该数据包已被损坏，循环冗余码可对一位或两位的错误进行100%的修复。

2. 错误处理

协议在硬件或软件级上提供对错误的处理。硬件的错误处理包括汇报并重新进行上一次失败的传输、传输中若遇到错误，USB主机控制器将重新进行传输，最多可再进行三次。若错误依然存在，则对客户端软件报告错误，客户端软件可用一种特定的方法进行处理

USB2.0系统配置

USB设备可以随时的安装和折卸，因此，系统软件在物理的总线布局上必须支持这种动态变化.

1. USB设备的安装

所有的USB设备都是通过端口接在USB上，网络集线器知道这些指定的USB设备，集线器有一个状态指示器指明在其某个端口上，USB设备是否被安装或拆除了，主机将所有的集线器排成队列以取回其状态指示。在USB设备安装后，主机通过设备控制通道激活该端口并以预设的地址值给USB设备.

主机对每个设备指定唯一的USB地址。并检测这种新装的USB设备是集线器还是功能部件。主机为USB设备建立了控制通道，使用指定的USB的地址和零号端口.

如果安装的USB设备是集线器，并且USB设备连在其端口上，那上述过程对每个USB设备的安装都要做一遍.

如果安装的设备是功能部件，那么主机中关于该设备的软件将因设备的连接而被引发.

www.cpit.com.cn

2. USB设备的拆卸

当USB设备从集线器的端口拆除后，集线器关闭该端口，并且向主机报告该设备已不存在。USB的系统软件将准确进行处理，如果去除的USB设备上集线器，USB的系统软件将对集线器反连在其上的所有设备进行处理

3. 总线标号

总线标号就是对连接在总线上的设备指定唯一的地址的一种动作，因为USB允许USB设备在任何时刻从USB上安装或拆卸，所以总线标号是USB的系统软件始终要作的动作，而且总线标号还包括对拆除设备的检测和处理



数据流种类

数据和控制信号在主机和USB设备间的交换存在两种通道：单向和双向。USB的数据传送是在主机软件和一个USB设备的指定端口之间。这种主机软件和USB设备的端口间的联系称作通道。总的来说，各通道之间的数据流动是相互独立的。一个指定的USB设备可有许多通道。例如，一个USB设备存在一个端口，可建立一个向其它USB设备的端口，发送数据的通道，它可建立一个从其它USB设备的端口接收数据的通道.

USB的结构包含四种基本的数据传输类型：

·控制数据传送：在设备连接时用来对设备进行设置，还可对指定设备进行控制，如通道控制；

·批量数据传送：大批量产生并使用的数据，在传输约束下，具有很广的动态范围；

·中断数据的传送：用来描述或匹配人的感觉或对特征反应的回馈。

·同步数据的传送：由预先确定的传送延迟来填满预定的USB带宽。

对于任何对定的设备进行设置时一种通道只能支持上述一种方式的数据传输，数据流模式的更多细节在数据流模型章节中详述

1 控制数据传送

当USB设备初次安装时，USB系统软件使用控制数据对设备进行设置，设备驱动程序通过特定的方式使用控制数据来传送，数据传送是无损性的.

2 批量数据传送

批量数据是由大量的数据组成，如使用打印机和扫描仪时，批量数据是连续的。在硬件级上可使用错误检测可以保证可靠的数据传输，并在硬件级上引入了数据的多次传送。此外根据其它一些总线动作，被大量数据占用的带宽可以相应的进行改变.

3 中断数据传输

中断数据是少量的，且其数据延迟时间也是有限范围的。这种数据可由设备在任何时刻发送，并且以不慢于设备指定的速度在USB上传送。 中断数据一般由事件通告，特征及座标号组成，只有一个或几个字节。匹配定点设备的座标即为一例，虽然精确指定的传输率不必要，但USB必须对交互数据提供一个反应时间的最低界限.

4 同步传输

同步数据的建立、传送和使用时是连续且实时的，同步数据是以稳定的速率发送和接收实时的信息，同步数据要使接收者与发送者保持相同的时间安排，除了传输速率，同步数据对传送延迟非常敏感。所以同步通道的带宽的确定，必须满足对相关功能部件的取样特性。不可避免的信号延迟与每个端口的可用缓冲区数有关。

一个典型的同步数据的例子是语音，如果数据流的传送率不能保持，数据流是否丢失将取决于缓冲区的大小和损坏的程度。即使数据在USB硬件上以合适的速率传送，软件造成的传送延迟将对那些如电话会议等实时系统的应用造成损害。

实时的传送同步数据肯定会发生潜在瞬时的数据流丢失现象，换句话说，即使许多硬件机制，如重传的引入也不能避免错误的产生。实际应用中，USB的数据出错率小到几乎可以忽略不计。从USB的带宽中，给USB同步数据流分配了专有的一部分以满足所想得到的传速率，USB还为同步数据的传送设计了最少延迟时间

5 指定USB带宽

USB的带宽分配给各个通道，当一个通道建立后，USB就分配给它一定的带宽，USB设备需要提供一些数据缓冲区。若USB提供了更多带宽，则需更多的缓冲区。USB的体系要保证缓冲引导的硬件的延迟限定在几毫秒内。

USB的带宽容量可以容纳多种不同的数据流，因此保证USB上可以连接大量设备，如可以容纳从1B+D直到T1速率范围的电信设备。同时USB支持在同一时刻不同设备具有不同比特率，并具有一个动态变动的范围.

USB规范对总线的每类转输规定的具体的原则.

USB设备

USB设备分为诸如集线器、分配器或文本设备等种类。集线器类指的是一种提供USB连接点的设备(详见USB集线器)，USB设备需要提供自检和属性设置的信息，USB设备必须在任何时刻执行与所定义的USB设备的状态相一致的动态.

1 设备特性

当设备被连接、编号后，该设备就拥有一个唯一的USB地址。设备就是通过该USB地址被操作的，每一个USB设备通过一个或多个通道与主机通讯。所有USB设备必须在零号端口上有一指定的通道，每个USB设备的USB控制通道将与之相连。通过此控制通道，所有的USB设备都列入一个共同的准入机制，以获得控制操作的信息。

在零号端口上，控制通道中的信息应完整的描述USB设备、此类信息主要有以下几类:

·标准信息：这类信息是对所有USB设备的共同性的定义，包括一些如厂商识别、设备种类、电源管理等的项目。设备设置、接口及终端的描述在此给出。关于这些具体的描述信息在第九章给出;

·类别信息：此类信息给出了不同USB的设备类的定义，主要反映其不同点。

·USB厂商信息：USB设备的厂商可自由的提供各种有关信息，其格式不受该规范制约。此外，每个USB设备均提供USB的控制和状态信息。

www.cpit.com.cn

2 设备描述

主要分为两种设备类：集线器和功能部件。只有集线器可以提供更多的USB的连接点，功能部件为主机提供了具体的功能.

1 集线器

在即插即用的USB的结构体系中，集线器是一种重要设备.从用户的观点出发，集线器极大简化了USB的互连复杂性，而且以很低的价格和高易用性提供了设备的健壮性。

集线器串接在集中器上，可让不同性质的设备连接在USB上，连接点称作端口。每个集线器将一个连接点转化成许多的连接点。并且该体系结构支持多个集线器的连接。

每个集线器的上游端口向主机方向进行连接。每个集线器的下游端口允许连接另外的集线器或功能部件，集线器可检测每个下游端口的设备的安装或拆卸，并可对下游端口的设备分配能源，每个下游端口都具有独立的能力，不论高速或低速设备均可连接。集线器可将低速和高速端口的信号分开。

一个集线器包括两部分：集线控制器（Controller）和集线放大器(Repeater)。集线放大器是一种在上游端口和下游端口之间的协议控制开关。而且硬件上支持复位、挂起、唤醒的信号。集线控制器提供了接口寄存器用于与主机之间的通信、集线器允许主机对其特定状态和控制命令进行设置，并监视和控制其端口.

中国普天信息技术研究院
www.cpit.com.cn
普天信息技术研究院

2 功能部件

功能部件是一种通过总线进行发送接收数据和控制信息的USB设备，通过一根电缆连接在集线器的某个端口上，功能设备一般是一种相互无关的外设。然而一个物理单元中可以有多个功能部件和一个内置集线器，并利用一根USB电缆，这通常被称为复合设备，即一个集线器连向主机，并有一个或多个不可拆卸的USB设备连在其上。

每个功能设备都包含设置信息，来描述该设备的性能和所需资源。主机要在功能部件使用前对其进行设置。设置信息包括USB带宽分配，选择设备的设置信息等。

下机列举了一些功能部件：

·定位设备：如鼠标或光笔；

·输入设备：如键盘；

·电信适配器：如ISDN.





USB主机：硬件和软件

USB的主机通过主机控制器与USB设备进行交互。主机功能如下：

·检测USB设备的安装和拆卸
·管理在主机和USB设备之间的控制流；
·管理在主机和USB设备之间的数据流；
·收集状态和动作信息；
·提供能量给连接的USB设备。

主机上USB的系统软件管理USB设备和主机上该设备软件之间的相互交互，USB系统软件与设备软件间有三种相互作用方式：

·设备编号和设置；
·同步数据传输；
·异步数据传输；
·电源管理
·设备和总线管理信息。

只要可能，USB系统软件就会使用目前的主机软件接口来管理上述几种方式.

www.cpit.com.cn

USB2.0体系结构的扩充

USB的体系结构包含主机控制驱动器和USB驱动器之间的接口的扩展，使多个主机控制器和主机控制驱动器的使用成为可能.





(五)USB数据流模型

USB提供了在一台主机和若干台附属的USB设备之间的通信功能，从终端用户的角度看到的USB系统，可简单地用下图表示



Figure 5-1. Simple USB Host/Device View

实际上的USB通信分为以下四个部分:

·USB物理设备(USB Physical Device)：USB上的一种硬件，可运行一些用户程序。

·客户软件(client software)：为一个特定的USB设备而在主机上运行的软件。这种软件由USB设备的提供者提供，或由操作系统提供。

·USB系统软件(USB system software)：此软件用于在特定的操作系统中支持USB，它由操作系统提供。与具体的USB设备无关，也独立于客户软件。

·USB主机控制器(USB Host Controller):总线在主机方面的接口，是软件和硬件的总和。用于支持USB设备通过USB连到主机上。

这四个USB系统的组成部分在功能上存在相互重叠的部分。为了支持主机与客户之间的坚固可靠的通信.

USB的实际应用领域如下图所示:

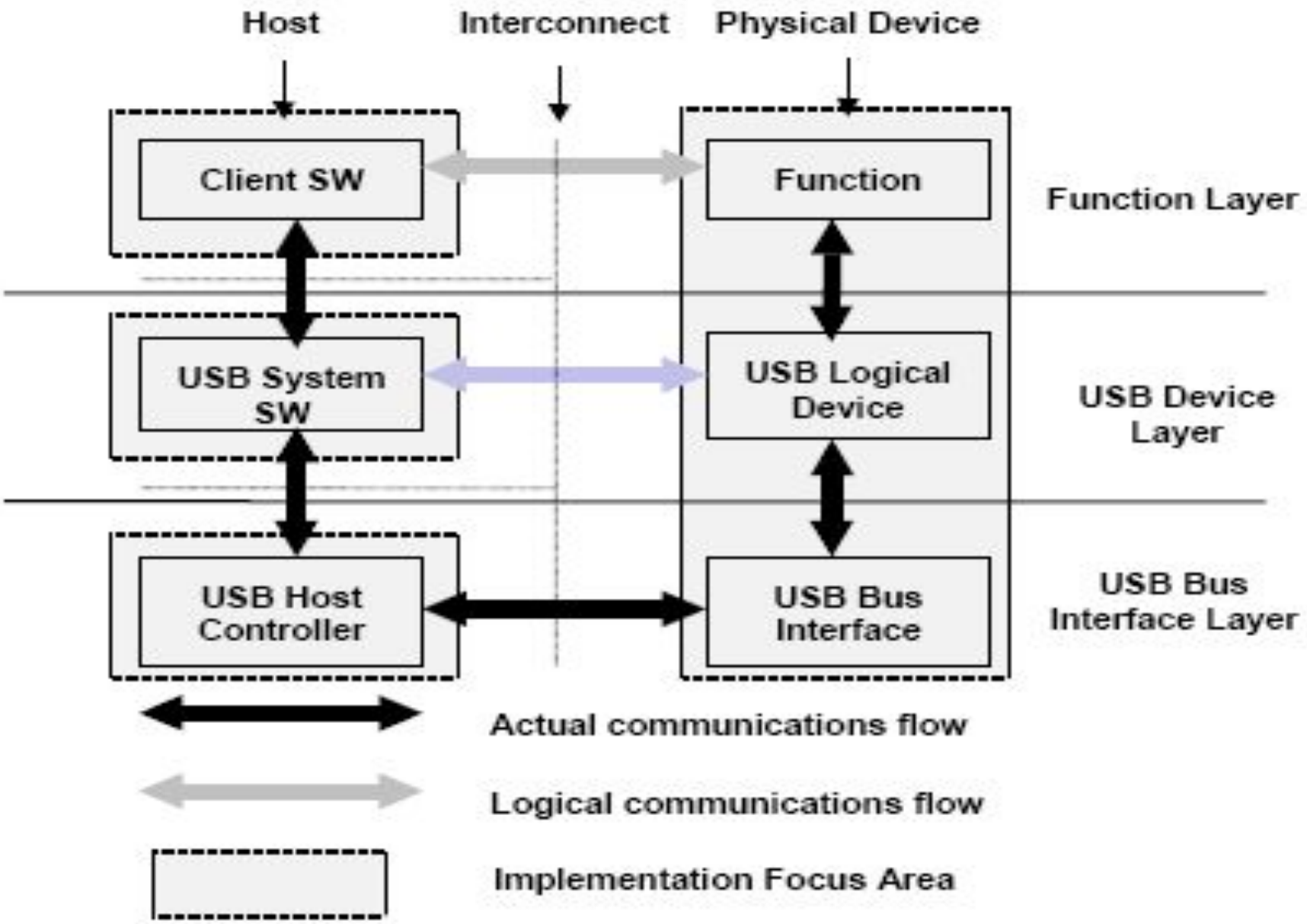


Figure 5-2. USB Implementation Areas

USB2.0的总线拓扑

总线拓朴结构包括四个重要的组成部分。

·**主机和设备：**USB系统的基础组成部分。

·**物理拓朴结构：**描述USB系统中的各组成部分是如何连接起来的。

·**逻辑拓朴结构：**描述USB系统中各种组成部分的地位和作用，以及描述从主机和设备的角度观察到的USB系统。

·**客户软件层与应用层的关系：**描述从客户软件层看到的应用层的情况，以及从应用层看到的客户软件层的情况。

Potevio
中国普天

主机和设备

1. 主机

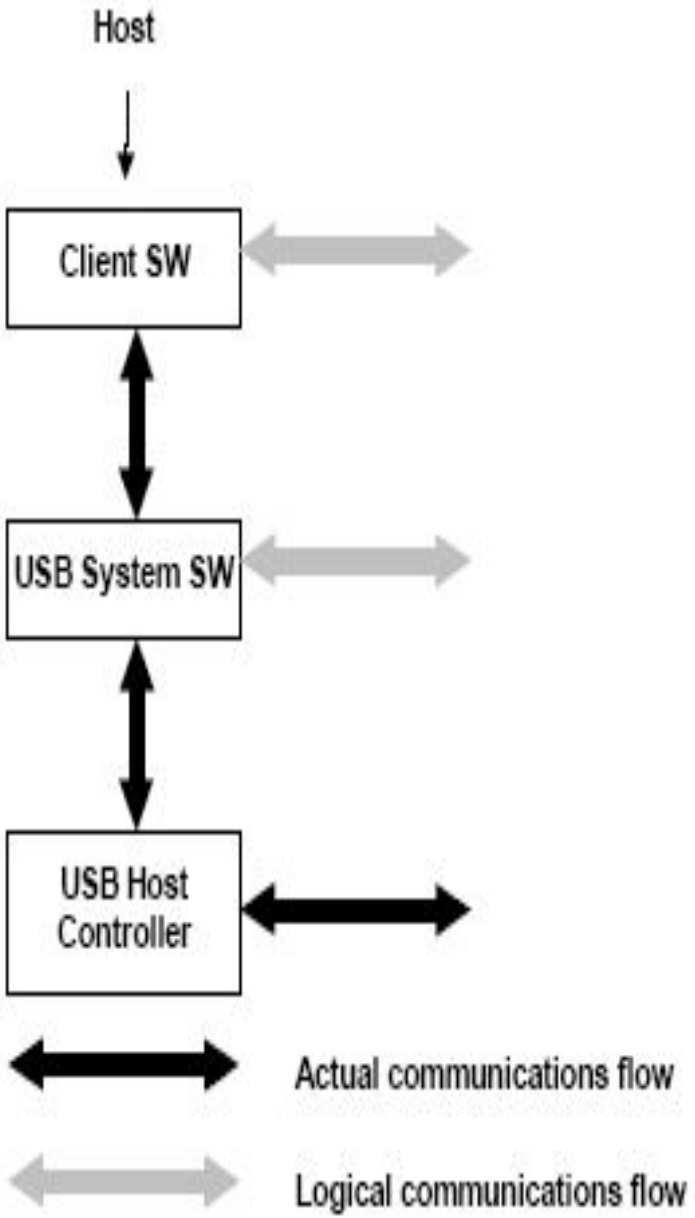
主机的逻辑结构如图包括:

· USB 主 机 控 制 器 （ USB Host Controller）

·USB系统软件集合：USB驱动程序，主机控制器的驱动程序，主机软件

·客户软件

USB主机在USB系统中是一个起协调作用的实体，它不仅占有特殊的物理位置，而且对于USB以及连到USB上的设备来说，还负有特殊责任。主机控制所有的对USB的访问。一个USB设备想要访问总线必须由主机给予它使用权。主机还负责监督USB的拓朴结构。



中国普天信息
www.cpit

USB设备

一个USB设备的逻辑结构如图所示，包括

- ·USB总线接口
- ·USB逻辑设备
- ·应用层

USB设备用于向主机提供一些额外的功能。USB设备提供的功能是多种多样的，但面向主机的接口却是一致的。所以，对于所有这些设备，主机可以用同样的方式来管理它们与USB有关的部分。

为了帮助主机辨认及确定USB设备，这些设备本身需要提供用于确认的信息。在某一些方面的信息，所有设备都是一样的；而另一些方面的信息，由这些设备具体的功能决定。信息的具体格式是不定的，由设备所处的设备级决定.

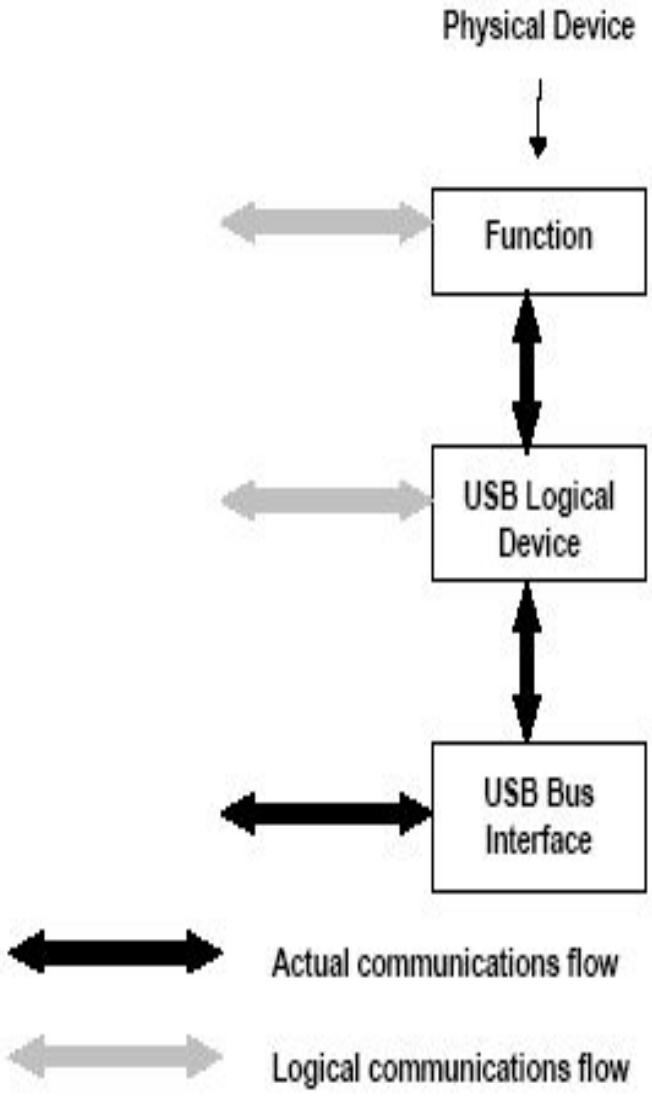


Figure 5-4. Physical Device Composition

中国普天信息技术
www.cpit.com.cn

Potevio
中国普天

总线的物理拓朴结构

USB系统中的设备与主机的连接方式采用的是星形连接，如图. 图中的Hub是一类特殊的USB设备，它是一组USB的连接点，主机中有一个被嵌入的Hub叫根Hub(root Hub)。主机通过根Hub提供若干个连接点。为了防止环状连接，采用星形连接来体现层次性，如图4-5。这种连接的形状很像一棵树。用于提供具体功能的设备叫应用设备。许多不同功能的设备放在一起被看作一个整体，叫包。例如，键盘和轨迹球可以被视作一个整体，在它的内部，提供具体功能的设备被永久地 接到Hub上，而这个Hub被接到USB上。所有这些设备及这个Hub被看作一个复合设备，而这个Hub又被看作这个复合设备的内部Hub。在主机看来，这个复合设备和一个带着若干设备的单独Hub是一样的。图中也标出了一个复合设备.

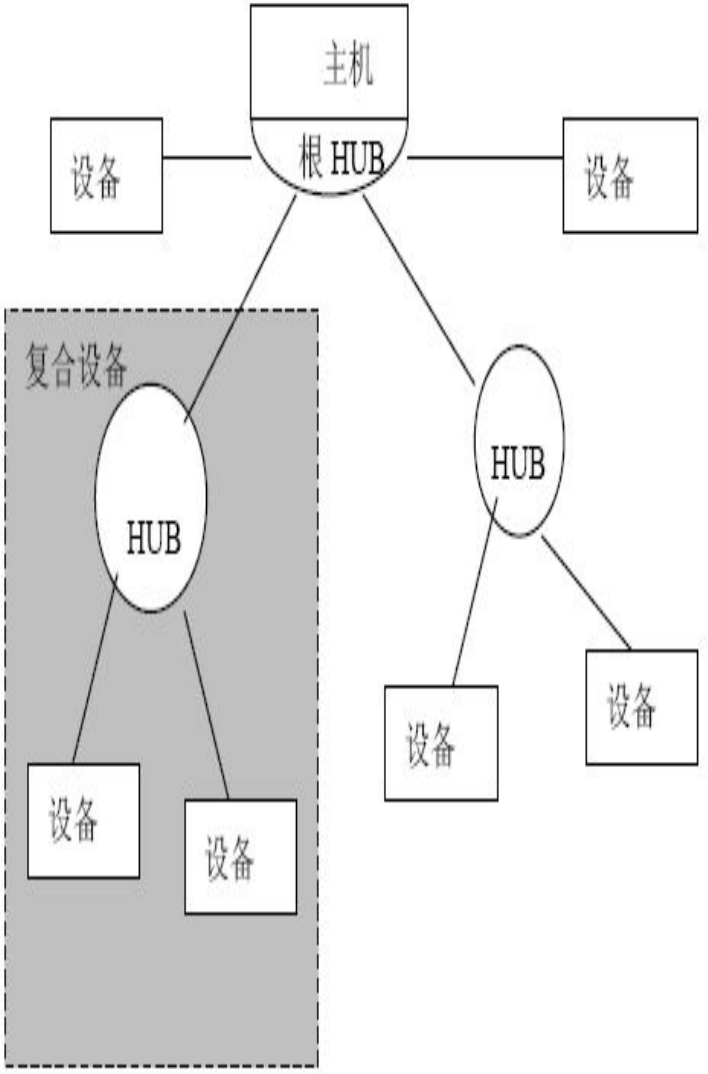
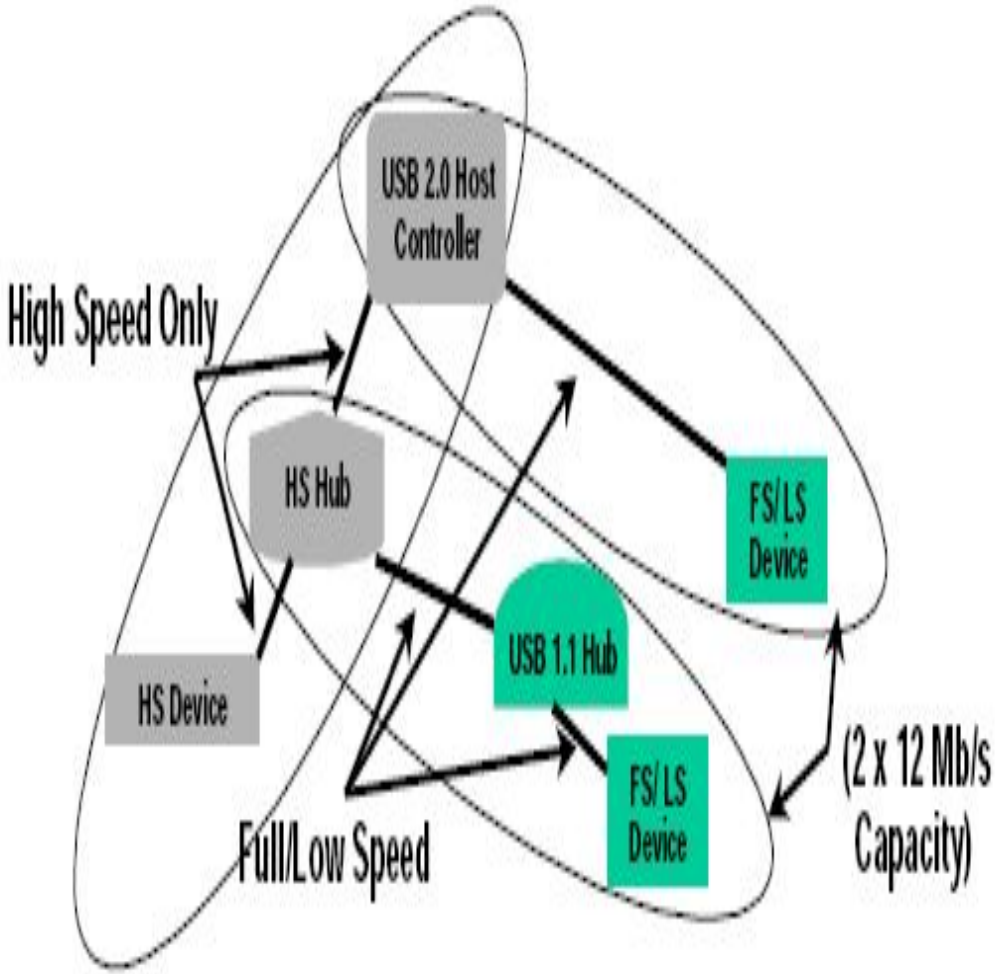


图 4—5 USB 物理总线的拓扑

www.cpit.com.cn

集线器在高速系统中扮演特殊的角色,集线器将全速/低速的信号环境和高速信号环境隔离开来.如图所示的是支持高速设备,并在高速下工作的集线器.集线器允许连接和操作全速/低速下的USB1.1集线器和其他只能进行全速/低速操作的设备.主机控制器也直接支持连接那些只能进行全速/低速操作的设备.每个高速运行的集线器可增加一条(或多条)额外的全速/低速总线,即每个集线器都支持额外的(任意多条)12Mb/s USB全速/低速宽带.这允许连接更多全速/低速总线,而不需要额外地增加主机控制器.尽管可以连接几条12MB/S的全速/低速总线,但是一台主机控制器最多只能连接127台USB设备.



USB逻辑总线的拓扑结构

在物理结构上，设备通过Hub连到主机上。但在逻辑上，主机是直接与各个逻辑设备通信的，就好像它们是直接被连到主机上一样。这个逻辑关系如图5-7所示。与之对应的物理结构就是图4-5中的结构。Hub也是逻辑设备，但在图5-7中，为了简化起见，未被画出，虽然USB系统中的工作都是从逻辑角度来看待的，但主机必须对物理结构有个了解。例如，在处理Hub被移去的情况时，当一个Hub被移出，通过它与主机相连的设备也应一起被移去，这是由其物理结构决定的.

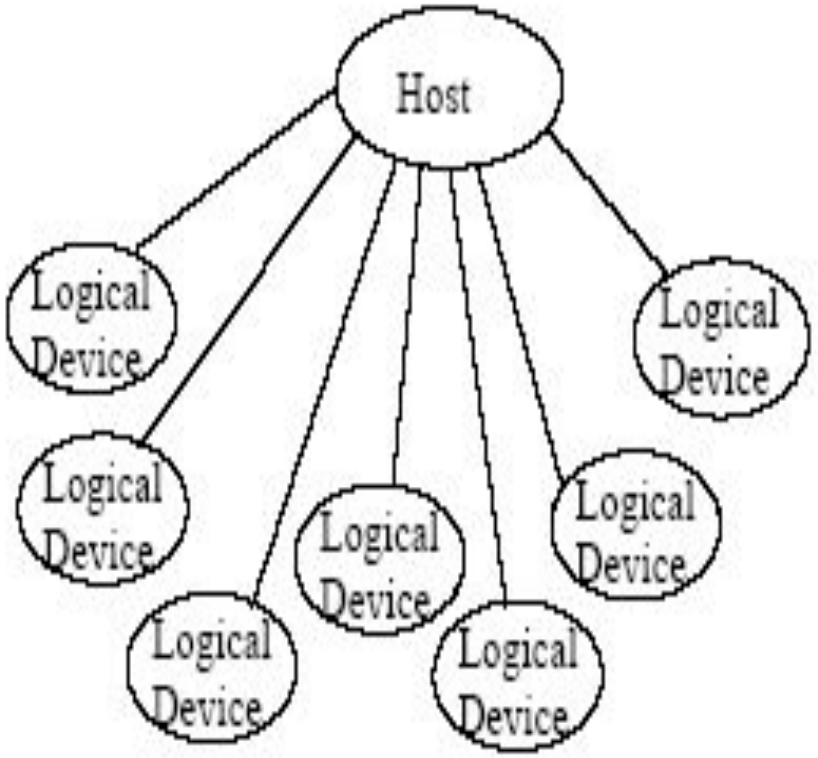


Figure 5-7. USB Logical Bus Topology

客户软件层与应用层的关系

USB系统的物理上、逻辑上的拓朴结构反映了总线的共享性。操纵USB应用设备的客户软件只关心设备上与它相关的接口，客户软件必须通过USB软件编程接口来操纵应用设备。这与另一些总线如PCI，EISA，PCMCIA等不同，这些总线是直接访问内存或I/O的。在运行中，客户软件必须独立于USB上的其它设备。这样，设备和客户软件的设计者就可以只关心该设备与主机硬件的相互作用和主机软件的相互作用的细节问题。图5-8说明了在图5-7的逻辑结构下，一个设备设计者看到的客户软件与相应应用的关系的视图.

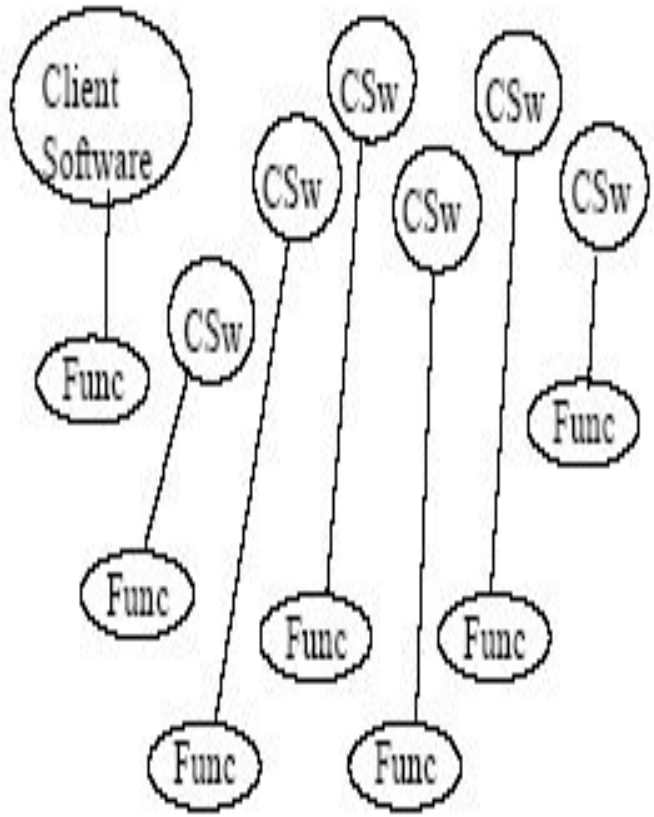


Figure 5-8. Client Software-to-function Relationships

www.cpit.com.cn

USB总线的通信数据流

USB是为主机软件和它的USB应用设备间的通信服务的，对客户与应用间不同的交互，USB设备对数据流有不同的要求。USB为此提供了更好的overall总线使用，它允许各种不同的数据流相互独立地进入一个USB设备。每种通信流都采取了某种总线访问方法来完成主机上的软件与设备之间的通信。每个通信都在设备上的某个端点结束.不同设备的不同端点用于区分不同的通信流. 图5-9是 详尽地描述了USB系统，支持了逻辑设备层和应用层间的通信流.

·主机控制器的驱动程序(HCD)：它位于USB主机控制器与USB系统软件之间。主机控制器可以有一系列不同的实现，而系统软件独立于任何一个具体实现。一个驱动程序可以支持不同的控制器，而不必特别了解这个具体的控制器。一个USB控制器的实现者必须提供一个支持它自己的控制器的主机控制器驱动器（HCD）实现.

·USB驱动程序(USBD)：USB系统软件与客户软件之间的接口，提供给客户软件一些方便的使用USB设备的功能。

一个USB逻辑设备对USB系统来说就是一个端点集合。端点可以根据它们实现的接口来分类。USB系统软件通过一个缺省的控制通道来管理设备。而客户软件用通道束管理接口。通道束的一端为端点，一端为缓冲区。客户软件要求通信数据在主机上的一个缓冲和USB设备上的一个端点之间进行。主机控制器或USB设备(取决于数据传送方向)将数据打包后在USB上传。由主机控制器(HC)协调何时用总线访问在USB上传递数据.

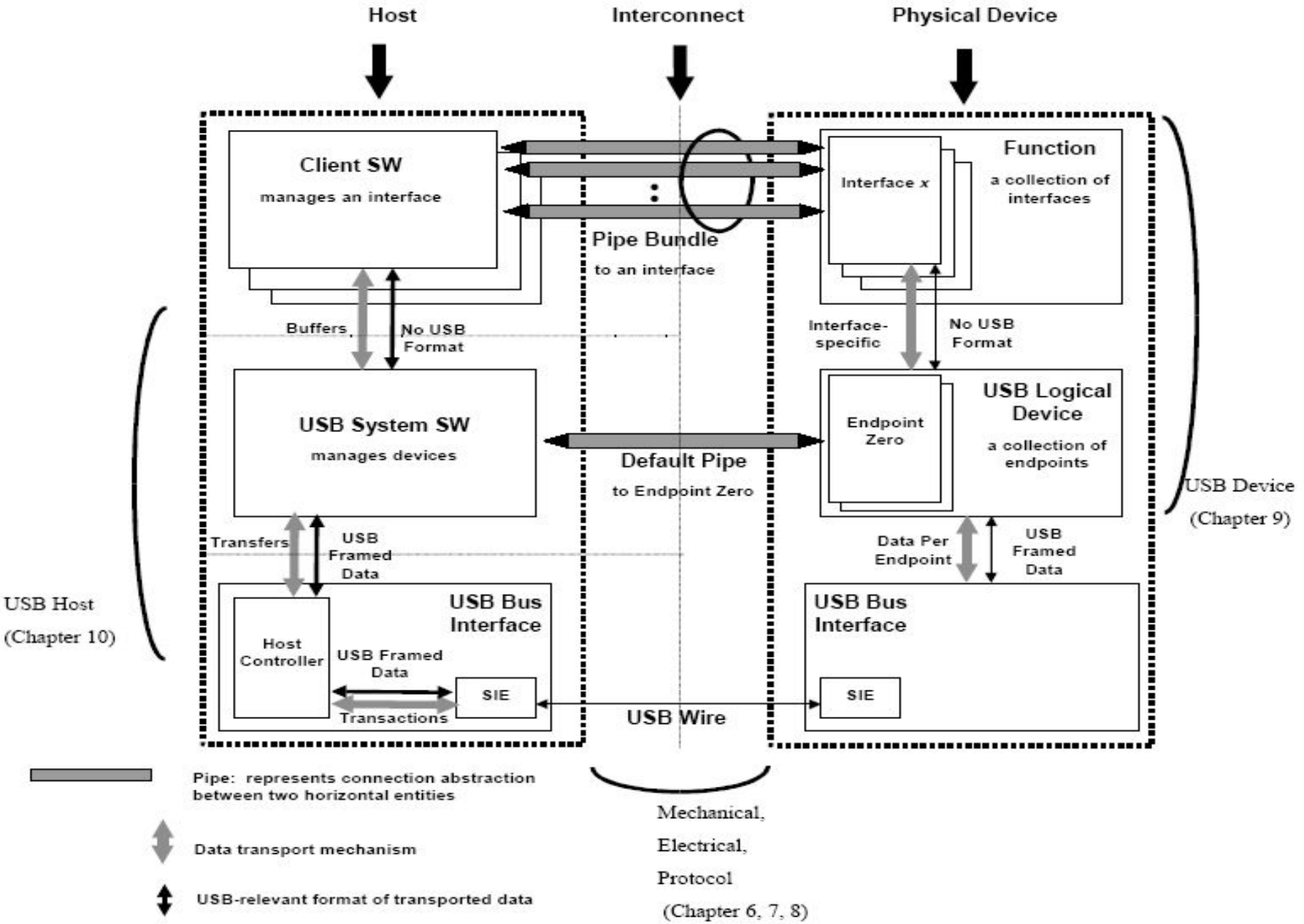


Figure 5-9. USB Host/Device Detailed View

图5-10说明了数据如何在主机侧中的内存缓冲和设备中的端点中传送。在后面，将逐步介绍端点、通道和通信流。主机上的软件通过一系列的通信流与逻辑设备进行通信。这一系列的通信流是由USB设备的软件和硬件设计者选择的，使设备能传送由USB提供的字符.

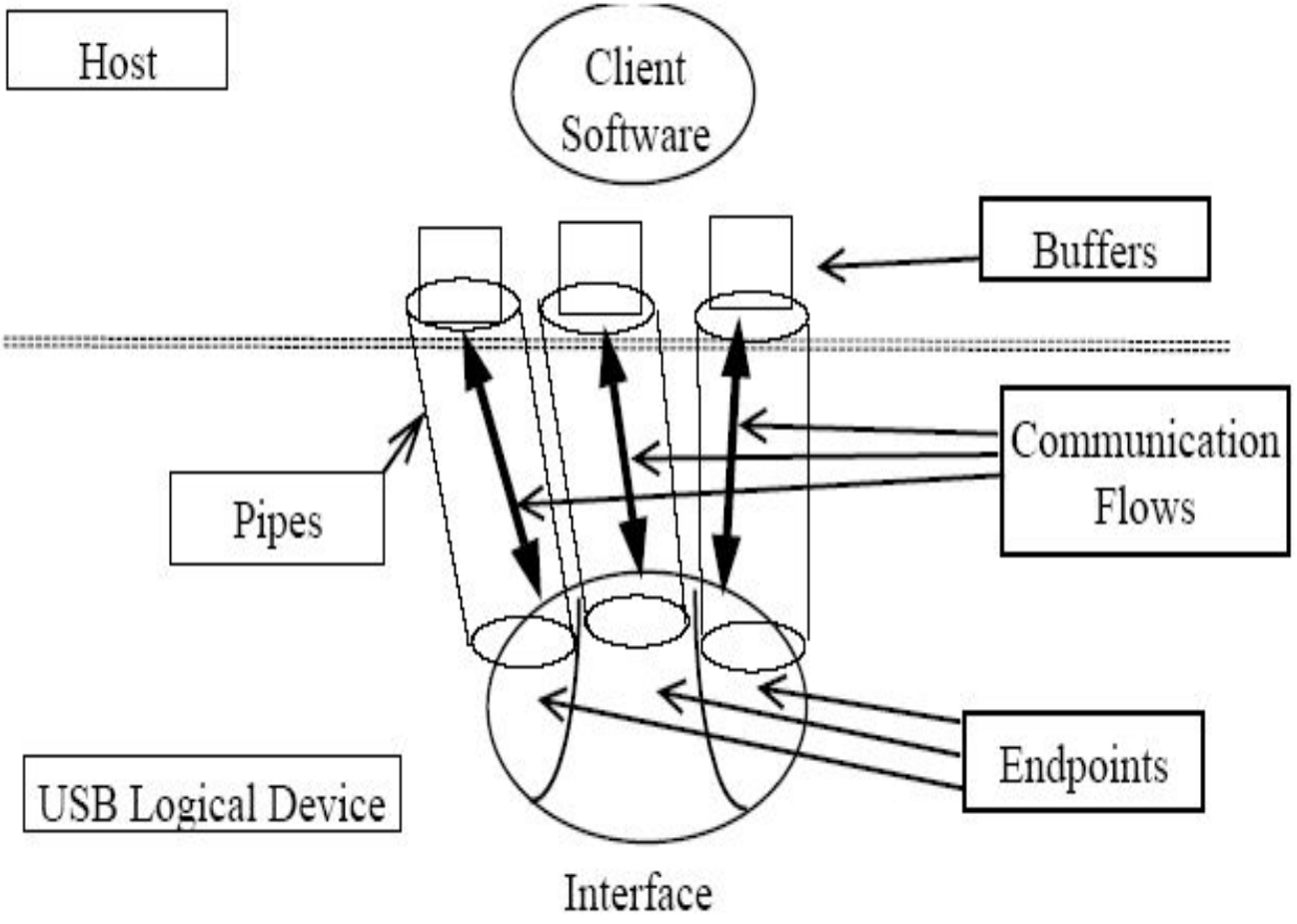
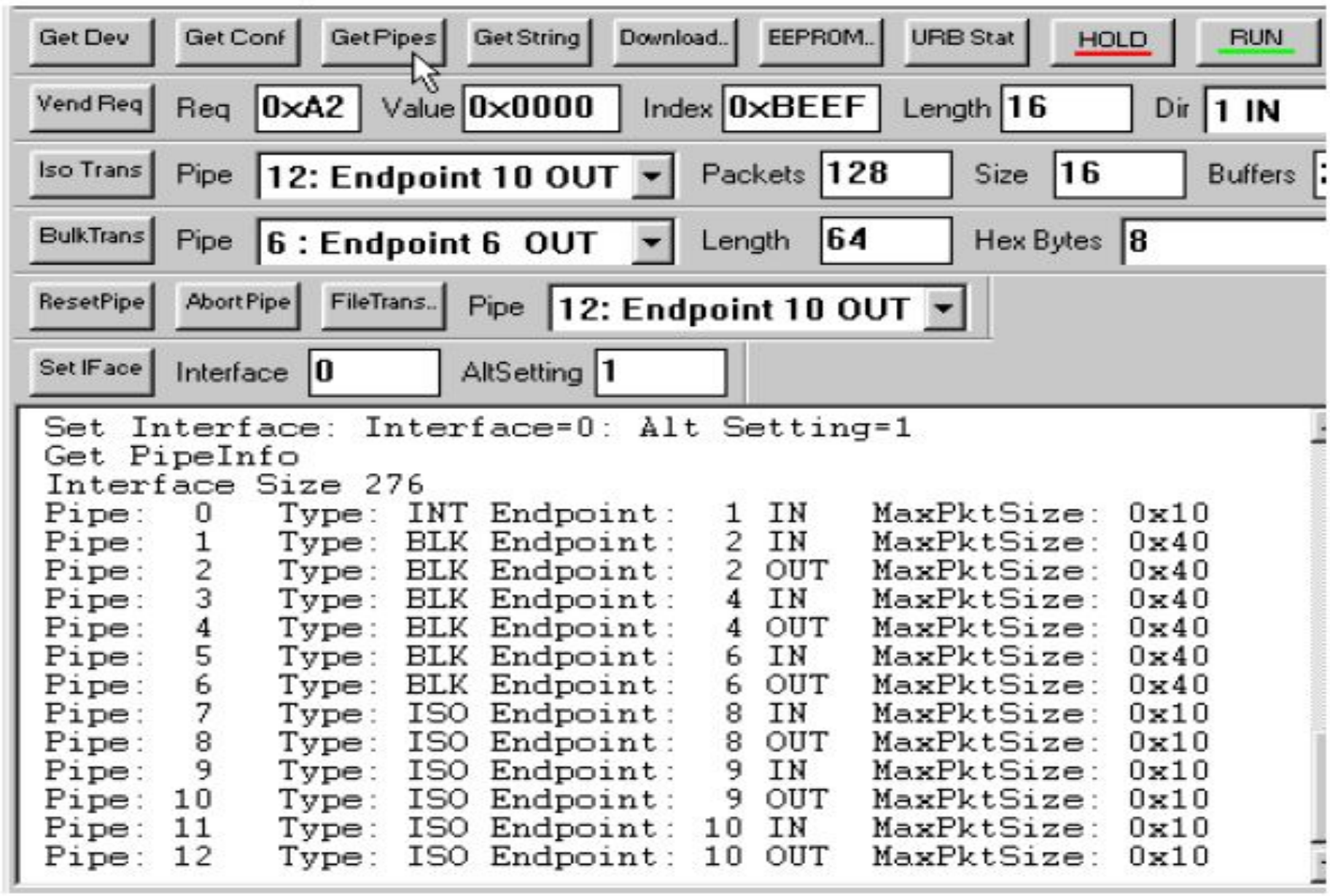


Figure 5-10. USB Communication Flow



CYPRESS核管道端点设定界面

中科普信息技术研究院 www.cpit.com.cn 普天信息技术研究院

USB设备端点

一个端点是一个可唯一识别的USB设备的Portion，它是主机与设备间通信流的一个结束点。一系列相互独立的端点在一起构成了USB逻辑设备。每个逻辑设备有一个唯一的地址，这个地址是在设备连上主机时，由主机分配的，而设备中的每个端点在设备内部有唯一的端点号。这个端点号是在设备设计时被给定的。每个端点都是一个简单的连接点，或者支持数据流进设备，或者支持其流出设备，两者不可得兼。

一个端点的特性决定了它与客户软件进行的传送的类型。一个端点有以下特性：

- **端点的总线访问频率要求**
- **端点的总线延迟要求**
- **端点的带宽要求**
- **端点的端点号**
- **对错误处理的要求**
- **端点能接收或发送的包的最大长度**
- **端点的传送类型出**
- **端点与主机的数据传送方向**

端点号不为0的端点在被设置前处于未知状态，是不能被主机访问的.

- **对0号端点的要求**

  所有USB设备都需要实现一个缺省的控制方法。这种方法将端点0作为输入端点，同时也将端点0作为输出端点。USB系统用这个缺省方法初始化及一般地使用逻辑设备(即设置此设备)。缺省控制通道 支持了对控制的传送 ，一旦设备接上，并加电，且又收到一个总线复位命令，端点0就是可访问的了.

  能在高速下工作的USB设备必须能在全速下工作,当设备连接到集线器并以全速工作时,设备必须:

  1.能成功地在全速下复位;

  2.能成功地响应标准请求,如设备和配置描述符的

  set_address,set_configuration,get_descriptor,并返回正确的信息.

  高速设备在全速下运行时可能或不支持这种功能.

- **对非0号端点的要求**

  设备可以有除0以外的其它端点，这取决于这些设备的实现。低速设备在0号输入及输出端点外，只能有2个额外的可选端点。而高速设备可具有的额外端点数仅受限于协议的定义(协议中规定，最多15个额外的输入端点和最多15个额外的输出端点)。

  除缺省控制通道的缺省端点外，其它端点只有在设备被设置后才可使用，对设备的设置是设备设置过程 的一部分.

www.cpit.com.cn

USB总线数据通道

一个USB通道是设备上的一个端点和主机上软件之间的联系。体现了主机上缓存和端点间传送数据的能力,有两不同的且互斥的通道通信格式。

·**流(Stream)：指不具有USB定义的格式的数据流。**

·**消息(Message)：指具有某种USB定义的格式的数据流。**

USB不解释在通道中传送的数据的内容。消息通道要求数据组织USB定义的格式，但它的内容，USB是不管的。特别地，有下列概念与通道相关：

·**对USB总线访问的申请(claim)，带宽的使用情况**

·**传送类型**

·**与通道相连的端点的特性**，例如：端点的数据传送方向,最大数据净负荷区的长度。数据净负荷是指在总线处理事务（transaction）中，数据包中数据区的数据 。由两个0号端点组成的通道叫缺省控制通道。一旦设备加电并复位后，此通道即可使用。其它通道只在设备被设置后才存在。USB系统软件在决定设备身份、设置要求和设置设备时使用缺省控制通道。当设备被设置后，这个设备的特定软件还可使用该通道。USB系统软件保留缺省控制通道的拥有权，协调其它客户软件对通道的使用。

一个客户软件一般都通过I/O请求包(IRP)来要求数据传送。然后，或者等待，或者当传送完成后被通知。IRP的细节是由操作系统来指定的。客户软件提出与设备上的端点建立某个方向的数据传送的请求，IRP就可简单地理解为这个请求。一个客户软件可以要求一个通道回送所有的IRP。当关于IRP的总线传送结束时，无论它是成功地完成，还是出现错误，客户软件都将获得通知说IRP完成了。

如果通道上没有正在传送的数据，也没有数据想使用此通道，这个通道就处于闲置状态。主机控制器对它不采取任何动作，也就是说，这个通道的端点会发现没有任何的总线动作是冲它而来的。只有当有数据在通道上时，该通道才能发现总线对它的动作.

如果一个非同步通道遇到一个迫使它给主机发STALL的情况(参见第7章)，或者在任一个IRP中发现3个总线错误。这个IRP将被中止。其它所有突出的IRP也一同被中止。通道不再接收任何IRP，直到客户软件从这个情况中恢复过来(恢复的方式取决于软件的实现)，而且承认这个中止或出现的错误，并发一个USBD Call来表明它已承认。一个合适的状态信息将通知客户软件IRP的结果——出错或中止。

一个IRP可能会需要多个数据净荷区来传递数据。这些数据区除最后一个外，都具有数据净荷区的最大长度，最后一个数据区包含了这个IRP中剩下的数据 .对这样的一个IRP，短包(也就是说未达到最大长度的数据区)在数据输入时无法填完IRP数据缓冲区。这可能会有二种不同解释，它依赖于客户软件的情况：

·如果该客户软件可以接受变长的IRP，那么，IRP数据缓冲区未被填满，可以看作一个分限，说明一个IRP已成功结束，主机控制器可以准备接收下一个IRP了。

·如果该客户软件只收定长的IRP。那么，我们认为发生了一个错误，这IRP将被中止，通道也会被阻塞，通道上的数据都中止。

因为对这两种情况，主机控制器会有不同的反应，而且采取何种措施不由控制器决定，所以对每个IRP都必须说明客户软件的具体要求。

通道的端点可以用NAK信号来通知主机自己正忙，NAK不能作为向主机反还IRP的中止条件。在一个给定的IRP处理过程中，可以遇到任意多个NAK，NAK不构成错误,而且不作为上面所说的3种错误之一.

- **流通道**

流通道中的数据是流的形式，也就是该数据的内容不具有USB要求的结构。数据从流通道一端流进的顺序与它们从流通道另一端流出时的顺序是一样的，流通道中的通信流总是单方向的。

对于在流通道中传送的数据，USB认为它来自同一个客户。USB系统软件不能够提供使用同一流通道的多个客户的同步控制。在流通道中传送的数据遵循先进先出原则。

流管流只能连到一个固定号码的端点上，或者流进，或者流出。(这个号码是由协议层决定的）而具有这个号码的另一个方向的端点可以被分配给其它流通道。

- **消息通道**

消息通道与端点的关系同流通道与端点的关系是不同的。首先，主机向USB设备发出一个请求；接着，就是数据的传送；最后，是一个状态阶段。为了能够容纳请求/数据/状态的变化，消息通道要求数据有一个格式，此格式保证了命令能够被可靠地传送和确认。

消息通道允许双方向的信息流，虽然大多数的通信流是单方向的。特别地，缺省控制通道也是一个消息通道。

USB系统软件不会让多个请求同时要求同一个消息通道。一个设备的每个消息通道在一个时间段内，只能为一个消息请求服务，多个客户软件可以通过缺省控制通道发出它们的请求，但这些请求到达设备的次序是按先进先出的原则的。设备可以在数据传送阶段和状态阶段控制信息流，这取决于这些设备与主机交互的能力 。正常情况下，在上一个消息未被处理完之前，是不能向消息通道发下一个消息的。但在有错误发生的情况下，主机会取消这次消息传送，并且不等设备将已收的数据处理完，就开始下一次的消息传送。在操作通道的软件看来，一个IRP中的错误，使这个IRP被取消，并且所有正排队等待的IRP一同也被取消。申请这个IRP的客户被通知IRP结束，且有出错提示。消息通道后有两个相同号码的端点，一个用于输入，一个用于输出。两个号码必须相同.

传送类型

USB通过通道在主机缓冲区与设备端点间传送数据。在消息通道中传递的数据具有USB定义的格式，它的数据净荷区中包含的数据允许具有设备指定的格式。USB要求任何在通道上传送的数据均被打包，数据的解释工作由客户软件和应用层软件负责。USB提供了多种数据格式，使之尽可能满足客户软件和应用软件的要求。一个IRP需要一个或多个总线处理事务来完成。

每个传送类型在以下的几个传送特征上会有不同：

- **USB规定的数据格式**
- **信息流的方向**
- **数据包的长度限制**
- **总线访问的限制**
- **延时的限制**
- **要求的数据顺序**
- **出错处理**

USB设备的设计者可以决定设备上每个端点的能力。一旦为这个端点建立了一个通道，这个通道的绝大多数传送特征也就固定下来了，一直到这个通道被取消为止。也有部分传送特征可以改变，对这样的特征，将会在介绍每个传送类型时作出说明. USB定义了4种传送类型：

1. **控制传送：可靠的、非周期性的、由主机软件发起的请求或者回应的传送，通常用于命令事务和状态事务 .**

2. **同步传送：在主机与设备之间的周期性的、连续的通信，一般用于传送与时间相关的信息。这种类型保留了将时间概念包含于数据中的能力。但这并不意味着，传送这样数据的时间总是很重要的，即传送并不一定很紧急。**

3. **中断传送：小规模数据的、低速的、固定延迟的传送。**

4. **批传送：非周期性的，大包的可靠的传送。典型地用于传送那些可以利用任何带宽的数据，而且这些数据当没有可用带宽时，可以容忍等待 .**

www.cpit.com.cn

一个(微型)帧 包含的最大处理数的表举例:

每张表包含以下内容:

- 指定传输类型(和速度)要求的协议开销
- 对某些采样数据有效负载大小为
  - 在这种情况可能保持的最大宽度;
  - 每个处理要求的(微型)帧百分比;
  - 在特定情况下, (微型)帧中的最大处理数;
  - 在特定情况下, (微型)帧中不要求的剩余字节;
  - 在特定情况下, 在单个(微型)帧传输的总的数据字节数;

典型的特殊传输类型处理要求多个包,每种处理的协议开销包括:

- 每个包的SYNC场为8位(全速/低速)或32位(高速);
- 每个包的PID字节包括PID和PID取反(校验)位;
- 每个包的EOP为3位(全速/低速)或8位(高速);
- 在令牌中还包括端点号,设备地址,和CRC场(共16位);
- 数据包包括CRC16场(共16位);
- 数据包可以包含任意数量的数据场(每字节8位);

- 对于多包的处理,要求有包间间隙或总线转向时间

  在这些计算中假设没有位填充的要求,用低速中断OUT作为例子,它在处理中共有5个包:

- – 一个PRE专用包;
- – 一个令牌包;
- – 一个PRE专用包;
- – 一个数据包;
- – 一个握手包;

其中,在数据和握手之间有个总线转向.因此,协议开销包括:5个SYNC,5个PID,端点+CRC5,CRC16,5个EOP和包间延时(一总线转向,包间的延时和2个集线器建立时间)

控制传送

控制传送允许访问一个设备的不同部分。控制传送用于支持在客户软件和它的应用之间的关于设置信息、命令信息、状态信息的传送。控制传送由以下几个事务组成：(1)建立联系，把请求信息从主机传到它的应用设备；(2)零个或多个数据传送事务，按照（1）事务中指明的方向传送数据；(3)状态信息回传。将状态信息从应用设备传到主机。当端点成功地完成了被要求的操作时，回传的状态信息为“success”.

USB设备必须实现缺省控制通道，并将它实现成一个消息通道。这个通道由USB系统软件使用。USB设备的确认信息、状态信息以及控制信息由该通道传送。如果需要的话，一个应用设备可以为端点实现额外的控制通道。

USB设备框架定义了标准的，设备级的或由销售商提供的请求，这些请求可操作设备的状态。USB设备框架又定义了一些描述器（descriptor），用于存放USB设备的各种信息。控制机制提供访问设备描述器和请求操作设备的机制。

控制传送只能通过消息通道进行。所以，使用控制传送的数据必须具有USB定义的数据格式 。

应用层和相应的客户软件不能为控制传送指定总线访问频率和带宽。这由USB系统软件从全局优化角度加以决定。USB系统软件会限制设备要求的访问频率和带宽.

1 控制传送类型的数据格式

Setup包的数据格式属于一个命令集，这个集合能保证主机和设备之间正常通信。这个格式也允许一些销售商对设备命令的扩展。Setup包后的数据传送也具有USB定义的格式，除非这个数据是销售商提供的信息。回传的状态信息仍然具有USB定义的格式。

2 控制传送的方向

控制传送使用的是消息通道上的双向信息流。所以，一旦一个控制通道被确认之后，这个通道就使用了具有某个端点号的两个端点，一个输入，一个输出。

3 控制传送包的大小的限制

控制传送的端点决定了它所能接收或发送的最大数据净负荷区长度。USB为高速设备定义的最大数据净负荷区长度为8、16、32或64字节，低速设备的数据净负荷区的长度只能是8字节。Setup后的所有数据包都要遵守这个规定，这个规定是针对这些数据包中的数据净负荷区的，不包括包中的协议要求的额外信息，Setup包实际上也是8字节。控制通道(包括缺省控制通道)总是使用w Max Packet Size的值。

端点在自己的设置信息中报告自己允许的最大净负荷区长度。USB不要求数据净负荷区必须达到最大长度，当长度不够时，不必填充到最大长度。

主机控制器对高速设备的控制通道端点支持8、16、32、64字节的最大长度，对低速设备支持8字节的长度。它不能支持更大的或更小的其它长度。

对于缺省控制通道的最大数据区长度，USB系统软件要从设备描述器的头8个字节中读出，设备将这8个字节放在一个包中发出，其中的七个字包含了缺省通道的wMaxPacketSize。对其它的控制端点来说，USB系统软件在它们被设置后，获得此长度，然后USB系统软件就会保证数据净负荷区不会超长。另外，主机总是认为数据净负荷区的最大长度至少为8。

端点所传的数据净负荷区长度必须小于或等于其wMaxPacketSize ，当一个数据区不能容纳所传数据时，就分几个区来传。除最后一个区外，其它区都应达到最大长度。最后一区包含最后剩下的数据。

当端点做了以下两件事时，控制传送的数据阶段可被认为结束：

·已传了由Setup阶段指定的数据量。

·传了一个数据包，它的长度为0或它的数据区长度小于最大长度。

数据阶段结束后，主机控制器进入状态阶段，而不是开始另一个数据传诵。如果它不这样做，端点会认为通道脱线而中止通道(通道脱线见4.3.2)。如果主机在状态阶段时，主机收到一个大于最大长度的数据区，那么请求这次传送的IRP将被中止。

当数据全部传完，主机与端点之间的控制传送的数据阶段结束。如果其间，端点收到了超过最大长度的数据区，它将中止通道。

www.cpit.com.cn

控制传送的总线访问的限制

无论低速设备还是高速设备都可以使用控制通道。

端点没法指明控制通道对总线访问频率的要求。USB权衡所有控制通道的总线访问频率和正等待的IRP，从全局优化，提供一个"最佳"传送方案。

USB要求数据帧中的一部分被留给控制传送使用。

- 如果被引发的控制传送(引发方式由实现决定)只用了数据帧的不到10%的时间，则剩余的时间留给批传送 。
- 如果一个控制传送被引发又被中止，则它的中止可在本次的帧内，也可在以后的帧内。也就是说，引发和中止不必在同一个帧内。
- 如果留给控制传送的时间不够用，但恰好有一些同步和中断传送的帧时间未用，则主机控制器利用这些时间进行额外的控制传送。
- 如果对可用的帧时间有太多的控制传送在等待，那么就对它们进行排序然后传送。
- 如果各个控制传送申请的是不同的端点，主机控制器根据公平访问原则决定它们的访问顺序。公平访问原则的具体内容决定于主机控制器的实现。
- 如果一个控制传送事务频繁地被中止，不能认为给它的总线访问时间是不公平的。

这些要求使得控制传送一般可以在总线上进行规则地、最优化地传送。

高速控制端点必须支持OUT处理的PING流控制协议

对某个端点的控制传送的速率是可以变化的，USB系统软件控制这些离散的变化。端点和其客户软件不能想当然的认为其有一个固定的传送速率，端点可能发现在一帧内有零个或若干个传送。一个端点和它相应的客户软件可占用的总线时间会因为其它设备进入或退出系统或者本设备上的其它端点进入或退出系统而改变。

总线频率和帧定时决定于一个帧内可传送的控制传送的最大个数。在任一个USB系统内，一个帧内的8字节高速数据区须少于29个，8字节低速数据区须少于4个。对于高速总线,控制传输数量限制为每个微型帧小于32个高速的64字节数据有效负载.表 是关于不同规格的高速的控制传送的情况，以及在一帧内可能的最大的传送数目。这张表有两个默认的前提，即控制传送有一个数据传送阶段而且这个数据传送阶段有一个长度为0状态阶段，表还指出了出现两个数据区都达不到最大长度的情况，表中不包括用于位填充的一些额外的位。

Table 5-3. High-speed Control Transfer Limits

	Protocol Overhead (173 bytes)		(Based on 480Mb/s and 8 bit interpacket gap, 88 bit min bus turnaround, 32 bit sync, 8 bit EOP: (9x4 SYNC bytes, 9 PID bytes, 6 EP/ADDR+CRC,6 CRC16, 8 Setup data, 9x(1+11) byte interpacket delay (EOP, etc.))			
	Data Payload	Max Bandwidth (bytes/second)	Microframe Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/ Microframe Useful Data
	1	344000	2%	43	18	43
	2	672000	2%	42	150	84
	4	1344000	2%	42	66	168
	8	2624000	2%	41	79	328
	16	4992000	3%	39	129	624
	32	9216000	3%	36	120	1152
	64	15872000	3%	31	153	1984
Max		60000000				7500

Table 5-2 lists information about different-sized full-speed control transfers and the maximum number of transfers possible in a frame. This table was generated assuming that there is one Data stage transaction and that the Data stage has a zero-length status phase. The table illustrates the possible power of two data payloads less than or equal to the allowable maximum data payload sizes. The table does not include the overhead associated with bit stuffing.

Table 5-2. Full-speed Control Transfer Limits

	Protocol Overhead (45 bytes)	(9 SYNC bytes, 9 PID bytes, 6 Endpoint + CRC bytes, 6 CRC bytes, 8 Setup data bytes, and a 7-byte interpacket delay (EOP, etc.))				
	Data Payload	Max Bandwidth (bytes/second)	Frame Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/Frame Useful Data
	1	32000	3%	32	23	32
	2	62000	3%	31	43	62
	4	120000	3%	30	30	120
	8	224000	4%	28	16	224
	16	384000	4%	24	36	384
	32	608000	5%	19	37	608
	64	832000	7%	13	83	832
Max		1500000				1500

因为一个帧内只留10%的时间给非周期性传送，所以当一个系统的总线时间被排满的时候，这个系统内的所有控制传送只能去竞争每个帧内的三个控制传送名额。因为除了客户软件会要求控制传送外，USB系统要用控制传送来传送设置信息，所以对某个客户和它的应用就不能指望它们的控制传送像它们想的一样进行。主机控制器可以自由地决定如何将某个具体的控制传送在总线上进行，可以在一个帧内，也可以跨几个帧。一个端点可能发现一个控制传送的各个总线处理事务在同一帧内或分在几个不连续的帧内。由于具体实现的不同，主机控制器可能不能提供理论上的每帧的最大控制传送数目。

低速控制传送与高速控制传送都是竞争同样多的可用帧时间。低速控制传送只是要用更多的时间来传送罢了。表5-1列出了不同规格的低速包的情况，以及一帧内允许的最大包数。这张表同样没包括进管理用的开销。无论低速与高速，由于一个控制传送都由几个包组成，所以都可能要用几个帧才能完成传送。

Table 5-1. Low-speed Control Transfer Limits

Protocol Overhead (63 bytes)		(15 SYNC bytes, 15 PID bytes, 6 Endpoint + CRC bytes, 6 CRC bytes, 8 Setup data bytes, and a 13-byte interpacket delay (EOP, etc.))				
	Data Payload	Max Bandwidth (bytes/second)	Frame Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/Frame Useful Data
	1	3000	26%	3	40	3
	2	6000	27%	3	37	6
	4	12000	28%	3	31	12
	8	24000	30%	3	19	24
Max		187500				187

控制传送的数据顺序

要进行控制传送，先要由主机向设备发一个总线建立（Setup）信息。它描述了控制访问的类型，设备将执行此控制访问。这个阶段之后，是零个或多个控制数据信息的传送，这是进行访问的具体信息。最后，由状态信息的传送来结束这次控制传送，允许端点将这次控传的状态回送给客户软件。这次控传完成之后，可以进行对这个端点的下一个控传， 每次控传何时在总线上进行由主机控制器的具体实现决定。

在数据传送阶段和状态信息回传阶段，可能由于设备自身的原因，设备处于"忙"状态。此时端点可设法表明自己正忙 ，主机将试着在稍后时间重传一次。

如果在上一个控传结束之前，端点又收到一个总线建立信息，设备将结束现未完成的传送，转而处理新的控传。正常情况下，是不会早发总线建立信息的，不过当上一个控传因错误而被中止后，主机可发下一个控传的总线建立信息。在端点看来，这是在上一个控传结束前过早发出的。

一旦主机遇到一个引起中止的条件或检测到一个错误，端点可以通过接收下一个Setup包的 PID来恢复，也就是说，不一定必须从别的通道进行恢复。对于缺省控制通道，如果端点收不到Setup的 PID时，最终会要求设备复位来清除中止条件或错误条件。

在控传中，USB提供了强大的错误检测功能和错误恢复和重传功能。传送器和接收器可以保持阶段的同步，既关于他们在控传的哪个阶段这个问题上保持同步。并且以最小的代价恢复。接收器可以识别一个数据重传包或状态信息重传包，因为包中带有数据重传的指示。一个发送器可以通过对方给它发的握手信息确知它发的数据重送包和状态信息包已被成功接收，除了Setup包以外，协议可以将一个重送的包与原来的包区分开来，Setup包可以因为出错而重传，但无法说明此包是重传的，还是原来的.

同步传送

在非USB的环境下，同步传送意味着恒定速率、错误容忍(error-tolerant)的传送。在USB环境下，要求同步传送能提供以下几点：

·固定的延迟下，确保对USB带宽的访问。

·只要数据能提供得上，就能保证通道上的恒定数据传送速度。

·如果由于错误而造成传送失败，并不重传数据。

当USB同步传送类型被用来支持同步的源和目的时，使用这个传送类型的软件并不要求是同步的，后面章节详细介绍USB上的同步数据的处理。.

1 同步传送的数据格式

对于同步传送的通道(同步通道)，USB并不对数据格式做要求.

2 同步传送的方向

同步通道是一种流通道，所以是单方向的。在对端点的描述中指明了与它相连的通道的数据流方向。如果设备要同步的双向流的话，只好用两个同步通道，一个流进，一个流出.

3 同步传送中包的大小的限制

同步通道的端点确定了数据区的最大长度，USB在设置端点期间，使用这一个信息，看是否可在每帧内为最大长度的数据区留下足够的时间。如果可以，设置端点成功；否则，不成功.

USB将全速同步端点的最大数据有效负载限制为1023字节.高速端点允许最大为1024字节的数据有效负载.高速,高带宽的端点指定了它在每个微型帧中要求两个或三个处理.表5.4列出了不同的全速同步处理和在一帧中可能的最大处理数的信息.表的阴影部分表示的全速端点(WMAXPACKETSIZE 不为0)不能是默认接口设置的一部分.表中不与位填充有关的信息.

www.cpit.com.cn

Table 5-4. Full-speed Isochronous Transaction Limits

Protocol Overhead (9 bytes)	(2 SYNC bytes, 2 PID bytes, 2 Endpoint + CRC bytes, 2 CRC bytes, and a 1-byte interpacket delay)				
Data Payload	Max Bandwidth(bytes/ second)	Frame Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/Frame Useful Data
1	150000	1%	150	0	150
2	272000	1%	136	4	272
4	460000	1%	115	5	460
8	704000	1%	88	4	704
16	960000	2%	60	0	960
32	1152000	3%	36	24	1152
64	1280000	5%	20	40	1280
128	1280000	9%	10	130	1280
256	1280000	18%	5	175	1280
512	1024000	35%	2	458	1024
1023	1023000	69%	1	468	1023
Max	1500000				1500

表5.5列出了不同的高速同步处理的微型帧中可能的最大处理数的信息.表的阴影部分表示的高速同步端点不能是默认接口设置的一部分.表中不包含与位填充有关的开销信息.

任何同步管道的给定处理不必是为端点指定的最大数据的有效负载.数据有效负载值有发送器(客户软件和功能设备)决定,而且可以根据各个处理的不同要求变换.USB确保主机控制器提出的任意数据有效负载都能在总线上传输.数据有效负载的实际值有数据发生器设定,而且可能小于预先协商的最大值.总线错误会改变接受器看到的实际包的值.这些错误可以通过数据的CRC场或接收器对所有处理要求的大小检测出来.

所有设备的默认接口设置不能包括任何数据有效负载不为0(通过端点描述的WMAX-PACKESIZE指定)的同步端点.其他接口配置可以为同步端点指定非零的数据有效负载.如果同步端点有大的数据有效负载,建议用其他配置或接口指定数据有效负载的范围.这增加了设备其他USB设备组合使用的机会.

www.cpit.com.cn

Table 5-5. High-speed Isochronous Transaction Limits

Protocol Overhead		(Based on 480Mb/s and 8 bit interpacket gap, 88 bit min bus turnaround, 32 bit sync, 8 bit EOP: (2x4 SYNC bytes, 2 PID bytes, 2 EP/ADDR+addr+CRC5, 2 CRC16, and a 2x(1+11)) byte interpacket delay (EOP, etc.))			
Data Payload	Max Bandwidth (bytes/second)	Microframe Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/ MicroFrame Useful Data
1	1536000	1%	192	12	192
2	2992000	1%	187	20	374
4	5696000	1%	178	24	712
8	10432000	1%	163	2	1304
16	17664000	1%	138	48	2208
32	27392000	1%	107	10	3424
64	37376000	1%	73	54	4672
128	46080000	2%	45	30	5760
256	51200000	4%	25	150	6400
512	53248000	7%	13	350	6656
1024	57344000	14%	7	66	7168
2048	49152000	28%	3	1242	6144
3072	49152000	41%	2	1280	6144
Max		60000000			7500

同步传送的总线方向限制

只有全速和高速设备可以使用同步方式 .

USB要求全速端点的任意帧为周期(同步和中断)传输分配的时间不超过90%,高速端点能为周期传输分配最多80%的微型帧时间.

同步端点必须指定它要求的总线访问周期.全速/高速端点必须指定要求的周期是(2bInterval-1)xF, 其中bInterval的范围是0~16（包括16），而F在高速指是125us,而在全速指是1ms.这就允许全速/高速同步传输的速率低于每（微型）帧一个处理。但是，一个同步传输端点必须准备好处理比指定快的轮询速率。主机不能在同步端点发出超过一个处理的（微型）帧，除非端点是高速，高带宽端点。当有比指定快的间隔请求数据而且数据不用时，同步IN端点必须返回0长度包。

高速端点能在每（微型）帧（或192Mb/s）传输高达3072字节。在每周期要求超过1024字节的高速同步端点称为高带宽端点。高带宽端点在每个微型帧使用多个处理。高带宽端点必须指定周期是1X125us（即bInterval的值是1）。

www.cpit.com.cn

总线错误或操作系统在调度客户软件出现的延时会导致没有包在（微型）帧中传输。此时，应将错误指示器作为一个状态信息返回给客户软件。设备通过跟踪SOF令牌也能检测出这种情况，而且它会指出在指定的总线访问周期模式有扰动出现。

总线频率和（微型）帧定时限制了任何USB系统的（微型）帧中最大同步处理数不仅要小于己151个全速1字节数据有效负载，而且要小于193个高速1字节数据有效负载，根据不同应用的需要，主机控制器不可能使每个（微型）帧的最大同步处理数达到理论值。

同步传送的数据顺序

同步传送不支持因总线错误而进行的重传。接收器可以判断是否发生了一个错误，低级的USB协议不允许有握手信号给同步通道的发送者。一般情况下，是可以有握手信号来通知发送者包是否被成功地接收。对于同步传送来说，定时比正确性和重传更重要。考虑到总线的错误率较低，协议就认为传送一般均能成功。同步接收者可以判断自己是否在一个帧内错过了一些数据，而且能知道丢失了多少数据。

因为没有用来指示引起中止的条件的握手信号，所以同步传送的端点从不途停止。虽然，错误信息可作为IRP的状态来报告，但同步通道不会因此停下。错误即使被查到，主机仍继续处理下一帧的数据。因为同步传送的协议不支持每次事务都进行握手，所以错误检测的功能可以相对弱一些。

中断传送

中断传送是为这样一类设备设计的，它们只传或收少量数据，而且并不经常进行传送，但它们有一个确定的服务周期，对中断传送有以下要求：

·通道的最大服务期得到保证。

·由于错误而引起的重发在下一服务期进行。

1 中断传送的数据格式

USB对中断通道上的数据流格式无要求。

2 中断传送的方向

中断通道是一种流通道，所以是单向的。端点描述信息指明了通道的数据流方向。

3 中断传送对包的长度的限制

中断通道的端点决定自己能接收和发送的最大数据区长度，全速设备允许最大不超过64字节(或更少)的数据区，高速端点允许的最大数据有效负载高达1024字节。高速，高带宽的端点指定了每个微型帧要求两个或三个处理。而低速设备只允许不超过8个(或更少)字的数据区 。这个最大值将应用到数据包的数据的有效负载，即指定的包数据场的大小，不包括其他协议要求的信息。USB不要求数据包正好是最大值，即如果数据包小于最大值，它不需要被填补成具有最大数据有效负载。

所有的主机控制器都要示支持高速设备的1024字节数据去和全速设备的64字节数据区和低速设备的8字节(或更少)的最大数据区，对超过最大值的数据区则不要求支持 。

USB系统软件设置中断通道的最大数据区长度。在设备设置期间，这一信息将被使用，只有此设置有效，这个数值是不会改变的。在设置有效期间，USB系统软件根据此数值来看分给这个通道的总线时间是否充分。如果充分，则通道建立，否则不建立。与控制通道不同，USB系统不为中断通道调整总线时间。所以对给定的USB系统，要么支持此通道，要么不支持。实际传送的数据区长度由发送器决定，可以小于最大长度。

端点所发的数据区中的数据长度不能超过端点的w Max Packet Size的值。而设备可以通过中断传送来传比此值多的数据。客户软件可以通过中断传送的IRP来接收这批数据，这个中断传送要求多个总线处理事务来完成，且要求每个事务后都有IRP完成的信号。可以设置一个缓冲区，它的长度为w Max Packet Size的整数倍，再加上一个零头。对需要的多个总线事务来说，除最后一个外，前面的事务都传递w Max Packet Size长度的包，后一个传剩下的零头。这些总线处理事务都在为通道建立的服务周期内进行。

www.cpit.com.cn

如果一个中断传送要传的数据不能放在一个数据区中，就分几个区，前几个区都是最大长度，最后一个包含剩下的长度。当出现以下情况时，认为中断传送结束：

·已传的数据量恰好与期望的数据量同。

·传了一个有一个数据区的包，此包的长度小于w Max Packet Size或传了一个长度为零的包。

如果一个中断传送完成，那么主机控制器结束当前的IRP，并开始下一个IRP。如果数据区的长度比预料的长，当前IRP中止，并且只有等到出错条件被确认且清除后，才能开始后面的IRP 。

中断传送对总线访问的限制

高速设备，全速设备和低速设备均可使用中断传送。高速端点可以分配几乎80%的微型帧进行周期传输，USB要求不能分配超过90%的帧给周期性的（同步和中断）全速/低速传输。

总线频率和（微型）帧定时限制了在任何USB系统的一个（微型）帧中成功的最大中断处理数小于108个全速1字节数据有效负载，或者小于10个低速1字节数据有效负载，或者小于134个高速1字节数据有效负载。根据不同的实际应用情况，主机控制器不可能提供上面所说每个（微型）帧的最大中断处理数。

表5.6列出了不同的低速中断处理和一帧中可能的最大处理数的信息.表5.7是全速的中断处理类的信息.表5.8是高速终端处理类的信息.表5.8的阴影部分表示高速中断端点不是默认接口设置的一部分.表中不包含与位填充有关的开销信息.

Table 5-6. Low-speed Interrupt Transaction Limits

Protocol Overhead (19 bytes)	(5 SYNC bytes, 5 PID bytes, 2 Endpoint + CRC bytes, 2 CRC bytes, and a 5-byte interpacket delay)				
Data Payload	Max Bandwidth (bytes/second)	Frame Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/Frame Useful Data
1	9000	11%	9	7	9
2	16000	11%	8	19	16
4	32000	12%	8	3	32
8	48000	14%	6	25	48
Max	187500				187

Table 5-7. Full-speed Interrupt Transaction Limits

	Protocol Overhead (13 bytes)	(3 SYNC bytes, 3 PID bytes, 2 Endpoint + CRC bytes, 2 CRC bytes, and a 3-byte interpacket delay)				
	Data Payload	Max Bandwidth (bytes/second)	Frame Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/Frame Useful Data
	1	107000	1%	107	2	107
	2	200000	1%	100	0	200
	4	352000	1%	88	4	352
	8	568000	1%	71	9	568
	16	816000	2%	51	21	816
	32	1056000	3%	33	15	1056
	64	1216000	5%	19	37	1216
Max		1500000				1500

Table 5-8. High-speed Interrupt Transaction Limits

Protocol Overhead		(Based on 480Mb/s and 8 bit interpacket gap, 88 bit min bus turnaround, 32 bit sync, 8 bit EOP: (3x4 SYNC bytes, 3 PID bytes, 2 EP/ADDR+CRC bytes, 2 CRC16 and a 3x(1+11) byte interpacket delay(EOP, etc.))			
Data Payload	Max Bandwidth (bytes/second)	Microframe Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/ Microframe Useful Data
1	1064000	1%	133	52	133
2	2096000	1%	131	33	262
4	4064000	1%	127	7	508
8	7616000	1%	119	3	952
16	13440000	1%	105	45	1680
32	22016000	1%	86	18	2752
64	32256000	2%	63	3	4032
128	40960000	2%	40	180	5120
256	49152000	4%	24	36	6144
512	53248000	8%	13	129	6656
1024	49152000	14%	6	1026	6144
2048	49152000	28%	3	1191	6144
3072	49152000	42%	2	1246	6144
Max	60000000				7500

中断管道的端点指定了它要求的总线访问周期.全速的端点要求的周期范围从1~255ms.低速端点被限制为10~255ms。高速端点要求的周期是（$2^{bInterval-1}$）X125us,其中bInterval的范围为1~16（包括16）。USB系统软件将在配置中使用这个信息以确定可以维持不变的周期。系统提供的周期可能比设备所要求的短，而且达到USB定义的最短周期（125us的微型帧或1ms的帧）。客户软件和设备只能依赖于这样的事实：主机保证端点两个处理尝试之间的持续时间不会比所要求的周期长。注意，总线错误会妨碍中断处理在总线上传输成功，因此超出了要求的周期。当客户软件有挂起的中断传输IRP时，才会轮询端点。如果到达执行中断传输的总线时间但没有挂起的IRP，那么端点将没有传输数据的机会。一旦IRP有效，它的数据将在下一个分配的周期传输。

高速端点在每个微型帧（或192Mb/s）能传输3072字节.要求每个周期传输超过1024字节的告诉端点被称为高带宽端点.高带宽端点在没个微型帧使用多个处理.高带宽端点必须指定周期1X125us (bInterval=1).

通过在每个指定的周期访问中断端点,可以在USB上进行中断传输.对于输入的中断端点,主机在不访问端点和请求中断传输的情况下不能确定端点是否中断的信源.如果当主机访问端点没有传输的中断数据,恻会以NAK响应.端点应只在中断等待时提供中断数据,避免客户软件错误地通知IRP结束.长度为0的数据有效负载是有效的传输,在一些应用上非常有效.

中断传输的数据顺序

中断传送可以利用0/1跳变位（toggle位）的机制，当成功的进行了一个传送，该位就跳变一次。

主机总是认为设备是遵守完备的握手协议和重发协议(参见第7章)。但如果无论传送成功否，设备都在Data1/Data 0间跳变 PID，就忽略主机发来的握手信号。但这时，客户软件会丢失一些包。因为有错误发生时，主机控制器会把设备发的下一个包当作上一个包的重发。

一旦在中断通道上检测到一个引起中止的条件，或收到设备发来的STALL握手信号，所有正等待的IRP都会中止。由软件通过独立的控制通道来消除中止条件。清除后，设备和主机都复位到Data 0的状态。如果总线上出现了一个影响传送的错误，则中断处理事务会停止.

www.cpit.com.cn

批量传送

为了支持在某些在不确定的时间进行的相当大量的数据通信，于是设计了批传送类型。它可以利用任何可获得的带宽。批传送有以下几点特性：

·以可获得带宽访问总线。

·如果总线出现错误，传送失败，可进行重发。

·可以保证数据必被传送，但不保证传送的带宽和延迟。

只当有可获得的带宽时，批传送才会发生。如果USB有较多的空闲带宽，则批传送发生地相对频繁，如果空闲带宽较少，可能有很长时间没有批传送发生.

1 批传送的数据格式

USB没有规定批通道上数据流的格式

2 批传送的方向

批通道是一种流通道，所以总是单方向的。如果要进行双向传送，必须用两个通道。

3 批传送对包长度的限制

批传送的端点决定自己可以接收或传送的最大数据净负荷区长度。USB规定最大的批数据净负荷区的长度为8、16、32或64字节,高速端点的最大值是512字节,而低速设备没有批量端点.这个最大长度是指数据包中数据区的最大长度，不包括协议要求的一些管理信息.批端点必须支持规定的最大长度中的一个，这个长度将在端点的设置信息中说明。USB并不要求每个数据净负荷区都达到最大长度，即如果不够长度的话，不必填充至最大长度。所有主机控制器必须分别支持8、16、32或64作为最大长度和高速批量端点512字节的最大包，而对更大或更小的长度可以不必支持。

在配置其间,USB系统软件读出了端点的数据有效负载最大值,并确保不会向端点发送比所支持的值还大的有效负载.　端点发送的数据负载其数据场必须小于或等于端点的wMaxPacketSize值。当批量IRP包含的数据超过数据有效负载的最大值时，要求所有数据有效负载都是最大值，但包含剩余数据的最后一个数据负载除外。当端点执行以下的操作时，批量传输完成：

www.cpit.com.cn

·已传的数据量恰好等于期望传送的量。

·传了一个不到wMax Packet Size长度的包或传了一个长度为0的包。

一旦批传送结束，主机控制器中止当前的IRP，并开始下一个IRP。如果收到的一个数据净负荷区超长，则所有在等待此端点的批传送IRP都将被中止/取消。

4 批传送对总线访问的限制

只有全速和高速设备可以使用批传送。

端点无法提出对批通道的总线访问频率的要求。USB会协调所有批传送和正等待的IRP的总线访问请求，以获得在客户软件和应用层之间的"最佳"传送效果。

总线上的控制传送的优先级比批传送高。对于控制传送，有可保证的传送时间，而对批传送，没有。只有当有可用的总线带宽时，批传送才发生。如果有段时间没有被用于其他目的，这段时间将用于批传送。如果正等待的各个批传送是要往不同的端点去的，主机控制器将根据公平访问原则，安排它们的顺序。至于公平访问原则的具体内容，由主机控制器的实现决定。系统中的所有批传送是竞争同一个可用的总线时间的，所以USB系统软件可以改变对某个特定端点进行的批传送所占有的总线时间。所以端点和它的客户软件不能够期望有一个特定的批传送的速度。当有设备被加进或移出USB系统或出现对其它设备上端点的请求时，端点和它的客户软件可获得的总线时间将起一定变化。但客户软件不能主观地认为批传送与控制传送的顺序，有时，批传送会在控制传送之前进行。

www.cpit.com.cn

高速批量OUT端点必须支持PING流控制协议。

总线频率和（微型）帧定制了USB系统中一个（微型）帧中成功的最大批量处理数要小于72个全速的8字节数据有效负载，或者小于14个高速的512字节数据有效负载。表5.9列出了不同的全速批量处理在一帧中可能的最大处理数的信息.表中不包含与位填充有关的开销信息.表5.10是高速批量处理的类私信息.

主机控制器能自由地决定特殊批量传输的某个总线处理如何在（微型）帧内和几个帧中传输.从端点能看到在一个（微型）帧或几个（微型）帧中展开的批量传输的所有总线处理.由于各种实际的原因,主机控制器不可能在每个（微型）帧提供上面所说的最大处理数.

5 批传送的数据顺序

批传送利用toggle位机制来保证接收器和发送器同步，即使在有错发生的情况下，也是如此。当端点被适当的控制传送设置过后，批传送被初始定位在DATA0，主机也将从DATA0开始第一个批传送。如果传送错误而导致出现一个能引起中止的条件或设备发了一个STALL握手信号，所有等待的IRP将被取消。软件通过一个独立的控制通道来清除中止条件，恢复之后，主机和设备的数据toggle都被定位在DATA0。

如果出现了一个影响事务的总线错误，批传送将被中止.

www.cpit.com.cn

Table 5-9. Full-speed Bulk Transaction Limits

Protocol Overhead (13 bytes)	(3 SYNC bytes, 3 PID bytes, 2 Endpoint + CRC bytes, 2 CRC bytes, and a 3-byte interpacket delay)

	Data Payload	Max Bandwidth (bytes/second)	Frame Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/Frame Useful Data
	1	107000	1%	107	2	107
	2	200000	1%	100	0	200
	4	352000	1%	88	4	352
	8	568000	1%	71	9	568
	16	816000	2%	51	21	816
	32	1056000	3%	33	15	1056
	64	1216000	5%	19	37	1216
Max		1500000				1500

Table 5-10. High-speed Bulk Transaction Limits

	Protocol Overhead (55 bytes)	(3x4 SYNC bytes, 3 PID bytes, 2 EP/ADDR+CRC bytes, 2 CRC16, and a 3x(1+11) byte interpacket delay (EOP, etc.))				
	Data Payload	Max Bandwidth (bytes/second)	Microframe Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/ Microframe Useful Data
	1	1064000	1%	133	52	133
	2	2096000	1%	131	33	262
	4	4064000	1%	127	7	508
	8	7616000	1%	119	3	952
	16	13440000	1%	105	45	1680
	32	22016000	1%	86	18	2752
	64	32256000	2%	63	3	4032
	128	40960000	2%	40	180	5120
	256	49152000	4%	24	36	6144
	512	53248000	8%	13	129	6656
Max		60000000				7500

高速,高带宽端点

USB支持某些要求数据速率达192Mb/s(即每个微型帧有3072个数据字节)的高速中断或同步端点.支持高带端点的一个微型帧内允许有一个,两个或三个高速处理.

高速中断或同步端点指出:当端点描述符w Max Packet Size场的第11和第12位不为0时,它要求每个微型帧超过1024字节. w Max Packet Size的低11位表示每个处理的数据有效负载,而第11和第12位表示可能的最大请求处理数.

主机控制器必须在每个微型内发送适当数量的高速处理.主机或总线的错误会导致主机控制器发送比端点请求少的处理数.如果提供的数据充足,第一个处理的数据有效负载必须与w Max Packet Size的低11位指定的大小一样,而最后一个处理包包含了有效负载小于或等于指定最大值的剩余数据.主机控制器可以向相同的端点一个接一个地发送处理(按提供的实际数据要求)或在高带宽端点的处理之间向其他端点发送处理.

Table 5-11. *wMaxPacketSize* Field of Endpoint Descriptor

Bits	15..13	12..11	10..0
Field	Reserved, must be set to zero	Number of transactions per microframe	Maximum size of data payload in bytes

Note: This representation means that endpoints requesting two transactions per microframe will specify a total data payload size in the microframe that is a multiple of two bytes. Also endpoints requesting three transactions per microframe will specify a total data payload size that is a multiple of three bytes. In any case, any number of bytes can actually be transferred in a microframe.

1. 高带宽中断端点

对于中断处理来说,如果端点在微型帧其间NAK一个处理,主机控制器就不能向该端点发送其他处理,直到进入下一个周期.

如果端点使处理超时,则主机控制器必须重发这个处理.端点指定了每个微型帧要求的最大处理数,如果不能到达,则主机控制器将在当前的微型帧其间立即重发该处理,因为这可以使对带宽敏感的设备影响最小;如果超过了微型帧的最大处理数,则主机必须在端点的下一个周期重发该处理.

主机控制器只在微型帧内的处理要求超过一个存储缓冲区时,才允许向端点发送处理数小于最大值的微型帧.

微型帧的每个中断处理都按正常的DATA0/DATA1数据触发顺序.

2. 高带宽的同步端点

对于同步处理来说,如果IN端点的数据有效负载最大值比其端点描述符指定的小,主机不会在微型帧中向该端点发送其他处理.

对于同步OUT端点来说,主机控制器必须发送实际数据要求的处理数,但不能超过端点描述符指定的最大值.发送的处理必须遵守端点描述符指定的最大有效负载.

同步端点不会执行重发操作

www.cpit.com.cn

高带宽同步端点(IN和OUT)必须支持数据PID序列,数据PID序列提供了数据接收器要求的支持,可以检测出微型帧中一个或多个丢失/被损坏的包.

高速,高带宽同步IN端点的数据的PID序列在微型帧各个处理的数据包中用DATA2,DATA1和DATA0 PID的重复序列.如果微型帧内只有一个处理,那么只使用DATA0数据的包的PID.如果微型帧内有两个处理,那么第一个处理的数据包用DATA2,第二个包用DATA1,第三个包用DATA0.在所有情况下, 数据PID序列在下一个微型帧重新开始.图5.11显示了在高带宽同步IN端点的微型帧内连续处理的数据包PID顺序.

当端点请求传输三个处理的数据时端点必须用DATA2响应第一个处理的IN令牌.当它请求两个处理时必须用DATA1响应,请求一个处理时用DATA0响应.在第一个处理后,端点如上所述接着发送数据PID序列.

主机知道IN端点每个微型帧允许的最大处理数.主机要求在第一个处理的响应中编码(通过数据包PID)端点的这个微型帧请求多小处理.如果主机没有收到发送给处理的无错,适当响应,主机就不能向端点的这个微型帧发送更多处理.当主机从端点接受到一个DA-TA0数据包时,它不能向该端点的微型帧发送更多的处理.



Figure 5-11. Data Phase PID Sequence for Isochronous IN High Bandwidth Endpoints



Figure 5-12. Data Phase PID Sequence for Isochronous OUT High Bandwidth Endpoints

www.cpit.com.cn

高速,高带宽同步OUT端点的数据PID序列与IN端点采用的序列不一样.当有一个处理时,主机必须发一个DATA0数据包;当微型帧有两个基本点处理时,主机必须为第一个处理发送一个MDATA,为第二个处理发送一个DATA1;当微型帧有三个处理时,主机必须发送两个MDATA处理和为第三个处理发送一个DATA2.这个顺序允许端点检测在微型帧内是否丢失/被破坏的处理.图5.12显示了高带宽同步OUT端点的微型帧连续处理的数据包PID顺序.

如果端点检测到不正常的OUT处理,必须像出错误一样处理所有在微型帧中传输的数据.注意,在微型帧有三个处理但缺少一个MDATA处理时,USB没有办法确定丢失了两个MDATA处理中的哪一个,可能有专用的应用方法更精确地确定丢失了什么数据,但USB在总线级不提供这类的方法.

分离处理

主机控制器和集线器支持一种叫分离处理的附加处理类型. 这种处理类型允许全速/低速设备连接到以高速运行的集线器. 这些处理只能在主机控制器和集线器使用, 他们对于设备来说是不可见的. 中断和同步传输的高速分离处理必须通过主机从微型80%的周期部分分配.

传输的总线访问

要完成主机与USB设备间的任何数据传送，必须要使用一定的USB带宽。要想支持从同步设备到异步设备的各种传送，必须要能满足它们对传送的不同要求。分配总线带宽给设备的工作叫做传送管理。主机上有几个部分是用于协调USB上的信息流的，它们是：客户软件、USB驱动器（USBD）和主机控制器驱动器（HCD）。实现这些部件必须要了解关于总线访问的一些核心概念：

- 传送管理：用于支持USB上信息流的各实体和各对象
- 事务跟踪：一种USB机制，跟踪在USB系统中的事务
- 总线时间：总线传一个信息包的时间
- 设备/软件缓冲区大小：支持一个事务所需要的空间。
- 总线带宽归还：被分配给其它传送的总线带宽未被使用时，可以重新给控制传送和批传送使用。

前几节主要着重于客户软件如何与它的应用层进行联系和什么是两实体之间的逻辑流。这节介绍主机上的不同部分如何相互协调工作来支持USB上的数据传送，对设备实现者来说，这个信息也是有用的，他们可以由此知道当客户请求传送时主机该做什么，以及传送请求是如何被发给设备的。

- **传输管理**

传输管理涉及以下几个为不同目标工作的部分，它们共同工作使数据能在总线上传送：

·客户软件：通过对USBD界面发出调用（calls）和响应调用（calls backs）的IRP请求，消费从应用端点来的数据或生产到消费端点去的数据。

·USBD(USB驱动器)：通过对合适的主机控制器驱动器(HCD)的调用（calls）和响应调用（callbacks），将从设备端点来的或到设备端点去的IRP中的数据进行一定转换。一个客户IRP可能会需要几个传送来完成。

·主机控制器驱动器(HCD)：将IRP转换成事务或将事务转换成IRP(按照主机控制器的要求)，并对它们进行组织，以使主机控制器进行操作。HCD和它的硬件间的互相作用与它的实现有关，不在USB说明的范围内。

·主机控制器：进行事务，为每个事务在总线上通过包传送数据

图5.13说明了当在客户软件和USB间有信息流动时，这些部分是如何工作的。它们之间的界面是我们了解的目标。

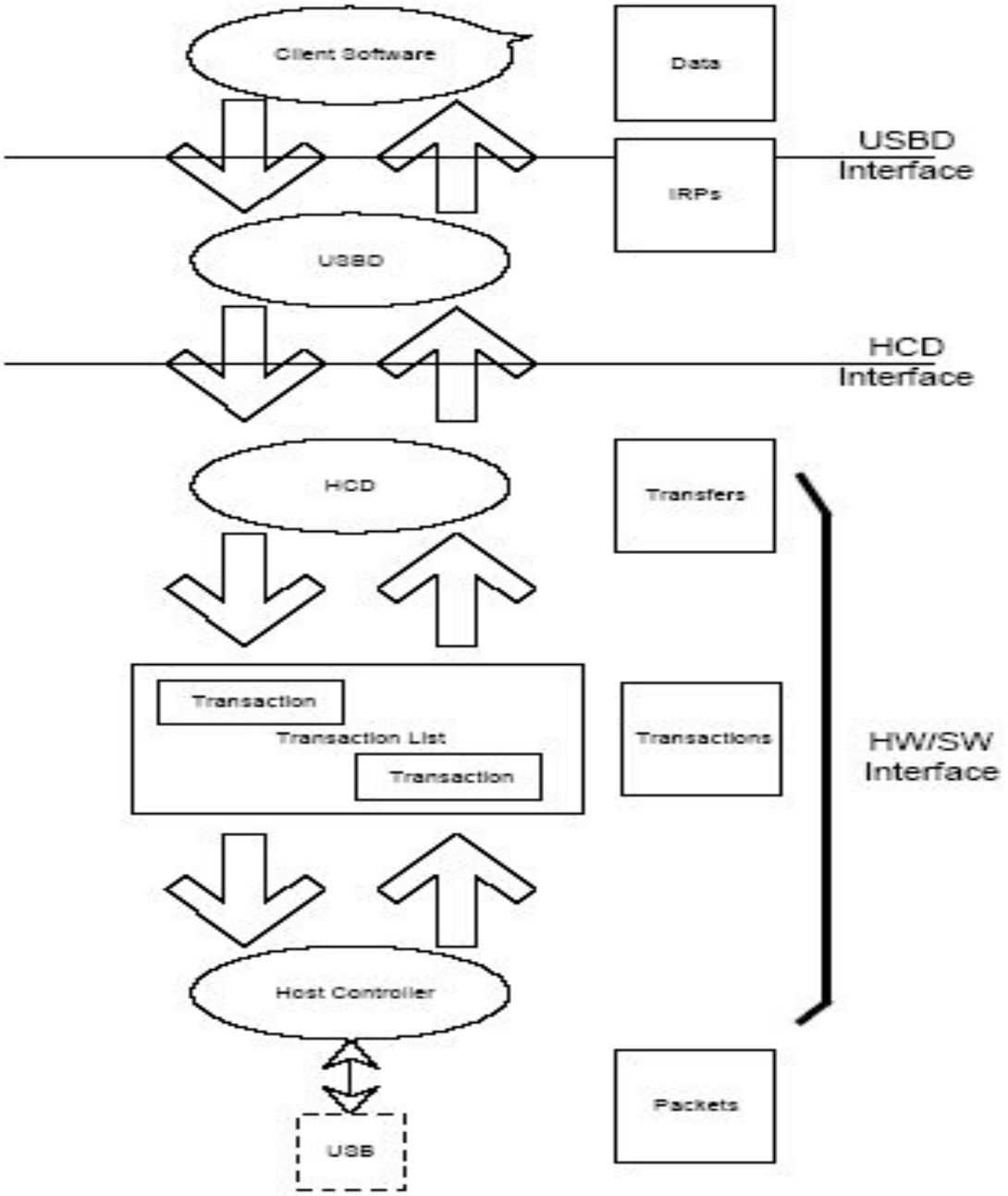


Figure 5-13. USB Information Conversion From Client Software to Bus

1 客户软件

客户软件决定某个应用应该用何种传送类型。它通过操作系统指定的界面来请求IRP。客户软件知道它用来操作应用的通道(接口)的存在；客户知道总线访问和带宽的限制，并且遵守它们。客户软件将自己的请求发给USB驱动器接口。

有些客户通过操作系统提供的另一些设备级接口来操作USB功能，而不直接进行USBD的调用（calls）。但总要有一些低级的客户进行直接的USBD 的调用（calls），以便将IRP传给USBD。所有这些提交的IRP必须遵守通道建立时定下的带宽限制。如果一个设备从一个非USB环境进入USB，客户软件将通过主存和I/O访问直接操作这个设备的硬件。USB系统中的最低级的客户软件和USBD相互作用，来操纵USBD的USB功能。

当客户软件要求一个与它的应用层间的数据传送，而且又被满足后，客户软件将收到一个关于IRP完成状态的通知。如果传送中有应用层到主机的数据，客户软件收到IRP完成的通知后，可在数据缓冲区中取到所要的数据。

www.cpit.com.cn

2 USB驱动器(USBD)

USBD将在两个主要的时间介入总线访问：

·在设备接上主机的设置期间

·在正常传送中

当设备被接上，并被设置时，USBD将确认设备的设置是否与总线兼容，USBD从设置软件处获得设置请求，它描述了要进行的设置：端点、传送类型、传送周期、数据长度等。USBD根据可用带宽的大小以及总线是否与请求的类型兼容来决定接受还是不接受这个设置。如果接收，USBD就为请求者建立一个相应类型的通道，自然也带有这个传送类型所应有的各种限制。在设备设置期间，不一定必须为周期性的端点作带宽分配。做过的带宽分配也可以放弃，而并不影响设备的设置。

对USBD的设置是典型地由操作系统决定的，而且反过来影响操作系统的设置特征。这样做可免除一些多余的接口。

一旦设备被设置，客户软件就可以通过IRP来请求与应用端点进行数据传送。

3 主机控制器驱动器(HCD)

HCD负责跟踪IRP，并确保USB带宽和帧最大时间不被突破。当有IRP要求通道时，HCD将它们加入事务表中，当IRP结束，HCD将把它的完成状态通报给发它的客户软件，如果IRP中涉及应用设备向主机的传送，来的数据将被放在客户指定的数据缓存中。

IRP的定义依靠于一定的操作系统。

4 事务表

事务表描述了当前需要被做的事务。它与HC（主机控制器）的实现有关。只有HCD和它的HC可以访问这些特殊的描述。这些描述包括了事务的各种参数。例如，数据长度(字节)。设备地址、端点号，以及发出数据和存放收到数据的主存区域。

事务表和HCD与HC间的接口是依赖于实现的，不能作为USB说明的一个部分而作出明确的定义。

5 主机控制器(HC)

HC可以访问事务表，并根据此表引起总线的动作。特别地，HC提供的报告机制使事务的状态(完成、等待、中止等)是可以知道的。HC将事务变成相应的总线动作(这种转换依赖于具体的实现)，这些动作又引起了在以根 Hub为根的总线拓朴结构上的USB包的传递。

HC可以保证协议规定的对总线访问的限制都被遵守了，例如：包间时限、超时等，HCD的接口向HC提供了参与决定是否允许一个新的通道访问总线的途径。这是因为HC的实现中有对两个事务间最小间隔的限制，这种限制支持了总线事务的组合。

事务表与HC的接口是被隐藏在HCD和HC的实现中的。

www.cpit.com.cn

- **事务的跟踪**

USB应用设备总线上的数据是以包的格式传输的。HC利用某种基于应用有关的表示方法来跟踪包，跟踪的信息包括这个包是什么包，到哪个端点去/从哪个端点来，在什么时间传送，有什么顺序。绝大多数客户软件不愿直接与被打了包的数据流打交道，因为这造成了一定的复杂性和对连接的依赖性，从而又限制了实现方法。USB系统软件(USBD和HCD)提供了将客户对数据运动的要求加到包上的方法。对组成一个客户软件和应用层间的数据传送的各个事务，HC利用IRP来跟踪它们。图5-14简要说明了事务是如何被组织成四种传送类型的IRP的。关于传送的详细协议规定，可见第8章。关于客户软件对IRP的使用的更详细信息可见第10章和特定操作系统的系统说明信息。

虽然IRP要跟踪传送数据的总线事务，HC仍可自由地选择如何传送这些事务，但必须遵守USB定义的限制，即一帧内只可有一个同步传送的事务。一般情况下，端点看到的各个事务的顺序与它们出现在IRP中的顺序是一样的，除非发生了错误。例如，图5-15表示了两个IRP，每个IRP有3个事务，用2个通道。对任何传送类型，HC可在第一帧内先传第一个IRP的第一事务，再传第二个IRP的第一个事务；同时在第二帧内先传第二IRP的第二事务，再传第一IRP的第二事务；如图中所示。

如果有同步传送，HC的自由度也就到此为止了。但对控制传送和批传送，HC可以在这两帧中的任一帧内多传或少传几个第一IRP或第二IRP的事务。

www.cpit.com.cn

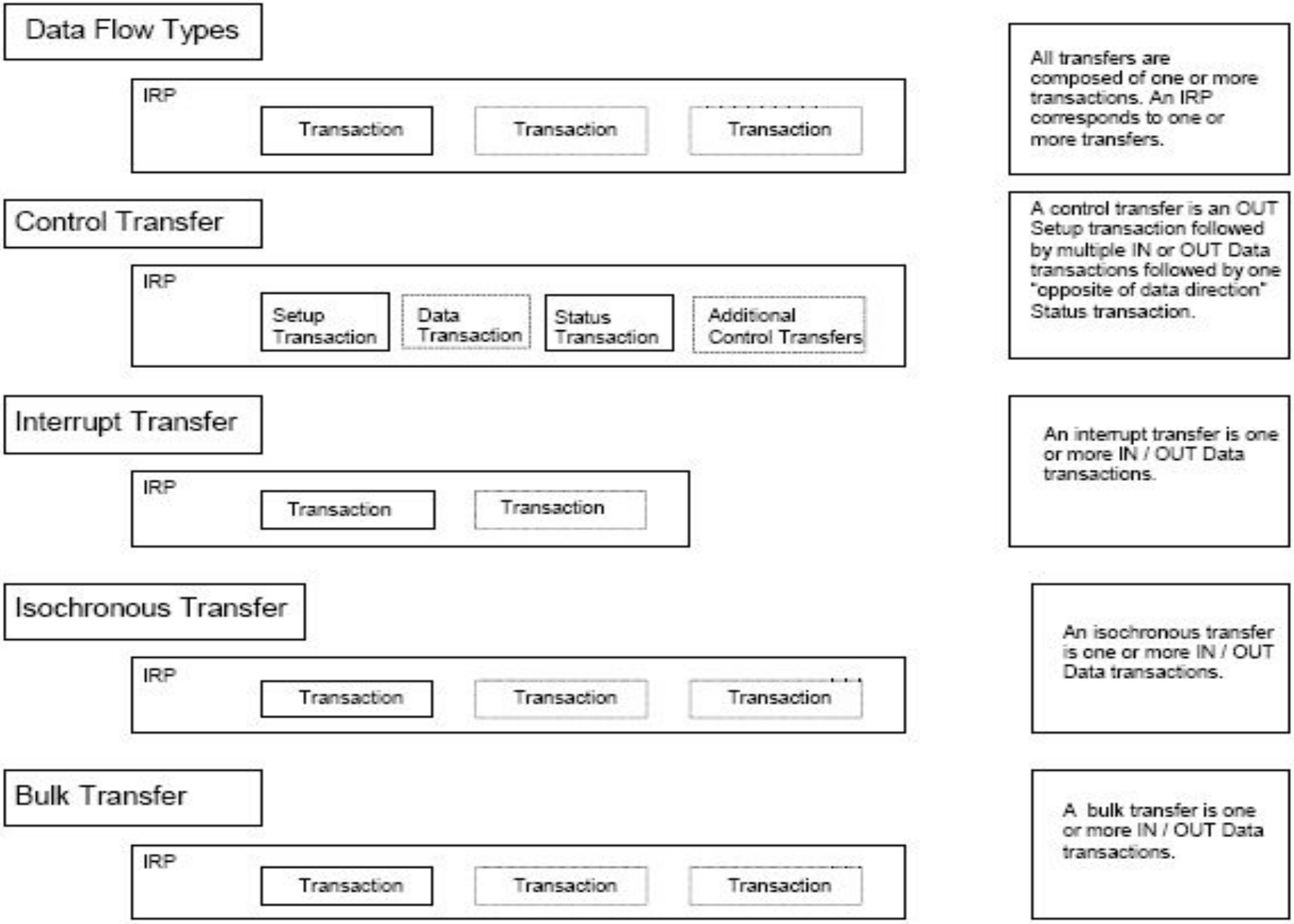


Figure 5-14. Transfers for Communication Flows

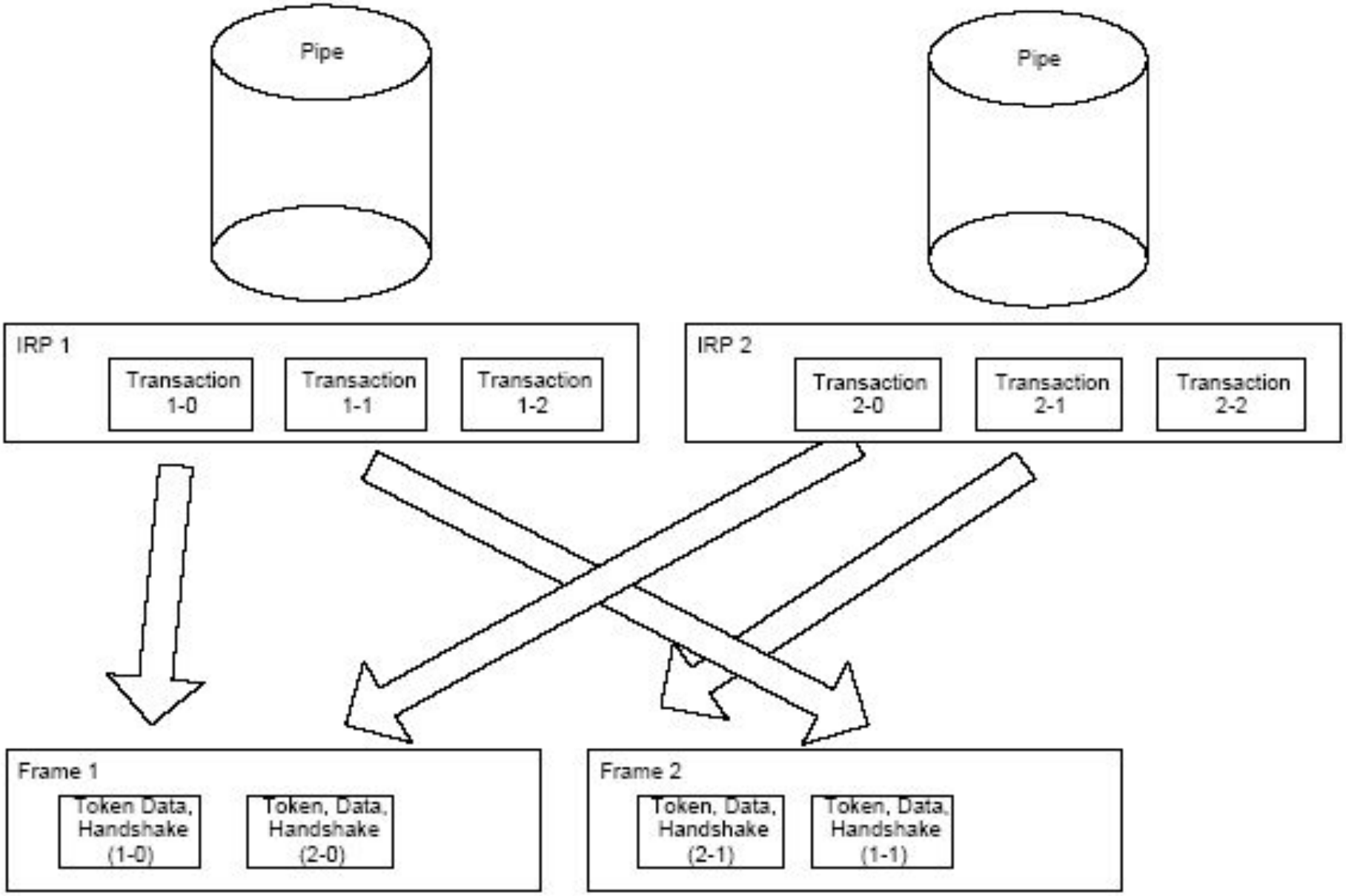


Figure 5-15. Arrangement of IRPs to Transactions/(Micro)frames

- 总线时间

当USB系统软件允许在总线上新建一个通道，它必须计算对一个给定的事务需要多少总线时间。这个时间是按照端点报告的最大包长度来计算的，同时也必须包括协议规定的开销、信号强加的位填充的开销、协议规定的包间时间的开销、事务间的时间的开销等等。计算的结果用来检查帧内可用的时间是否够用的，以下是计算用的各方程。

关键字:

Data_bc	数据有效负载的字节数
Host_Delay	主机或处理转换器准备或从传输恢复所要求的时间;主机控制器是基于应用的
Floor()	参数的整数部分
Hub_LS_Setup	主机控制器为集线器使能低速端口提供的时间;延时的测量从PRE PID的结束到低速SYNC的开始;最小值是4个全速位时间
BitStuffTime	计算由于发送位填充信号在理论上要求的额外时间的函数;最差的情况是(1.1667 X 8 X Data_bc).

www.cpit.com.cn

High-speed (Input)

Non-Isochronous Transfer (Handshake Included)
= (55 * 8 * 2.083) + (2.083 * Floor(3.167 + BitStuffTime(Data_bc))) + Host_Delay

Isochronous Transfer (No Handshake)
= (38 * 8 * 2.083) + (2.083 * Floor(3.167 + BitStuffTime(Data_bc))) + Host_Delay

High-speed (Output)

Non-Isochronous Transfer (Handshake Included)
= (55 * 8 * 2.083) + (2.083 * Floor(3.167 + BitStuffTime(Data_bc))) + Host_Delay

Isochronous Transfer (No Handshake)
= (38 * 8 * 2.083) + (2.083 * Floor(3.167 + BitStuffTime(Data_bc))) + Host_Delay

Full-speed (Input)

Non-Isochronous Transfer (Handshake Included)
= 9107 + (83.54 * Floor(3.167 + BitStuffTime(Data_bc))) + Host_Delay

Isochronous Transfer (No Handshake)
= 7268 + (83.54 * Floor(3.167 + BitStuffTime(Data_bc))) + Host_Delay

Full-speed (Output)

Non-Isochronous Transfer (Handshake Included)

$$= 9107 + (83.54 * \text{Floor}(3.167 + \text{BitStuffTime}(\text{Data_bc}))) + \text{Host_Delay}$$

Isochronous Transfer (No Handshake)

$$= 6265 + (83.54 * \text{Floor}(3.167 + \text{BitStuffTime}(\text{Data_bc}))) + \text{Host_Delay}$$

Low-speed (Input)

$$= 64060 + (2 * \text{Hub_LS_Setup}) + (676.67 * \text{Floor}(3.167 + \text{BitStuffTime}(\text{Data_bc}))) + \text{Host_Delay}$$

Low-speed (Output)

$$= 64107 + (2 * \text{Hub_LS_Setup}) + (667.0 * \text{Floor}(3.167 + \text{BitStuffTime}(\text{Data_bc}))) + \text{Host_Delay}$$

由以上方程计算出的总线时间的单位是纳秒，而且计算结果包含了由于主机和设备之间的延迟而造成的传播延时，这些方程是典型的用于计算总线时间的方程，但实现时可选用一些较粗糙的近似计算。

某个特定的事务实际所用的总线时间总是小于计算出的总线时间，因为位填充的开销是与数据有关的，最糟的情况是位填充的时间是原时间的1.1667(7/6)倍(方程中用8*1.1667乘以Data-bc)。这就是说，当所有规则安排的事务完成后，总有些总线时间是未用的(服从于数据模式的指定)，这些未用的时间可以被重新用于别处.。

方程中的Host-Delay项是与HC及系统有关的，它允许HC由于申请访问造成的延迟或其它与实现有关的延迟的原因多要求一点额外时间。通过HCD接口提供的传送管理功能，这项被包括在方程的实现中。方程的实现可采用USBD和HCD软件的共同工作来完成，这些方程的结果决定了一个传送或通道是否被USB设置所支持。

- **应用层及软件对缓冲区大小的计算**

  客户软件和应用设备必须给正在等待传送的数据事务提供缓冲区。对非同步传送，这个缓冲区的大小必须恰好能装下下一个数据包，如果不止一个事务在等待同一个端点，必须为一个事务提供数据缓冲区。由于在应用层与客户软件间的事务的出现，应用可能需要一定大小的缓冲区，关于这个区的精确的最小绝对长度的计算是不在USB说明的范围之内的。

  HC能够支持的等待事务的数量应是不受限的，但实际上，要受限于可用内存缓冲的大小和描述器的空间大小等。对同步通道，4.10.4节描述了影响主机侧和设备侧的缓冲要求的一些细节。一般说来，缓冲区应容纳约等于1ms中能传的数据量的两倍大小的数据。

- **总线带宽归还**

  SB带宽和总线访问的允许都是从最糟的情况考虑的。由于不同传送类型的限制以及被算作常数的位填充的时间实际是数据相关，所以在每帧内，与计算出的时间相比，总有些时间剩余。为了提高带宽利用率，这些时间可以用于控制传送或批传送。HC具体如何去做是与实现相关的。HC可以考虑等待的IRP的传送类型和剩余时间的情况来决定如何使用这些被归还的时间。

关于同步传送的一些特别考虑

系统的同步传送的能力是由USB支持的。在USB上进行可靠的同步数据传送必须注意一些细节。同步传送可靠性由几个USB部分分别负责：

. 设备/应用

. 总线

. 主机控制器(HC)

. 一个或多个软件代理商

因为时间对于同步传送是很重要的，所以USB的设计者必须了解USB中的这些部分是如何处理时间问题的。

所有的同步设备必须以各自的描述器的形式来报告自己的能力。同时这些能力也要以某种形式向客户说明，让他们决定该设备是否能解决他们的问题。设备各有不同的专门功能，所以它们的价格也有差异。

在任何通信系统内，发送器和接收器必须同步，也能保证数据传送正确。如果是异步系统，发送器可检测接收器是否收到数据，并且可在出错时，进行简单的重发

而如果是同步系统，接收器和发送器必须保持时间上和数据上的同步，才能使数据传送正确。USB不支持同步数据的重发，所以可以使分给同步传送的带宽达到最小。也不会因为重发而丢失同步。然而，重要的是，接收器与发送器不仅在正常情况下要保持同步；在错误发生时，也要能保持同步。

在许多系统，使用一个全局时钟，所以部件与它同步。一个例子就是PSTN(公共交换电话网)，考虑到接到USB上的设备有不同的固有频率，不可能有一个时钟在满足大众化PC产品的价格要求的同时，还能满足系统上所有设备和软件的同步要求。USB定义了一个时间模式，在有较合理价格的前提下，使各色设备在总线上共存。

本节提供了几种选择供端点使用，使其能够尽量缩小一个设备的非USB实现与USB实现间的差异，后面有一例子说明设备的非USB应用与USB应用间的相似和相异。

本节最后还介绍以下一些重要概念：

·USB时间模型：USB系统中使用的时钟，它影响着同步数据传送。

·USB帧时钟与应用时钟同步的方法：USB帧时钟如何与应用时钟发生联系。

·SOF跟踪：关于SOF令牌和USB的帧，以及同步端点的责任和机会。

·数据预缓冲：在数据生成、传送和消费前，对数据积累的要求。

·出错处理：对于同步传送的出错处理的细节。

· 为速率匹配而做的缓冲：可用方程计算同步端点所需的缓冲区大小。

1 典型的非USB同步应用

这个例子是具一般性的例子。更复杂或更简单的情况都是有可能的。相关的USB特性可能用的不太适当。

例子中有一个8KHz的单声道麦克风，通过一个送进入数据流的混合器驱动器（Mixer Driver）连上一个44KHz的立体声扩音器。混合器期望发送和接收的数据有一个确定的取样速率和编码。在输入、输出口的速率匹配器驱动器（Rate Matcher Driver）将固有取样频率和编码变为混合器期望的取样速率和编码，图5-16说明了这个例子。

PC中的主控时钟(由被实时时钟驱动的软件提供)用来提醒混合器向输入源要输入数据和向输出提供输出数据。这个例子中，假设此时钟每20ms提醒一次。麦克风和扬声器的采样时钟各不同步，且与主控混合器时钟也不同步。麦克风的固有速率为1秒8000次取样，一次取样结果为一个字节，它以此速度生产数据。扬声器的固有频率为1秒44100次采样，一次采样结构为4字节，它以此速度消费数据，系统中的这三个时钟会产生浮动和抖动，速率匹配器的时钟可以与这三种时钟(混合器时钟，源时钟，目的地时钟)均不相同。

速率匹配器一直监视设备的数据速率，不时插入一个额外采样或将两个采样合并为一个，来调整设备的时钟，向主控混合器时钟看齐。调整工作可以每两秒做一次，但一般都是不定期的，不频繁的。速率匹配器提供几个额外的缓冲来完成速率匹配工作.

注意：有些应用不能进行采样的调整。这时或者用一些方法来消除主时钟与设备时钟的不一致；或者用一些方法使两时钟同步，不让不一致发生

混合器总是精确地从它的输入设备处收到一个服务周期的数据(20ms)或为它的输出设备产生一个服务周期的数据。对混合器的延迟不能超过一个服务周期的长度。混合器认为这些延迟不会积累起来。

当输入设备和输出设备处理完一个服务周期的数据后，要能对DMA发的硬件中断做出发数据或收数据的反应。他们总是发或收一个服务周期的数据。输入设备每个服务周期(20ms)产生160个字节(10个取样)。输出设备每20ms的服务周期消费3528个字节(882个采样)。DMA控制器在主机缓存和设备传一个采样的速率比这些设备的采样速率都要快。

输入、输出设备都提供能容纳两个服务周期数据的系统缓存。一个给DMA控制器处理，另一个备用，以防第一个被取空。如果当前的缓存正被逐步取空，硬件中断将提醒设备驱动器，该驱动器要求速率匹配器给它一个缓存。在当前缓存被取空以前，设备驱动器会为新给的缓存申请一个新的IRP。

设备提供的数据缓存两个取样长度那么长，以保证当系统区对前一个采样做反应时，总可以为下一个采样同时做处理。

驱动器的服务周期可在操作系统中的中断延迟之后继续。不同的操作系统环境要求不同的服务周期来保证可靠的操作。服务周期长度的选择要能够使系统的中断负荷最小，因为系统中还有别的软件会要求处理时间。

中国普天信息技术研究院
www.cpit.com.cn

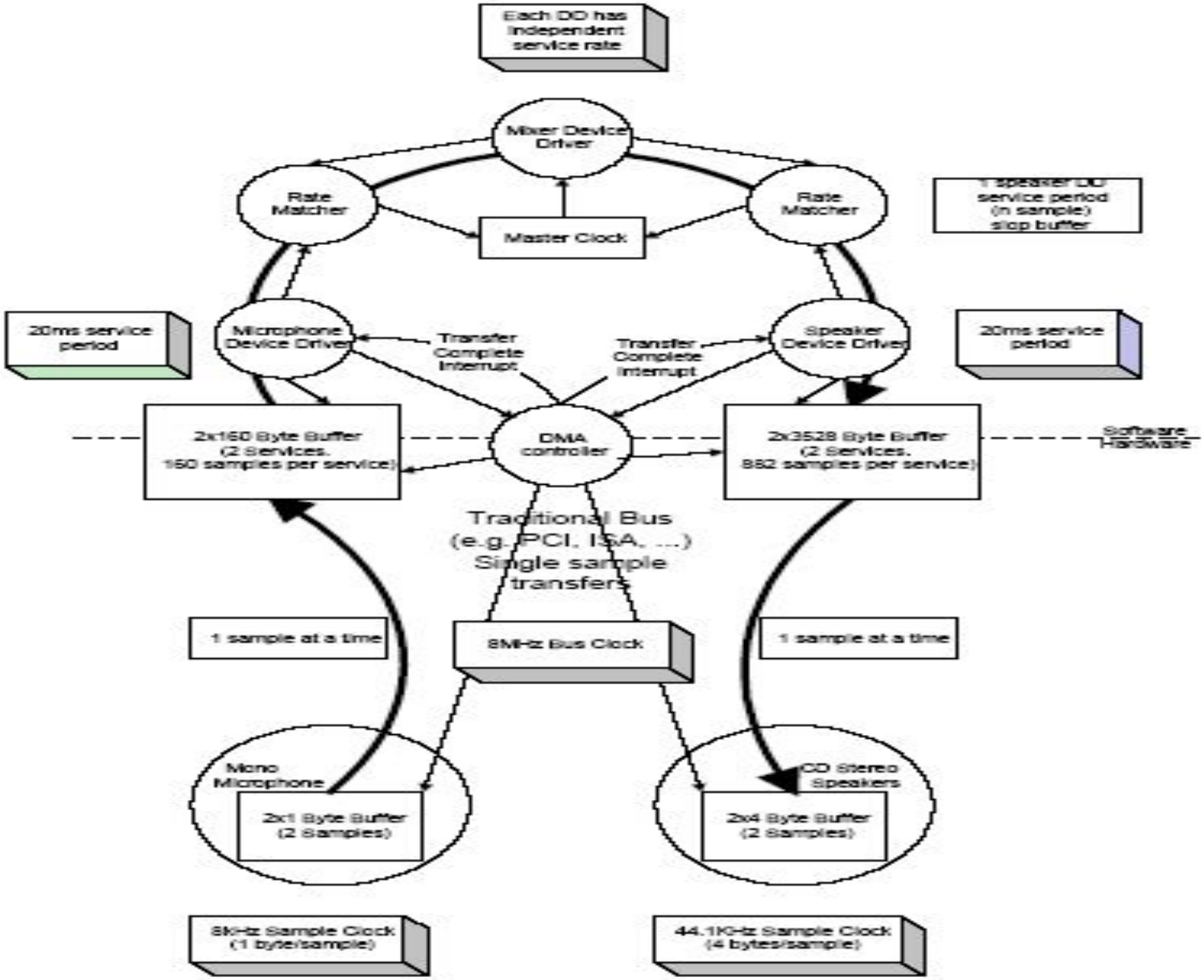


Figure 5-16. Non-USB Isochronous Example

2 USB时钟模型

USB系统中的时间是由时钟体现。实际上，USB系统中有多个时钟：

·取样时钟：这个时钟决定了客户软件与应用设备间传数据取样的固有频率。在非USB和USB实现中，这个时钟不必不一样。

·总线时钟：这个时钟频率为1KHZ。总线上SOF包的时间体现了此时钟。这个时钟相当于非USB的例子中的8KHZ时钟。在USB系统中，总线时钟一般比采样时钟频率低。在非USB系统中，总线时钟一般比采样时钟频率高。

·服务时钟：它取决于客户软件服务IRP的速度，这些IRP可能在两次执行之间积压下来。在非USB和USB系统中，这个时钟可以相同。

如果每个设备驱动器在高速取样的每次取样后都要被中断，现存的操作系统是无法支持变化很大的各种同步通信流的。因此，一般等有多个取样或多个取样包后，再将其交由客户软件处理，然后送给HC，根据事先约定的总线访问要求在总线上排队。图5-17提供了一个USB系统时钟环境下的例子，与图5-16的非USB系统例子相对照。

图5-17表示了一个典型的信息环路，麦克风作为输入设备，扬声器作为输出设备。涉及的时钟、包、缓存也标在了图中。在以后的几节后，图5-17将被进一步详细地说明。

这个例子的重点是突出USB与前一个非USB例子的不同。不同之处在于缓存的区域，USB总线时钟的同步和延迟，。设备驱动器上的客户软件在大多数情况下可不受影响。

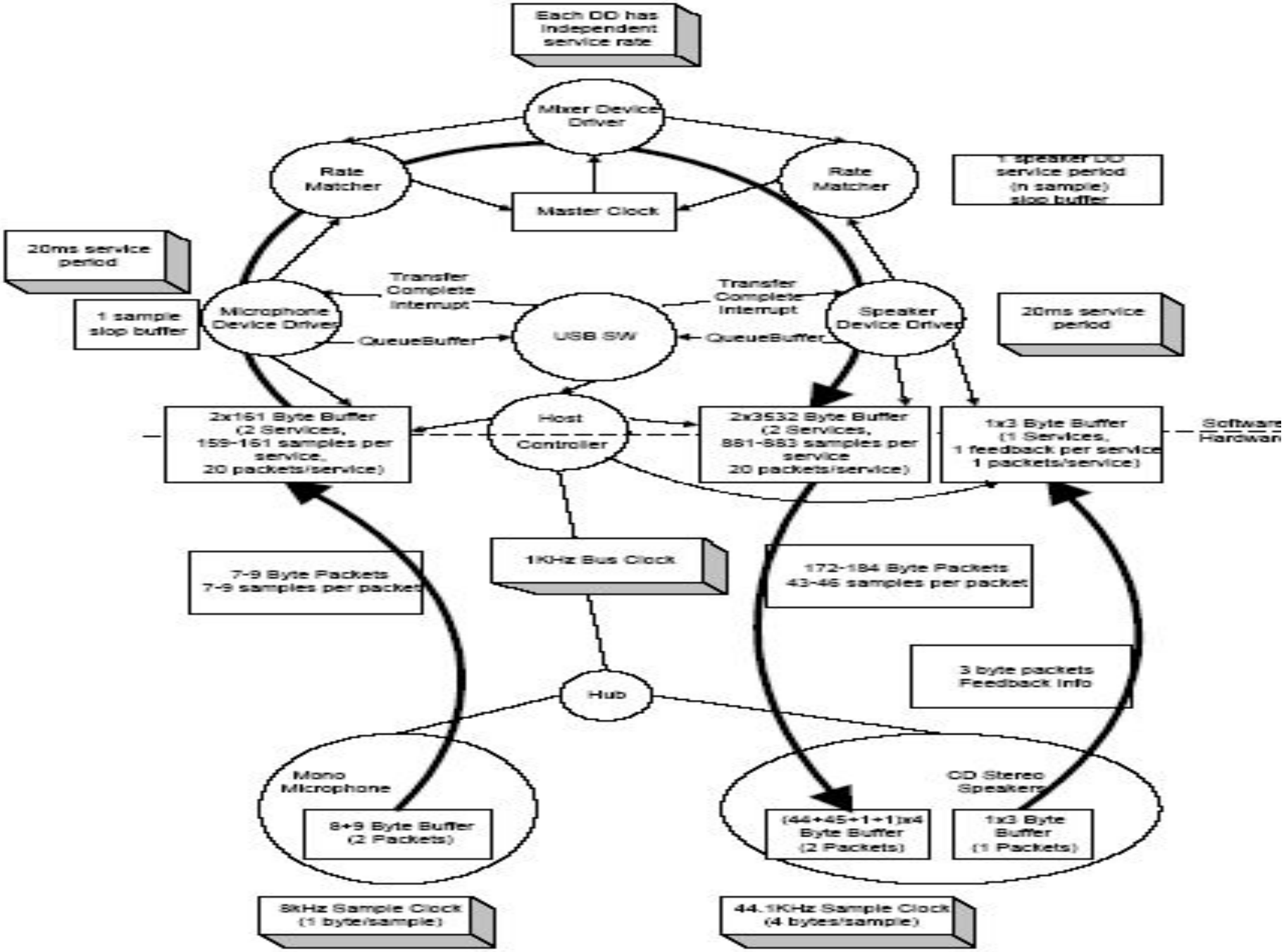


Figure 5-17. USB Full-speed Isochronous Application

3 时钟同步

为了可靠地传送同步数据，前面提到的三个时钟必须同步。如果不这样，就会出现一些不希望看到的时钟间的问题：

·时钟漂移：两个应正常同步的时钟，因为实现上的问题，在走很长一段时间后，可能会走得快慢不一。如果不及时校正，则与其中一个作为标准的时钟相比，另一个时钟就会导致数据过多或过少。

·时钟抖动：由于温度等的原因，时钟频率会发生改变，也造成了数据实际发出的时刻与它被期望发出的时刻不同。

·时钟间相位差异：如果两个时钟不被锁相，由于时钟的错位，在不同的时间点上会收到不一样多的数据。这会造成量化/取样的不真。

总线时钟提供了一个中心时钟，USB硬件设备和软件可以向它同步。但在大多数PC的操作系统中，仅靠实时调度的支持，软件并不能精确地锁相、锁频到总线时钟上。软件知道USB上的数据是打包的。对同步传送，在一帧内只传一个包，而帧时钟是相对精确的，软件可以凭实际花费的帧时间调整它处理数据的量。

www.cpit.com.cn

通信未来 普天制造

4 同步设备

USB为同步设备提供了一个框架，定义了同步类型及同步端点如何提供数据速率反馈，以及同步端点如何被连结到一起。同步设备包括被取样的模拟设备(如：声音设备、电话设备)和同步数字设备。对端点同步类型的分类的依据是它们将自己的时钟同步到与它们相连设备的能力。反馈信息能指出什么是要求的时钟频率。进行连接的能力依赖于想要的连接质量、端点的同步类型，以及进行此连接的主机应用程序的能力。额外的设备级信息是需要的，要多少取决于应用程序。

注意：数据是个一般性的词，可代表取样过的模拟信号(如声音)，也可指抽象的信息。数据速率可以指模拟信号被取样的速率，也可以指数据时钟的速率。

以下信息用于决定如何连接同步端点：

·同步类型

——异步：不同步、但目的（sink）能提供数据速率反馈

——同步：同步到USB的SOF时钟

——可调：用反馈或feed forward 的数据速率信息实现同步

·可获得的数据速率 ·可获得的数据格式

同步类型和数据速率信息决定了源和目的之间的速率是否匹配，如果有可接受的转换过程存在，就允许它们相连。应用程序有责任决定在可获得的处理资源的限制和其它限制(如延迟)下，是否支持一个连接。具体的USB设备级定义了如何描述同步类型和数据速率。

数据格式匹配和转换也是需要的，但不是仅对同步连接才要求。关于数据格式转换的细节在其它文件中可找到。

中国普天信息技术研究院 www.cpit.com.cn 普天信息技术研究院

- **同步类型**

  有三个明显的同步类型，表5-12列出了它们各自的端点的同步特征(作为源或目的)。这三个类型的排列是根据它们的能力逐步增强的顺序进行的.

Table 5-12. Synchronization Characteristics

	Source	Sink
Asynchronous	Free running Fs Provides implicit feedforward (data stream)	Free running Fs Provides explicit feedback (isochronous pipe)
Synchronous	Fs locked to SOF Uses implicit feedback (SOF)	Fs locked to SOF Uses implicit feedback (SOF)
Adaptive	Fs locked to sink Uses explicit feedback (isochronous pipe)	Fs locked to data flow Uses implicit feedforward (data stream)

1 异步

异步不能向SOF或其它USB范围内的时钟同步。它们的源和目的的同步数据流都是在一个固定的数据速率上(单数率端点)，有一个有限的数据率的个数(32KHZ,44KHZ,48KHZ)，或一个连续的可编程的数据率。如果可编程，是在同步端点的初始化时候进行的，异步设备需在它们的端点描述器中报告自己的可编程能力。数据速率是锁到一个USB的外部时钟或一个内部的自由时钟上的。这些设备将进行速率匹配的任务留给了USB环境中的其它部分。异步源端点通过它们每帧的生产的采样个数隐式地体现了自己的数据速率信息，异步目的端点必须用显式地反馈向一个可调的驱动器说明自己的数据率.

异步源的一个例子是CD播放器，它依据自己的内部时钟或共振器来提供数据，另一个例子是数字声音广播(DAB)的接收器或数字卫星接收器(DSR)，它们的取样速率取决于广播侧而不是USB的控制，异步目的端点可以是低价格的扬声器，用自己的内部取样时钟。

另一个情况USB上有两个或更多的设备要求对SOF的产生拥有控制权以便能够作为同步设备运作。例如有两个电话设备，有各自的外部时钟。一个电话可被接到Private Branch Exchange(PBX)上，PBX不与ISDN同步。另一个电话直接接到ISDN上，每个设备向网上产生或接收数据都是用自己的外部驱动速率。因为只有一个设备可获得SOF的控制权，另一个设备的数据速率就只能与SOF异步。这个例子表明，每个有能力控制SOF的设备都可能被迫成为异步设备。

2 同步

同步端点可有自己的时钟系统，但此系统可被SOF同步所控制。这些端点可以：

·将自己的取样时钟作为SOF时钟（1ms 1 次）的奴隶(通过一个可编程的PCL)。

·控制USB的SOF产生速率，显而易见，这样的话，它们的时钟自然与SOF同步。如果这些终点不被给予SOF的控制权，它们降级到异步模式。

同步端点发出或接收同步数据流的数据速率有以下几种情况。可以使用一个固定的数据速率(单频率端点)，或有限个数的几个速率(32KHZ，44.1KHZ，48KHZ)，或一个连续的可编程的数据速率。如果可编程，在同步端点的初始阶段进行设定。在一系列的USB帧中产生的采样数或数据单元数是可确定的和周期性的。同步设备须在端点描述器中报告自己的可编程能力。

同步源的一个例子是数字麦克风。它向SOF同步自己的取样时钟，并在每个帧内产生同样多的声音采样。另一种可能是从ISDN 的“Modem”中来的64kb/s的位流。如果USB的 SOF产生时钟被同步到PSTN的时钟(也许通过同一个ISDN设备)，数据的产生也会被同步到SOF，且端点将产生一个稳定的64kb/S的数据流，以SOF时钟为参照。

www.cpit.com.cn

3 可调

可调的端点是能力最强的端点，它们可以在自己的操作范围内产生和接收任意速率的数据。对可调的源端点来说，数据产生速率是被数据的目的端点所控制的。目的给源一个数据速率的反馈 。这样源就知道了目的要求的速率。可调的端点可以和任何类型的目的通信。对可调的目的端点来说，数据速率的信息是被包含在数据流中的。在一个确定的平均时间段内接收的平均取样数目决定了瞬时的数据速率。如果在操作中，这个数目发生了变化，则数据速率就作出相应的调整。

数据速率的变化范围是以某个速率为中心(例如8KHZ)，它是从几个可编程的速率或自动检测的数据速率(32KHZ，44.1KHZ，48HZ)中选择出来的，速率也可以是一个或几个范围(如5KZ-12KHZ或44KHZ-49KHZ)，可调端点也必须在它们的端点描述器中报告自己的可编程能力。

可调源的一个例子是CD播放器。它包括一个完全可调的取样速率转换器(SRC)，所以输出取样的速率不必为44.1HZ，而可以是在SRC控制范围内的任何值。可调目的端点包括高端（High-end）数字扬声器，耳机等。

www.cpit.com.cn

- **反馈**

异步目的向可调源提供一个反馈，来表明它想要的数据速率(Ff)是多少。要求的数据速率必须精确到1秒1个取样(1HZ)以上，这才能够使源速率达到一个高质量，并能容忍反馈中的延迟和错误。

Ff的值包括一个小数部分和一个整数部分，小数部分是为了达到1KHZ频率帧的分辨率，整数部分给出了每帧内的最小取样数目。在1KHZ的帧频率下，小数部分要求有10位来表示一个取样(1000/2^10=0.98)。这是一个十位的小数部分，可以用无符号的定点二进制0.10格式表示。整数部分也需要10个位(2^10=1024)来表示每帧内1023个单字节取样。10位的整数用无符号的定点二进制格式10.0表示。两部分合在一起的Ff值可用无符号的定点二进制格式10.10。此格式需要3字节(24 bits)。因为最大的整数值是1023，数10.10格式的数被向左靠齐成为24位，所以它的格式为10.14，小数点后的头十位被承认，剩下的低4位被可选地用于扩展精度，或者可以被看作0。

每帧中，可调源将Ff加到从前一帧中剩下的小数部分上，然后产生总数中整数部分大小的取样，将小数部分的取样再算到下一帧。源可以通过许多帧的Ff的情况来确定一个更精确的速率。

目的端点可以通过以下方法确定Ff。它在2^(10-P)帧的时间内(P为整数)，以Fs*2^P的频率数时钟的周期。P的经验取值在［0，10］的范围内，因为没有点（point）用比Fs更慢的时钟，且没有点（point）企图在一帧内更新不止一次。计数器被读入Ff,并且每2^(10-P)帧时间后复位一次。只要没有时钟被漏掉，计数在很长一段时间内将是很精确的。端点只要保证计数器的位数即可，这个位数要能容纳它的最大Ff。

数字电话端点通过将64KHZ的时钟分频(P=3)来得到自己的8KHZ Fs。64KHZ是它用于数据流串行的时钟。64KHZ的时钟相位可以作为一个额外的位来提高精确度，相当于取了P=4。这使Ff每2^(10-4)=64帧刷新一次。所以就需要一个13位的计数器来保存Ff,3位表示每帧8个取样，10位表示小数部分。13位的格式为3.10，因为它在10.14 Ff值格式下，所以其余的位置设为0。

P的选择是与端点有关的，选P有以下的指导原则：

·P在［1，9］的范围内

·P的值越大越好，因为这样可以减少帧计数器的大小而提高Ff刷新的速度。高的刷新速度会保证对源端数据速率的更好控制，减少了用于处理Ff变化的缓存的大小。

·P必须比10小，这样保证了至少两帧以上才会刷新一次，避免了SOF的抖动。

·P不能为0，这样可保证在一次Ff值丢失的情况下，与产生的采样数的偏离小于1。

从反馈寄存器中读Ff由同步传送来完成。每2^(10-P)要保告一次反馈信息。Ff在每个刷新周期必须被报告至少一次。如果一个刷新周期内将一个同样的Ff值报告一次以上，不会有什么用处。端点只有在Ff值与上次比有变化时才报告一下。

www.cpit.com.cn

经过一个很长的阶段，源可能会多发一个或少发一个采样。这可能是因为测量Ff的错误或误差的积累。目的端必须有足够的缓冲能力处理这些情况。一旦目的端发现这些情况，应当将报告的Ff调整正确。对时钟的相对漂移，调整也是必须的，这种校正处理的实现与端点有关，不被指定。

在双向通话的连接中，可调源可以获得可调目的的数据速率，但该源是采用目的端的身分获取此速率的，所以，此时不需要反馈通道。

- **隐含的反馈**

在某些情况下,需要避免使用独立的明确反馈端点.如果一个设备实现了一组紧密相关的同步数据端点,而且有以下几种情况:

1.组内的所有端点同步(即使用从公共主机时钟取得的采样时钟);

2.组内在一个方向上包含一个或多个通常需要明确反馈的同步数据端点

3.组内在相反方向包含至少一个同步数据端点

那么在这些情况下,设备会选择不使用有独立明确反馈的同步端点,取而代之的是反馈信息可通过观察方向相反的数据端点的数据速率取得,此时会发生下面两种情况:

1.一个或多个异步信宿端点附带一个异步信源端点.信源端点的数据数率可以作为隐含的反馈信息调整信宿端点的数据速率.

2.一个或多个自适应信宿端点附带一个自适应信源端点.信源端点在信宿端点接收到的数据速率的基础上可以调整自己的数据速率.

本规范如上面所说明的提供了必要的框架实现同步.但是,如何从隐含反馈端点的数据速率取得要求的数据速率Ff由应用决定。

- **联接**

连接模型被用来详细描述源到目的的连接过程。这个模型涉及到的各个不同的组成部分及它们如何互相作用来完成连接。

模型提供了多个源和多个目的的情况，图5-18说明了一个典型的情况(高度浓缩的及不完全的)，物理设备通过不同的物理层、软件层接到主机应用软件上(USB说明中描述)。在客户界面层、应用层看到一个虚拟设备。从应用层的角度看，只有虚拟设备存在。物理设备与虚拟设备的关系由设备驱动器和客户软件决定。

设备制造商(或操作系统销售商)必须提供设备驱动器软件和客户界面软件，以便将物理实现转变为一个USB需要的软件实现(虚拟设备)。如前所述，以加入软件的功能为基础，虚拟设备可以展现物理设备的不同同步行为。然而，同步的分类对物理设备和虚拟设备是一样的。所有的物理设备都必须属于3种类型中的一种。所以，加入设备驱动器或客户软件的功能应与物理设备的功能一样。名词“应用”必须被换成“设备驱动器/客户软件”，当物理源被联系到虚拟源，“虚拟源设备”应被换成“物理源设备”，“虚拟目的设备”应被换成“虚拟源设备”。当虚拟目的联系到物理目的，“虚拟源设备”必须被换成“虚拟目的设备”，“虚拟目的设备”应被换成“物理目的设备”。

www.cpit.com.cn

将速率调节器(RA)功能加到设备驱动器/客户软件层上有明显的好处，它可将所有应用独立起来，使设备不必关心与速率调节有关的一些规定和问题。多速率的应用也会退变为简单一些的单速率系统。

注意：这个模型并不局限于USB设备。例如，一个包含44.1KHZ的声音的CD-ROM驱动器可以作为异步、也可作为同步或可调的源。异步操作时，CD-ROM以它读盘的速率来填充自己的缓冲区、驱动器根据它的USB服务的间隔来取数据。同步操作时，驱动器根据USB服务的间隔(10ms)和名义上的取样速率(44.1KHZ)决定每个USB服务周期时输出441个取样。可调操作时，建一个取样速率转换器来匹配CD-ROM的输出速率和各种不同的目的端的取样速率。

使用这个参照模型，可以定义在不同的源和目的间建立连接所必须的操作。而且，这个模型还指出了这些操作必须或可以在哪层发生。第一阶段，物理设备被映射到虚拟设备，这由驱动器和/或客户软件完成，基于软件的能力，物理设备可被映射成具有完全不同的同步类型的虚拟设备。第二阶段，是对虚拟设备的应用，在软件栈的驱动器/客户层加入数据匹配功能后，与虚拟设备通信的应用就可以不必为连上它的每个设备作速率匹配工作。一旦虚拟设备的特性定了下来，实际的物理设备的特性就不必关心了。

作为一个例子，考虑在源侧将一个混合器应用连到不同的源上，每个源以各自的频率和时钟工作。在混合发生前，所有的流都必须被转换成同一个频率，并锁到一个参考时钟上。这个工作可以在物理到虚拟的映射层做，也可以在应用层为每个源单独处理。相似的工作在目的侧也要做，如果应用将混合的数据流送到不同的目的设备，它可以为每个设备做速率匹配的工作，也可以把这个工作留给驱动器/客户软件去做(如果可能的话)。

www.cpit.com.cn

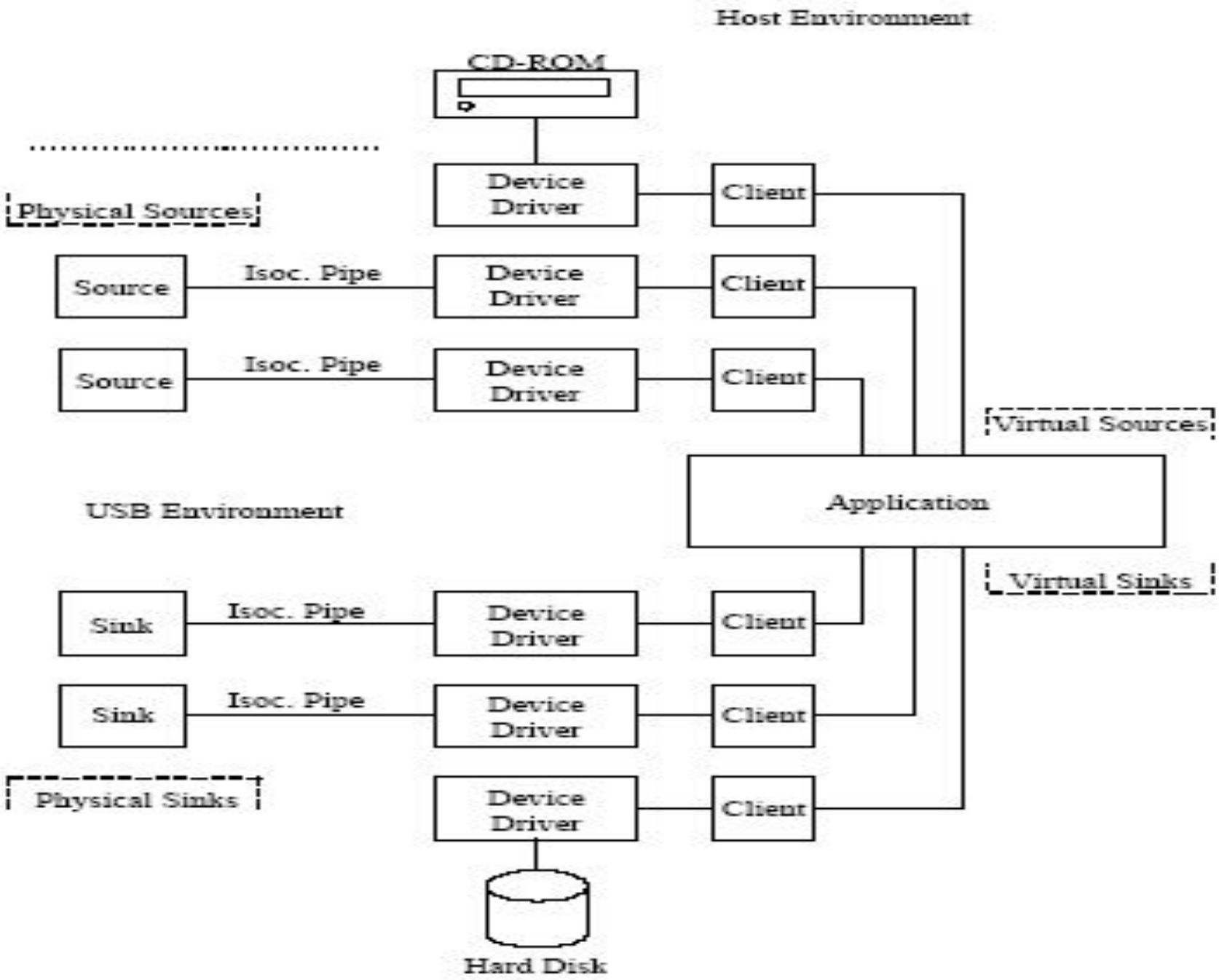


Figure 5-18. Example Source/Sink Connectivity

Table 5-13. Connection Requirements

	Source Endpoint		
Sink Endpoint	**Asynchronous**	**Synchronous**	**Adaptive**
Asynchronous	Async Source/Sink RA See Note 1.	Async SOF/Sink RA See Note 2.	Data + Feedback Feedthrough See Note 3.
Synchronous	Async Source/SOF RA See Note 4.	Sync RA See Note 5.	Data Feedthrough + Application Feedback See Note 6.
Adaptive	Data Feedthrough See Note 7.	Data Feedthrough See Note 8.	Data Feedthrough See Note 9.

Notes:

1. Asynchronous RA in the application. Fs_I is determined by the source, using the feedforward information embedded in the data stream. Fs_O is determined by the sink, based on feedback information from the sink. If nominally $Fs_I = Fs_O$, the process degenerates to a feedthrough connection if slips/stuffs due to lack of synchronization are tolerable. Such slips/stuffs will cause audible degradation in audio applications.
2. Asynchronous RA in the application. Fs_I is determined by the source but locked to SOF. Fs_O is determined by the sink, based on feedback information from the sink. If nominally $Fs_I = Fs_O$, the process degenerates to a feedthrough connection if slips/stuffs due to lack of synchronization are tolerable. Such slips/stuffs will cause audible degradation in audio applications.
3. If Fs_O falls within the locking range of the adaptive source, a feedthrough connection can be established. $Fs_I = Fs_O$ and both are determined by the asynchronous sink, based on feedback information from the sink. If Fs_O falls outside the locking range of the adaptive source, the adaptive source is switched to synchronous mode and Note 2 applies.
4. Asynchronous RA in the application. Fs_I is determined by the source. Fs_O is determined by the sink and locked to SOF. If nominally $Fs_I = Fs_O$, the process degenerates to a feedthrough connection if slips/stuffs due to lack of synchronization are tolerable. Such slips/stuffs will cause audible degradation in audio applications.

5. Synchronous RA in the application. Fs_I is determined by the source and locked to SOF. Fs_O is determined by the sink and locked to SOF. If $Fs_I = Fs_O$, the process degenerates to a loss-free feedthrough connection.

6. The application will provide feedback to synchronize the source to SOF. The adaptive source appears to be a synchronous endpoint and Note 5 applies.

7. If Fs_I falls within the locking range of the adaptive sink, a feedthrough connection can be established. $Fs_I = Fs_O$ and both are determined by and locked to the source.
   If Fs_I falls outside the locking range of the adaptive sink, synchronous RA is done in the host to provide an Fs_O that is within the locking range of the adaptive sink.

8. If Fs_I falls within the locking range of the adaptive sink, a feedthrough connection can be established. $Fs_O = Fs_I$ and both are determined by the source and locked to SOF.
   If Fs_I falls outside the locking range of the adaptive sink, synchronous RA is done in the host to provide an Fs_O that is within the locking range of the adaptive sink.

9. The application will use feedback control to set Fs_O of the adaptive source when the connection is set up. The adaptive source operates as an asynchronous source in the absence of ongoing feedback information and Note 7 applies.

在需要RA但RA不可得时，速率调节过程只能被模拟一下，即采用漏掉一两个采样或填充一两个采样的方法。此时，连接有可能被建立，但可能会有一个对质量不佳的警告；或者，连接根本不能建立 。

- **音频连接**

当以上所说的一般连接应用到音频连接上时，RA过程就被取样速率转换所取代，这是一种速率调整的特殊形式。不用错误控制，而用某种形式的取样插补来匹配输进和输出的取样速率。依靠这种手段，谈话的声音质量(扭曲(distortion)、信噪比等)均有很大改善。一般地来说，更强的处理能力带来更好的质量。

- **同步数据连接**

在同步数据连接中，我们使用RA。对许多实现了差错控制的应用来说，偶尔的slips/stuffs是可以接受的，差错控制包括差错检测和丢弃(discard)，差错检测和重发，或者前向纠错。slips/stuffs的发生速率主要看源的时钟和目的时钟误差的大小，它们是信通上最主要的差错来源。如果差错控制足够强大的话，连接仍可建立)。

www.cpit.com.cn

- **数据预缓存**

USB要求设备在处理和传送数据前要将数据预缓存一下，这样主机在处理各个通道的事务时就具有更大的灵活性。

从设备到主机的传送中，端点必须在帧x期间积累采样，直到收到帧X+1的SOF信号。它将帧x的数据锁到它的包缓冲中，准备好将包含这些取样的包在帧x+1期间发出去。至于在帧内的何时将发送这些数据只能由HC(主机控制器)决定，而且对每一帧的情况可以不同。

从主机到设备的传送中，在帧Y期间，端点收到从主机来的包。当它收到帧Y+1的SOF信号后，它才可以开始处理在帧Y期间收到的数据。

这个机制允许端点将SOF令牌用作打入时钟(stable clock)，而且抖动和/或漂移都很小。当HC在总线上传包时，这个机制也允许HC可以精确地改变在一帧内何时将包传出。与到每帧的SOF距离相同的总线访问相比，这种预缓存会造成端点准备好数据和数据真正在总线上传送的时间之间的额外延迟。

图5-19说明了设备到主机之间的传送的时间序列(IN process)，数据D0在Ti处的帧Fi期间积聚，在帧Fi+1期间发给主机。相似的，从主机到设备的传送(OUT process)，数据Do由端点在帧Fi+1期间收到，在帧Fi+2期间处理。

www.cpit.com.cn

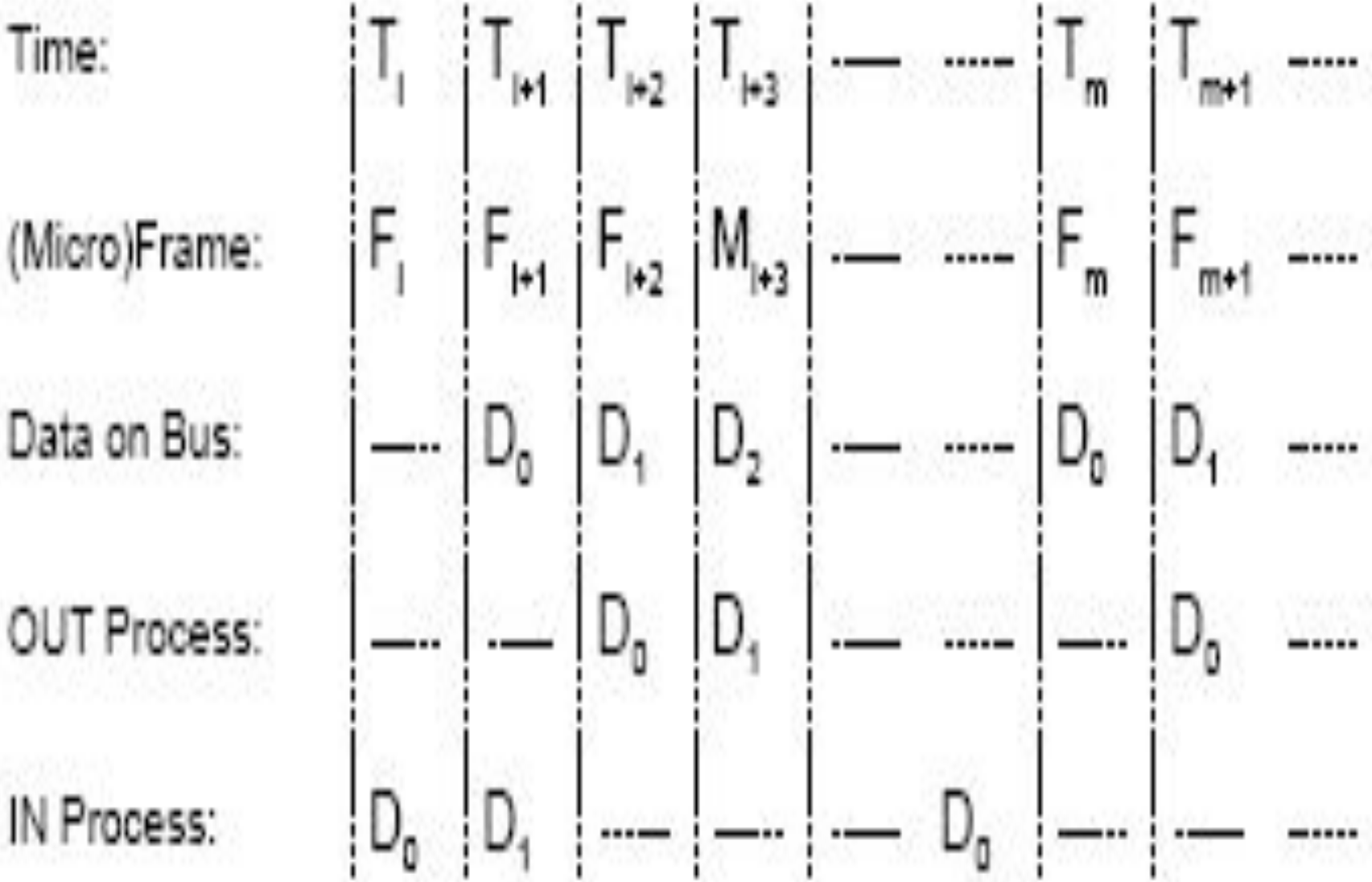


Figure 5-19. Data Prebuffering

- **SOF跟踪**

支持同步通道的应用设备必须要能够接收和理解SOF令牌才能支持预缓存。由于SOF可能会被破坏，所以设备必须能从从被破坏的SOF中恢复。这个限制使得只有高速设备才可使用同步传送，因为低速设备不能发现总线上的SOF。因为SOF包可能在传送中被毁坏，所以支持同步传送的设备需有合成SOF的能力，这个SOF由于总线错误而不能被他们看到。

同步传送要求适当的数据在适当的帧内被传送。USB要求当同步传送被显示给HC时，它必须指出它的第一帧的帧号。HC不会在指定的帧前开始传第一个事务。如果当前的帧内无事务在等待传送，HC在同步通道上就什么都不传。如果同步传送要求的帧已经过去了，HC会跳过(即不传)前面的事务直到对应当前帧的事务到来。

- **差错处理**

同步传送不进行数据包的重发(即接收方不发握手信号给发送方)，所以数据传送的同步不会被打乱。但数据传送的代理商有责任知道何时发生了一个错误以及该错误如何影响通信流。特别地，对于数据包序列(A，B，C，D)，USB有够的信息。所以一个漏掉的包(A，，C，D)是可以被测的，并且不会在不知道的情况自动变成不正确的数据或时间序列(A，C，D)或(A，，B，C，D)。协议提供了4种机制来支持这个想法：一帧一包；SOF；CRC以及总线事务超时。

- 同步传送要求在正常的操作中，一帧只能有一个数据事务。USB并不干涉每帧内是什么样的数据。数据发送器（源）决定提供什么样的数据。这种数据每帧(data-perframe)结构很规则，它提供了一个框架，这个框架是丢失数据的错误检测的基础。在总线传送期间事务的任何一个阶段都可能有破坏。第7章将介绍每种错误对协议的影响。
- 因为每帧前都有一个SOF，接收器会在总线上看到SOF。接收器可以凭此发现在两个SOF之间，它期望的帧没有出现。特别地，SOF也有可能被破坏，所以设备必须能够重建被漏掉的SOF(见4.10.6)。
- 数据包可能在总线上被损坏，所以CRC保护可以让一个接收器发现它收到的数据包是否被破坏了。
- 当接收器成功收到一个令牌包后，如果发现有总线事务超时，它可以不准备去接收数据包。这些细节都在协议中有定义。

一旦接收器决定不再接收某个数据包，它必须知道被丢掉的这个包的数据长度，才能在做出相应的错误后的恢复。如果每帧的数据流都是一样的长度，这个值就是一个已知的常数。然而，有些情况，帧与帧的数据长度不同。这时，接收器和发送器要有一个机制来决定丢失的包的长度，这个机制是与它俩的具体实现有关 。

总的来说，不管一个事务是否成功地被传送了，接收器和发送器均会以每帧一个事务的速度继续它们的数据/缓冲流。这样才能保证数据在时间上的同步。以上描述的一些机制，用于检测、跟踪、和报告被破坏的事务，让设备或它的客户软件可以对这种破坏以合适的方式做出反应。关于设备和应用对此如何反应是不在USB说明的范围内的。

- **为匹配速率而做的缓冲**

如果USB上有许多不同的时钟在影响着同步数据流，为了使USB上的数据流的速率的相互匹配，缓冲是必不可少的。设备上的每个端点都必须有缓存。主机上的客户软件也必须拥有缓存，数据在被传送之前可以存放在缓存中。如果已知设备的数据速率，需传的数据包的最大长度就是可计算的，图5-20给出了几个方程，用于决定设备和主机上的缓存大小以及在给定的数据速率下可能出现的最大包长。

设备和客户软件利用这些方程来设计服务时钟速率(变量x)，取样时钟速率(变量C)，和取样的长度(变量S)。USB只允许一个总线时钟对应一个事务。这些方程提供的信息可成为端点选择合适的包长，或信息/端点和它的客户软件选择合的缓存的依据。图4-14就是了一个典型的同步例子的实际缓存值、包长、时钟值。

USB数据模式认为设备有自己的取样的大小和取样的速率。USB支持包的传送，这个包应当是取样长度的整数倍，这样有利于对总线上被损坏的同步事务的错误恢复。如果一个设备没有固有的取样长度或它的取样长度超过了一个包的长度，它应将它的取样长度描述为一字节。如果一个取样被分为几个包，当任一个包被丢掉后，对错误的恢复比一般情况要困难。在有些情况中，数据同步会丢失，除非接收器知道取样的各个部分将在帧号为几的帧中被传。而且，如果由于时钟校正造成取样的个数发生变化(例如一个non-derived的设备时钟)，将很难知道取样的一个部分何时被传，因此USB不会将一个取样分几个包传。

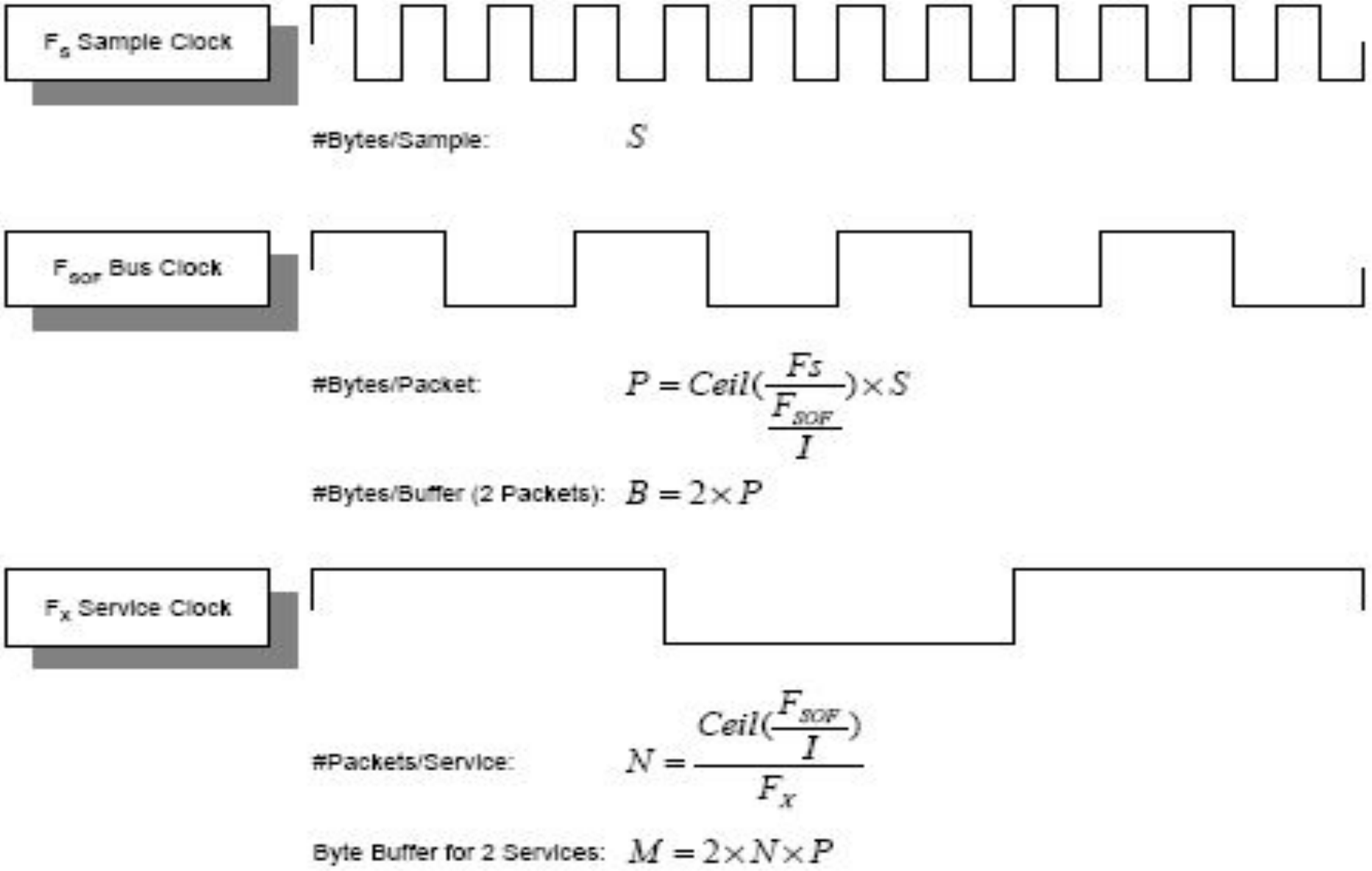


Figure 5-20. Packet and Buffer Size Formulas for Rate-matched Isochronous Transfers

(七)电气规范

USB 串行数据是用 NRZI 进行编码的，编码过程是在通过 USB 数据线进行传输之前进行的。图 5-1 就是在通过 USB 数据线段进行的信息传输时包含的步骤。NRZI 编码首先由 USB 代理执行，它负责发送信息。接下来，编码后的数据被放入 USB 数据线，这是由差分驱动程序完成的。接收器放大传来的差分数据，并把 NRZI 数据发送到解码器，对数据进行编码和采用差分信号进行传输有助于确保数据的完整性和消除噪声干扰。



图 5-1 在 USB 上采用 NRZI 编码和差分信号的传输方式

检测设备连接和速度

在进行信息传输之前，无论数据是发送给 USB 设备还是来自给定的 USB 设备，主机软件首先都必须检测到 USB 设备是否存在。所有这些设备的设计思想是，当 USB 设备连接到集线器的端口上的时候，该设备就会自动被检测到。反之，当设备和系统断开连接时，系统也能够自动的检测到设备已经不存在了。用于检测设备连接所采用的机制同时也提供一种途径，可以用于检测该设备是一个全速还是低速设备。

一个 USB 的集线器通过监视差分数据线来检测设备是否已连接到自己的一个端口上。当然，前提条件是数据线的电源已经加在这个端口上了。参见图 5-2，当没有设备连接到 USB 端口时，和 D+和 D− 线相连的下拉电阻就能够保证了两条数据线都是近地的。USB 设备必须至少在 D+和 D− 线的任意一条上有一个上拉电阻（这取决于该设备的速度），如图 5-2 所示。当一个设备连接好以后，电流就会流过分压器，分压器是由集线器的下拉电阻器和 D+或 D−线上的上拉电阻组成。由于下拉电阻为 15 千欧，而设备的上拉电阻为 1.5 千欧，所以数据线上将会有升高将近 90%的 Vcc 电压。当集线器检测到一条数据线电压接近 Vcc 的时候，而其他保持近地电压，那么他就知道该设备已经连接好了。

www.cpit.com.cn

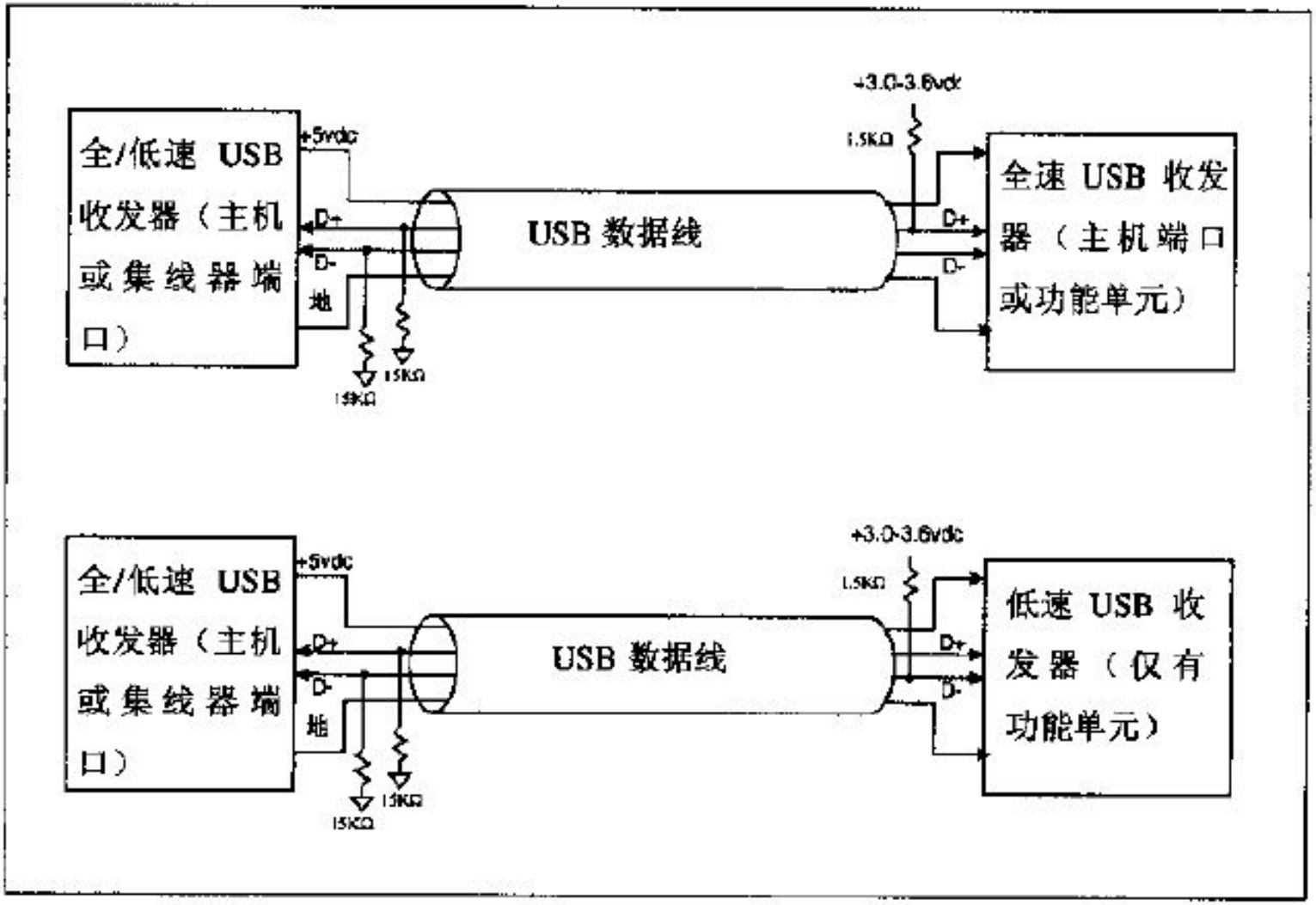


图 5-2　集成器和设备的电阻连接

从图中可以看到，全速设备在 D+线上有一个上拉电阻，而低速设备则在 D− 线上有一个上拉电阻，这样就可以识别设备的速度。图 5-3 所示的就是 USB 设备连接和

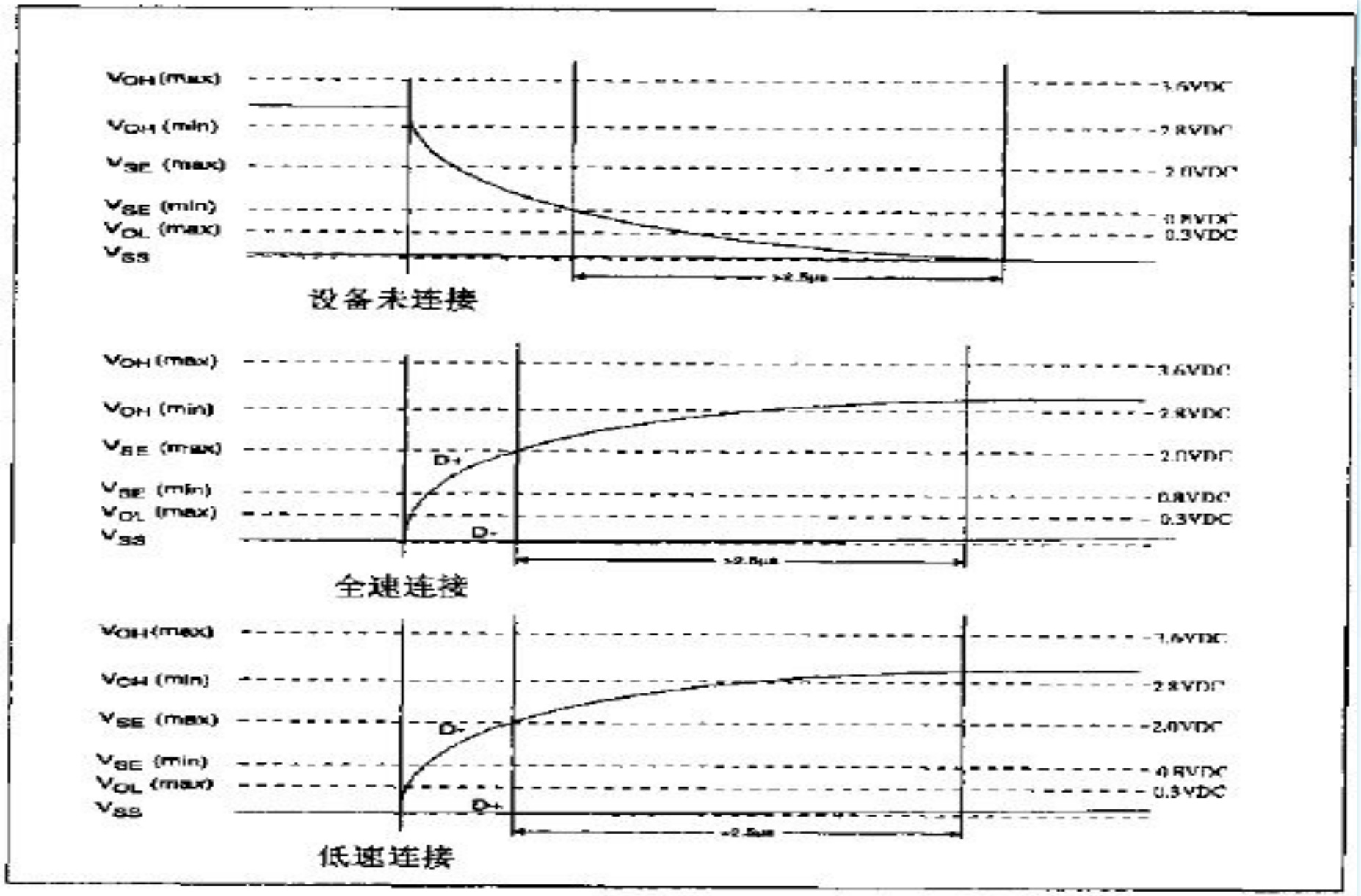

VOH (max)
VOH (min)
VSE (max)
VSE (min)
VOL (max)
VSS
3.6VDC
2.8VDC
2.0VDC
0.8VDC
0.3VDC
>2.5μs
设备未连接
VOH (max)
VOH (min)
VSE (max)
VSE (min)
VOL (max)
VSS
D+
D-
3.6VDC
2.8VDC
2.0VDC
0.8VDC
0.3VDC
>2.5μs
全速连接
VOH (max)
VOH (min)
VSE (max)
VSE (min)
VOL (max)
VSS
D-
D+
3.6VDC
2.8VDC
2.0VDC
0.8VDC
0.3VDC
>2.5μs
低速连接

图 5-3　设备连接与分离

断开连接时的电压的分级。当 D+和 D—的电压都下降到 Vse（min）以下，或者说只有直流 0.8V 以下，而且这种情况持续了 2.5 微秒以上的话，就认为该设备器断开连接了。当 D+或 D—升高到 Vse（max）以上，或者说直流 2.5 伏以上的状态，并且这种情况超过了 2.5 微秒时候，集线器就认为该设备已经连接上了。当它检测到该设备已经连接上以后，集线器就在它的端口状态寄存器中设置适当的状态位，当它发现设备断开连接以后，它就把相应的状态位复位。主机软件周期性地检查每个集线器，以检查是否有设备和集线器进行连接或者断开连接。

USB 设备有两种信号状态，对于低速和高速设备，这两种状态是相反的：

- J 状态；
- K 状态。

当一个设备一开始和 USB 相连的时候，它的系统数据线接近 Vcc，而另一个则近地，该状态被称为“J”信号状态，也就是该设备的闲置状态。当信号跳变发生时，两条数据线发生状态切换，导致上面的“J”状态转化为“K”状态。

注:高速设备的连接和检测同全速设备

www.cpit.com.cn

NRZI编码

数据通过 USB 传输采用的编码方式是 NRZI（反向非归零）编码方式。这种编码方式既能够确保数据发送的完整性，又不需要独立的时钟信号和数据一起发送。NRZI 编码绝非一种新的编码方式，人们使用它已经有几十年了，所以它的应用领域十分广阔。

图 5-4 所示的就是一个串行数据流和相应的经过 NRZI 编码的数据，在 NRZI 编码的数据流中的电平跳变代表 0，而没有电平跳变则代表 1。NRZI 编码器必须和传来的数据流保持同步，这样才能正确地对数据进行采样。NRZI 编码数据流必须在一个数据窗口内被采样，以检测是否从前一位时间到现在发生了电平的跳变。解码器在每一位时间对数据流进行采样，以检测是否发生了跳变。

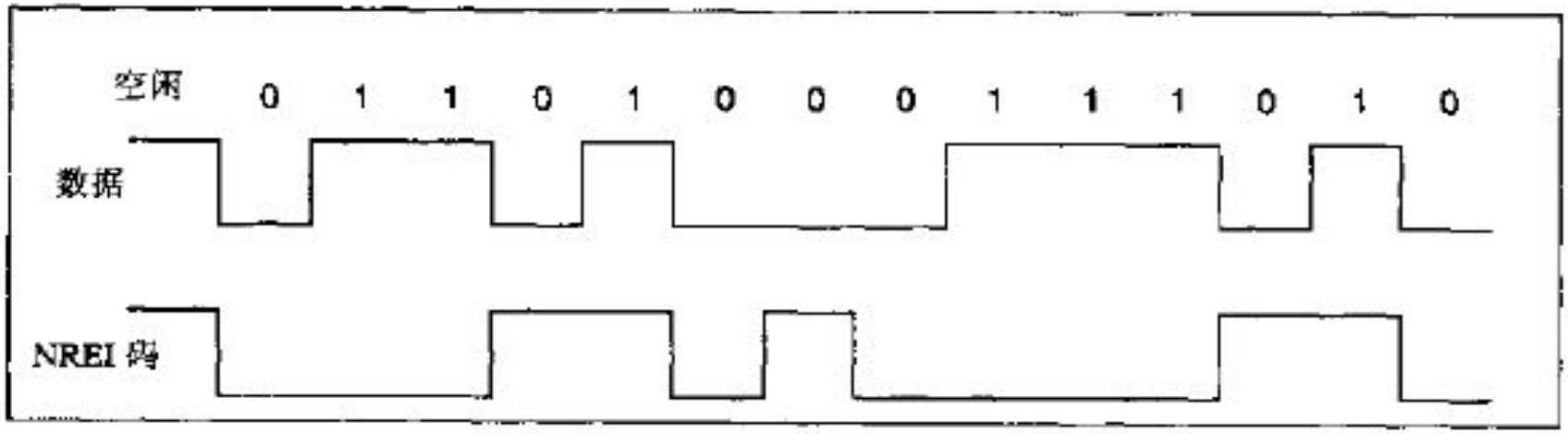


图 5-4 NRZI 编码的数据

www.cpit.com.cn

数据流的跳变使解码器可以和收到的数据保持同步，因而可以不使用一个独立的时钟信号。但是要注意，一长串的连续 1 将会导致无电平跳变，从而引起接收器最终丢失同步信号，解决办法是采用位填充的办法。

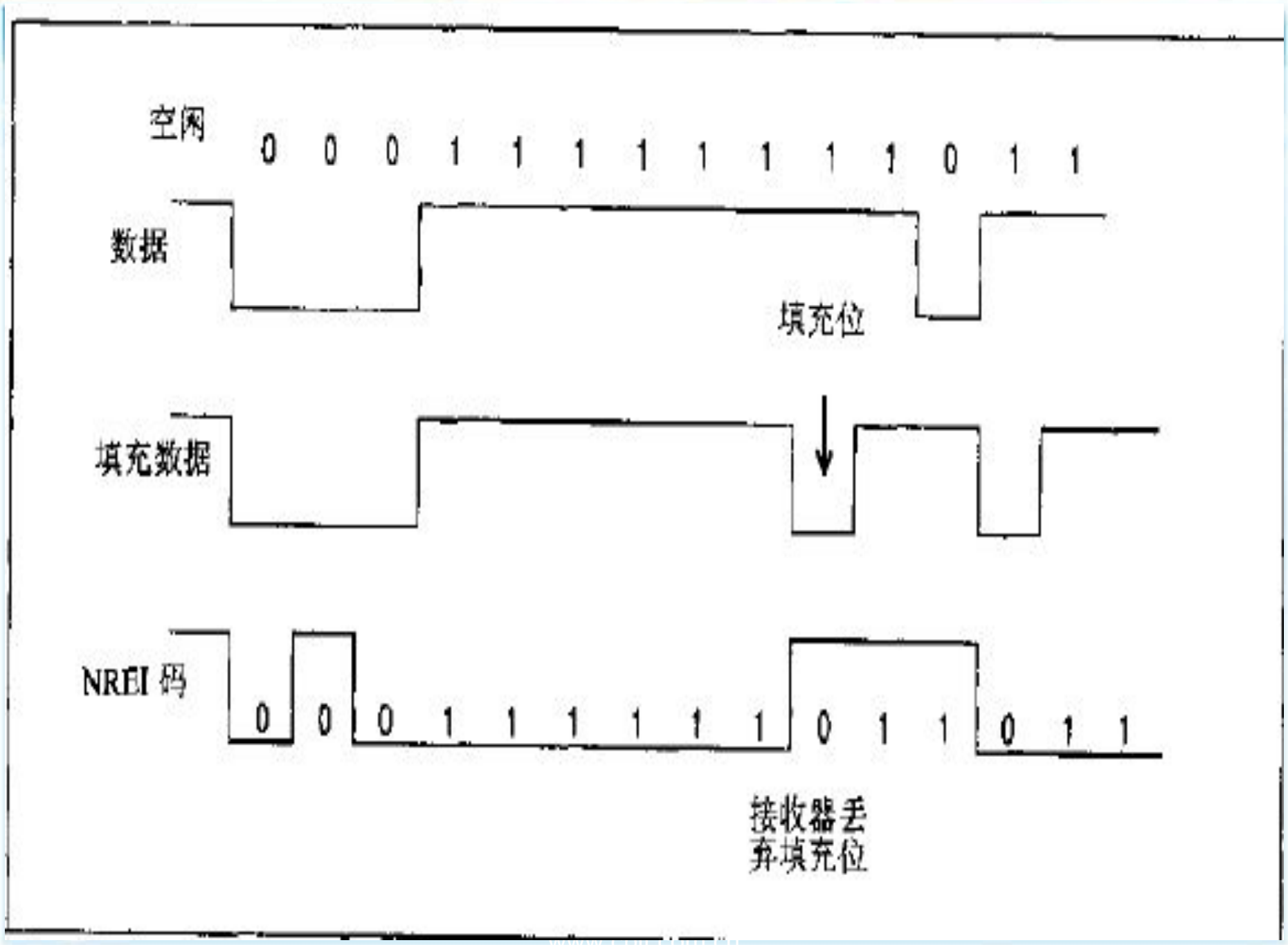
注:USB总线是以差模驱动的方式来进行数据传输的，但在资料包发送之前， USB 协议规定必须使用 NRZI 的编码方式来对资料进行编码。当然，在 USB 外设中，用于译码的器件对外设来说也是必不可少的。 NRZI 的编码协议其实很简单，它采用的是逢“1”保持，逢“0”跳变的原则(上图)，而 NRZI 的译码则采用相反的操作。

位填充

位填充的做法是这样的，在连续传输 6 个 1 的情况下，强制在 NRZI 编码的数据流中加入跳变。这就确保接收器至少可以在每 7 个位的时间间隔内从数据流中会检测到一次跳变。这就使接收器和传送的数据保持同步。NRZI 数据的发送器负责在 NRZI 数据流中插入一个 0（填充位）。接收器必须设计为能够在 6 个连续 1 之后自动知道将有一个跳变发生，并且把跟在这 6 个连续 1 之后的那个 0 抛弃掉。

图 5-5 中，上面的线表示发送到接收器那里的原始数据。注意数据流包含了一个连续 8 个 1 的长串。第二条线代表加入了填充位之后的原始数据。可以看到在数据流中，这个填充位被插到第 6 个和第 7 个 1 之间。第 7 个 1 的发送延迟了一个数据位的时间，这就是为了使填充位可以被插入。接收器知道第 6 个连续的 1 之后将会是一个填充位 0，所以就把它忽略了。需要注意的是，如果原始数据第 7 位是 0，填充位还是会被加入的，而且还是加在同样的位置，这就导致了在填充后的数据流中会有两个连续的 0。

www.cpit.com.cn


空闲
0 0 0 1 1 1 1 1 1 1 1 0 1 1
数据
填充位
填充数据
NRZI 码
0 0 0 1 1 1 1 1 1 0 1 1 0 1 1
接收器丢
弃填充位

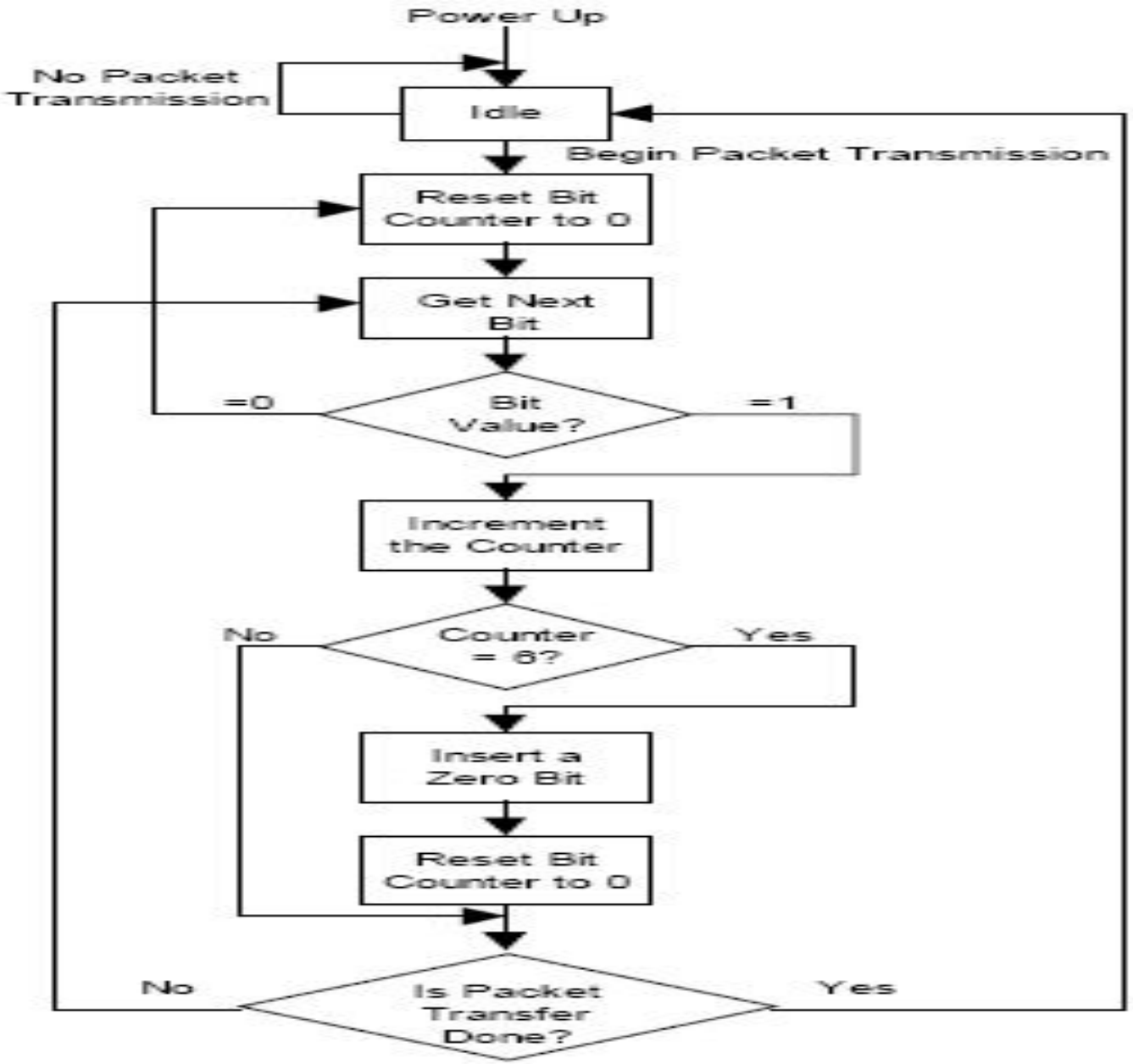


Figure 7-34. Flow Diagram for Bit Stuffing

1.全速/低速

全速/低速信号在包中无例外地使用填充位. 如果接收器在包的任何地方看到7个连续的1, 就表示产生位填充错误, 包应被忽略. 但EOP前面的时间间隔外. EOP前的最后一个数据位可以被集线器的开关时滞延长. 众所周知, 只延长了一点, 情况如下图所示, 这一点点的延长引入了第六个位, 因此不需要 位填充, 所以接收器必须能接收在EOP前的6个完整的位时间内没有跳边的包.

2. 高速

高速信号也在包中使用位填充, 除故意使用的高速EOP位填充错误.

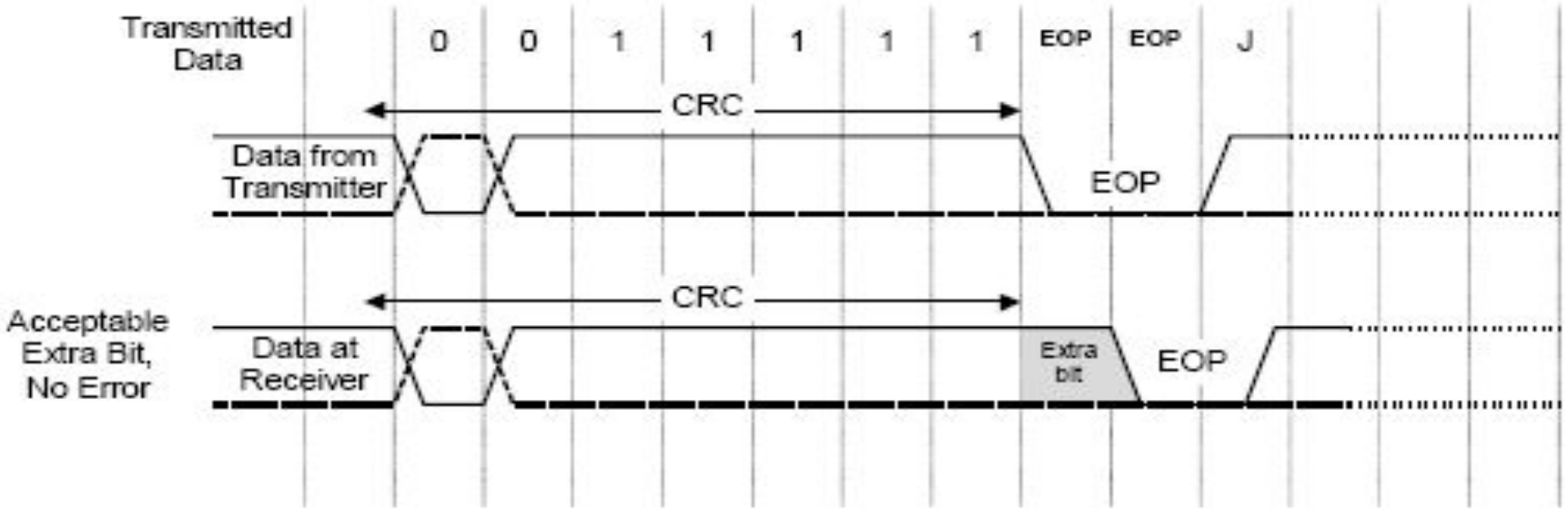


Figure 7-33. Illustration of Extra Bit Preceding EOP (Full-/low-speed)

www.cqit.com.cn

SYNC结构

低速/全速传输的SYNC结构要求有3个KJ对后接着2个K,共8个符号.图7.35显示了在每个低速/全速包前的NRZI数据编码位结构.

高速传输的SYNC结构要求有15个KJ对,接着2个K,共32个符号.当重发包时,集线器最多可以丢弃SYNC结构开始的4个位.但集线器不能破坏SYNC场的任何重复位.因此在被5个集线器重发后,包的SYNC场就只有12位了.

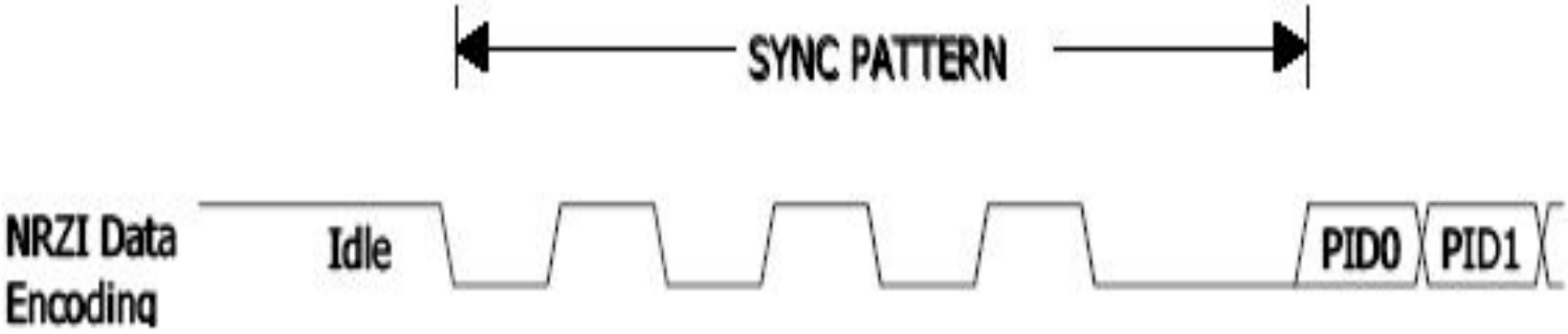


Figure 7-35. Sync Pattern (Low-/full-speed)

www.cpit.com.cn

(八)USB2.0的协议层

这章从场（Field）和包（Packet）的定义开始，从底向上地展示USB（Univeral Serial Bus）协议。接着是对不同事务（Transaction）类型的包事务格式的描述。然后是链路层（Link layer）流程控制（Flow Control）和事务级别的故障恢复（Fault recovery）。本章的最后将讨论复执同步化（Retry synchronization），超时干扰（Babble）和总线活动丧失（Loss of bus activity）的恢复。

1 位定序

数据位被发送到总线的时候，首先最低有效位（LSb），跟着是下一个最低有效位，最后是最高有效位（MSb）。在以后图表中的，包以下列形式给出，即包中单个的位和字段从左到右的顺序就是它们通过总线的顺序。

在标准描述符,请求和响应中的多字节场按从LSB到MSB的顺序解释以及在总线上传输.

2 SYNC场

所有的包都从同步（SYNC）场开始的，同步场是产生最大的边缘转换密度（Edge TransitionDensity）的编码序列 .输入电路用它将输入数据和本地时钟对齐.

同步场作为空闲状态出现在总线上，后面跟着以NRZI编码的二进制串“KJKJKJKK” .全速/低速的SYNC场被定义为8位,高速的SYNC场被定义为32位.接收到的SYNC场只是作为一个同步机制,并不在接着的包图中表现出来,SYNC场的最后两个位是一个识别SYNC场的结束或者以此推断PID的开始.

3 包的场格式

在后面几节将描述标记，数据和握手包的字段格式。包中位的定义是以未编码的数据格式给出。为了清楚起见，在此不考虑NRZI编码和位填充（Bit Stuffing）的影响。所有的包都分别有包开始（Start-of-Packet）和包结束（End-of-Packet）分隔符。包开始（SOP）分隔符是同步字段的一部分，而包结束（EOP）分隔符在电气规范中描述.

(1) 包的标识符场

所有USB包的同步字段后都紧跟着包标识符（PID）。如图8-1所示，包标识符由4位的包类型字段和其后的4位的校验字段构成。包标识符指出了包的类型，并由此隐含地指出了包的格式和包上所用错误检测的类型。包标识符的4位的校验字段可以保证包标识符译码的可靠性，这样包的余项也就能被正确地解释。包标识符的校验字段通过对包类型字段的二进制的求反码产生的。如果4 个PID检验位不是它们的各自的包标识符位的补，则说明存在PID错。



Figure 8-1. PID Format

主机和所有功能部件都必须对得到全部PID字段实行完整的译码。任何收到包标识符，如果含有失败的校验字段，或者经译码得到未定义的值，则该包标识符被假定是被损坏的，而且包的余项将被包接收机忽略（Ignore）。如果一个功能部件（Function）收到了包含了它所不支持的事务类型或方向的合法包标识符，则不必应答。例如，只能输入的端口（IN-only Endpoint）必须忽略输出标记（Token）。表8-1列出了包标识符类型，编码及其描述。

包标识符被分为4个编码组：标记，数据，握手和专用。包标识符传送的前2位（PID<0： 1>）指出了其属于哪个组。这说明包标识符编码的分布。

www.cpit.com.cn

Table 8-1. PID Types

PID Type	PID Name	PID<3:0>*	Description
Token	OUT	0001B	Address + endpoint number in host-to-function transaction
	IN	1001B	Address + endpoint number in function-to-host transaction
	SOF	0101B	Start-of-Frame marker and frame number
	SETUP	1101B	Address + endpoint number in host-to-function transaction for SETUP to a control pipe
Data	DATA0	0011B	Data packet PID even
	DATA1	1011B	Data packet PID odd
	DATA2	0111B	Data packet PID high-speed, high bandwidth isochronous transaction in a microframe (see Section 5.9.2 for more information)
	MDATA	1111B	Data packet PID high-speed for split and high bandwidth isochronous transactions (see Sections 5.9.2, 11.20, and 11.21 for more information)
Handshake	ACK	0010B	Receiver accepts error-free data packet
	NAK	1010B	Receiving device cannot accept data or transmitting device cannot send data
	STALL	1110B	Endpoint is halted or a control pipe request is not supported
	NYET	0110B	No response yet from receiver (see Sections 8.5.1 and 11.17-11.21)
Special	PRE	1100B	(Token) Host-issued preamble. Enables downstream bus traffic to low-speed devices.
	ERR	1100B	(Handshake) Split Transaction Error Handshake (reuses PRE value)
	SPLIT	1000B	(Token) High-speed Split Transaction Token (see Section 8.4.2)
	PING	0100B	(Token) High-speed flow control probe for a bulk/control endpoint (see Section 8.5.1)
	Reserved	0000B	Reserved PID

*Note: PID bits are shown in MSb order. When sent on the USB, the rightmost bit (bit 0) will be sent first.

通信未来 普天制造

(2)设备地址场

用功能设备地址场和端点场可以对功能设备的端点寻址.功能设备需要完全译码地址场和端点场.地址或端点不允许被混淆,任一个场的不匹配都会使令牌不忽略.访问未经初始化的端点也会使令牌被忽略.

. 地址场

功能部件地址（ADDR）字段通过其地址指定功能部件，至于是数据包的发出地还是目的地，则取决于标记PID的值。如图8-2所示，ADDR<6 ： 0>指定了总共128个地址 . ADDR场规定在IN,SETUP和OUT令牌以及PING和SPLIT特许令牌中使用.根据定义,每个ADDR值都定义了一个功能设备.在复位和上电后,功能设备地址的默认值是0,必须在枚举过程由主机进行编程.功能设备地址0是保留的默认地址,不能分配用于其他方面.

. 端点场

如图8-3所示，附加的4位的端口（ENDP）字段在功能部件需要一个以上端口时候允许更灵活的寻址。除了端口地址0之外，端口个数是由功能部件决定的。端口字段只对输入，建立和输出标记PID有定义。所有的功能部件都必须在端口0提供一个控制管道（缺省控制管道）。对于低速(Low Speed)设备，每个功能部件最多提供3个管道：在端口0的控制管道加上2个附加管道（或是2个控制管道，或是1个控制管道和1个中断端口，或是2个中断端口）。全速(Full Speed)功能部件可以支持最多可达16个的任何类型的端口。



Figure 8-3. Endpoint Field

www.cpit.com.cn

- **帧号码场**

  帧号码场是一个11位的场，主机在每个帧的基础上加1。帧码场在到达最大值7FFH后开始循环，它只在每个（微型）帧开始的SOF令牌中发送。

- **数据场**

  数据场的范围是0~1024字节，而且必须是整数个字节。图8.4是多字节的格式.每个字节的数据位都是由LSb首先移出.数据包的大小随着传输类型而变化.

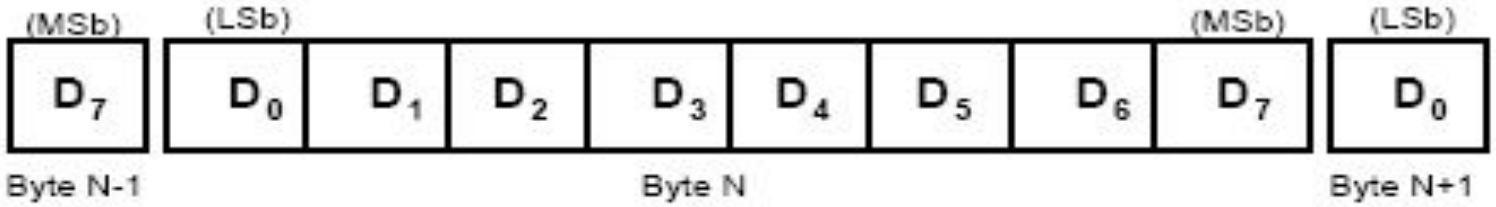


Figure 8-4. Data Field Format

- **循环冗余校验**

循环冗余校验（CRC）被用来在标记和数据包中保护所有的非PID字段。在上下文中，这些字段被认为是保护字段。PID不在含有CRC的包的CRC校验范围内。在位填充之前，在发送器中所有的CRC都由它们的各自的字段产生。同样地，在填充位被去除之后，CRC在接收器中被译码。标记和数据包的CRC可100 %判断单位错和双位错。失败的CRC指出了保护字段中至少有一个字段被损坏，并导致接收器忽略那些字段，且在大部分情况下忽略整个包。为了CRC的发生和校检，发生器和检验器里的移位寄存器置成为全1（All-ones）型。对于每个被发送或者被收到的数据位，当前余项的最高一位和数据位进行异或（XOR），然后，余项是左移1位，并且，最低一位置零。如果异或的结果是1，余项和生成多项式作异或。当检查的字段的最后的一位被发送的时候，发生器里的CRC被颠倒，再以最高位（MSb）在前发给检验器。当检验器收到CRC的最后的一位,且不发生错误的时候，余项将等于多项式的的剩余。如果剩余与包接收器中最后计算出的检验和余项（Checksum remainder）不匹配，则存在CRC误差。对于CRC，必须满足位填充的要求，且如果前6位都是1的话，这包括在CRC的最后插入零

1.令牌CRC

标记使用了5位的CRC字段，它覆盖了IN，SETUP和OUT令牌的ADDR和ENDP场或SOF标记的时间场。生成多项式如下：

$$G(X) = X^5 + X^2 + 1$$

用二进制的位方式表示多项式表示这个多项式就是00101B.如果令牌的所有位都被正确接收,那么接收器的5位余数是01100B.

2.数据CRC

数据CRC是一个16位的多项式，应用在数据包的数据场。其生成的多项式如下：

$$G(X) = X^{16} + X^{15} + X^2 + 1$$

这个多项式的二进制位组合是1000000000000101B。如果全部的数据和CRC位被准确无误地收到，16位剩余将是1000000000001101B.

4 包的格式

这节展示令牌，数据和握手包的包格式。包中的场在图中按位移动到总线的循序显示。

（1）令牌包

图8-5显示了令牌包的字段格式。令牌由PID，ADDR和ENDP构成，其中PID指定了包是IN，OUT还是SETUP类型。对于OUT和SETUP事务，地址和端口字段唯一地确定了接下来将收到数据包的端口。对于IN事务的，这些字段唯一地确定了哪个端口应该传送数据包。只有主机能发出标记包。输入PID定义了从功能部件到主机的数据事务。IN和SETUP PID定义了从主机到功能部件的数据事务。

如下图所示，标记包包括了覆盖地址和端口字段的5位CRC。CRC并不覆盖PID,因为它有自己的校验字段。标记和帧开始(SOF)包是由3个字节的包字段数据后面的包结束（EOP，End of Packet）界定的。如果包被译码为合法标记或SOF，但却没有在3个字节之后以EOP终止，则它被认为是无效的，并被接收器忽略。

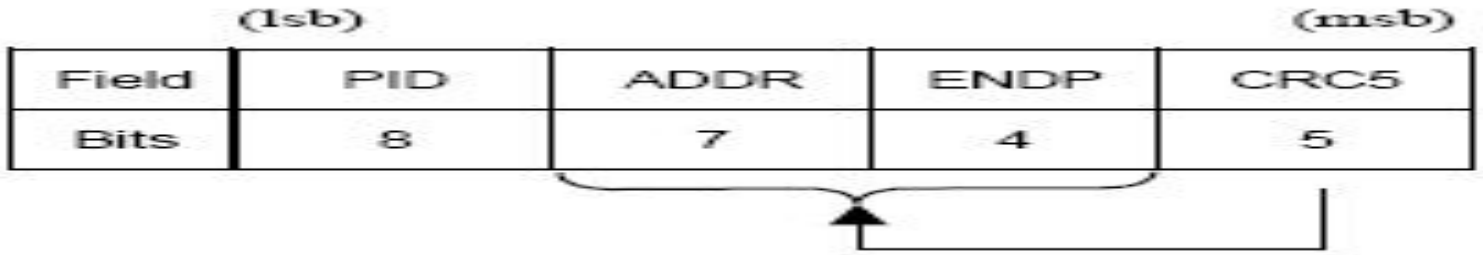


Figure 8-5. Token Format

（2）分离处理特殊令牌包

USB2.0为分离处理定义了一个特殊的令牌:SPLIT。与其他一般是3子节的命令牌相比，它共有4字节。分离处理令牌包通过额外的专用处理信息提供额外的处理类型。分离处理令牌用于支持主机控制器与在高速工作但下行口连接了一些全速/低速设备的集线通信的分离处理。共有两个分离处理使用了SPLIT特殊令牌定义：开始分离处理（SSPLIT）和完成分离处理（SPLIT）。

- 分离处理

高速分离处理只在当集线器连接了全速/低速设备时，在主机控制器和集线器之间使用。这种高速分离操作用于通过集线器和一些全速/低速设备的端点初始化一个全速/低速处理。高速分离处理也允许在总线上检索全速/低速处理，然后继续其他的高速处理，不需要在慢速下等待全速/低速处理进行/完成。

www.cpit.com.cn

高速分离处理由两部分组成：开始分离和完成分离。分离处理只在主机控制器与集线器之间定义使用。其他高速或全速设备不会使用用分离处理。

图8.6所示为一般的开始分离处理包的构成.在令牌阶段有2个包:SPLIT特殊命令牌和全速/低速令牌.根据数据传输的方向以及处理类型是否定义了握手,令牌阶段后面可以跟数据包和握手包.开始分离处理可以由2,3或4个包组成,由特定的传输类型和数据方向决定.

图8.7所示为一般的完成分离处理包的构成.在令牌阶段有2个包:SPLIT特殊命令牌和全速/低速令牌.根据数据传输的方向以及特定的处理类型,令牌后面可以跟数据包或者握手包.完成分离处理可以由2个或3个包组成,有特定的传输类型和数据方向决定.

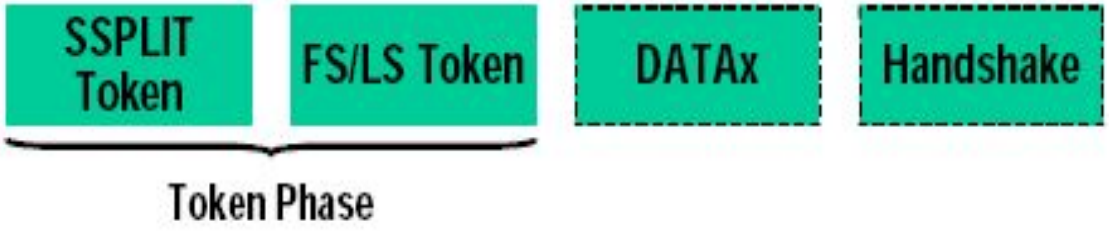


Figure 8-6. Packets in a Start-split Transaction

www.cpit.com.cn



Figure 8-7. Packets in a Complete-split Transaction

分离处理操作的结果由完成分离处理操作返回.图8.8所示为一个中断IN传输类型事例在概念上的“转变”.主机向集线器发出一个分离(用1表示)信号,然后继续进行其他的高速处理.开始分离处理会使集线器内在一段时间内发出一个全速/低速IN令牌(用2表示).(在这个例子中)设备用一个数据包响应IN令牌,而集线器用握手包响应设备.最后,主机在一段时间内发出完成分离(用3表示)信号,取得设备提供的数据.注意:在这个例子中,集线器在完成分离前向设备提供的是全速/低速握手(在这个例子中是ACK),完成分离不向集线器提供高速握手.

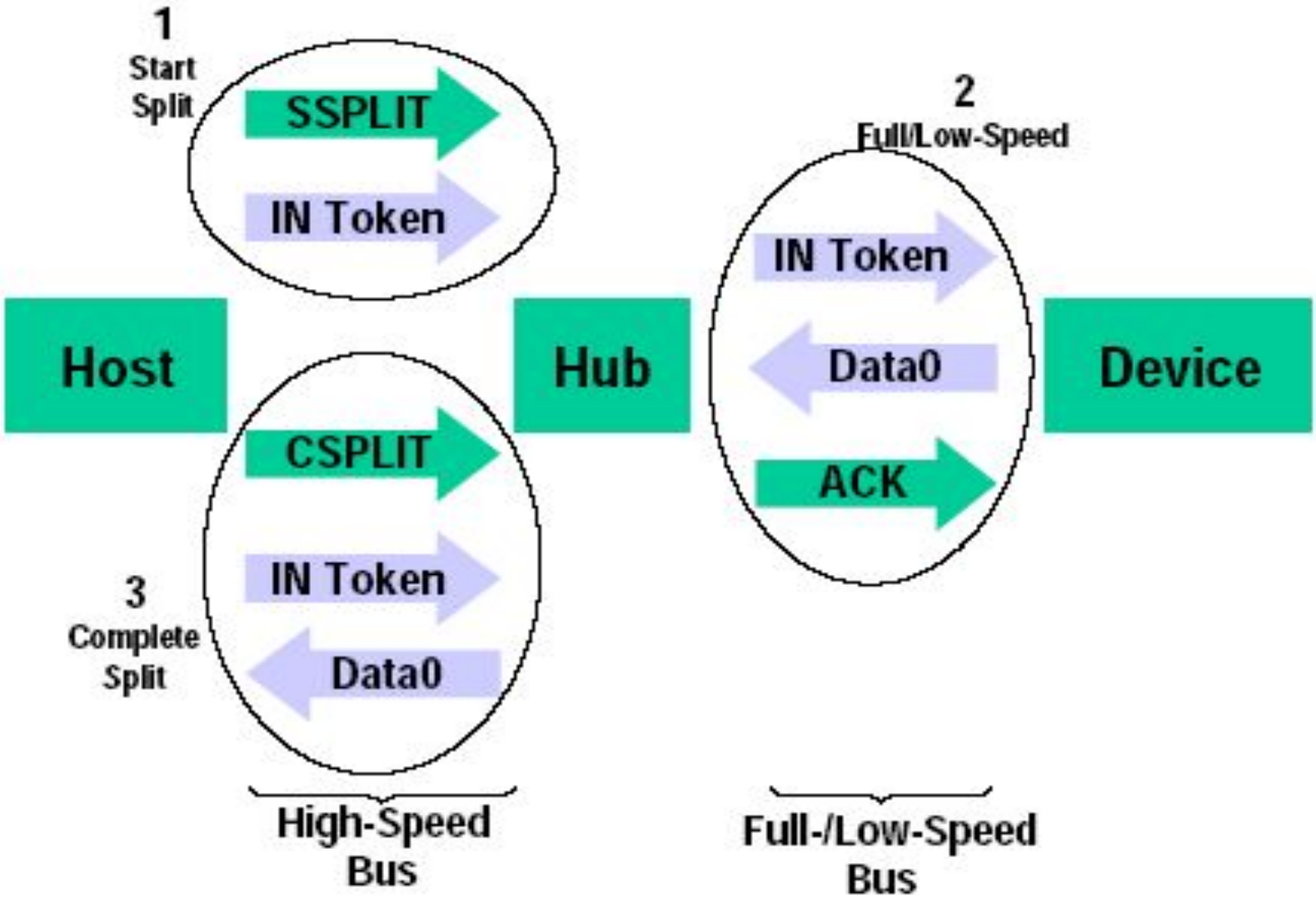


Figure 8-8. Relationship of Interrupt IN Transaction to High-speed Split Transaction

www.cpit.com.cn

一般的全速/低速OUT处理也可以相似地从概念上“转换”到开始分离和完成分离处理.图8.9所示为一个中断的OUT处理类型“转化”事例.主机发出一个包含1个SSPLIT特殊令牌,1个OUT令牌和1个DATA包的开始分离处理.集线器在一段时间后在全速/低速总线发送OUT令牌和DATA包.设备用一个握手信号响应.段时间后,主机发送完成分离处理操作,集线器用设备提供的结果(全速/低速数据或者握手)响应.

SPLIT特殊令牌使用一般的令牌格式,从PID场(在SYNC后)开始,以CRC5场(和EOP)结束.开始分离和完成分离令牌都有4个字节.SPLIT处理只能由主机产生.

- **开始分离处理令牌**

图8.10所示为开始分离处理令牌包的场.

集线器地址场包含了集线器在这个全速/低速处理中支持的全速/低速设备的USB设备的地址.这个场与8.3.2中定义的ADDR场有相同的定义.

SC(开始/完成)场为0的SPLIT特殊令牌包表示它是一个开始分离处理(SSPLIT).

www.cpit.com.cn

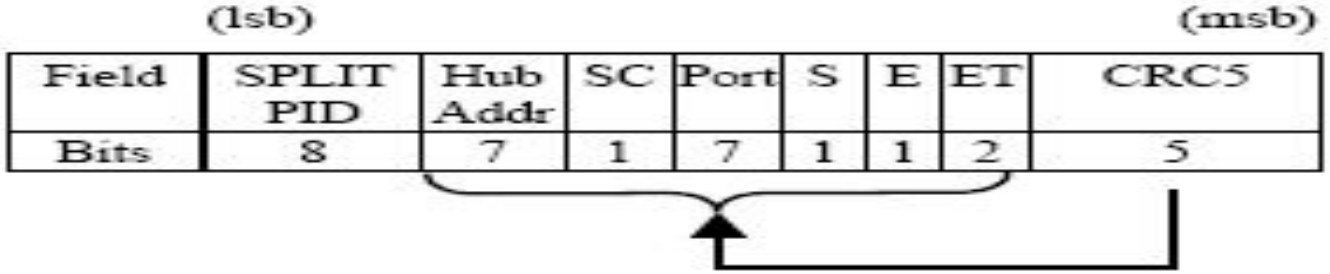


Figure 8-10. Start-split (SSPLIT) Token

端口场包含了全速/低速处理要送往的目标集线器端口号码.如图8.11所示,用PORT<6:0>总共指定128个端口.主机必须为单个或多个TT集线器的应用正确设置端口号.单个TT集线器的应用可以忽略端口场.

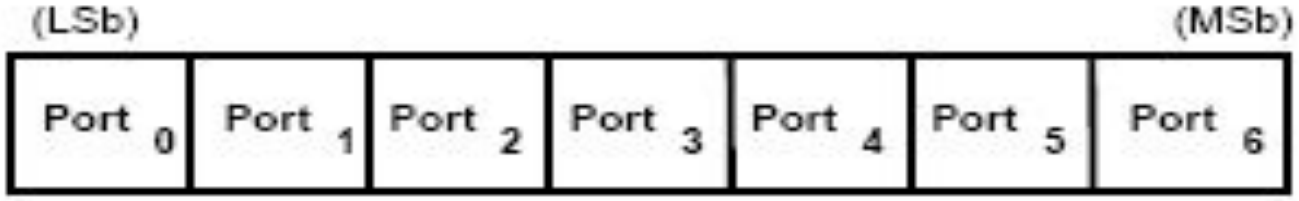


Figure 8-11. Port Field

S(速度)场指定了中断或控制处理的速度，它的含义如下：

- 0——全速
- 1——低速

S场在批量IN/OUT和同步IN开始分离中必须设为0。在批量/控制IN/OUT，中断IN/OUT和同步IN开始分离中，E场必须设为0。

对于全速同步OUT开始分离来说，S（开始）和E（结束）场指定了高速数据有效负载如何对应于全速数据包的数据通信，如表8.2所列.

同步的OUT开始分离处理使用这些编码使集线器能检测各种错误情况.这些错误情况由端点的数据有效负载请求多个开始分离信号,但没有接收到开始分离处理造成.例如,大的全速数据有效负载可能请求3个开始分离处理:开始分离/开始,开始分离/中间和开始分离/结束.如果集线器没有到任何处理,他将忽略全速处理(如果没有收到开始分离/开始信号)或者错误来响应全速处理(如果没有收到其余两个处理中的一个).在微型帧其间不用接收开始分离信号,也可以检测其他错误情况.

ET(端点类型)场如表8.3列出了全速/低速处理的端点类型.这个场告诉集线器对全速/低速处理采取那种分离处理状态机.

全速/低速设备的地址和端点信息包含在SPLIT特殊令牌包后面的一般令牌包中。

Table 8-2. Isochronous OUT Payload Continuation Encoding

S	E	High-speed to Full-speed Data Relation
0	0	High-speed data is the middle of the full-speed data payload
0	1	High-speed data is the end of the full-speed data payload
1	0	High-speed data is the beginning of the full-speed data payload
1	1	High-speed data is all of the full-speed data payload.

Table 8-3. Endpoint Type Values in Split Special Token

ET value (msb:lsb)	Endpoint Type
00	Control
01	Isochronous
10	Bulk
11	Interrupt

- **完成分离处理令牌**

图8.12所示为完成分离处理令牌包的场.

(lsb) (msb)

Field	SPLIT PID	Hub Addr	SC	Port	S	U	ET	CRC5
Bits	8	7	1	7	1	1	2	5

Figure 8-12. Complete-split (CSPLIT) Transaction Token

SC场是1的SPLIT特殊令牌包表示这是一个完成分离处理(CSPLIT).

U位是保留的/不使用位,必须设置为0(0B).

完成分离令牌包的其他场与开始分离令牌包的场有相同的定义.

(3)帧开始(SOF,Start-of-Frame)包

帧开始(SOF)包由主机以每(1.00±0. 0005)ms(全速总线)一次和每(125 ±0. 0625)us(高速总线)一次的标称速率进行发送.SOF包包含一个指出包类型的PID场,后面紧跟一个11位的帧号码场,如图8.13所示.

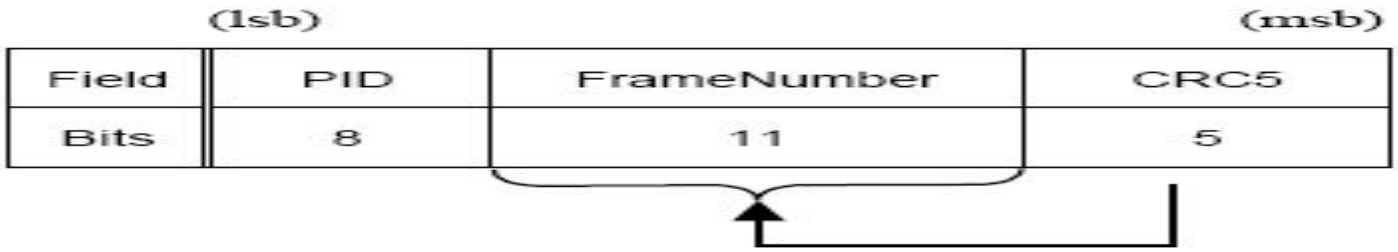


Figure 8-13. SOF Packet

SOF令牌由只对令牌进行处理的操作完成.它在对应每帧开始的精确定时间隔中分配SOF标记以及帧号码.所有高速的全速功能设备(包括集线器都接收SOF)包.SOF令牌不会使任何正在进行接收的功能设备产生一个返回的包,因此不能保证SOF被传输到任何指定的功能设备.

SOF包发送两条定时信息.当功能设备检测到SOF PID后,就知道产生一个SOF.此时,不需要跟踪帧号码.对帧定时敏感的功能设备(例如全速运行的集线器),只需要译码SOF PID；它们能忽略帧号码及其CRC。如果功能设备需要跟踪帧号码，则必须译码PID和时间标记。不是特别需要总线定时信息的全速设备可以忽略SOF包。

USB帧和微型帧

USB定义了一个全速1 ms的帧时间，用带定义的抖动容差及每隔1 ms的帧开始（SOF）包来表示。USB还定义了带相应抖动容差，125 us帧时间的高速微型帧。SOF包在全速链路下（由主机控制器或集线器处理转换器）每1 ms产生一次。高速链路的SOF包每隔7个125 us产生一次。

Figure 8-14 shows the relationship between microframes and frames.

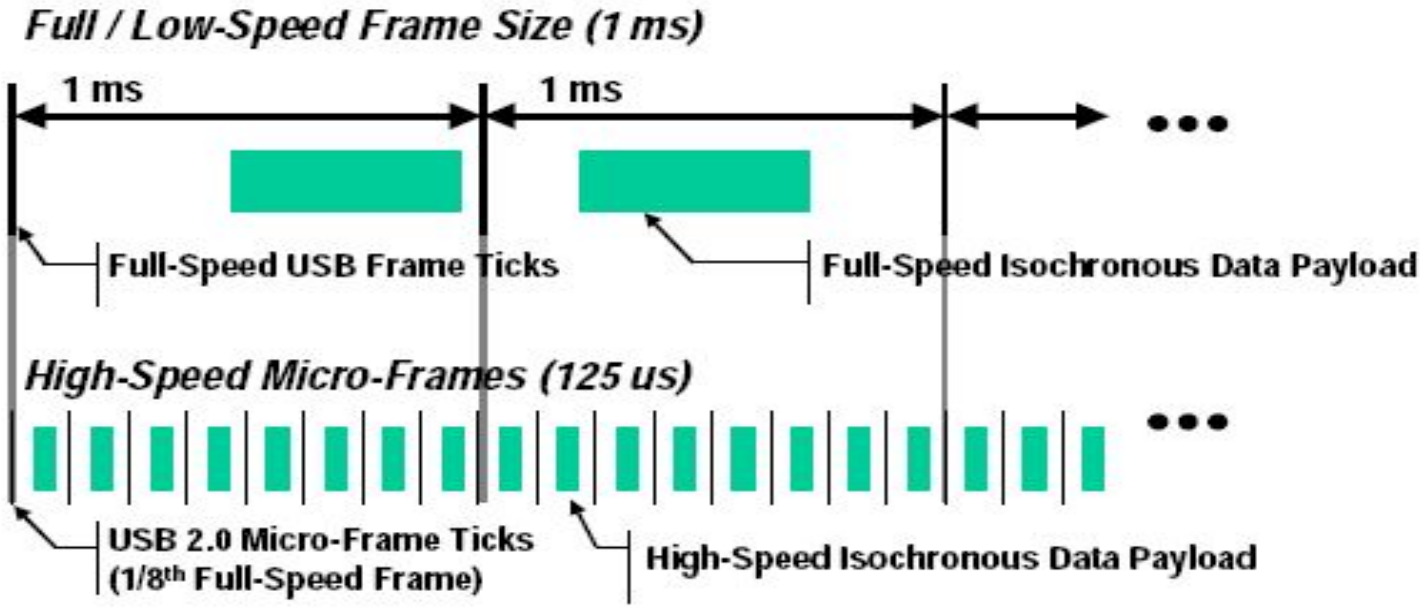


Figure 8-14. Relationship between Frames and Microframes

高速设备在每1ms的周期内能看到相同的帧号码8次（每125 us ）。如果要求的话，高速设备检测到SOF的帧号码与前一个SOF的帧号码不同时，可以在本地确定待定微型帧“号码”，而且这个微型帧被看作第0号微型帧。接着的7个有相同帧号码的微型帧被看做微型帧1~7。

（4）数据包

数据包有1个PID场，包含0个或多个数据字节的数据场以及1个CRC场组成，如图8.15所示.数据包共有4中类型,分别用不同的PID识别:DATA0,DATA1,DATA2和MDATA.两个数据包PID(DATA0和DATA1)定义用于支持数据触发同步.所有4个数据PID都可以在高带宽,高速同步端点的数据PID序列中使用.3个数据PID(MDATA,DATA0,DATA1)可以在分离包中使用.

数据必须以整数字节进行发送.数据CRC只对包的数据场进行计算,而不包括PID场,因为PID有自己的校验场.

低速设备允许的最大数据有效负载是8字节.全速设备的最大数据有效负载是1023字节.高速设备的最大数据有效负载是1024字节.

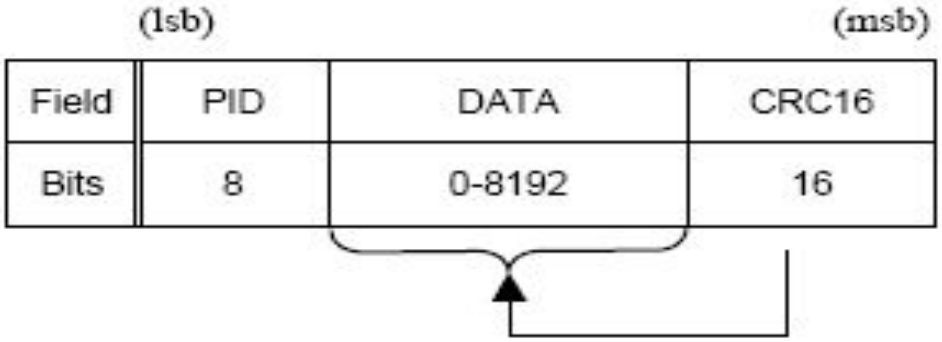


Figure 8-15. Data Packet Format

(5)握手包

如图8.16所示,握手包只有一个PID组成.握手包用于报告数据处理的状态,还能返回表示成功接收数据,命令被接受或拒绝.流控制以及停止的情况值.只有支持流控制的处理类型能返回握手.握手一般在处理的握手阶段返回,可以替代数据在数据段返回.握手包与EOP通过一个字节隔开.如果包被译成另一个有效的握手,但不是在一字节后以EOP终止,那么它被认为无效且被接收器忽略.

有4种握手包类型和1种特殊的握手包.

- ACK　表示数据包没有位填充或数据字段上的CRC错，并且数据PID被正确收到。ACK在下列的情况下被发放，当时序位（Sequence Bit）匹配且接收器能接受数据的时候，或者当时序位失配，但发送方和接收器互相之间必须再同步（Resynchronize）（在第8.6节详细提到）的时候。ACK握手信号只适用于数据被传送且期待握手信号的事务中。对于IN事务，ACK由主机返回，而对于OUT或SETUP或PING事务则由功能部件返回；

	(lsb)	(msb)
Field	PID	
Bits	8	

Figure 8-16. Handshake Packet

www.cpit.com.cn

- NAK 表示功能部件不能从主机接受数据（对于OUT事务），或者功能部件没有传输数据到主机（对于IN事务）。NAK仅由功能部件在IN事务的数据时相返回，或在OUT事务或PING事务的握手时相返回。主机决不能发出NAK。出于流控制的目的，NAK用于表示功能部件暂时不能传输，或者接收数据，但是最终还是能够在不需主机干涉的情况下而传输或接收数据.
- **STALL** 作为IN事务的回应或响应PING处理或者在OUT事务的数据阶段之后由功能部件返回（见图8-30和图8-38）。STALL表示功能部件不能传输，或者接收数据，或者不支持一个控制管道请求。功能设备返回STALL后的状态(对于除默认端点为的所有端点)没有定义. 在任何条件下都不允许主机返回STALL。 STALL握手由设备用于在两个不同的场合之一。第一种情况，是当设置了与端口相联系挂起特征（Halt feature）的时候，称为“功能STALL（functional stall）”（挂起特征在这文档的第9章中详细说明）。功能STALL的特殊情况是“命令STALL（commanded stall）”。如同在第9章中详细叙述的那样，命令STALL发生在主机显式地设置了端口的挂起特征的时候。如果功能部件的端口被挂起，则功能部件必须继续返回STALL，直到引起停止的条件通过主机干涉而被清除。

  第二种情况称为“协议STALL（protocol stall）”。协议STALL对于控制管道是唯一的。协议STALL和功能STALL在意义和持续时间上是不同。协议STALL在控制传送（Control Transfer）的数据或状态阶段（Stage）被返回，并且，STALL条件在下一个控制传送的开始终止（建立事务）。这节的剩下的部分将提到功能STALL的一般情况。

- NYET 是只在高速中出现的握手,它在两种情况下返回.当高速端点是PING协议的一部分时,会返回NYET握手;当集线器在全速/低速处理仍未完成分离处理或者不能进行分离处理操作时,也会返回NYET握手.
- ERR 只在高速度中出现的握手,它的返回允许高速集线器报告的全速/低速总线错误.它只能由高速集线器作为分离处理协议的一部分返回.

(5)握手响应

传输和接收功能部件必须根据从表8-4中到表8-6详细叙述的优先顺序返回握手。不是所有的握手都是被允许，依赖于事务类型和功能部件或主机是否发出握手。如果标记在传输到功能部件的阶段里发生了错误，则功能部件将不以任何包回应，直到下一个标记被收到并成功地译码。

1. 功能部件对IN事务回答

表8-4显示了功能部件作为对输入标记的反应而可能做的回答。如果由于停止或流控制条件,功能部件不能发送数据，它将发出STALL或NAK握手。如果功能部件能发出数据，它就发出数据。如果收到的标记被损坏，则功能部件不应答。

www.cpit.com.cn

Table 8-4. Function Responses to IN Transactions

Token Received Corrupted	Function Tx Endpoint Halt Feature	Function Can Transmit Data	Action Taken
Yes	Don't care	Don't care	Return no response
No	Set	Don't care	Issue STALL handshake
No	Not set	No	Issue NAK handshake
No	Not set	Yes	Issue data packet

2. 主机对IN事务回答

表8-5显示了主机对输入事务回答。主机只能返回1种类型的握手：ACK。如果主机收到了损坏的数据包，它把数据丢弃且不应答。如果主机不能从功能部件接受数据，则（出于类似内部缓冲溢出的问题）这条件被认为是错误，并且主机不应答。主机能接受数据，并且如果数据包是完整无错地被接收到，则主机接受数据并发出ACK握手。

Table 8-5. Host Responses to IN Transactions

Data Packet Corrupted	Host Can Accept Data	Handshake Returned by Host
Yes	N/A	Discard data, return no response
No	No	Discard data, return no response
No	Yes	Accept data, issue ACK

3. 功能部件对OUT事务回答

对输出事务的握手回答由表8.6表示。假设令牌译码成功，功能部件收到数据包后，可以返回3种握手类型中的任何一种。如果数据包被损坏，功能部件不返回握手。如果数据包是被完整无错地接收到，而功能部件的接收端口被停止，则功能部件返回STALL。如果事务正维持着时序位同步而探测到失配号，那么功能部件返回ACK，并丢弃数据。如果功能部件能够接受数据并完整无错收到数据，它返回ACK。如果由于流控制的原因，功能部件不能接受数据包，它返回NAK。

4. 功能部件对SETUP处理的响应

SETUP定义了特殊的主机-功能部件的数据事务，它允许主机初始化端口的同步位为主机的同步位。一收到SETUP 令牌，功能部件就必须接受数据。功能部件不能对SETUP令牌用STALL或NAK应答，并且，接收功能部件必须接受SETUP 令牌后的数据包。如果非控制端口收到SETUP 令牌，它必须忽略事务且不应答。

(5)处理包的序列

包事务格式根据端口类型而变化。有4种端点类型：批处理（Bulk），控制（Control），中断（Interrupt）和同步（Isochronous）。]

主机控制器和设备都要求用不同的状态机能正确地按顺序组织各种处理类型.下面的图显示了状态机如何对每种类型处理正确定义包的顺序.这些图表不应在要求的应用中使用,但可以指定要求的行为.

图8.17所示为状态机的图例.

www.cpit.com.cn

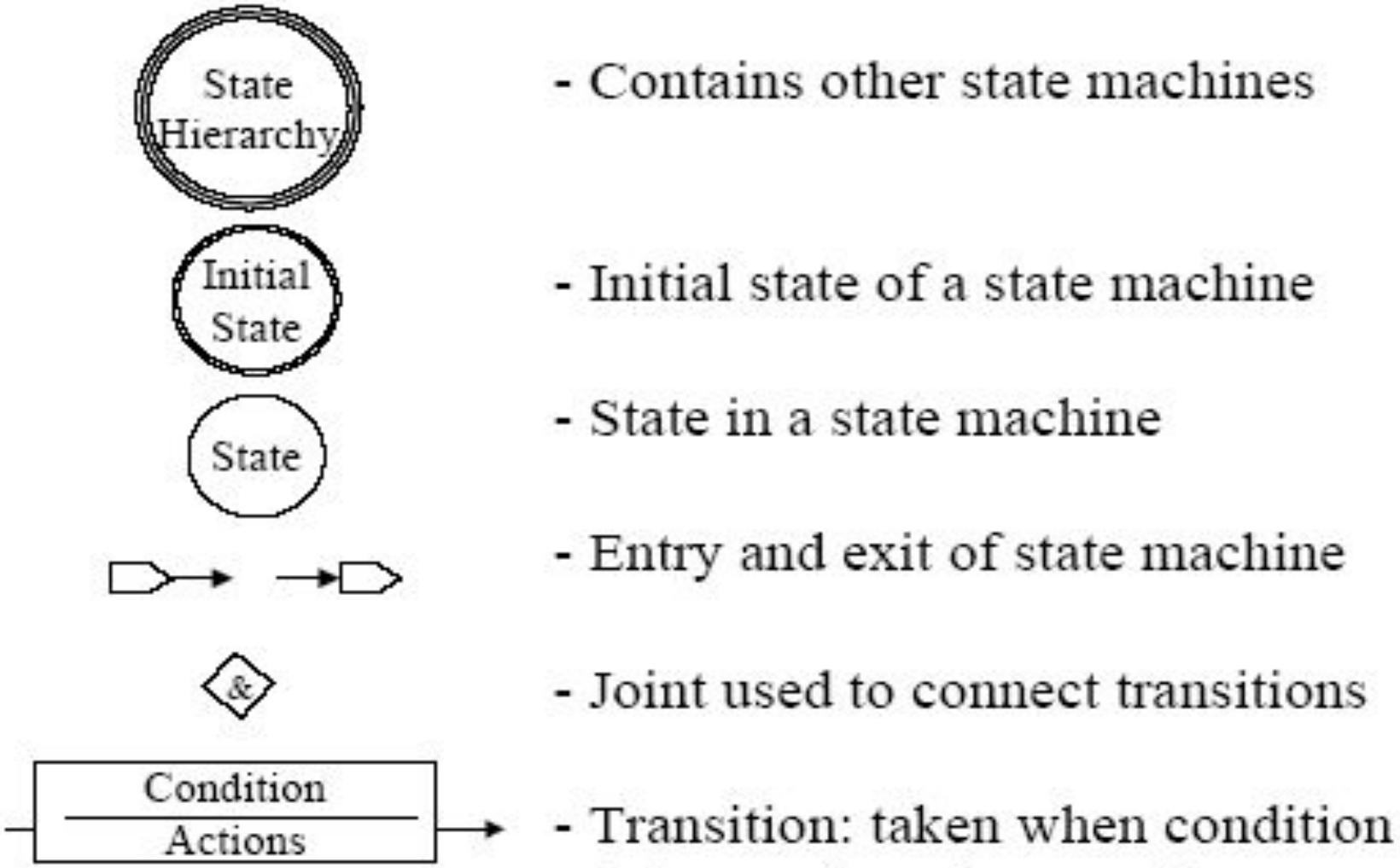


Figure 8-17. Legend for State Machines

www.cpit.com.cn

图中,又3条边界线的圆表示引用了另一个(层次的)状态机.有两条边界线的圆表示初始状态.只有一条边界线的圆表示单个状态.带箭头的图标(图例中)分别表示状态机的入口或出口.入口/出口与在状态机层次结构中的更高层状态机的另一个状态有关.菱形(接合点)用于将几个处理连接到一个公共接点.接合点允许有单个输入处理,多个输出处理,或者多个输入处理.单个输出处理.当要使用路径时,在路径上所有处理的条件包括接合点都必须为真.路径只是一个包括一个或多个接合点的处理序列.过渡状态用一个巨形来标记,其中间的一条线分隔了条件(上面)和操作(下面).要执行过渡状态时,条件必须为真.行为和条件的语法是VDHL.如果出现过渡状态,那么就会执行操作.圆圈中包含粗体的名字以及在进入状态前可以执行的一个或多个操作.

主机控制器和设备状态机的前后关系如图8.18所示.主机控制器决定了端点要运行的下一个处理并命令(HC_cmd)发送到主机控制器状态机。这就使主机控制器状态机向下行总线（HSD1）发送一个或多个包。

设备总线（HSD2）接收这些包，处理接收到的包，并通过对应的端点状态（在EP_array）与它的功能设备互相作用.然后设备会向上行总线(HSU1)发送一个包来响应.主机控制器状态机可以从总线(HSU2)接收到包,并向主机控制器(HC_resp)提供处理的结果.在总线上发什么包由端点的传输类型和状态机要执行什么总线活动决定.

www.cpit.com.cn

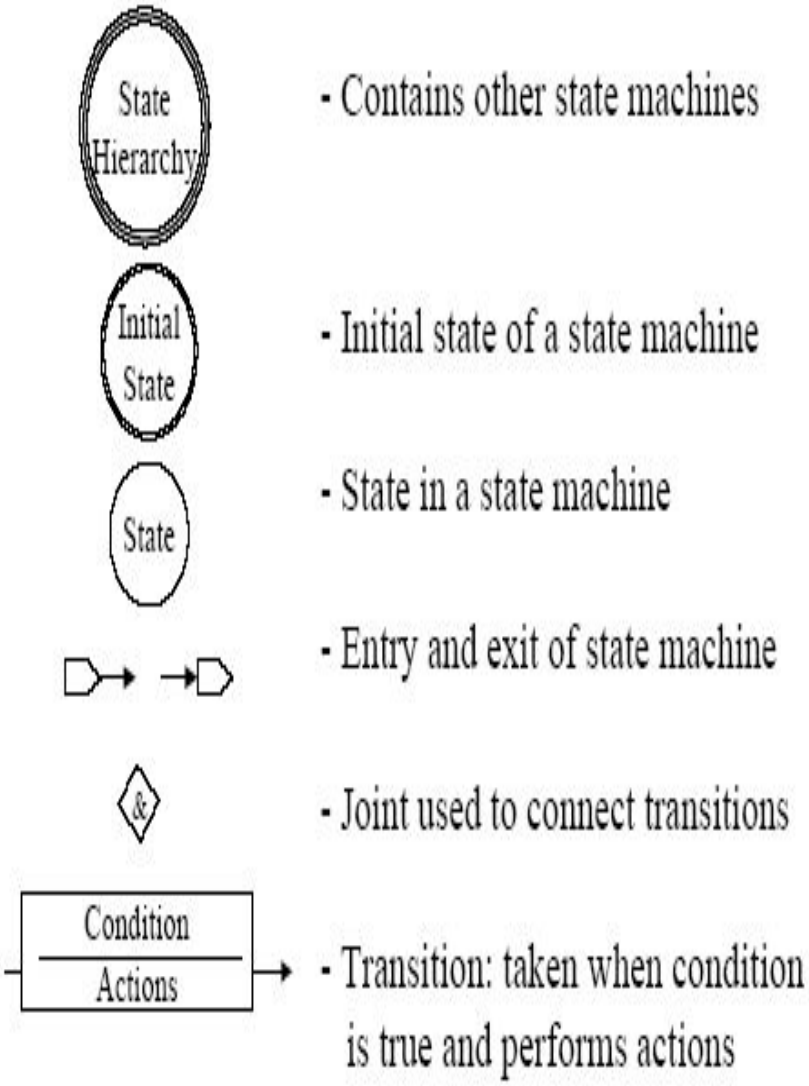


Figure 8-17. Legend for State Machines

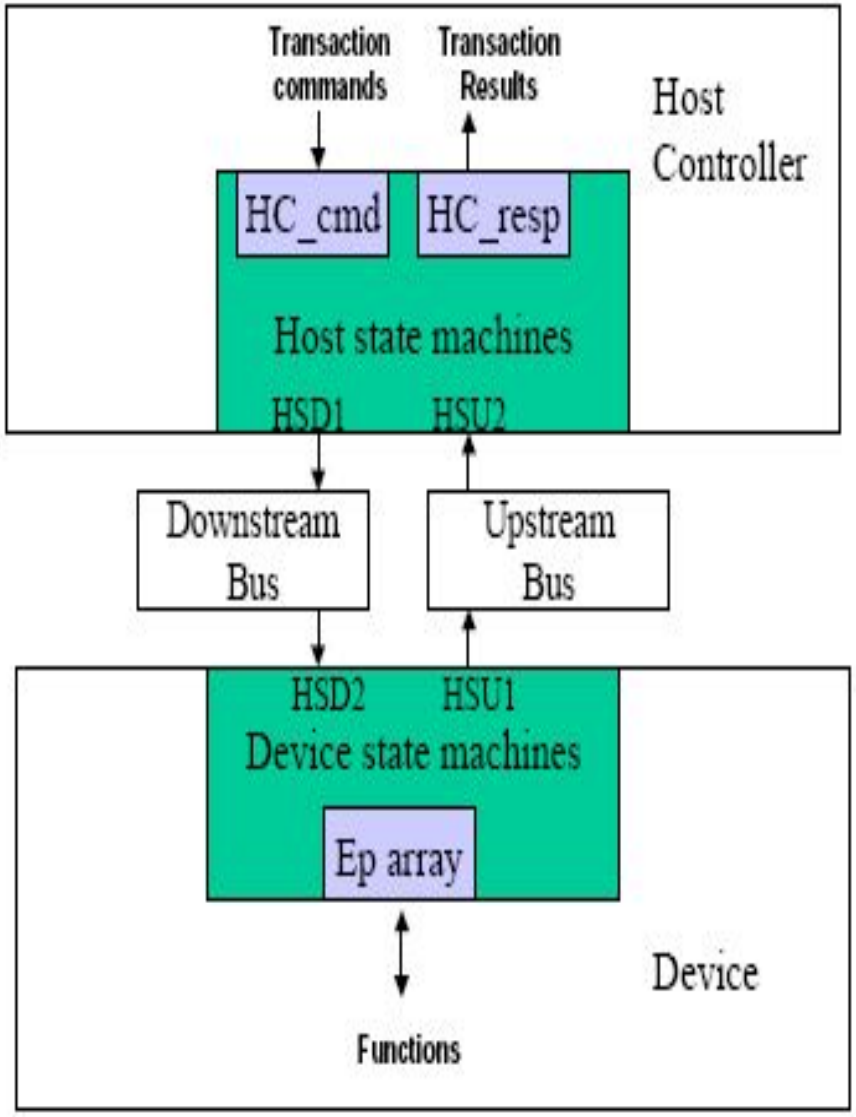


Figure 8-18. State Machine Context Overview

状态机用层次结构的形式表示.图8.19所示为主机控制器的顶层状态机.在本章其他部分出现的都是非分离处理操作.包处理的状态机(HC_Do_start和HC_ Do_complete)将在第11章介绍.

主机控制器的状态机在主机控制器中.主机控制器向下行发送包(标记为HSD1)并接收上行的包(标记为HSU2).

主机控制器的命令可以告诉它向端点发送的下一个处理是什么.主机控制器可以跟踪 几个端点的处理.主机控制器的状态机按顺序确定主机控制器下一步要对当前的端点执行什么操作.设备的每个端点都有一个状态.设备的状态机按顺序确定设备要对处理作什么反应.

图8.20所示为非分离处理类型的主机控制器的状态机层次结构的总体概况.图8.21所示为设备状态机的层次结构.首先说明的是各种端点类型的公共状态机.底层端点类型的详细状态机将在下面的端点类型部分说明.

图8.22-图8.26分别介绍了设备状态机各个层次的结构概况.

www.cpit.com.cn

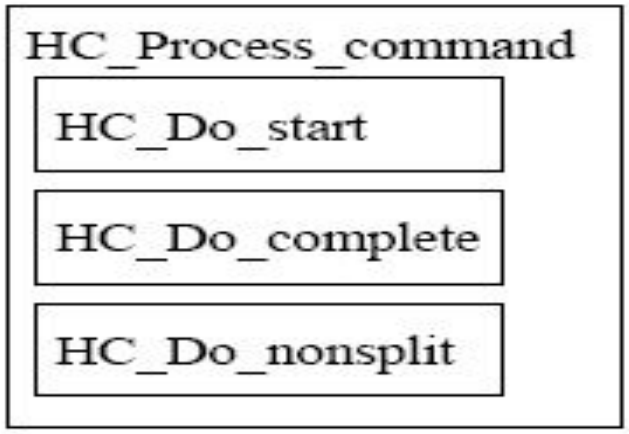


Figure 8-19. Host Controller Top Level Transaction State Machine Hierarchy Overview

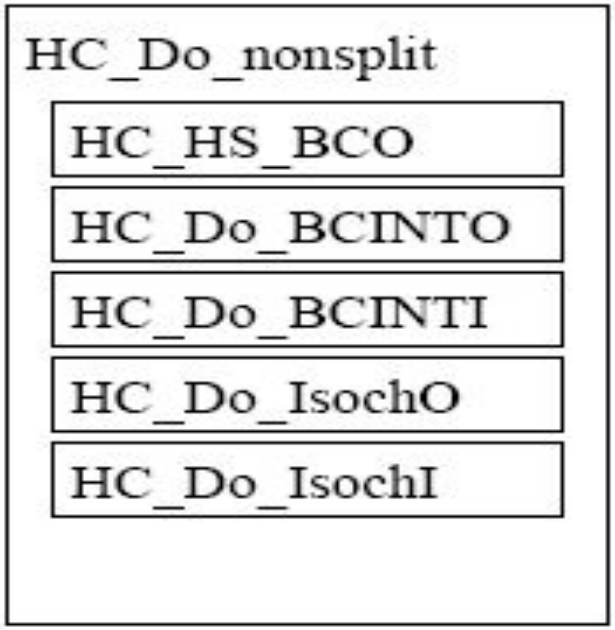


Figure 8-20. Host Controller Non-split Transaction State Machine Hierarchy Overview

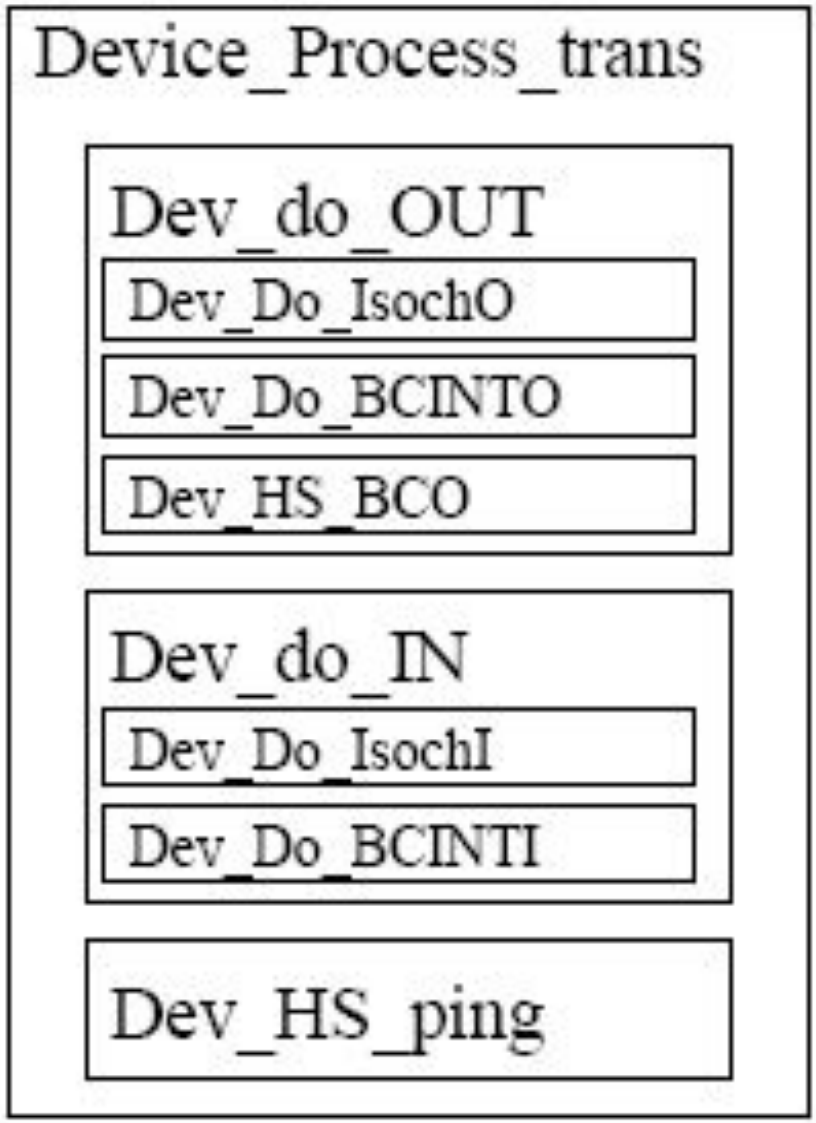


Figure 8-21. Device Transaction State Machine Hierarchy Overview

Global Actions Concurrent Statements Architecture Declarations Signals Status State Register Statements

Package List

```
ieee        std_logic_1164
ieee        numeric_std
usb2statemachines behav_package
```

```
SIGNAL  SCOPE DEFAULT
hwul    OUT   (BULK,NAK,0,0,ok,in_dir,TRUE,ALLDATA,FALSE,FA
device  INT   '0'
token   INT   '0'
```

Process Declarations

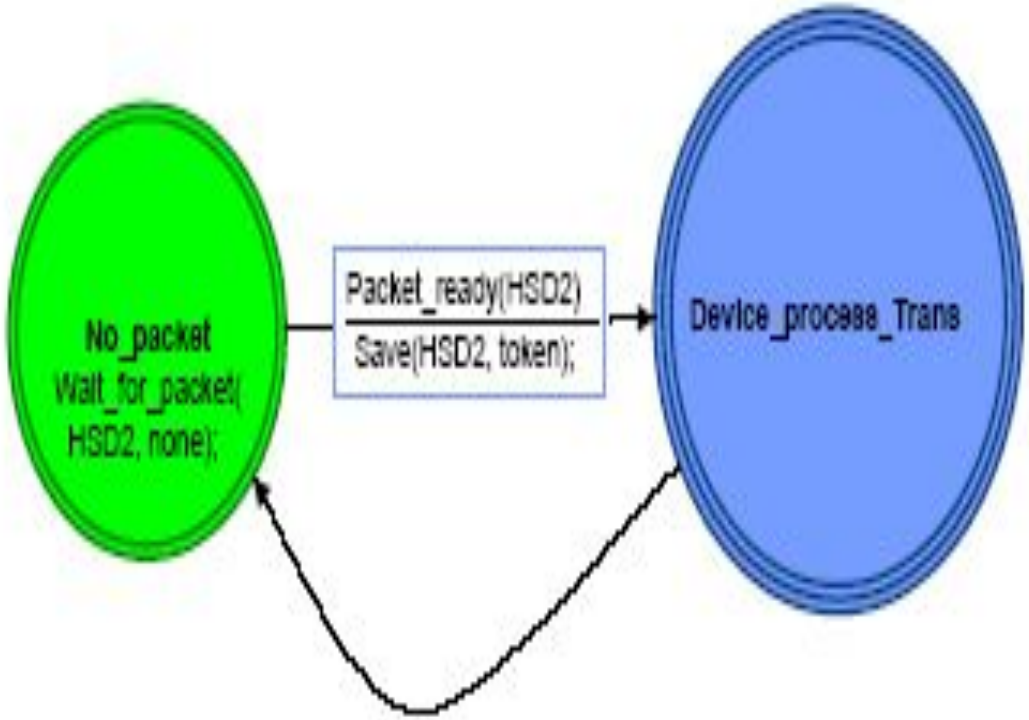


Figure 8-22. Device Top Level State Machine

www.cpit.com.cn

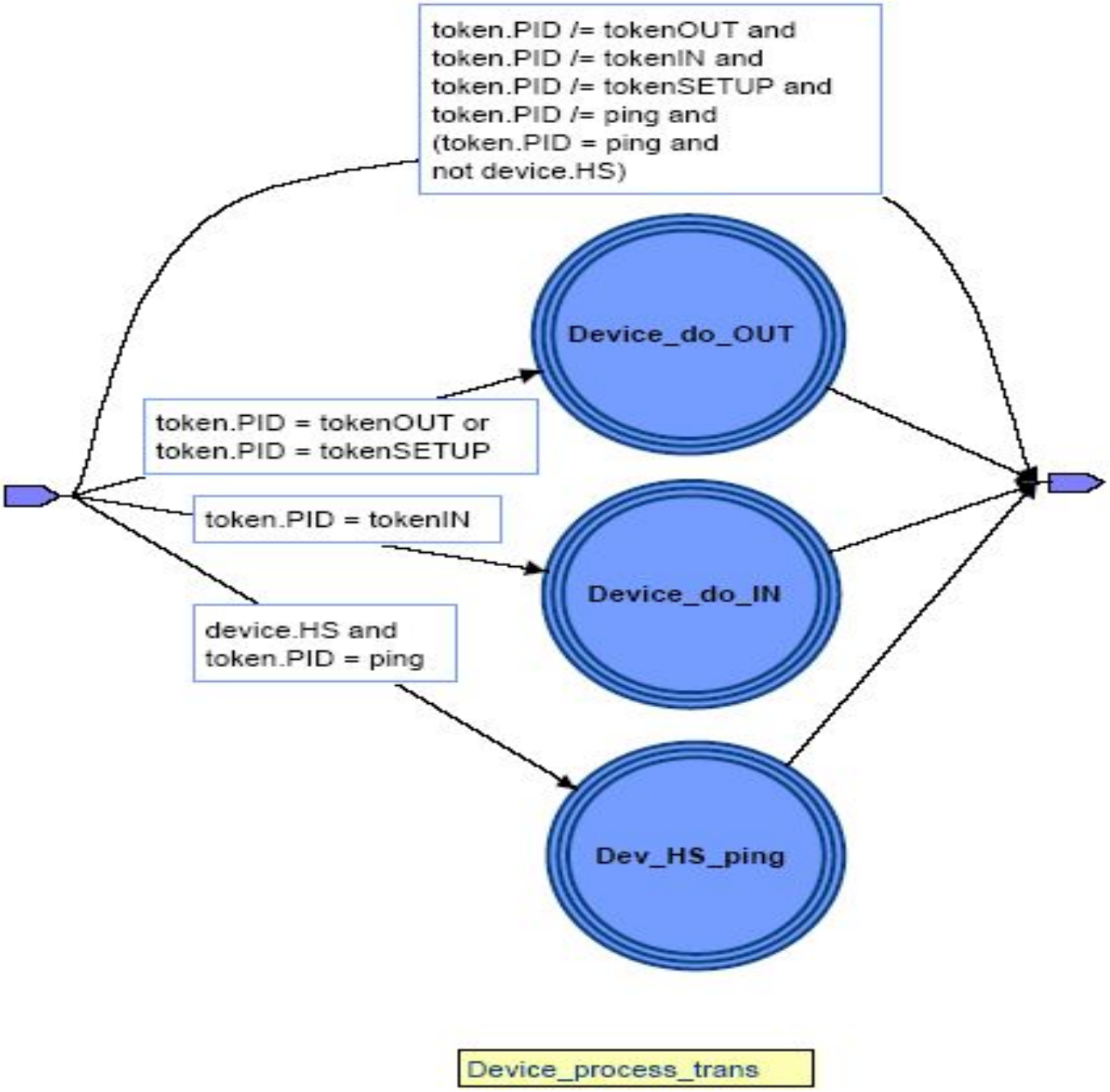
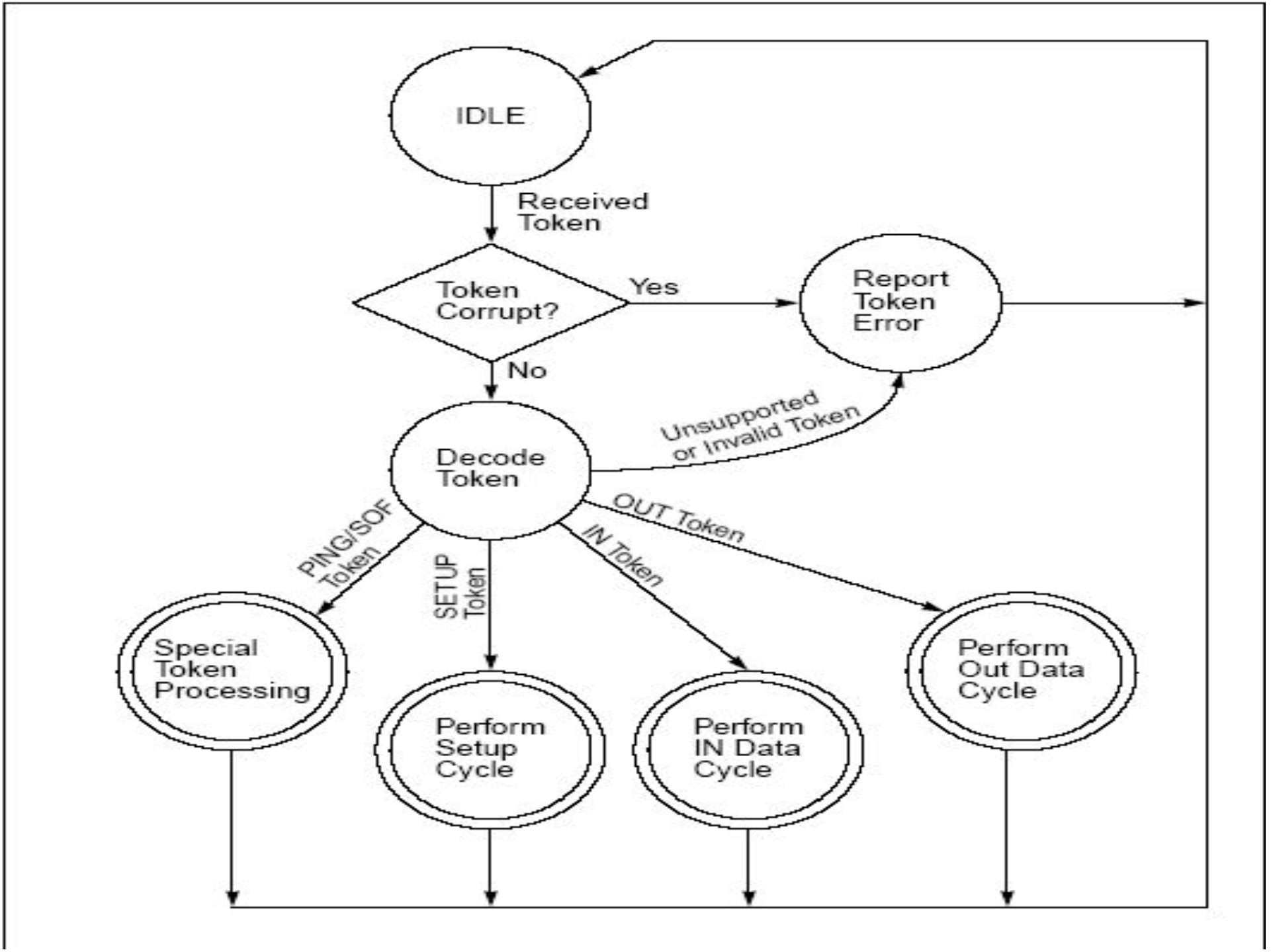


Figure 8-23. Device_process_Trans State Machine


IDLE
Received Token
Token Corrupt?
Yes
Report Token Error
No
Decode Token
Unsupported or Invalid Token
PING/SOF Token
SETUP Token
IN Token
OUT Token
Special Token Processing
Perform Setup Cycle
Perform IN Data Cycle
Perform Out Data Cycle

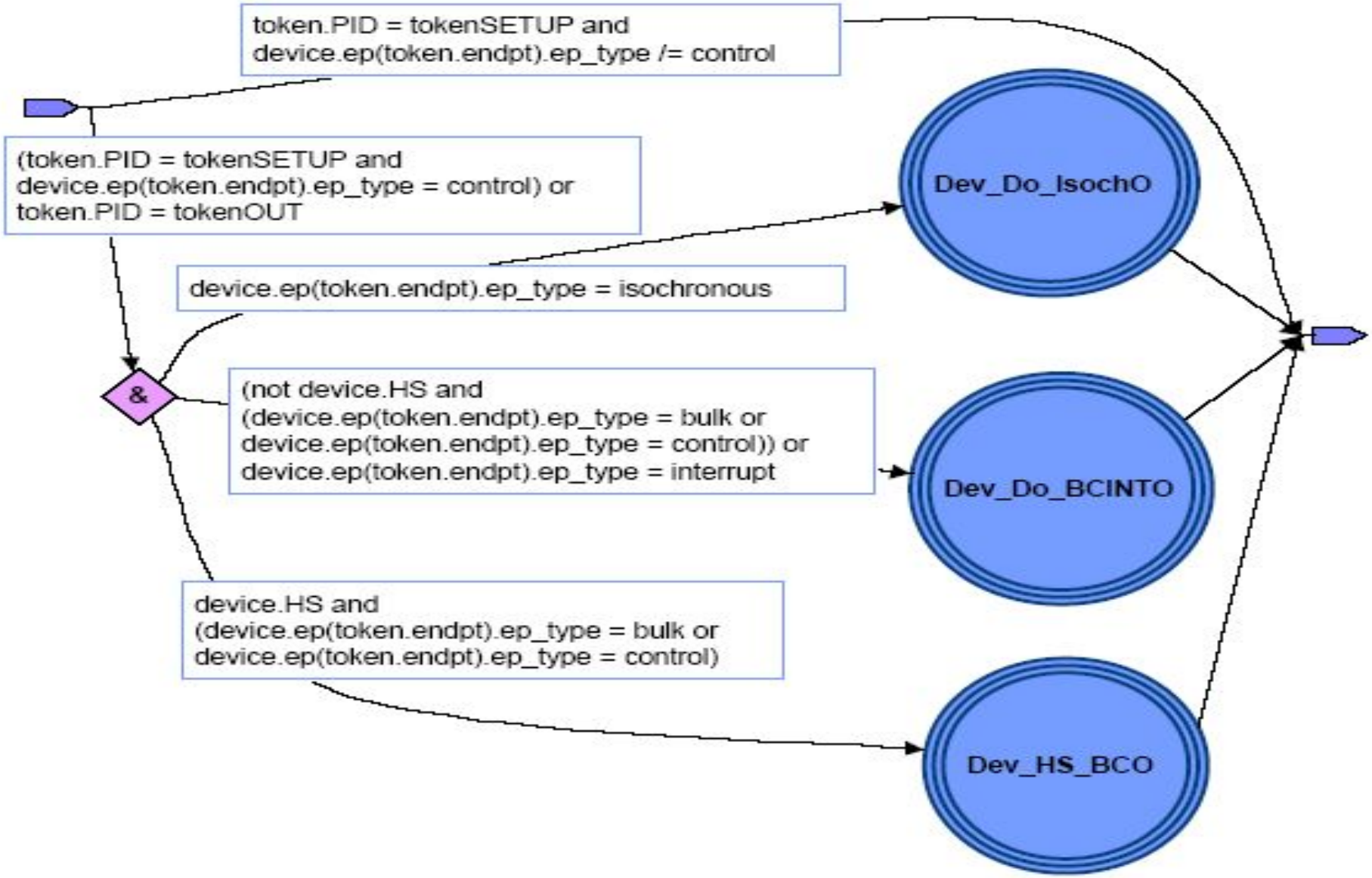


Figure 8-24. Dev_do_OUT State Machine

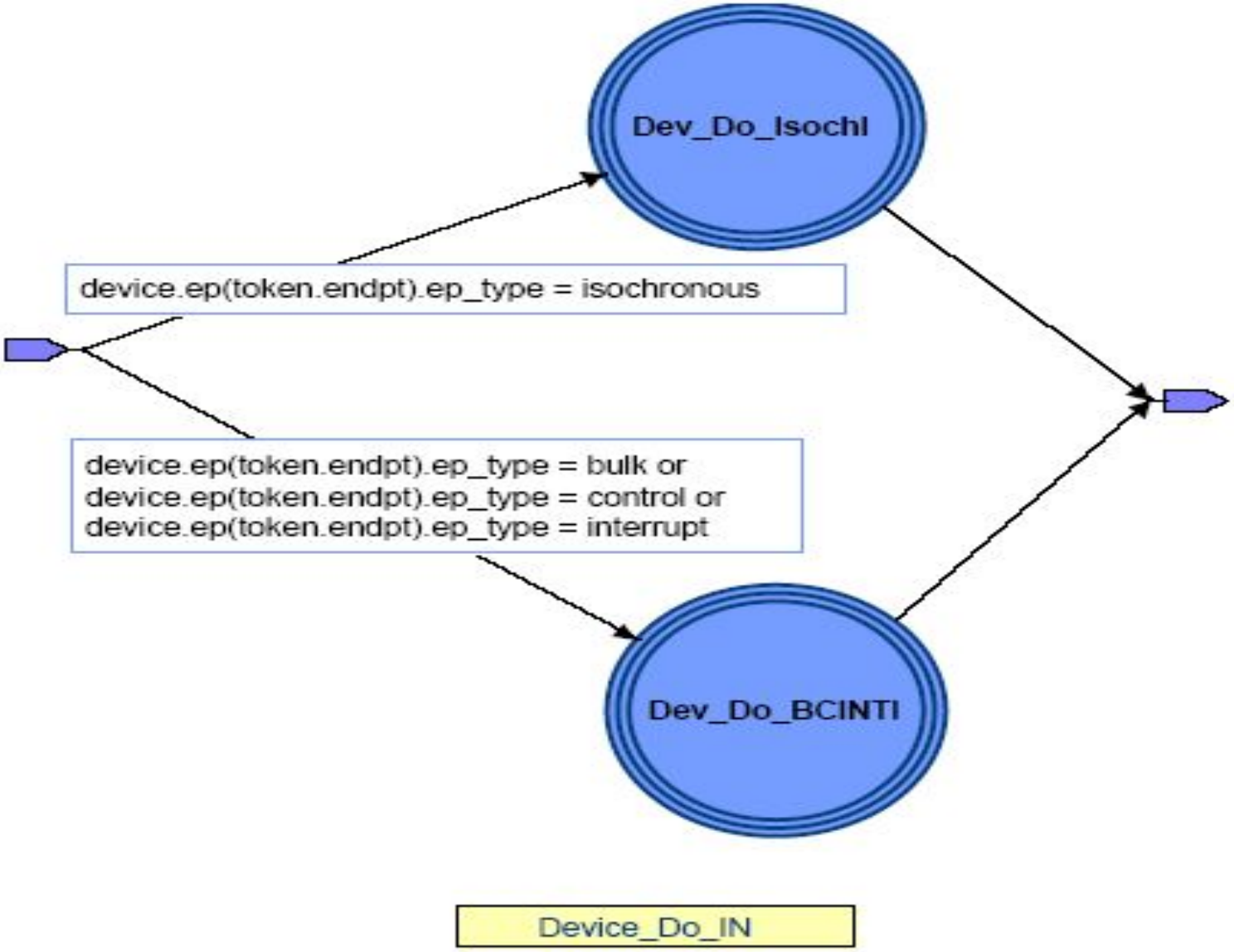


Figure 8-25. Dev_do_IN State Machine

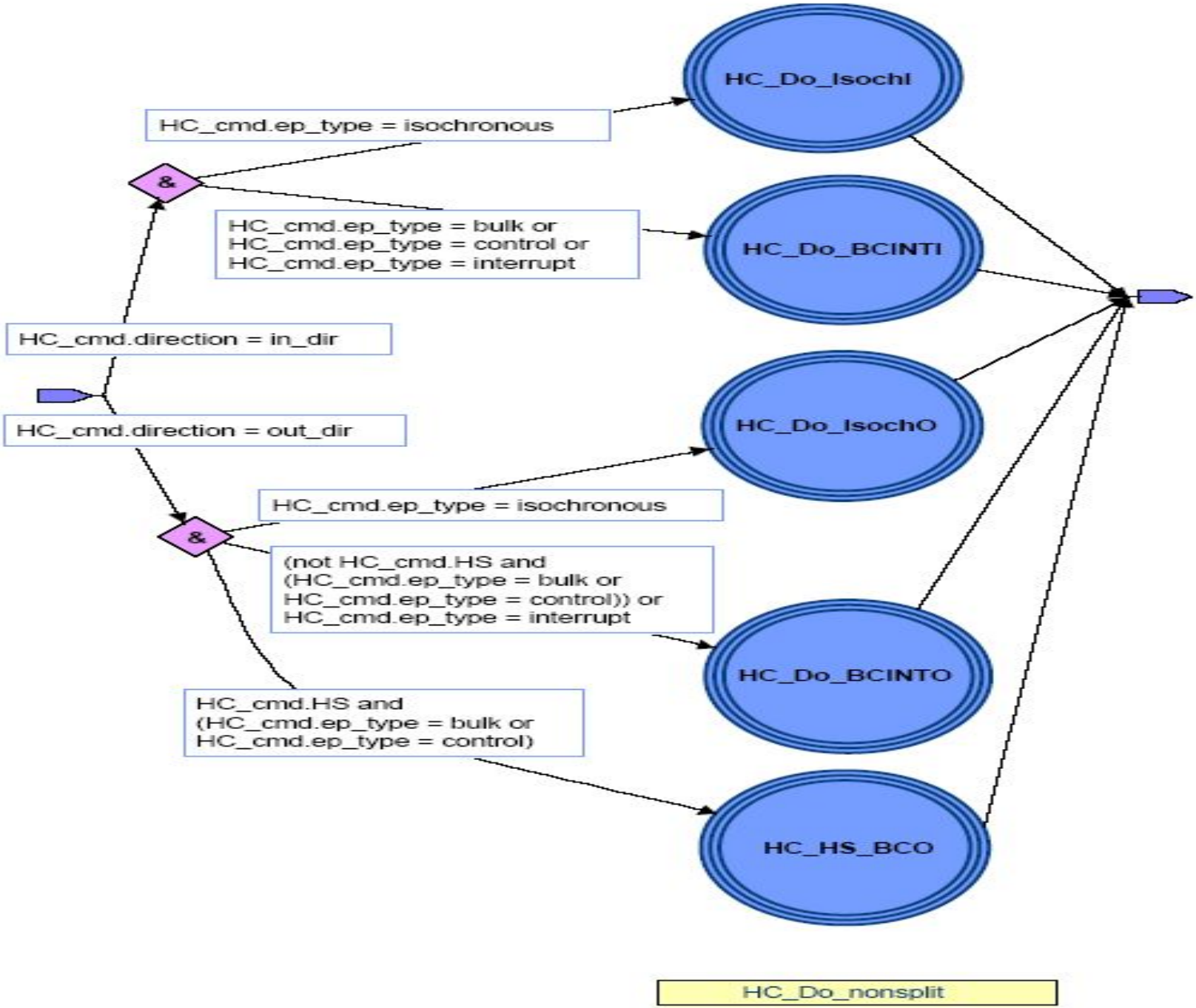


Figure 8-26. HC_Do_nonsplit State Machine

1. 通过PING流控制限制NAK

全速/低速设备可以包含需要花时间处理数据的批量/控制端点，并因此使用NAK握手响应OUT处理. 这个握手响应指出，由于端点没有数据间，因此它不接收数据. 主机控制器希望在端点有可用的空间的时候重试这个处理. 当端点NAK时，处理的很多全速/低速的总线路时间已被使用. 这就是说，当高频地使用NAK响应OUT处理时，全速/低速总线的利用率很低.

高速设备必须支持改善的NAK机制以满足批量OUT，控制端点以及处理的需要.

控制端点必须在OUT处理的数据和状态阶段支持该协议. 控制建立阶段不能支持PING协议.

该机制允许设备告知主机控制器它是否有足够的端点空间供下一个OUT处理使用. 如果设备端点没有空间，主机控制器可以选择延迟该端点的处理，以其他处理代替. 这可以改善总线的利用率. 该机制使主机控制器只有在知道端点有数据空间的情况下，才使用总线时间发送数据.

主机控制器用PING特殊令牌询问高速设备的端点. PING特殊令牌包是一如图8. 5所示的标准令牌包. 端点可以用NAK，STALL或ACK握手响应PING令牌.

NAK握手表示端点没有空间存放大小是wMaxPacketSize的数据有效负载.主机控制器会在一段时间后重发PING令牌再次询问端点.设备可以在很长的时间内用NAK响应PING令牌,NAK响应不是主机控制器撤消传输要求的原因.

如果设备在一个微型帧内使用NAK响应OUT处理,主机控制器可以选择在端点指定的下一个bInterval时间间隔发送下一个处理.但是,设备喜讯准备像接收连续处理一样接收PING令牌,即一个接着另一个.

ACK握手表示端点有空间存放大小是wMaxPacketSize的数据负载。主机控制器必须产生一个带DATA阶段的OUT处理作为端点的下一个处理。主机控制器在这个端点的OUT/DATA处理可以对其他或端点产生其他处理。

端点用ACK握手响应OUT/DATA处理，即端点成功接收数据，并且有空间存放另一个大小是wMaxPacketSize的数据有效负载。只要主机控制器要产生处理，它就要继续进行OUT/DATA处理（不要求是总线的下一个处理）。

端点用NYET握手响应OUT/DATA处理，即端点接收数据，但没有空间存放另一个大小是wMaxPacketSize的有效负载。主机控制器必须重新使用PING令牌直到端点表示有空间。

在PING协议中，端点也可以用NAK握手响应OUT/DATA处理。这意味着端点不接收数据，而且此时没有空间存放大小是wMaxPacketSize的数据有效负载.主机控制器必须重新使用PING令牌直到端点表示它有空间.

用NAK响应OUT处理被认为是一种不正常的情况.高速批量/控制端点必须在它的端点描述符指定它的最大NAK速率.端点允许每个bInterval周期最多一个响应OUT处理的NAK.NAK表示端点使用了不适当的握手响应前一个OUT或PING,或者表示端点切换到了一个(暂时)不能接收数据的状态.端点可以用为0的bInterval来表示它不回发出NAK.即端点永远不会发送NAK,它也必须一直能从主机接收PING.

如果在数据阶段产生超时,主机必须重新使用PING令牌.注意:切换回PING状态不会影响处理操作数据阶段的数据触发状态.

图8.27所示为PING与OUT/DATA令牌之间主机控制器状态机的相互作用和过渡过程,以及PING机制 允许的ACK,NAK,STALL或NYET握手.

图8.28所示为高速处理的状态机.

图8.29所示为端点有缓冲空间时设备端点的PING状态机.

全速/低速设备/端点不支持PING协议.主机控制器不支持全速/低速设备的PING协议.

注意: PING协议也不是分离处理协议定义的一部分,一些分离处理有等效的流控制,而无须使用PING.其它分离处理不会从定义的PING得益.在任何情况下,返回NAK握手的分离处理数据有效负载很小,对高速总线的影响很小.集线器必须在它们的控制端点支持PING协议,但PING不在全速/低速设备通信的集线器分离处理中定义.

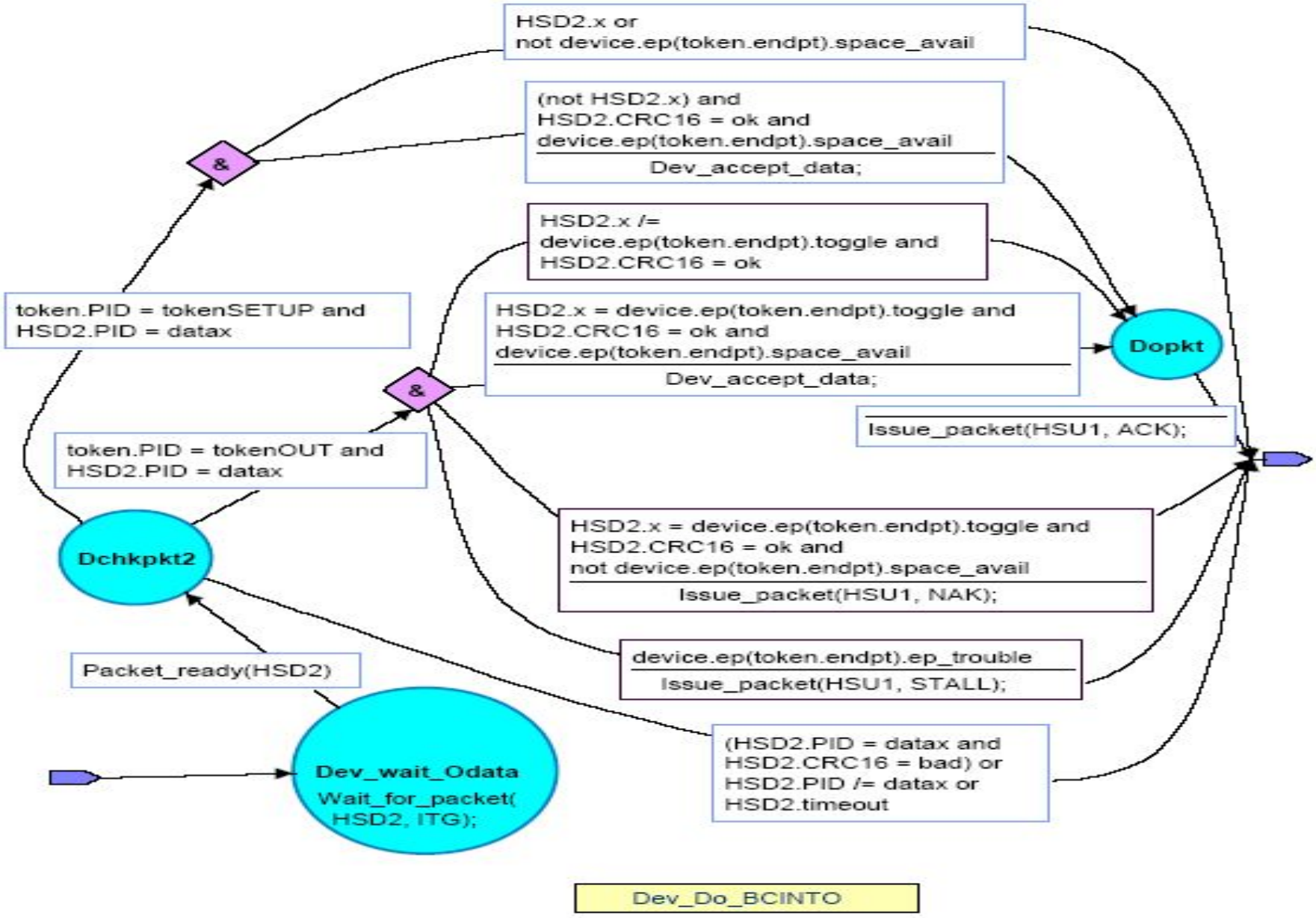


Figure 8-27. Host High-speed Bulk OUT/Control Ping State Machine

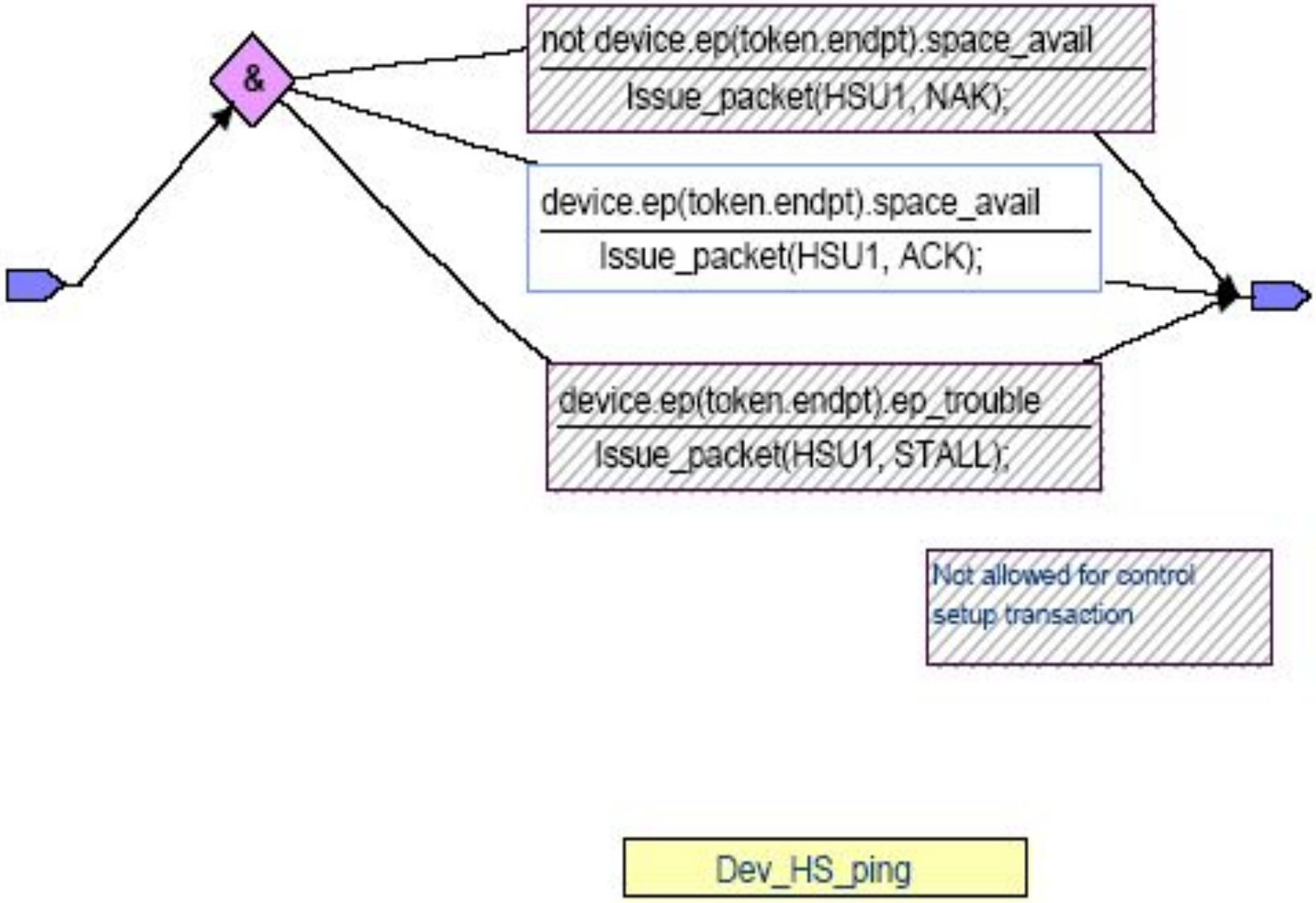


Figure 8-28. Dev_HS_ping State Machine

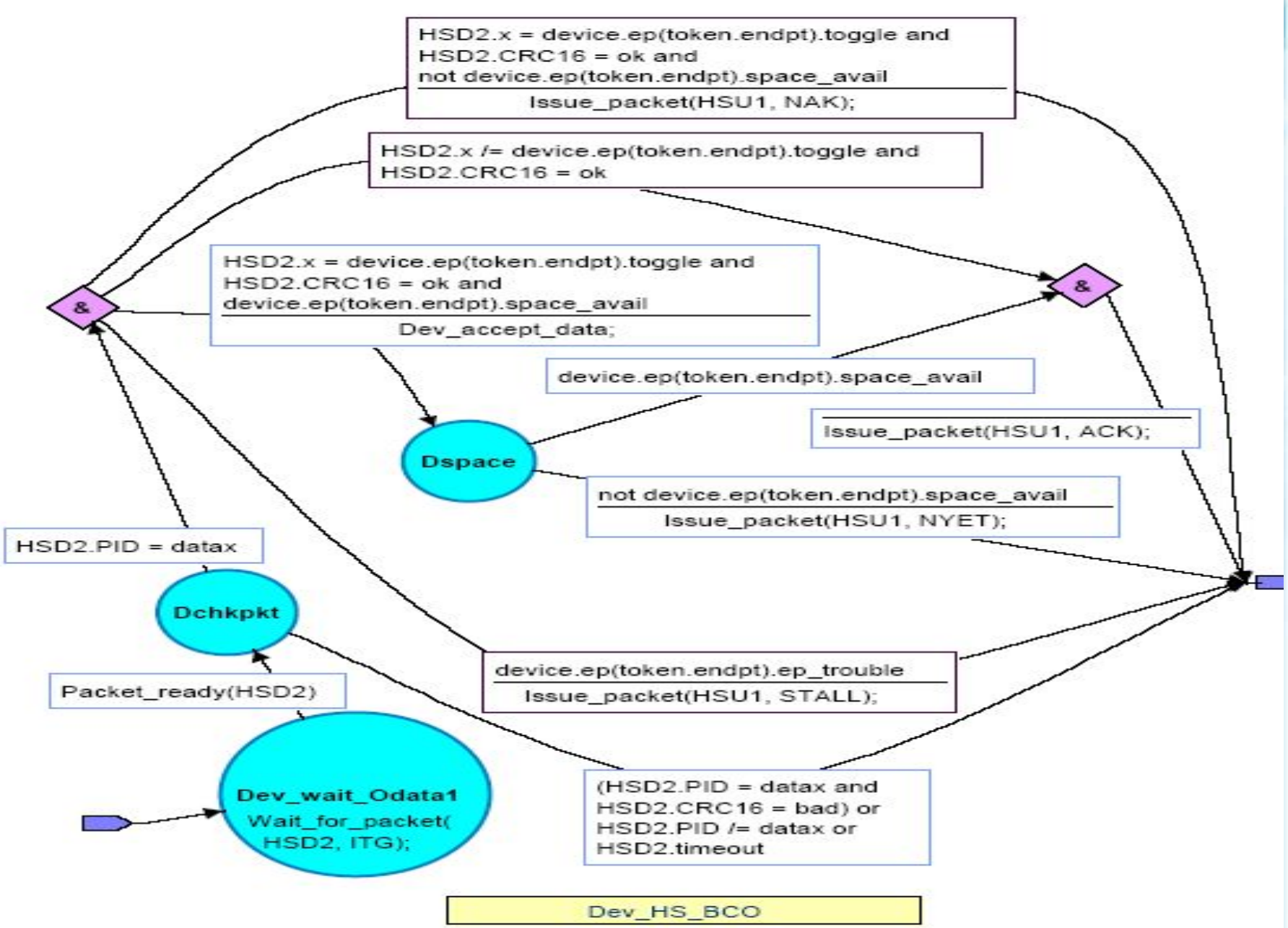


Figure 8-29. Device High-speed Bulk OUT /Control State Machine

2.批量处理

批处理事务类型的特点是以错误检测和重试的方式保证主机和功能部件之间的数据的无错传输的能力.批处理共有3个阶段的处理:令牌,数据和握手包.如图8.30所示.在某些流控制和停止情况下,数据阶段会被握手阶段代替,结果处理只有另个阶段,没有数据被传送.PING和NYET包只能在高速操作的设备中使用.

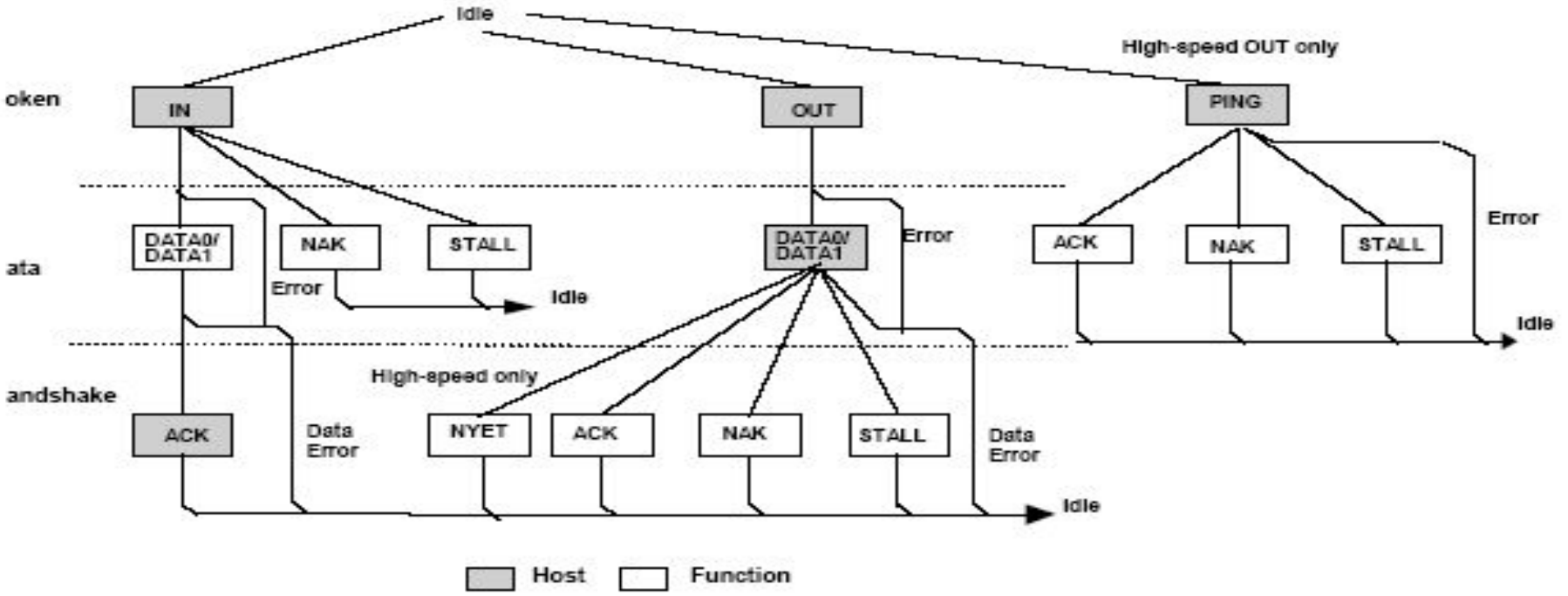


Figure 8-30. Bulk Transaction Format

当主机准备接收批量数据时,它发送一个IN令牌.功能设备的端点通过返回一个数据包或者NAK或STALL握手(如果不能返回数据)响应.NAK表示功能设备暂时不能返回数据,STALL表示端点永久停止,要求USB系统软件进行干涉.如果主机接收到一个有效的数据包,它会用ACK握手响应.如果主机在接收数据时检测到错误,它就不能向功能设备返回握手.

当主机准备发送批量数据时,它首先发送一个跟有数据包的OUT令拍包(或者PING特殊令牌包).如果数据被功能设备无错地接收,它会返回3个(或者4个,对于高速操作的设备包括NYET)握手中的一个:

- ACK表示数据包被无错接收,并通知主机它可以发送序列中的下一个包
- NAK表示数据被无错接收,但由于功能设备处于暂时的阻碍接收数据情况(例如缓冲器满),主机应重发数据.
- 如果端点停止,它将返回STALL,表示由于功能设备上存在错误的情况,主机不应重试传输.

如果接收的数据包有CRC或位填充错误,端点不会返回握手.

图8.31和图8.32所示分别是批量,控制和中断OUT全速/低速处理的主机和设备状态机;图8.33和图8.34所示分别是批量,控制和中断IN处理的主机和设备状态机.

www.cpit.com.cn

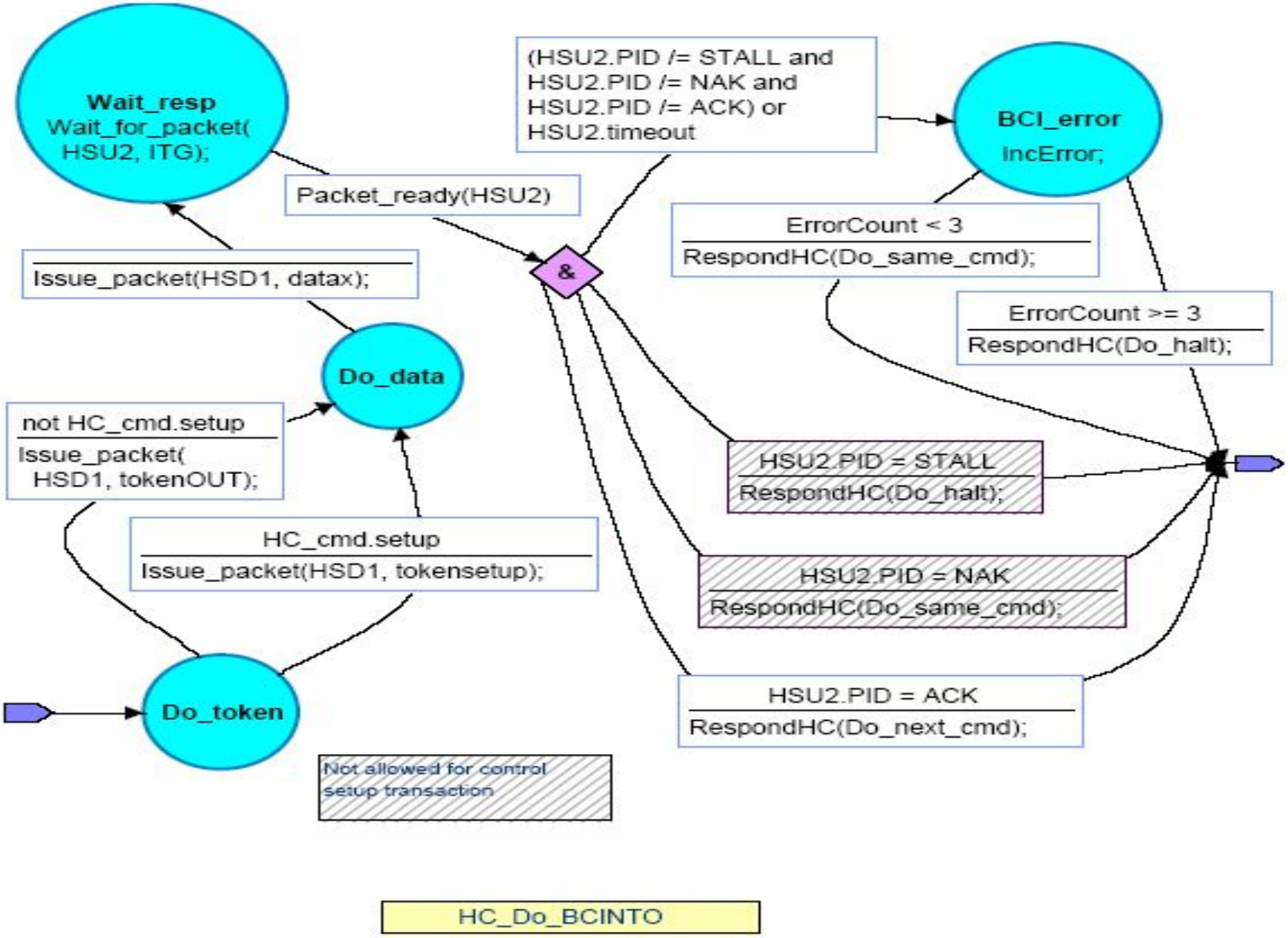


Figure 8-31. Bulk/Control/Interrupt OUT Transaction Host State Machine

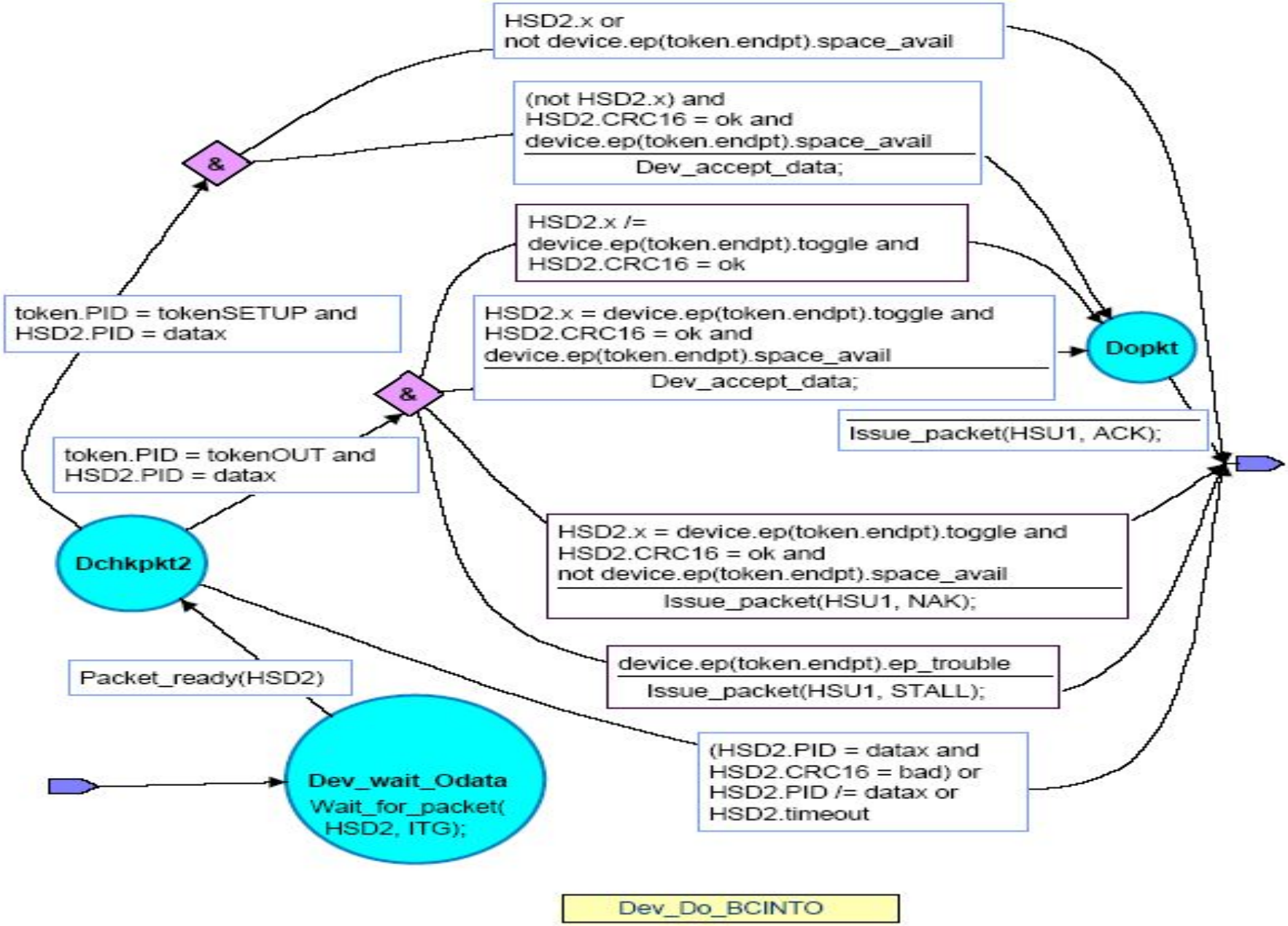


Figure 8-32. Bulk/Control/Interrupt OUT Transaction Device State Machine

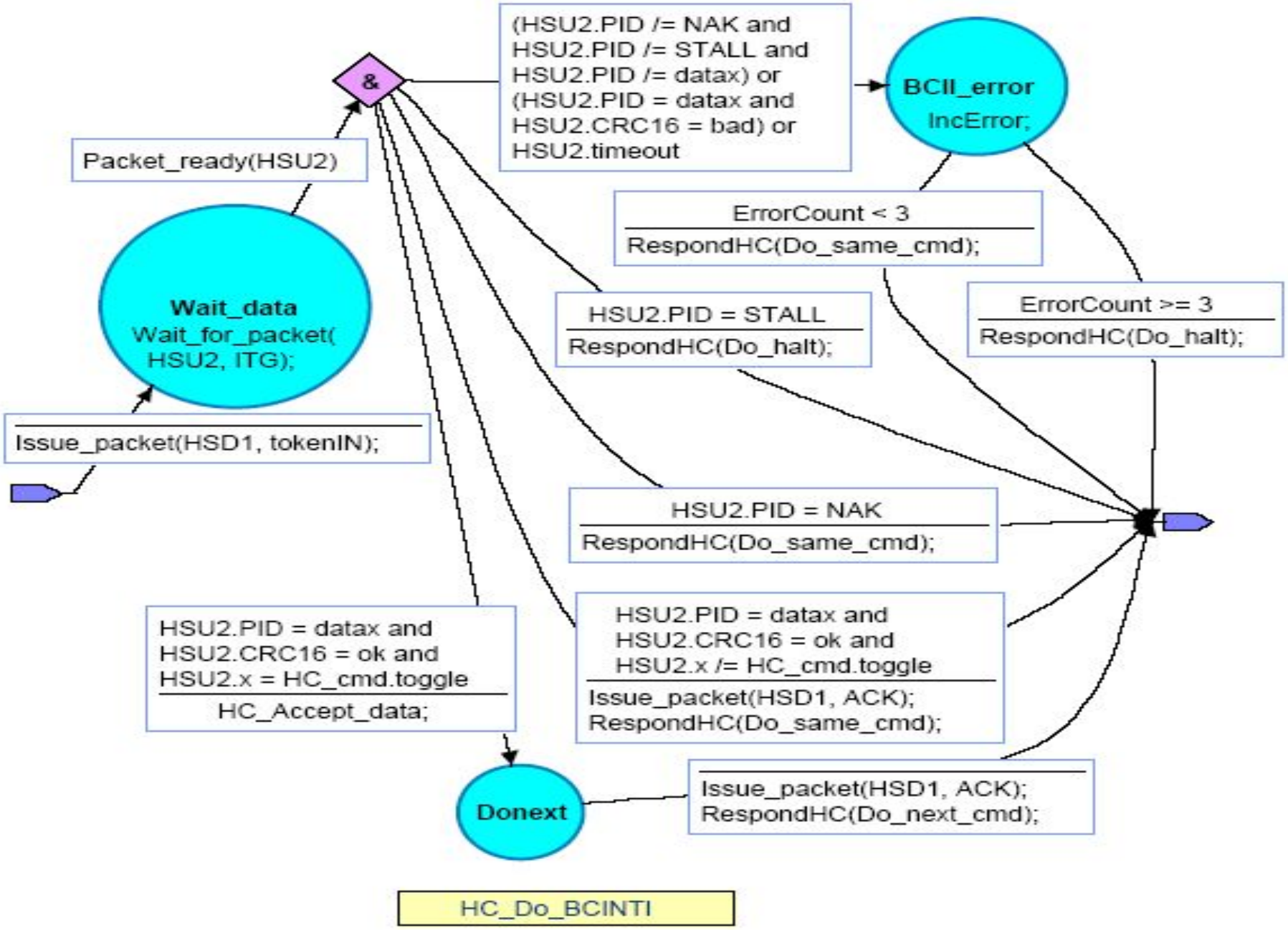


Figure 8-33. Bulk/Control/Interrupt IN Transaction Host State Machine

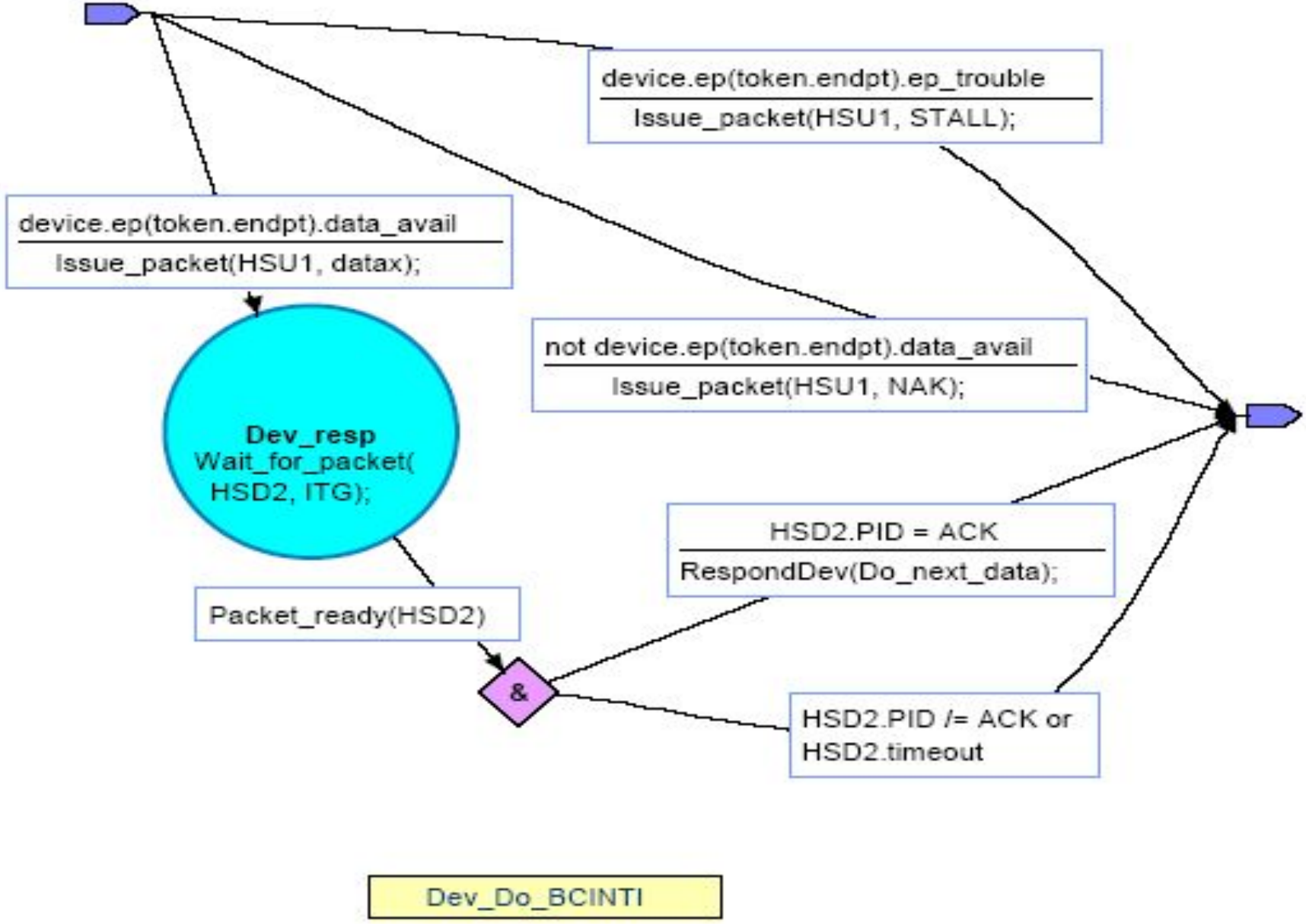


Figure 8-34. Bulk/Control/Interrupt IN Transaction Device State Machine



Figure 8-35. Bulk Reads and Writes

图8.35所示为批量读和写的位序列和数据PID用法.数据包同步通过使用数据序列触发位和DATA0/DATA1 PID实现,当端点经历了任何配置事件后,批量端点的触发序列被初始化为DATA0.端点的数据触发不会被初始化为短包传输或者撤销IRP的直接结果.

主机通常将总线传输的首个处理初始为配置事件的DATA0 PID.第二个处理使用DATA1 PID,接下来的数据传输交替地使用剩下的批量传输,接收到ACK后,数据包发送触发器;接收了一个有效的数据包,接收器触发.

3.控制传输

控制传送最少有2个事务阶段：建立和状态。控制传送可以有选择性地包括建立和状态阶段之间的数据阶段。在建立阶段里，SETUP事务用于向功能部件的控制端口传输信息。SETUP事务在格式上类似于输出，但是使用的是SETUP而不是OUT的PID。图8-36说明了SETUP事务的格式。SETUP总是在SETUP事务的数据时相上使用DATA0 PID。收到SETUP的功能部件必须接受SETUP数据并用ACK应答，如果数据被损坏，则丢弃数据且不返回握手。

控制传送的数据阶段(如果存在)，由一个以上的IN或OUT构成，遵守和批处理传送相同的协议规则。所有的数据阶段里的事务都必须有相同的方向（即全部IN或者全部OUT）。在数据时相中要发送的数据的数量和其方向在建立阶段里被指定。如果数据的数量超过了先前确定的数据包大小，数据在支持最大的包大小的多个事务中被发送（IN或者OUT）。任何剩下的数据都作为剩余在最后的事务中被发送。

控制传输的状态阶段是序列的最后一个处理. 状态阶段处理尊从与批量处理相同得不偿失协议序列. 以高速操作的设备状态阶段通常包括PING协议.状态阶段用从上一个阶段的数据流方向改变来描绘.通常总使用DATA1 PID.例如,如果数据阶段包含OUT,那么状态阶段就是一个单独的IN处理;如果控制序列没有数据阶段,那么它就由一个建立阶段和紧接的一个包的IN处理的状态阶段组成.

图8.37所示为处理顺序,数据位序列的值以及读写序列的数据PID类型,位序列在圆括号使用.

当处于控制传输的数据或状态阶段的控制端点发送STALL握手时,对该端点的所有后续访问也必须返回STALL握手,直到接收到SETUP PID.端点在接收到后续的SETUP PID后不要求返回STALL握手.对于默认的端点来说,如果对SETUP处理返回ACK握手,主机要求端点已经导致产生STALL的情况自动恢复,而且端点必须正常工作.

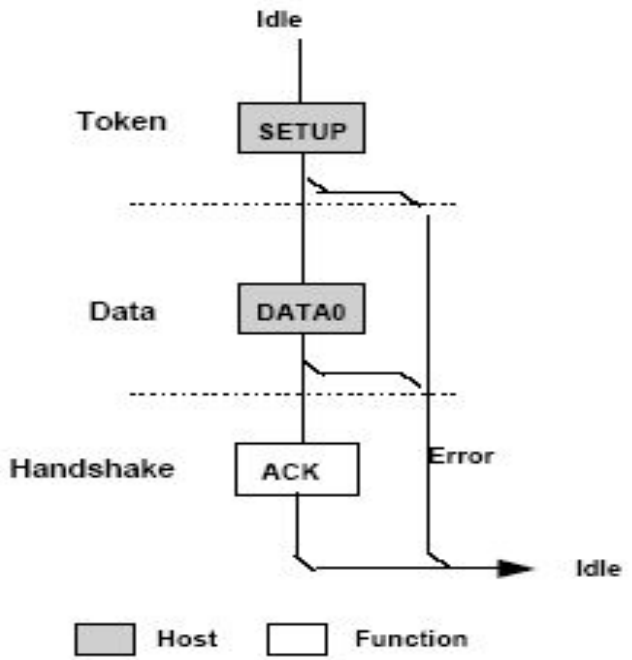


Figure 8-36. Control SETUP Transaction

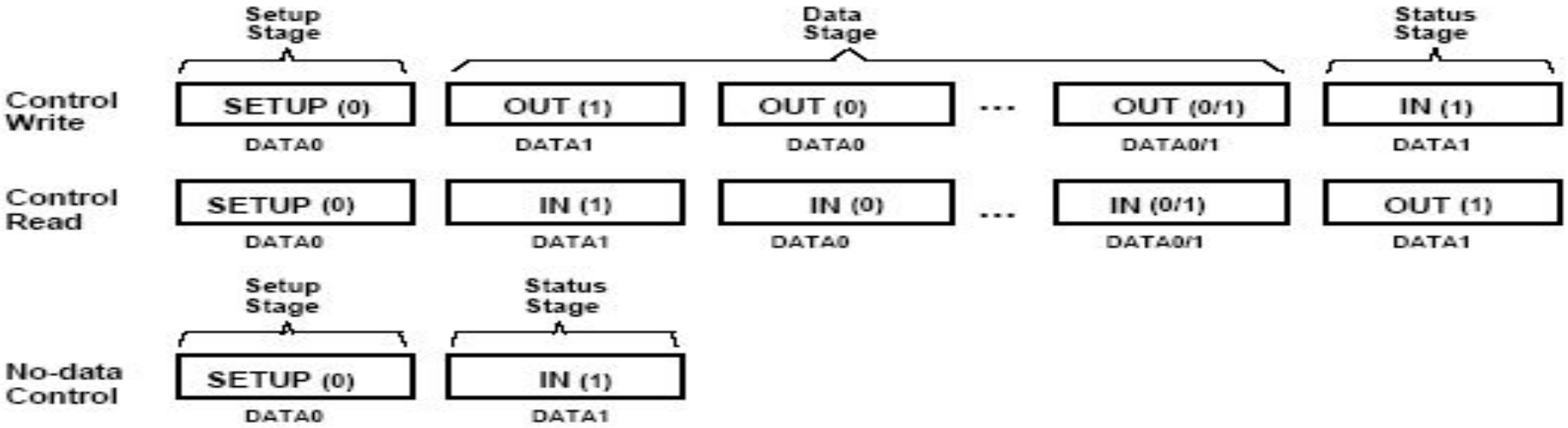


Figure 8-37. Control Read and Write Sequences

- **报告状态结果**

状态阶段向主机做汇报传送中的先前的建立和数据阶段的结果。可能返回三种结果：

命令序列成功地完成了；

命令序列没能完成；

功能部件还在忙于完成指令。

汇报状态总是从功能部件到主机的方向。表8-7概括了每一种所需的应答类型。控制写传送在状态阶段的事务的数据时相返回状态信息。而对于控制读传送，主机在状态阶段事务的数据时相中发出零长度的数据包之后，功能部件在握手时相返回状态信息。

Table 8-7. Status Stage Responses

Status Response	Control Write Transfer (sent during data phase)	Control Read Transfer (sent during handshake phase)
Function completes	Zero-length data packet	ACK handshake
Function has an error	STALL handshake	STALL handshake
Function is busy	NAK handshake	NAK handshake

www.cpit.com.cn

对于控制读来说,主机必须想控制管道发送OUT令牌或PING特殊令牌(对于在高速操作下的设备),以初始化状态阶段.主机可以在这个阶段发送一个0长度的数据包,但功能设备能接收任意长度的数据包作为有效的状态查询.管道对这个数据包的握手响应指出了设备的当前状态.NAK表示功能设备仍在处理命令,主机应应延续状态阶段.ACK表示功能设备已经完成命令,并且准备好接收下一个新的命令.STALL表示功能设备有错误,不能完成命令.

对于控制写，主机发送IN令牌到控制管道用以启动状态阶段。功能部件使用握手或零长度的数据包应答以说明其现状。NAK说明功能部件还在事务指令，并且主机应该继续状态阶段；返回零长度的包表明指令正常完成；而STALL表示功能部件不能完成指令。功能部件期待主机在状态阶段中对数据包以ACK应答。如果功能部件收不到ACK，它仍处于指令的状态阶段中，并且只要主机再发IN令牌，它将继续返回零长度的数据包。

如果在命令管道的数据阶段发送了过多数据或者请求返回比在建立阶段指定的还要多的数据,那么功能设备应返回STALL握手.如果控制管道在数据阶段返回STALL,则此控制传输没有状态阶段.

- **长度可变的数据阶段**

控制管道可以放进可变长度的数据时相，在可变长度的数据时相中主机请求比指定的数据结构中所能包含的更多的数据。当全部的数据结构被返回到主机的时候，功能部件应该向管道返回一个短于包最大长度（MaxPacketSize）的包以表明数据阶段的结束。如果数据结构是数据是管道包最大长度（wMaxPacketSize）整数倍，功能部件将返回零长度的包以表明数据阶段的结束。

- **最后数据事务的出错处理（Error Handling）**

  如果输入事务的ACK握手损坏，功能部件和主机在事务是否成功上这个问题上将暂时不一致。如果此事务的后面跟着另一个输入事务，切换重试机制（Toggle Retry Mechanism）将检测失配并从错误中恢复原状。如果此ACK是数据阶段最后的输入事务上的，则不能使用切换重试机制而必须使用另一种机制。

  成功收到最后输入数据的主机将会发送ACK，然后主机将发出输出标记以启动传送的状态阶段。如果功能部件收不到结束数据阶段的ACK，功能部件将好象主机成功地收到数据那样去解释状态阶段的开始。而控制写则没有这种模棱两可的情况。如果输出事务上的ACK握手损坏，主机将不进入状态阶段而是重发最后的数据 .

- **控制管道返回的STALL握手**

www.cpit.com.cn

控制管道具有在控制传送中根据功能部件的问题返回STALL握手的独特能力。如果设备不能完成指令，它将在控制传送的数据和（或）状态阶段中返回STALL。与功能STALL的情况不同，协议STALL并不表示设备错误。协议STALL条件一直持续到接受到下一个SETUP事务，且功能部件将对于任何IN或OUT出事务返回STALL直到管道上的SETUP事务被收到为止。一般来说，协议STALL表示请求或者其参数不能被设备所理解，这样提供用来扩展USB请求的机制。

另外， 控制管道也可以支持功能STALL，但是不推荐使用。因为控制管道上的功能STALL表示它失去和主机通信的能力，所以这是一种退化的情况。如果控制管道的确要支撑功能STALL，它必须拥有能被主机设置或者清除的停止特征。第9章详细叙述事务在控制管道上停止特征的特殊情况的方法。设计良好的设备将把其全部功能和停止特征,与非控制端口联系起来。而控制管道应该预留用于为USB请求提供服务

4.中断处理

中断事务可由IN或OUT构成。一收到IN令牌，功能部件便可返回数据，NAK或STALL。

www.cpit.com.cn

如果端口没有新的中断信息（即没有等待事务的中断）可供返回，功能部件在数据时相里返回NAK握手。如果中断端口的停止特征被设置了，功能部件将返回STALL握手。如果中断是等待事务的，功能部件象数据包那样返回中断信息。作为对数据包接收的反应，主机如果数据无错地被接受则发出ACK握手，或者如果数据包损坏则不返回握手。图8-38说明了中断事务格式。

中断端点在一个微型帧内使用了多个处理. 每个高带宽端点的处理都尊从图8. 38的处理格式.

当端点要在实际的中断数据使用中断传输机制时, 它必须遵守数据触发协议. 着就允许功能设备知道数据已经被主机接收, 而且事件条件被清除. 这“保证”了事件传输允许功能设备只发送中断信息直到它被主机接收, 而不是在每次功能设备被查询的时候发送中断数据直到USB系统软件清除了中断的情况为止. 当在触发模式中使用时, 中断端点必须通过端点的所有配置事件初始化为DATA0 PID, 而且它的运行方式与图8. 35所示的批量处理相同.

www.cpit.com.cn

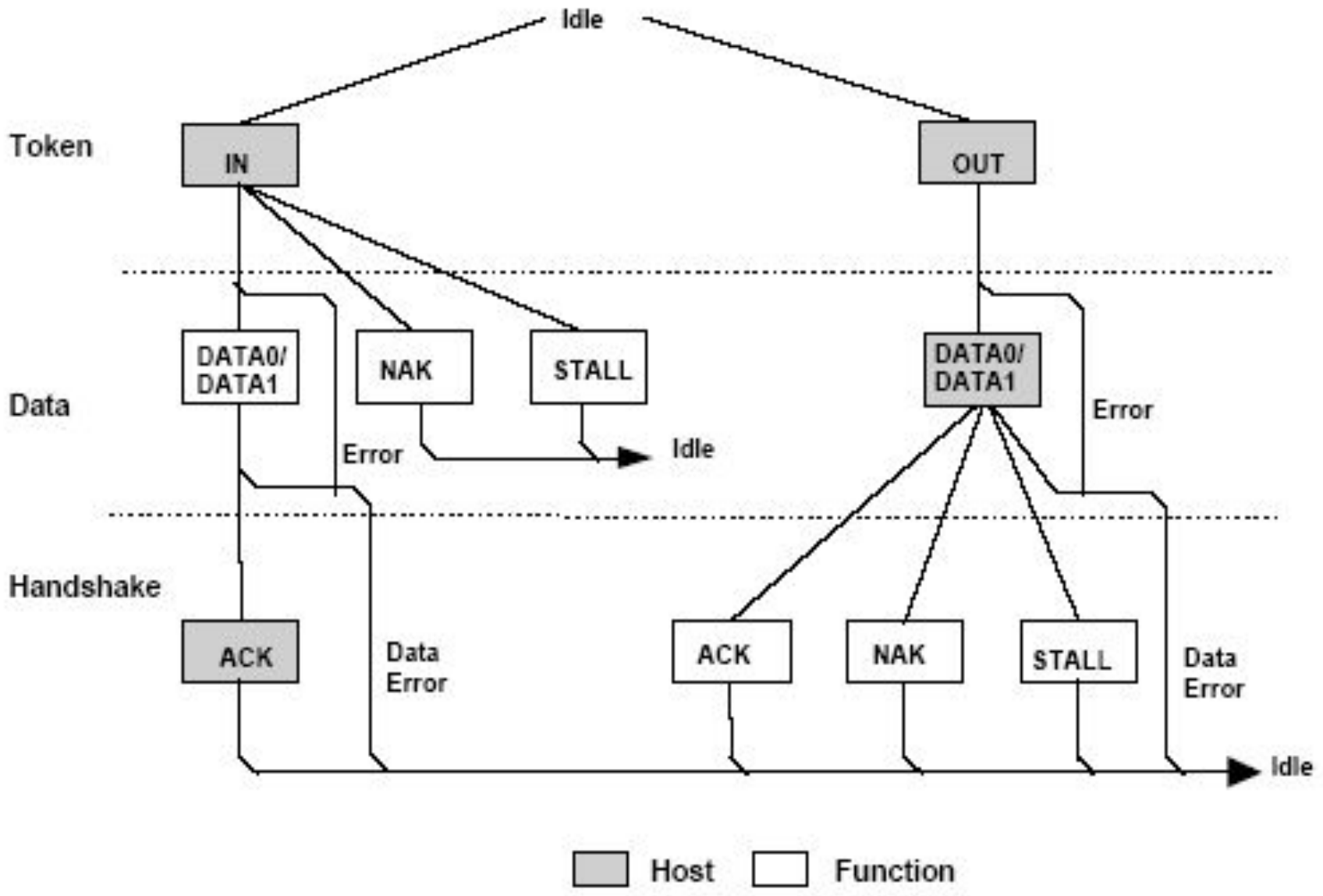


Figure 8-38. Interrupt Transaction Format

www.cpit.com.cn

5. 同步处理

同步处理有令牌和数据阶段,但没有握手阶段,如图8.39所示.主机发送IN或OUT令牌,后面近跟数据阶段.数据阶段包括端点(IN)或主机(OUT)要发送的数据.同步处理不支持握手阶段或重试性能.

全速同步处理不支持触发序列.每个微型帧有一个处理的高速同步处理不支持触发序列.高带宽,高速同步处理支持数据PID序列.

图8.40和图8.41分别是同步OUT处理的主机和设备状态机.图8.42和图8.43分别是同步IN处理主机和设备状态.

注意:全速设备或主机控制器应能接收数据包中的DATA0或DATA1 PID.全速设备或主机控制器应只能在数据包中发送DATA0 PID.高速主机控制器必须能接收和发送数据包中的DATA0,DATA1,DATA2或MDATA PID.每个微型帧最多有1个处理的高速设备只能在数据包中发送DATA0 PID.有高带宽端点的高速设备(例如:每个微型帧包含超过一个处理)必须能接收和/或发送数据包中的DATA0,DATA1,DATA2,或MDATA PID.

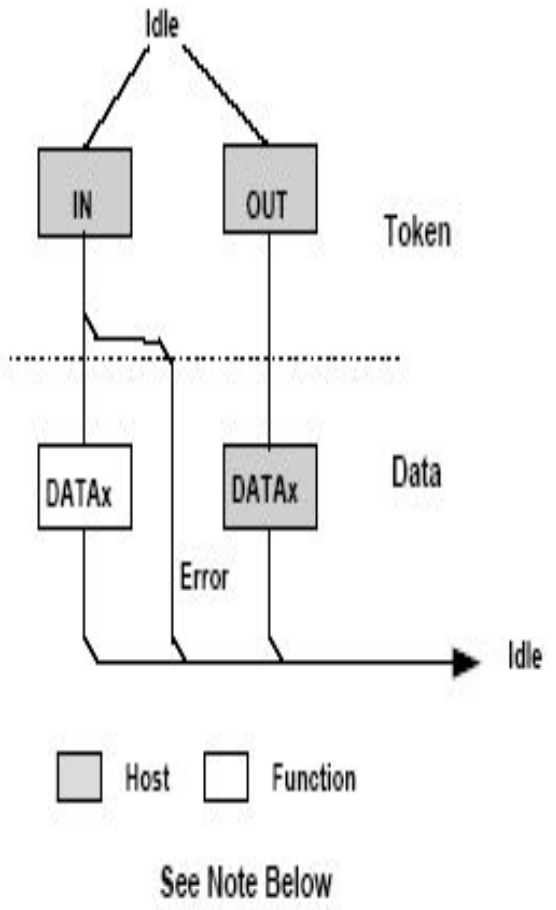


Figure 8-39. Isochronous Transaction Format

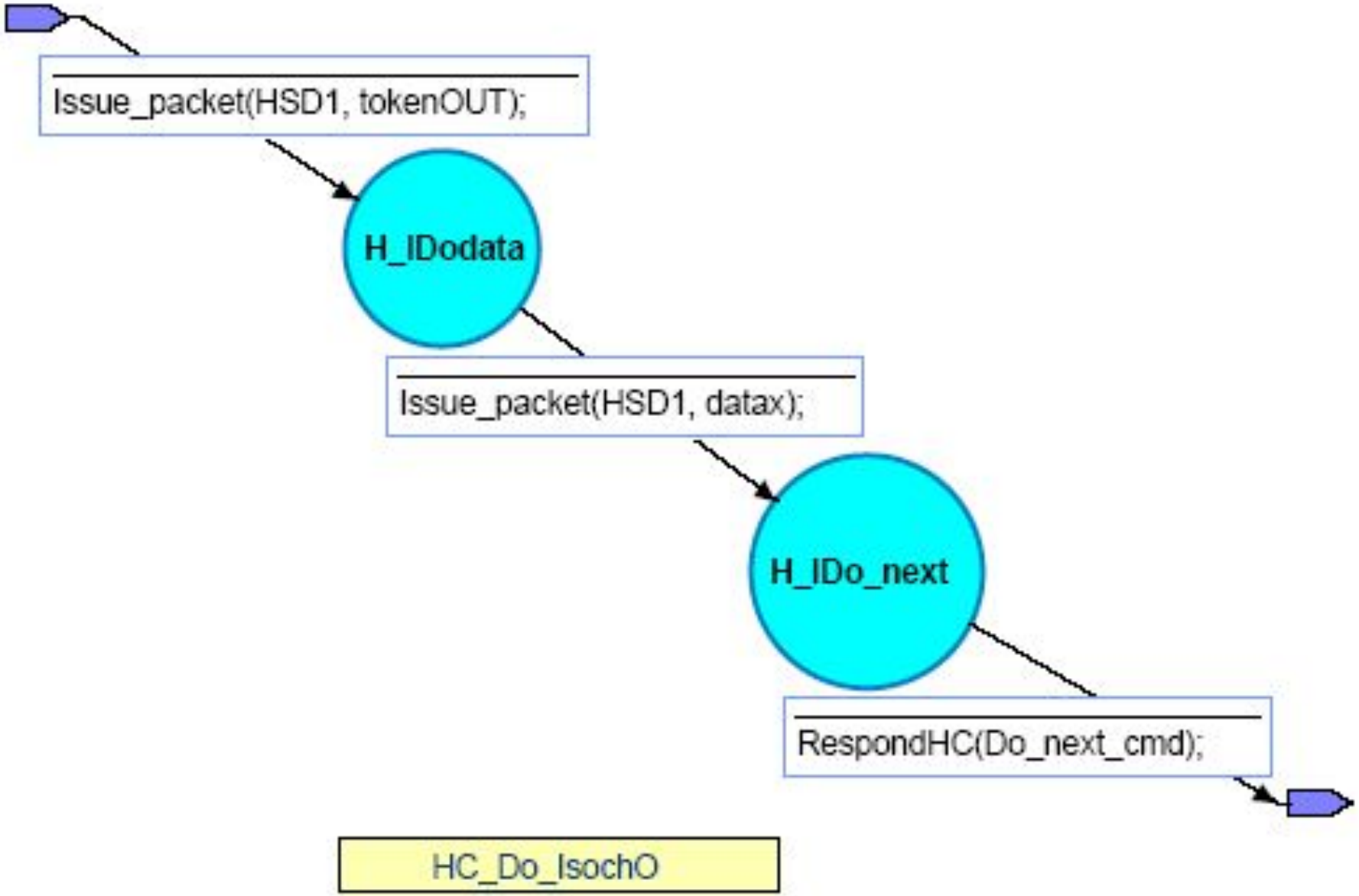


Figure 8-40. Isochronous OUT Transaction Host State Machine

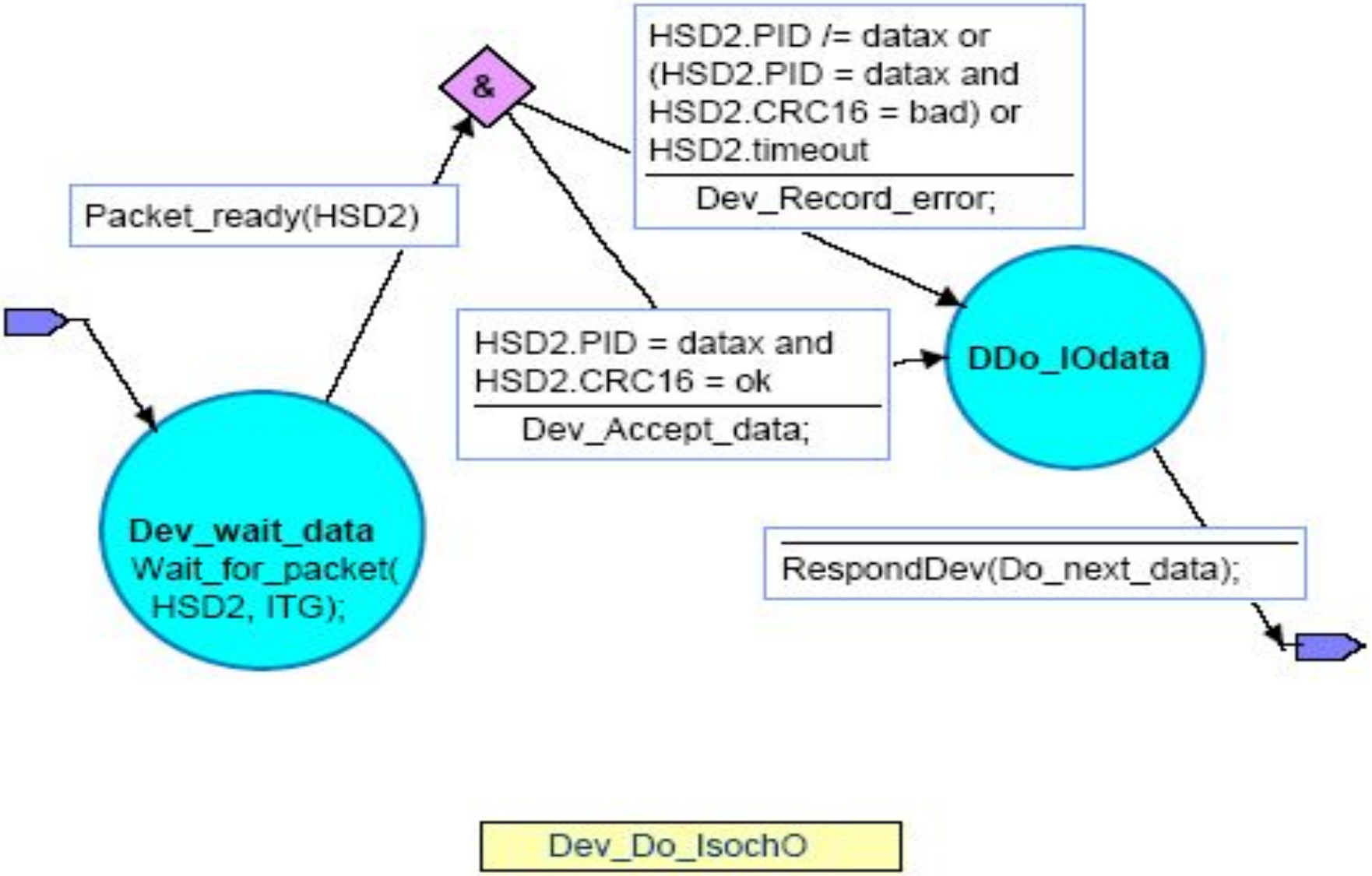


Figure 8-41. Isochronous OUT Transaction Device State Machine

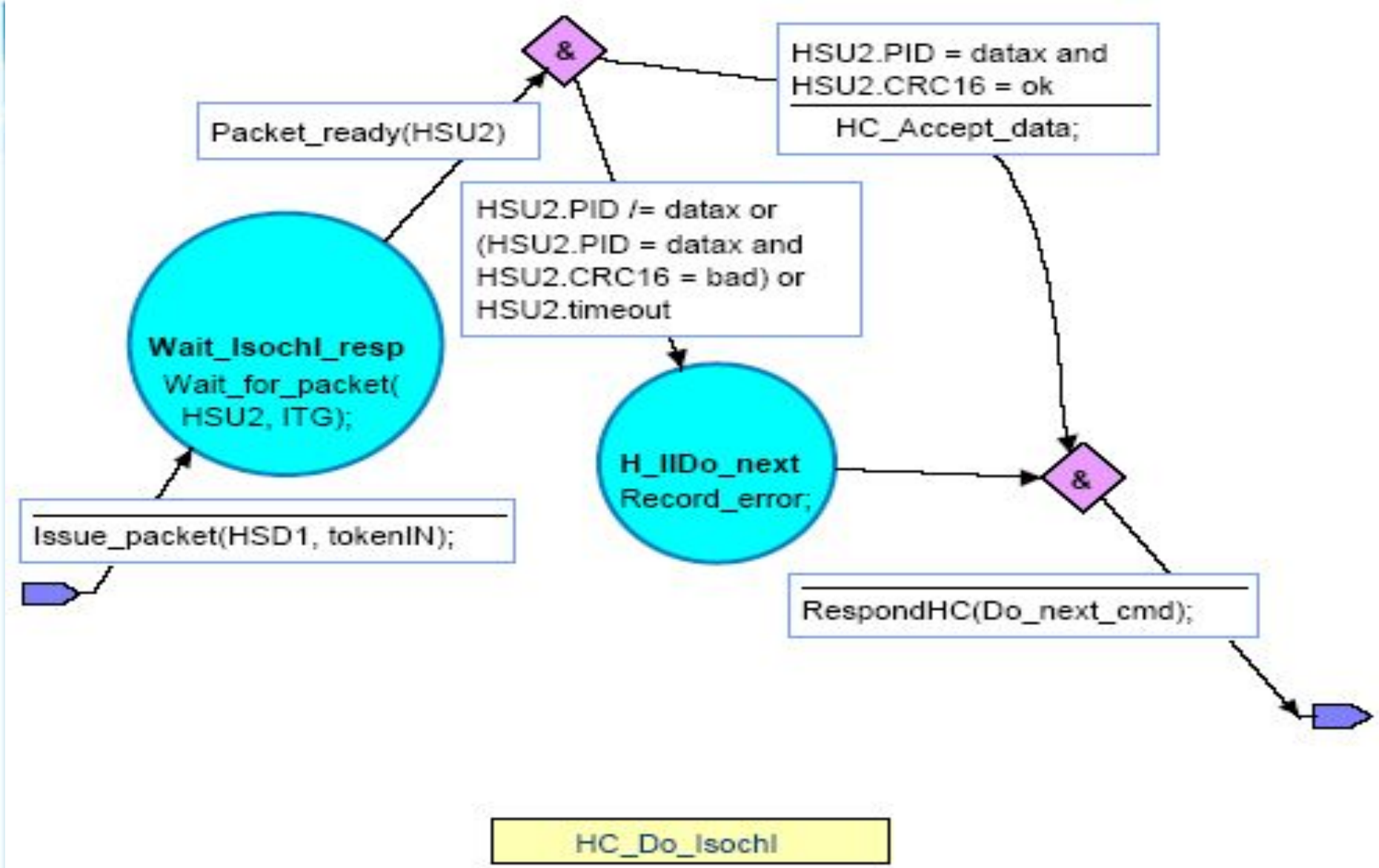


Figure 8-42. Isochronous IN Transaction Host State Machine

www.cpit.com.cn

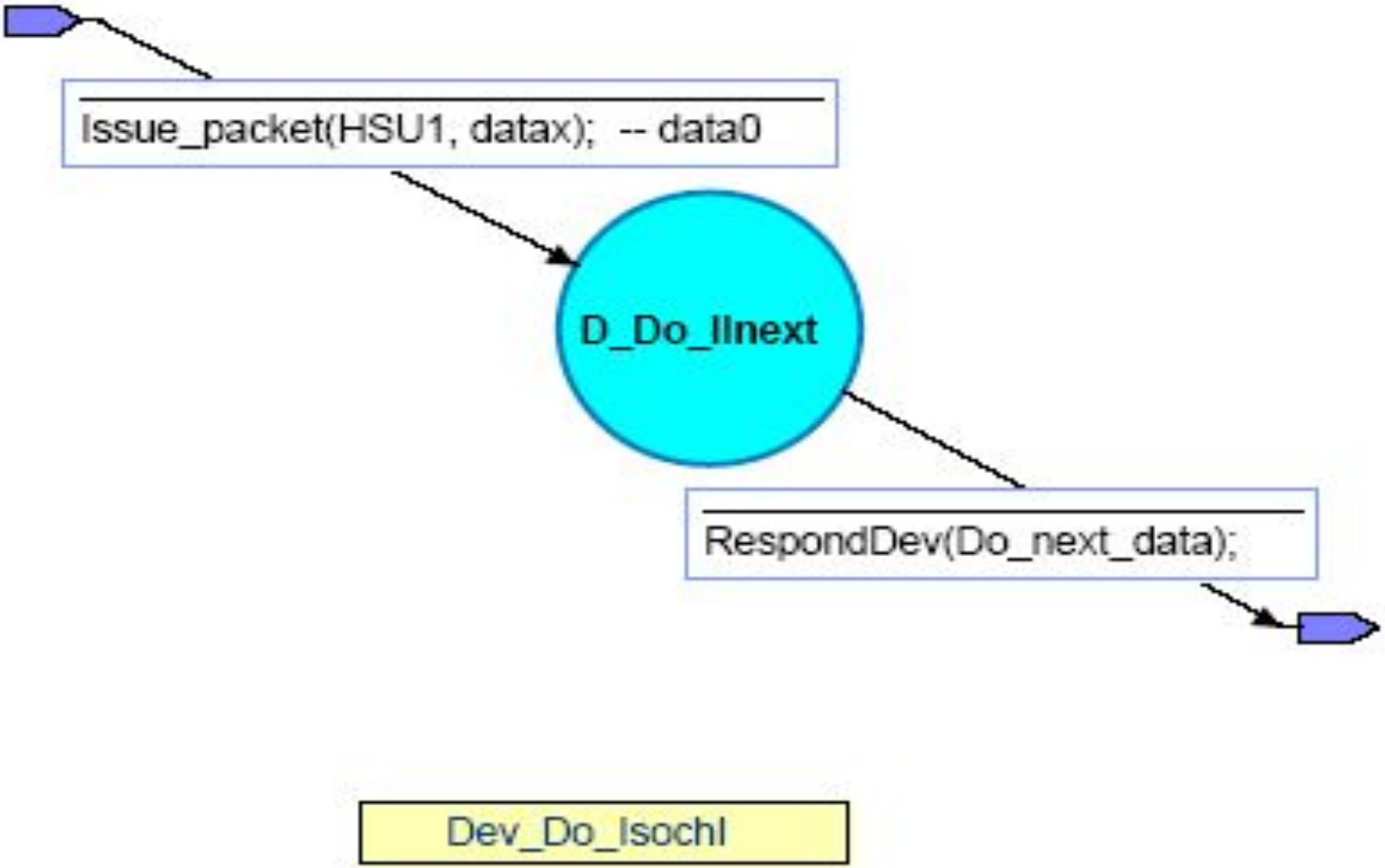


Figure 8-43. Isochronous IN Transaction Device State Machine

www.cpit.com.cn

6. 数据触发同步和重试

USB提供了一种机制保证多个处理之间的数据发送器和接收器的数据序列同步. 这种机制提供了一种方法保证处理的握手阶段能被发送器和接收器的正确解释. 通过在发送器和接收器中使用了DATA0和DATA1 PID与分离的数据触发位序列可以实现同步. 接收器的位序列在接收器能接收数据, 并接收数据PID正确的无错数据包时触发; 发送器的位序列只在数据数据发送器接收到一个有效的ACK握手触发. 数据发送器和接收器必须在处理开始的时候使位序列同步. 同步机制随着处理的类型变化. 同步传输不支持数据触发同步.

本章和第11章的状态机以非常简单的形式说明了数据触发同步. 它使用值"DATAx"来表示DATA0/DATA1 PID, 而不是明确的DATA0和DATA1表示. 在一些情况下, 指定数据PID非常重要; 此时要用另一种可变的标记"x". 其中, 值0表示DATA0, 值1表示DATA1.

高速, 高带宽的同步端点支持相似但不相同的数据同步技术-数据PID序列. 这种技术用于代替数据触发同步.

www.cpit.com.cn

(1)通过SETUP令牌初始化

控制传输用SETUP令牌初始化主机和功能设备的位序列.图8.44所示为主机向功能设备发送SETUP包后,再发送OUT处理情况.圆圈内的编号表示发送器和接收器的为序列.功能设备必须接收数据并返回ACK.当功能设备接收处理后,它必须设置它的位序列,那么主机和功能设备的位序列在SETUP处理结束的时候等于1.

(2)连续的数据处理

图8.45是产生两个连续处理的情况.对于数据发送器来说,它在接收到ACK后触发它的位序列.接收器只在它接收了一个有效的数据包,并且包的数据PID与其位序列的当前值一致时,触发其位序列;发送器只在接收到数据包的ACK后触发其位序列.

在每次处理中,接收器都将接收到的位序列与发送器的位序列比较(将数据包PID编码成DATA0或者DATA1).如果不能接收数据,接收器必须发送NAK,而且发送器和接收器的位序列不变;如果能接收数据并且接收器的位序列与PID位序列相等,那么数据被接收且位序列触发.没有数据包的两处理阶段发送器和接收器的位序列不变.

www.cpit.com.cn

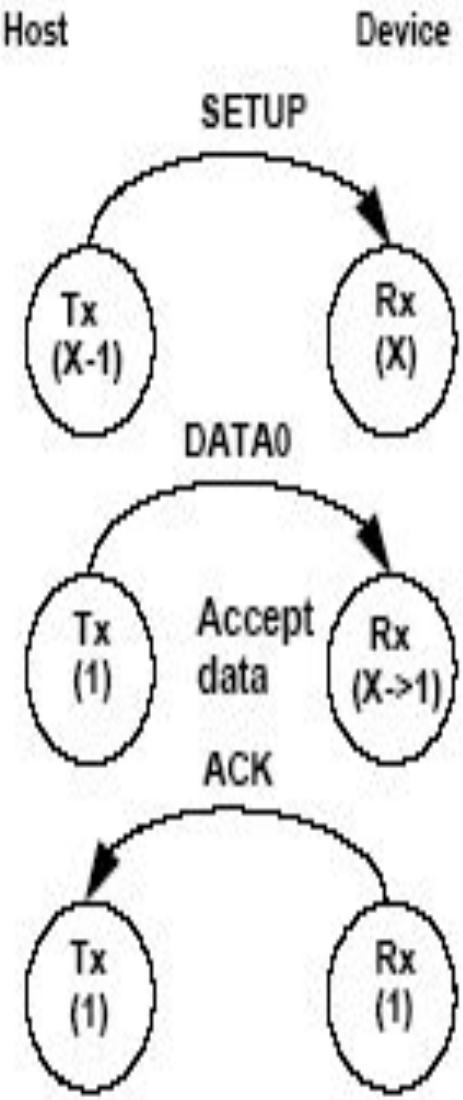


Figure 8-44. SETUP Initialization

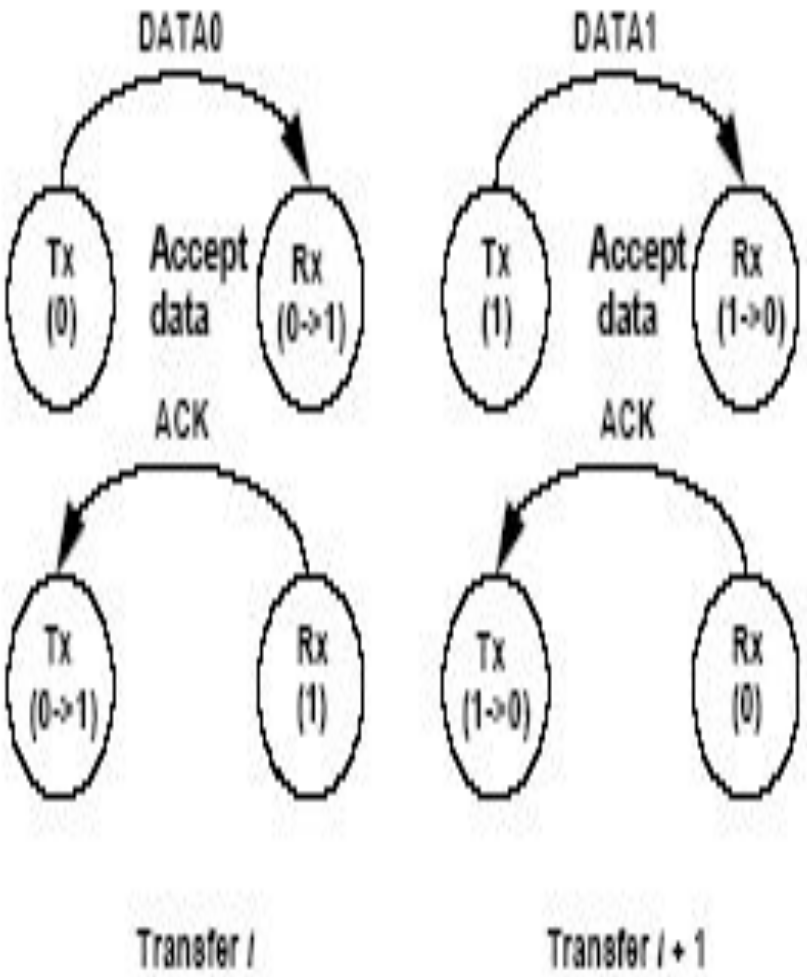


Figure 8-45. Consecutive Transactions

www.cpit.com.cn

(3)数据被破坏或不能接收

如果数据不能被接收或者接收到数据包被破坏,接收器会根据环境发送NAK或STALL握手或者超时,接收器不会触发它的位序列.图8.26是处理被NAK然后重试的情况.任何非ACK握手或者超时都会产生相似的重试操作.没有收到ACK握手的发送器不会触发它位序列.结果,错误的数据包处理使发送器和接收器位序列同步但不被触发.然后,处理将被重试,如果重试成功,发送器和接收器的位序列都将触发.

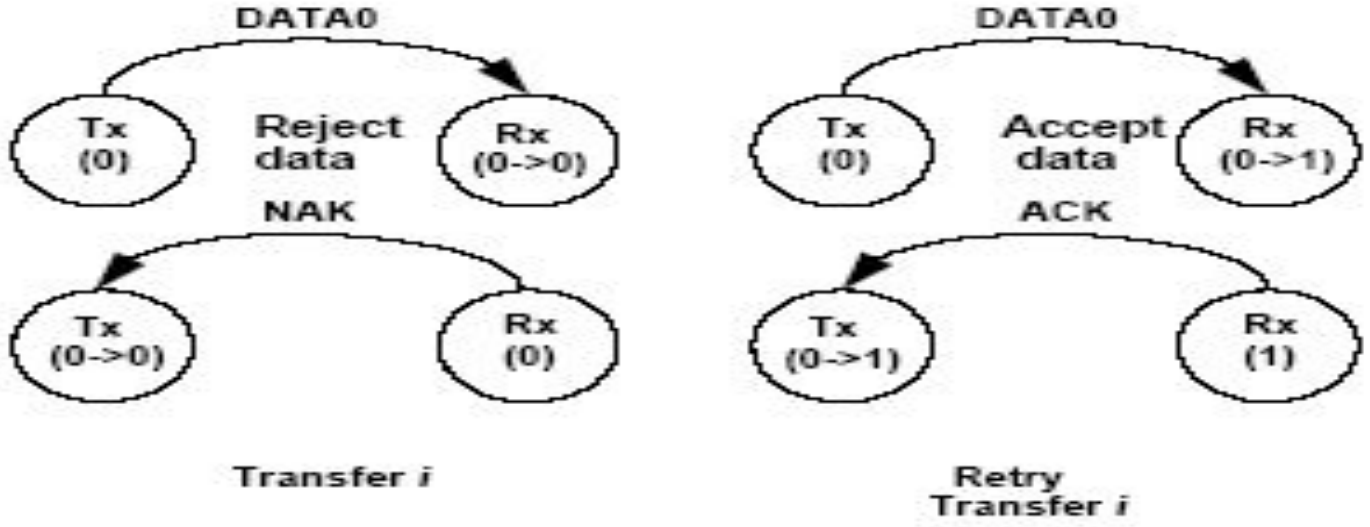


Figure 8-46. NAKed Transaction with Retry

(4)被破坏的ACK握手

由于发送器能接收ACK握手,因此它是最后一个而且唯一能确定处理是否成功完成的工具.丢失或被破坏的ACK握手会导致发送器和接收器之间暂时丢失同步,如图8.47所示.发送器在这里发送一个有效的数据包,而且接收器已经成功获得这个包;但是ACK包却被破坏了.

在处理I结束时，发送器和接收器之间会暂时丢失一致性。这可以分别从它们的位序列不一致清楚地看出。接收器已经接收了完好的数据，但发送器不知道他是否成功发送数据。在下一处理中，发送器会用前一个DATA0 PID重发前面的数据。由于接收器的位序列和数据PID不一致，那么接收器知道它在前面接收了这个数据。因此它丢弃了输入的数据包且不触发它的位序列；然后接收器发送ACK，使发送器认为重试处理成功完成。接收ACK会导致发送器触发它的位序列。在处理I+1开始时，位序列已被触发且被再次同步。

数据发送器必须保证任何重试的数据包与原始发送的处理一样（相同的长度和内容）。如果数据发送器由于诸如缓冲器下溢等原因不能发送与原始数据包一样大小的数据，则它必须通过产生违反全速/低速的位填充来终止处理。高速错误通过取当前计算的CRC，并在发送前取反来强制产生。这就在接收器产生了可检测的错误，而且保证一个局部的包不会被解释为完整的包。发送器不应通过发送一个固定的，被破坏的CRC迫使在接收器上产生错误。接收器会将带“错误”CRC的错误包组合解释为一个完好的包。

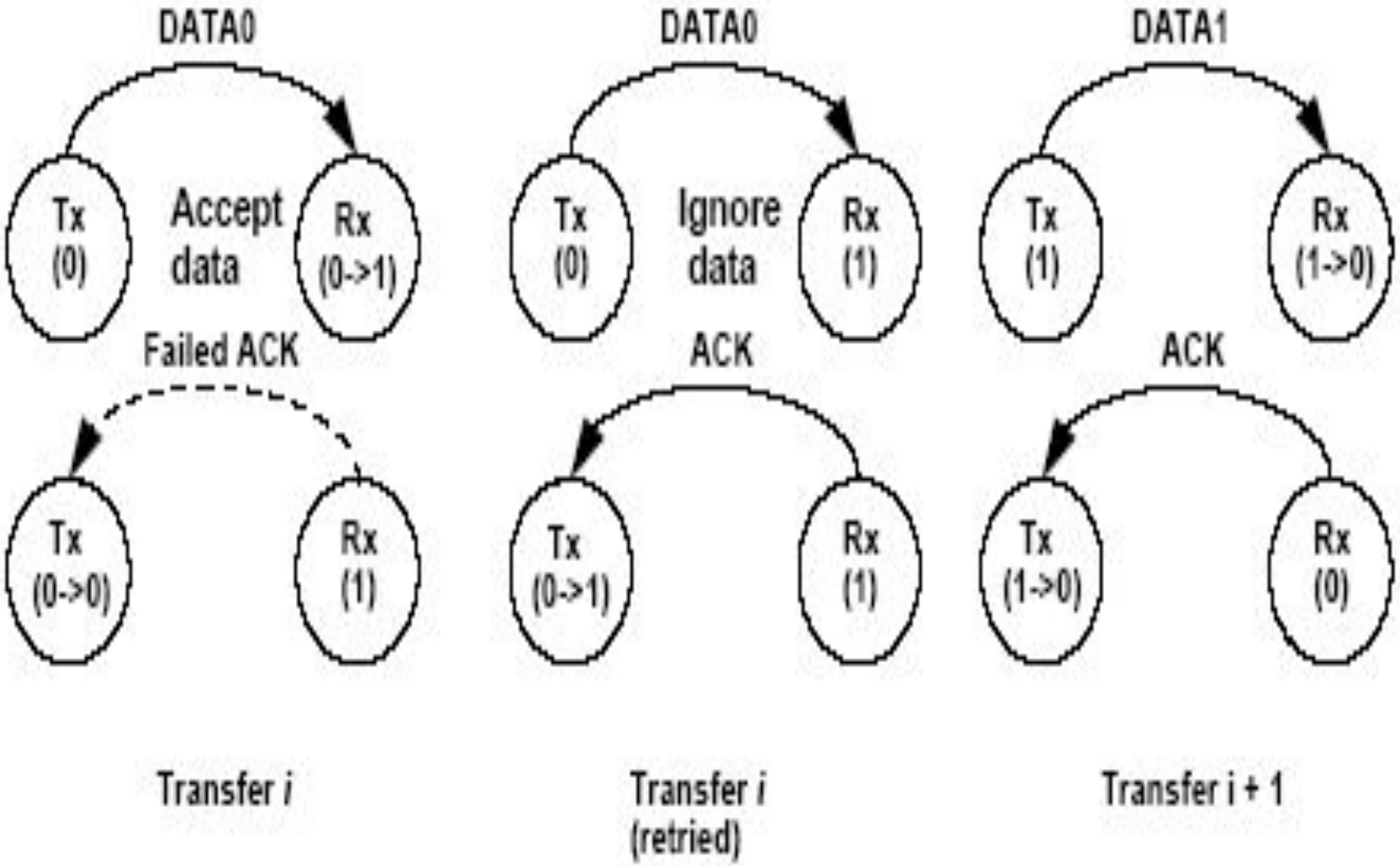


Figure 8-47. Corrupted ACK Handshake with Retry

www.cpit.com.cn

（5）低速处理

USB支持在3种速度下发送信号：480Mb/s的高速信号12.0 Mb/s的全速信号以及1.5 Mb/s的低速信号.集线器将高速信号环境与全速/低速信号环境分隔开来.

在全速/低速信号环境中,集线器禁止在全速下行信号环境中与连接低速设备端口的下行通信.这是由于EMI原因以及防止低速设备将下行的全速包曲解是对它的寻址而作出的要求.

图8.48所示为一个IN低速处理.其中,主机(或TT)发送令牌和握手.并且接收数据包.

在全速/低速信号环境中,所有发送到低速设备的下行包都要求有前同步信号.前同步信号不在高速信号环境中使用.前同步信号是由一个SYNC紧接着一个PRE PID组成,它们都以全速发送.集线器必须理解PRE PID;而其他USB设备可以忽略它,并把它认为是未定义的令牌.在前同步PID结束时,主机(或者TT)驱动总线进入空闲状态,并持续至少一个全速位时间.在这个集线器建立间隔其间,集线器必须驱动它的全速和低速端口,使它们分别进入各自的空闲状态.集线器必须准备好在集线建立间隔前在低速端口重发低速信号.低速连接规则总结如下:

www.cpit.com.cn

- 低速设备在连接过程中被识别出来,而且它们所连接的集线器端口被认为是低速端口.
- 所有下行低速包必须以前同步信号开始(以全速发送).它打开了低速集线器端口的输出缓冲器.
- 低速集线器端口的输出缓冲器在收到EOP后关闭,直到检测到一个前同步PID时才再次打开.
- 集线器端口是全速还是低速不影响上行连接.

低速信号以主机(或者TT)在低速下发送SYNC信号为开始,接着是包的剩余部分.包的结束通过包结束(EOP)来识别.此时,所有集线器都断开连接,并禁止连接低速设备的所有端口.集线器不会关闭上行信号的端口;全速和低速信号中上行方向的低速端口保持使能.

低速和全速处理仍有高度的协议公共性.但是低速信号仍有某些限制:

- 数据有效负载的最大限制为8字节.
- 只支持中断和控制传输类型.
- 低速设备不接收SOF包.

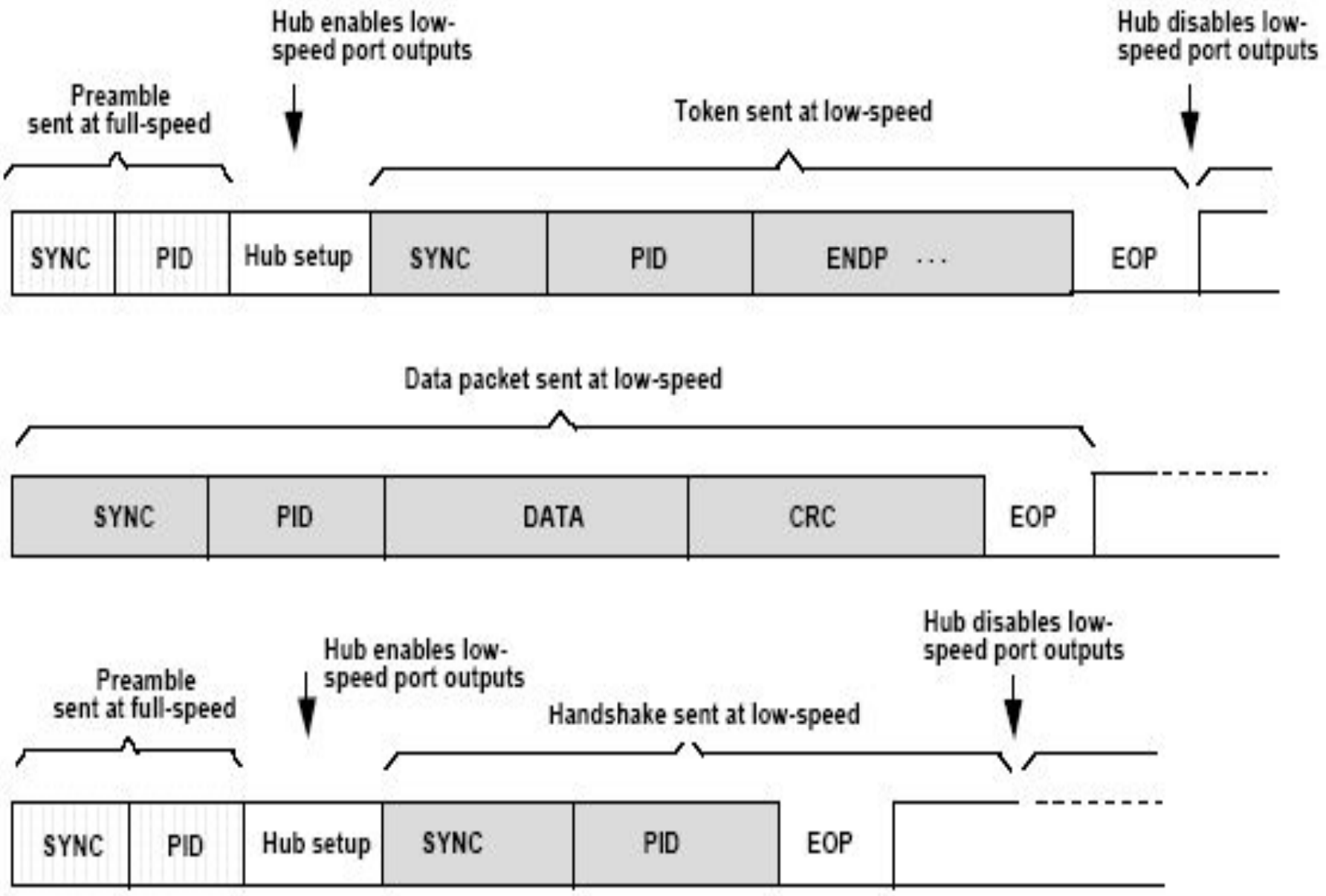


Figure 8-48. Low-speed Transaction

7. 错误的检测和恢复

USB允许可靠的端到端（End-to-end）的通信,这种通信容许产生物理信号层上的错误。这包括可靠地检测到大量可能的错误的能力和基于事务类型的从错误中恢复的能力。例如，控制事务需要高度数据可靠性；它通过使用错误检测和重试的方法支持端到端的数据完整性。出于同步事务的带宽（Bandwidth）和等待时间（Latency）的要求的原因，它不允许进行重试，且必须对未纠正的错误有较高的容忍程度。

(1) 包的错误分类

USB使用3种错误检测机制：位填充违反，PID检验位和CRC.

除了SOF令牌外,接收器会忽略所有收到但被破坏的包,并丢弃该包带的所有数据和其他场信息.表里8.8列出了错误检测机制都被应用到哪些类型的包以及相应的包接收器响应.

Table 8-8. Packet Error Types

Field	Error	Action
PID	PID Check, Bit Stuff	Ignore packet
Address	Bit Stuff, Address CRC	Ignore token
Frame Number	Bit Stuff, Frame Number CRC	Ignore Frame Number field
Data	Bit Stuff, Data CRC	Discard data

(2)总线周转（Turn-around）时间

设备和主机都不会发出指示以指出其收到的包有错误的。不作肯定答复则被认为是有错误的.作为这种错误汇报的方法的结果，主机和USB功能部件需要知道从发送器发完包的时候算起直到它开始收到应答为止过了多少时间。这一段时间被称为总线周转时间.

对于全速/低速处理,定时器在EOP选通脉冲的SE0到J的跳变沿开始计数,在检测到空闲到K的SOP跳变时结束计数;对于高速处理,定时器在线数据返回到静噪电平时开始计数,在数据线离开静噪电平结束计数.

设备的总线转向时间由最差情况的往返延时加上最大的设备响应延时定义.如果在这个最差情况的超时时间内没有接收到响应,那么发送器认为包的传输失败.

许多传输类型都使用超时并将踏解释为错误情况.如果主机期望通过超时指出处理的错误情况,它在发送一个令牌前必须等待完整的总线转向时间,以确保所有下行设备已经超时.

如图8.49所示,设备在令牌与数据或者数据与握手阶段之间使用自己的总线转向定时器.主机在数据与握手或者令牌与数据阶段之间使用定时器.

如果主机接收到一个被破坏的数据包,它会要求在发送下一个令牌前有额外的时间.这个额外等待时间间隔保证主机正确处理错误的EOP.

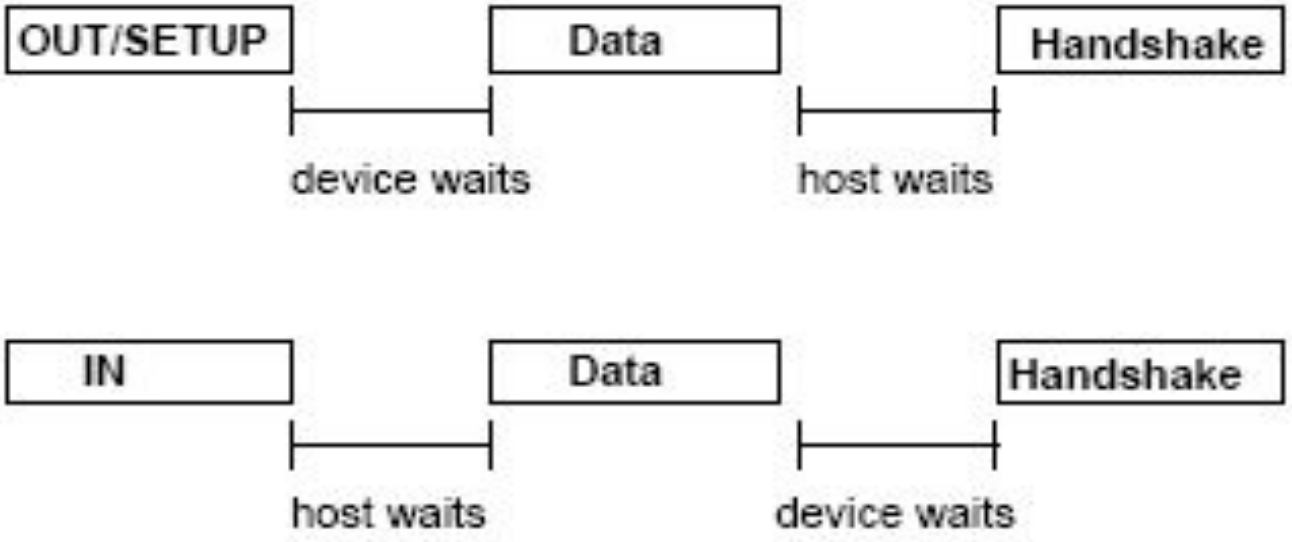


Figure 8-49. Bus Turn-around Timer Usage

www.cpit.com.cn

(3)错误的EOP

错误的EOP必须使用一种方法来处理.这种方法保证当前正在处理的包在主机或其他设备尝试发送一新的包前完成处理.如果发生这样的事件,它会产生总线冲突并会破坏最多两个连续的处理.错误的EOP的检测依靠有错误的EOP的包像一带CRC错误的截短包来进行.(数据包的最后16位是正确的CRC有极低的可能性.)

主机和设备处理错误EOP的情况不同.当设备接收到一被破坏的数据包时,它不发送响应,而是等待主机发送下一个令牌.这种方法保证设备在机仍然发送数据包时不会尝试返回握手.如果出现错误的EOP,主机数据包将最后结束,设备能检测到下一令牌.如果设备发送的数据包被错误的EOP破坏,主机会忽略包且不发送握手.等待接收主机握手信号的设备会处理超时.

如果主机接收到一个被破坏的全速/低速数据包,它假设会产生错误的EOP并且等待16个位时间看是否有后续的上行通信.如果在这16个位时间内没有检测到总线跳变而且总线仍处于空闲状态,主机会发送下一个令牌.

否则,主机等待设备发送完剩余的全速/低速包.等待16个位时间保证了两种情况.

第一种情况是确保设备完成包的发送,这由超时间间隔(没有总线跳变)大于最差情况下6个单位时间的位来填充间隔来保证.

第二种情况是必须保证发送设备的总线转向定时器超时.

注意: 超时间隔对处理速度很敏感.对于全速处理来说,主机必须等待全速位时间;对于低速处理来说,它必须等待低速位时间.

如果主机接收到被破坏的高速数据包,它回忽略所有数据直到数据现在发送下一个令牌前,返回到静噪电平.对于高速处理来说,主机不需要在线数据返回到静噪电平后等待额外的时间(超过正常的处理间隔时间).

如果主机接收到带有效CRC的数据包,它假设包完成而且在发送下一个令牌时不要求额外的延时(超过正常的处理间隔时间).

(4)串扰和丢失活动恢复能力

USB必须能检测无限地等待全速/低速EOP或者在(微型)帧的结束时总线不是处于空闲状态的情况,并从这些情况中恢复.

- **全速/低速丢失活动(LOA)的特点是:SOP后面没有总线活动(总线仍被驱动为J或K)而且在帧的结束没有EOP.**
- **全速/低速串扰的特点是:数据线在微型帧结束后处于非静噪电平的状态.**

LOA和串扰都能死锁总线或者延迟下一个(微型)帧的开始.这两种情况都是不可接收的,而且必须防止发生.作为负责控制连接的USB部件,集线器将负责串扰/LOA的检测以及恢复.对于在(微型)帧的结束仍不能完成传输的所有USB设备,通过禁止它连接的最近一个集线器的端口,防止它在超过(微型)帧结尾时继续发送.

(九)USB设备的结构

USB设备的3个层：

- 底层是传送和接收数据包的总线接口
- 中间层处理总线接口与不同端点之间的数据路由端节点是数据的终结提供处或使用处，它可被看作数据源或数据接收端(Sink)
- 最上层的功能由串行总线设备提供，比如鼠标，或ISDN接口。

本章描述的是USB设备中间层的通用属性与操作。这些属性与操作由设备的特定功能的部分用于通过总线接口最终与主机(host)的通信。

Table 9-1. Visible Device States

Attached	Powered	Default	Address	Configured	Suspended	State
No	--	--	--	--	--	Device is not attached to the USB. Other attributes are not significant.
Yes	No	--	--	--	--	Device is attached to the USB, but is not powered. Other attributes are not significant.
Yes	Yes	No	--	--	--	Device is attached to the USB and powered, but has not been reset.
Yes	Yes	Yes	No	--	--	Device is attached to the USB and powered and has been reset, but has not been assigned a unique address. Device responds at the default address.
Yes	Yes	Yes	Yes	No	--	Device is attached to the USB, powered, has been reset, and a unique device address has been assigned. Device is not configured.
Yes	Yes	Yes	Yes	Yes	No	Device is attached to the USB, powered, has been reset, has a unique address, is configured, and is not suspended. The host may now use the function provided by the device.
Yes	Yes	--	--	--	Yes	Device is, at minimum, attached to the USB and is powered and has not seen bus activity for 3 ms. It may also have a unique address and be configured for use. However, because the device is suspended, the host may not use the device's function.

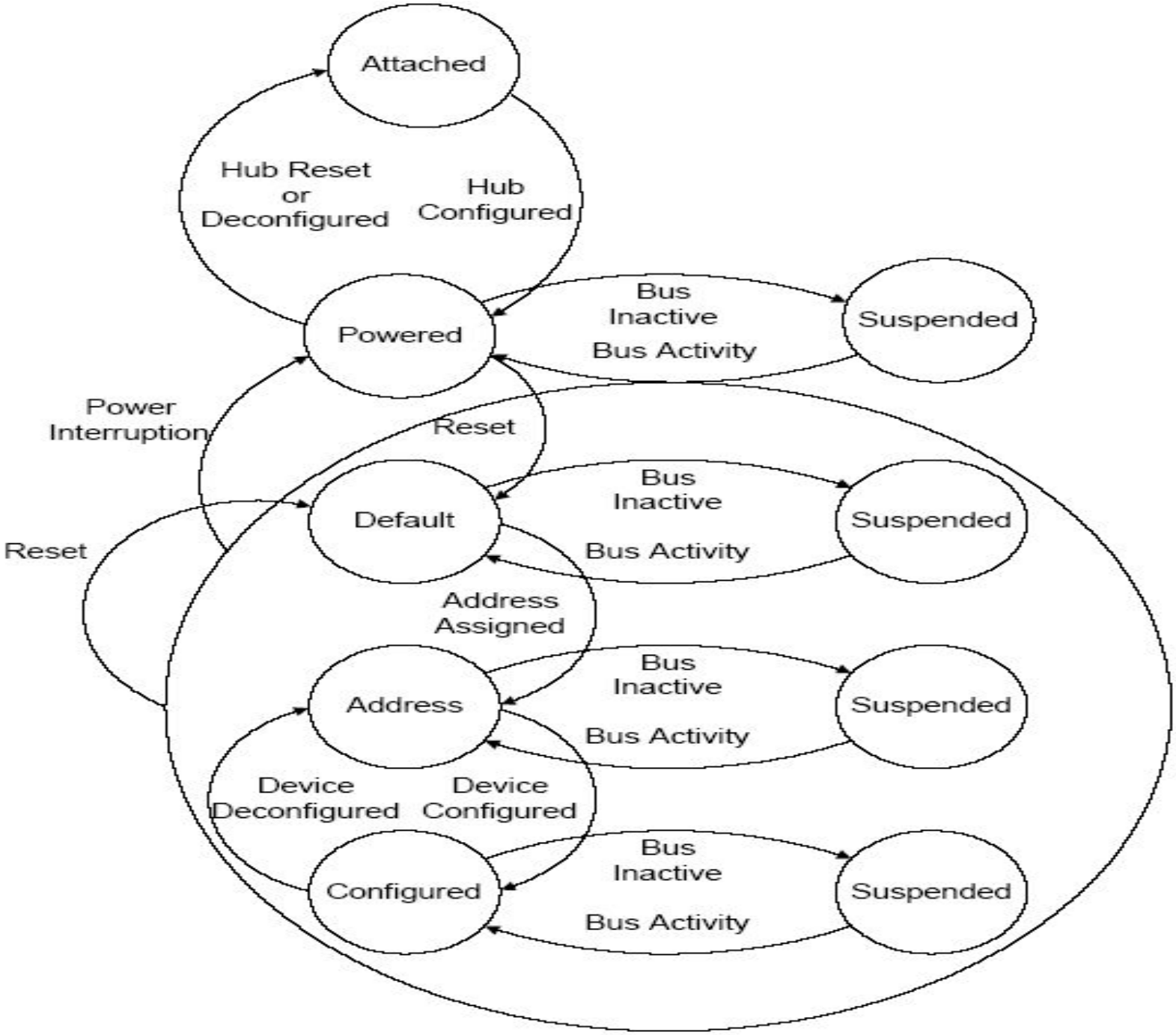


Figure 9-1. Device State Diagram

USB设备状态

- 可见的设备状态

外部可见的USB设备的状态如表9.1所列.图9.1所示位可见的USB设备状态图.

注:USB设备用复位操作响应上行口的复位信号.当复位信号结束后,USB设备复位.

1.连接

设备可以连接到USB或者从USB上拔出.USB设备从总线上拔出后的状态在规范没定义,只说明一旦USB连到总线要求的操作以及属性.

2.上电

USB设备的电源可来自外部电源，也可从USB接口的集线器而来。电源来自外部电源的USB设备被称作自给电源式的(self-powered)。尽管自给电源式的USB设备可能在连接上USB接口以前可能已经带电，但它们直到连线上USB接口后才能被看作是加电状态(Powered state)。而这时候VBUS已经对设备产生作用了.

一个设备可能有既支持自给电源的，同时也支持总线电源式的配置。有一些支持其中的一种，而另一些设备配置可能只有在自给电源下才能被使用。设备对电源支持的能力是通过配置描述表(configuration descriptor)来反映的。当前的电源供给形式被作为设备状态的一部分被反映出来。设备可在任何时候改变它们的供电来源，比如说：从自给式向总线式改变，如果一个配置同时支持两种模式，那此状态的最大电源需求就是指设备在两种模式下从VBUS上获取电能的最大值。设备必须以此最大电源作为参照，而究竟处于何状态是不考虑的。如果有一配置仅支持一种电源模式，那么电源模式的改变会使得设备失去当前配置与地址，返回加电状态。如果一个设备是自给电源式，并且当前配置需要大于100mA电流，那么如果此设备转到了总线电源式，它必须返回地址状态(Address state)。自给电源式集线器使用VBUS来为集线控制器(Hub controller)提供电源，因而可以仍然保持配置状态(Configured state)，尽管自给电源停止提供电源。

3.默认状态

设备上电后,它不响应任何总线处理,直到总线接收到复位信号为止.接收到复位信号后,用默认的地址可以对设备寻址.

当用复位过程完成后,USB设备在正确的速度下操作(即低速/全速/高速).低速和全速的数据选择由设备的终端电阻决定.能进行高速操作的设备决定它是否在复位的过程的一部分执行高速操作.

能进行高速操作的设备在全速的电气环境中操作时,必须能以全速成功复位.设备成功复位后,设备必须成功响应设备和配置描述符请求,并且返回适当的信息.当在全速下工作时,设备可能或者不能支持预定义的功能.

4.地址

所有的USB设备在加电复位以后都使用缺省地址。每一设备在连接或复位后由主机分配一个唯一的地址。当USB设备处于挂起状态时，它保持这个地址不变。

USB设备只对缺省通道(Pipe)请求发生响应，而不管设备是否已经被分配地址或在使用缺省地址。

5. Configured (配置状态)

在USB设备正常工作以前，设备必须被正确配置。从设备的角度来看，配置包括一个将非零值写入设备配置寄存器的操作。配置一个设备或改变一个可变的设备设置会使得与这个相关接口的终端结点的所有的状态与配置值被设成缺省值。这包括将正在使用(date toggle)的结点(end point)的 (Date toggle)被设置成DATA0。

6 挂起状态

为节省电源，USB设备在探测不到总线传输时自动进入中止状态(参见第七章)。当中止时，USB设备保持本身的内部状态，包括它的地址及配置。

所有的设备在一段特定的时间内探测不到总线活动时必须进入中止态，这一特定的时间在第7章中进行了说明。不管设备是被分配了非缺省的地址或者是被配置了，已经连接的设备必须在任何加电的时刻随时准备中止。总线活动的中止可能是因为主机本身进入了中止状态。另外，USB设备必须在所连接的集线器端口失效时进入中止态。这就是所指的选择性中止(Selective suspend)。

USB设备在总线活动来到时结束中止态。USB设备也可以远程唤醒的电流信号来请求主机退出中止态或选择性中止态。具体设备具有的远程唤醒的能力是可选的，也就是说，如果一个设备有远程唤醒的能力，此设备必须能让主机控制此能力的有效与否。当设备复位时，远程唤醒能力必须被禁止。

- **Bus Enumeration总线 枚举**

当USB设备接上或从USB设备移开的时候，主机启动一个被称作总线标识(bus enumeration)的进程，来标识并管理设备状态的改变，当USB设备接上一个加电端口时，系统当采取以下操作：

1. USB设备所连的集线器通过其通向主机的状态改变通道向主机，汇报本USB设备已连接上。(参照11.13.3节)。此时，USB设备处于加电状态，它所连接的端口是无效的。
2. 主机通过寻问集线器决定此次状态改变的确切含义。
3. 主机一旦得知新设备已连上以后，它至少等待100ms以使得插入操作的完成以及设备电源稳定工作。然后主机发出端口使能及复位命令给那个端口.
4. 集线器将发向端口的复位信号持续10ms(见11.5.15节)。当复位信号撤消后，端口已经有效了。这时USB设备处于缺省状态，并且可从VBUS汲取小于100mA的电能，所有设备寄存器及状态已经被复位，设备可对缺省地址产生响应。
5. 主机给设备分配一个唯一的地址，设备转向编址状态。(Address state)。

6. 在USB设备接受设备地址之前，它的缺省控制通道(Default Control Pipe)在缺省地址处自然是可寻址的，主机通过读取设备描述表，判决设备缺省通道的实际净数据负载。
7. 主机从设备读取配置信息要从配置0读到配置n-1，其中n为配置个数，此操作须花费几个毫秒。
8. 基于从设备取来的配置信息及设备如何被使用的信息，主机给设备一个配置值，此刻，设备就处于配置状态(Configured state)并此配置有关的所有端节点，都按照配置各就各位，USB设备现在可以从VBUS得到描述中所要求的电量了。从设备的角度来讲，它已经准备就绪了。

当UBS设备被取走时，集线器同样会通知主机，断开一个设备连接会使得设备所连接的端口无效，一收到断开通知后，主机就会更新的拓扑信息。

普通USB设备的操作

1 动态插接与拔开

USB设备必须在任意时刻允许被插接与拔开。提供连接点或端口的集线器应当负责汇报端口的状态改变情况。

当主机探测到连接操作后，会使得所连的集线器端口生效，设备也会因此而复位，一个被复位了的USB设备有如下特性：

·对缺省USB地址发生响应

·没有被配置

·初始状态不是挂起

当设备从一个集线器端口移去时，集线器会使得原来连接的端口失效，并且通知主机设备已移去。

2 地址分配

当USB设备连接以后，由主机负责给此设备分配一个唯一的地址，这个操作是在设备复位及端口使能操作以后。

3 配置

USB设备在正常被使用以前，必须被配置，由主机负责配置设备。主机一般会从USB设备获取配置信息后再准定此设备有哪些功能。

作为配置操作的一部分，主机会设置设备的配置值，并且，如果必要的话会选择合适的接口的备选设置。

只须一个简单配置，一个设备可能支持多重接口。一个接口是一组端结点集合，它们代表了设备向主机提供的单一的功能或特性，用来与这组相关端结点通信的协议以及接口内各端结点的目的可以作为一个设备类的一部分或者由厂商制定具体定义。

另外，一个配置中的结口可能有备选设置。这些备选设置会重定义相关端结点的数目或特性。如果是这样的话，设备必须支持GetInterface(接口请求)与SetInterface(接口设置)请求，来汇报及选择指定的接口的设备选设置。

在每个设备配置下，每个接口描述表可能包括用来标识接口的及备选设置的域，接口被从0～N-1编号。n为配置所支持的能同时使用的接口数目，类似的设置的编号也从0开始。当设备初始化配置后，缺省设置是备选设置0。

为了支持通用的设备驱动程序管理一组相关的BUS设备，设备与接口描述表中包含了类(Class)，子类(Sub class)，及协议(Protocol)域。这些域用来标识一个设备的功能及用于通信的协议。

一个类值被分配给一组按照特性划分成USB类说明一部分的设备。一个类的设备可进一步划分成子类，并且在一个类或子类中，一个协议代值可定义主机软件是怎样与设备通信的。

注意：类、子类、与协议值必须一致，但在本说明范围之外。

4 数据传送

数据可能以四种方式在USB设备端结点与主机之间传送。四种传送方式参见第五章。在不同设置下，一个终端结点可能被用于不同的传输方式，但一旦设置选定，传送方式就选定了。

5 电源管理

USB设备的电源管理包括以下说明部分的几条。

1 电源

USB总线电源是一个有限的资源，在设备标识(device enumeration)阶段，主机估测电源的需求。如果电源的需求量超过USB总线所能提供的电量，主机软件则不能选择那个配置。

USB设备应将电源需求量限制在一个单元以下，直到被配置。中止(挂起)的设备，不管是否已经配置过了，应将总线耗电降到第7章定义的标准以下。视接到设备的端口电源负载能力而定，USB设备在配置了以后可从VBUS汲取达5个单元的电量。

2 远程唤醒

远程唤醒能力参许一个被挂起的USB设备发达信号给处于挂起状态的主机。这个信号会使得主机醒来，处理触发事件。USB设备通过配置描述来向主机汇报其远程唤醒的能力。USB设备的远程唤醒能力应能被禁止的。远程唤醒能力通过7.1.7.7节中电信号的方式来达到的。

www.cpit.com.cn

6 请求处理

除SetAddress( )请求以外，在安装完成返回ACK信号以后，设备就开始处理请求。在某一状态成功结束以前，设备应当“完成”对请求的处理。许多请求费时较多，像这样的请求，该设备类应定义一个方法而不是等待交换状态信息阶段(StatusStage)的结束来表示该操作已经完成。像这样的操作有：集线器端口的复位至少需10ms来完成。当端口复位产生时，SetPortFeature (PORT-RESET)(见11章)请求就结束了。当端口状态改变并表明此端口已经生效时，一个信号就会产生表明复位信号已经结束。这种技术可以防止当主机知道某一个请求费时较长的情况一直探测此请求是否已完成。

(1) 请求处理的定时处理

所有的设备应当及时处理请求，USB给定一个5秒的命令处理的时间上界。这个限制并不是对所有情况都适用的。这些限制在接下来的部分给予描述，应当说明的是，下面的限制包括实现的很大的范围。如果所有设备都采用最大的请求处理允许时间的话，用户是无法忍受的。应此，具体实现应当尽可能快地完成请求的处理。

(2) 复位/继续 恢复时间

当一个端口被复位或从中止态继续的时候，USB系统软件应当等待一个10ms的恢复时间才能确保端口对数据传输产生响应。

一旦恢复时间段结束(从reset信号结束，或resume信号结尾的EOP结束开始计时)设备必须在任意时刻都能对数据传输作出响应。

www.cpit.com.cn

(3) 设置地址的处理

在reset/resume恢复时间段以后，如果设备收到SetAddress( )请求，设备必须能在50ms内完成请求的处理，并完成状态的转换，在SetAddress ( )的请求下，当设备发出0长度的状态数据包或设备收到状态数据包的响应信号ACK就表明状态转换结束了.

在状态转换结束后，设备有2ms的SetAddress ( )恢复时间。在这段时间结束以后，设备必须能在新地址处接受Setup数据包，并且，必须确保此时设备不对旧地址的信号产生响应。(当然，除非新旧地址是一样的).

(4) 标准设备请求

对于不须传送数据标准的设备请求，一个设备必须在收到请求的50ms以内结束对请求的处理及状态的转换.

对于需要数据传输的标准设备请求，设备必须在收到请求的500ms以内返回第一个数据包。接下来的数据包必须在前一个数据包发送起的500ms以内开始发送。设备必须在最后一个数据包返回以后的50ms以内结束状态的转换.

对于需要数据传输的标准设备请求，5秒的限制就起作用了。这意味着设备必须能在主机以设备最大能接受的速率发送数据包的情况下接收所有的数据包并且完成状态切换，数据包之间时延是主机让设备完成请求处理而加入的.

(5) 与类有关的请求

除非在类文档中特别说明，所有的类有关的请求必须按照标准请求的时间限制。

分类说明文档可能要求设备反应比这部分讲的要快。标准设备请求与与类有关的设备请求可被要求反应更快 .

(6) 由速度决定的描述符

能进行高速操作的设备可以在全速或高速中操作.作为复位过程的一部分设备要正确管理它的收发器.设备通常都知道它的操作速度.完成复位序列后,设备在单一的速度下工作.特殊情况是在正常操作中没有速度转换.但是,能进行高速操作的设备有速度决定的配置.也就是说,它有某些配置只能在高速下使用,某些配置只能在全速中运行.能进行高速操作的设备必须支持报告它们这些由速度决定的配置.

能进行高速操作的设备用当前的操作速度有效的描述符信息响应.例如,当设备请求配置描述符时,它只能返回当前操作速度(例如全速)的那些描述符.但是,必须有一种方法确定高速和全速操作的性能.

有两个描述符允许进行高速操作的设备报告其他操作速度的配置信息.这两个描述符是:(其他速度的)设备限定符((other_speed)device_qualifier)描述符以及其他速度配置描述符（ other_speed _configuration）.这两个描述符是由主机通过使用GetDescriptor请求以及相应的描述符类型值取得。

注意：除非主机明确地发出相应的GetDescriptor请求，否则不能取得这些描述符。如果没有这两个请求，设备应表现为一个单一速度的设备。

能进行高速操作的设备必须将其描述符bcdUSB场的版本号设置为0200H. 这表示这个设备支持USB2. 0定义其他速度(other_speed)请求. 如果描述符版本号小于0200H的设备接收了其他速度请求, 它应产生请求错误响应. 因此, 不应向USB1. 设备(设备描述符版本号小于0200H的设备)发送其他速度请求.

7 请求错误

如果一设备收到一个请求，它或是在设备中无定义，或是不适用于当前设置，或是数值不对，这时就会产生一个请求错误。设备在下一个数据传输阶段或状态交换阶段(Status stage)返回一个表明错误的STALL PID信号，一般在下一个数据传输返回更好，这样可减少不必要的总线活动

www.cpit.com.cn

USB设备的请求

Table 9-2. Format of Setup Data

Offset	Field	Size	Value	Description
0	*bmRequestType*	1	Bitmap	Characteristics of request: D7: Data transfer direction 0 = Host-to-device 1 = Device-to-host D6...5: Type 0 = Standard 1 = Class 2 = Vendor 3 = Reserved D4...0: Recipient 0 = Device 1 = Interface 2 = Endpoint 3 = Other 4...31 = Reserved
1	*bRequest*	1	Value	Specific request (refer to Table 9-3)
2	*wValue*	2	Value	Word-sized field that varies according to request
4	*windex*	2	Index or Offset	Word-sized field that varies according to request; typically used to pass an index or offset
6	*wLength*	2	Count	Number of bytes to transfer if there is a Data stage

所有的USB设备在设备的缺省控制通道(Default Control Pipe)处对主机的请求发出响应。这些请求是通过使用控制传输来达到的，请求及请求的参数通过Setup包发向设备，由主机负责设置Setup包内的每个域的值。每个Setup包有8个字节. 见表9-2 .

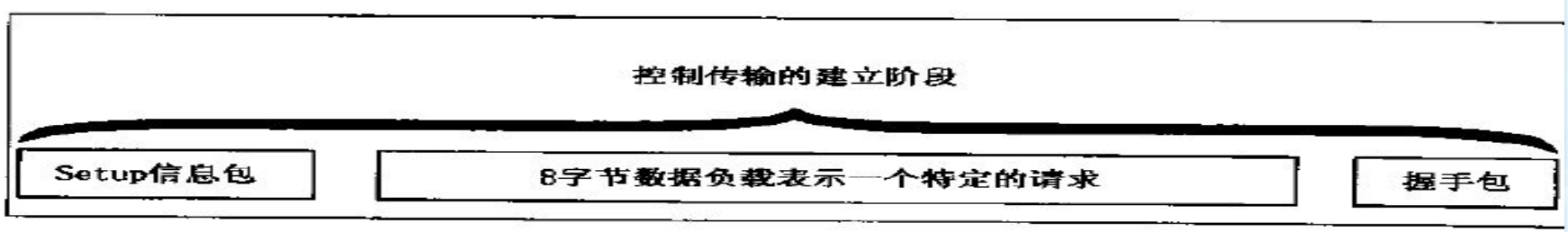


图 15-1　指定被执行的设备请求的建立事务处理的格式

1 bmRequestType域

这个域表明此请求的特性。特别地，这个域表明了第二阶段控制传输方向。如果wLength域被设作0的话，表明没有数据传送阶段，那Direction位就会被忽略.

USB说明定义了一系列所有设备必须支持的标准请求。这些请求被例举在表9-3中。另外，一个设备类可定义更多的请求。设备厂商也可定义设备支持的请求.

请求可被导引到设备，设备接口，或某一个设备端结点(endpoint)上。这个请求域也指定了接收者。当指定的是接口或端结点(endpoint)时，wIndex域指出那个接口或端节点.

2 bRequest域

这个域标识特别的请求。bmRequestType域的Type啦可修改此域的含义。本说明仅定义Type 字位为0即标准设备请求时bRequest域值的含义 .

www.cpit.com.cn

3 wValue域

此域用来传送当前请求的参数，随请求不同而变。

4 wIndex域

wIndex域用来表明是哪一个接口或端结点，图9-2表明wIndex的格式(当标识端结点时)。Direction位在设为0时表示出结点，设为1时表示是入结点，Endpoint Number是结点号。图9-3表明wIndex用于标识接口时的格式。

D7	D6	D5	D4	D3	D2	D1	D0
Direction	Reserved (Reset to zero)			Endpoint Number			
D15	D14	D13	D12	D11	D10	D9	D8
Reserved (Reset to zero)							

Figure 9-2. wIndex Format when Specifying an Endpoint

D7	D6	D5	D4	D3	D2	D1	D0
Interface Number							
D15	D14	D13	D12	D11	D10	D9	D8
Reserved (Reset to zero)							

Figure 9-3. wIndex Format when Specifying an Interface

5.wLength域

这个域表明第二阶段的数据传输长度。传输方向由bmRequstType域的Direction位指出。wLength域为0则表明无数据传输。在输入请求下，设备返回的数据长度不应多于wLength，但可以少于。在输出请求下，wLength指出主机发出的确切数据量。如果主机发送多于wLength的数据，设备做出的响应是无定义的 .

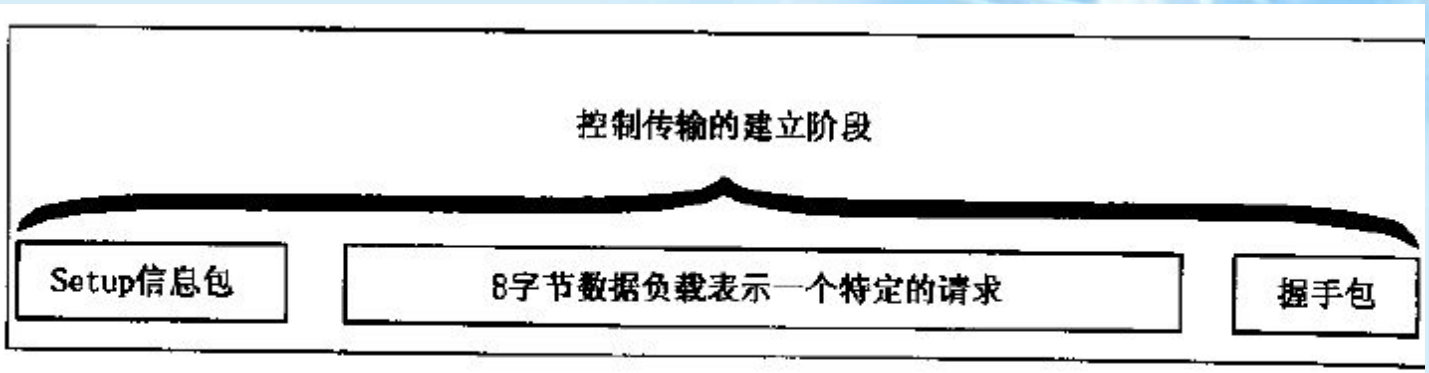


图 15-1　指定被执行的设备请求的建立事务处理的格式

标准的设备请求

这部分描述的所有USB设备都定义的标准设备请求，表9-3将它们列出，而表9-4、9-5分别结出了对应的标准请求码及描述表类型.

不管设备是否被分配了非缺省地址或设备当前是被配置了的，它们都应当对标准请求产生响应.

特性选择符被用来设置特性或使特性生效。比如说某个设备、接口、或结点的远程唤醒功能，特性选择符的值在表9-6中得到了说明.

对于非法请求的处理从前所述。但是非法请求并不会使得控制通道设置Halt特性。如果因为某种原因，设备因为一个错误状态不能通过缺省控制通道来与主机通信，设备必须被reset来清除错误状态并重启缺省通道.

Table 9-6. Standard Feature Selectors

Feature Selector	Recipient	Value
DEVICE_REMOTE_WAKEUP	Device	1
ENDPOINT_HALT	Endpoint	0
TEST_MODE	Device	2

Table 9-3. Standard Device Requests

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B 00000001B 00000010B	CLEAR_FEATURE	Feature Selector	Zero Interface Endpoint	Zero	None
10000000B	GET_CONFIGURATION	Zero	Zero	One	Configuration Value
10000000B	GET_DESCRIPTOR	Descriptor Type and Descriptor Index	Zero or Language ID	Descriptor Length	Descriptor
10000001B	GET_INTERFACE	Zero	Interface	One	Alternate Interface
10000000B 10000001B 10000010B	GET_STATUS	Zero	Zero Interface Endpoint	Two	Device, Interface, or Endpoint Status
00000000B	SET_ADDRESS	Device Address	Zero	Zero	None
00000000B	SET_CONFIGURATION	Configuration Value	Zero	Zero	None
00000000B	SET_DESCRIPTOR	Descriptor Type and Descriptor Index	Zero or Language ID	Descriptor Length	Descriptor
00000000B 00000001B 00000010B	SET_FEATURE	Feature Selector	Zero Interface Endpoint	Zero	None
00000001B	SET_INTERFACE	Alternate Setting	Interface	Zero	None
10000010B	SYNCH_FRAME	Zero	Endpoint	Two	Frame Number

Table 9-4. Standard Request Codes

bRequest	Value
GET_STATUS	0
CLEAR_FEATURE	1
Reserved for future use	2
SET_FEATURE	3
Reserved for future use	4
SET_ADDRESS	5
GET_DESCRIPTOR	6
SET_DESCRIPTOR	7
GET_CONFIGURATION	8
SET_CONFIGURATION	9
GET_INTERFACE	10
SET_INTERFACE	11
SYNCH_FRAME	12

Table 9-5. Descriptor Types

Descriptor Types	Value
DEVICE	1
CONFIGURATION	2
STRING	3
INTERFACE	4
ENDPOINT	5
DEVICE_QUALIFIER	6
OTHER_SPEED_CONFIGURATION	7
INTERFACE_POWER[1]	8

www.cpit.com.cn

1.清除特性(ClearFeature())

这个请求是被用来清除或禁能一个特性。

wValue中的特性选择符的值必须根据接收者来设定适当的值。接收者是设备要用设备特性选择符，是接口就必须用接口特性选择符，是端结点就要用端节点特性描述符.

参照表9-6来看选择符与接收者的对应关系.

一个ClearFeature( )请求所指的特性如果不能被清除，不存在或指的是不存在的接口或结点号，会产生RequestError错误 .

如果wLength不为0，设备响应无定义

缺省状态：当设备处于缺省状态时，对此请求的反应无定义。

地址状态：在设备处于地址状态时这个请求是合法的，但如果该请求指的是接口或是非零号端结点，会引起请求错误。

配置状态：在此状态下，该请求合法

注意:ClearFeature()请求不能清除Test_Mode特性。

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B 00000001B 00000010B	CLEAR_FEATURE	Feature Selector	Zero Interface Endpoint	Zero	None

2.取得配置

这个请求返回当前设备的配置值.

此请求返回当前设备配置值。

如果返回0值表明设备未配置。

如果wValue，wIndex，wLength的值与上面表中不同，设备响应无定义。

缺省状态：该请求响应无定义

地址状态：返回0值

配置状态：非0的 bConfigurationValue值被返回

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10000000B	GET_CONFIGURATION	Zero	Zero	One	Configuration Value

3.取得描述符

这个请求返回存 在的描述符.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10000000B	GET_DESCRIPTOR	Descriptor Type and Descriptor Index	Zero or Language ID (refer to Section 9.6.7)	Descriptor Length	Descriptor

wValue域的高一字节标识描述表类型，低一字节表示描述符号的索引(参见表9-5)。当将几个类型相同的描述符应用到设备中时，描述符索引用于选择一个特定的描述符（只能是配置和字符串描述符）。例如：设备可以用几个配置描述符。对于其他可以通过GetDescriptor()请求取得的标准描述符来说，描述符的索引须为0. 描述符索引值的范围从0开始到设备使用该类型描述符的数量减1.

wIndex域标识字串描述表的语言(Language ID)如果是其它语言的话就设为0，wLength表示要返回多少字节。如果描述表长度大于wLength域值，那么只有描述表的初始部分被返回。如果描述表比wLength域值小，没发送一个短包来标志传输的结束。一个短包被定义成一个长度短于最大负载长度或一个空(NULL)包。

www.cpit.com.cn

这个标准请求包括3种描述符：设备(也就是设备的限定符)、配置(也就是其他速度配置)、及字符串.能进行高速操作的设备支持设备限定描述符返回有关设备不支持的速度信息(包括默认端点的wMaxPacketSize和其他速度配置的数量).其他速度配置用与配置描述符相同的结构返回信息,但如果设备在其他速度下操作则返回配置信息.请求配置描述符将返回配置描述符,所有接口描述符和在这个请求中所有接口的端点描述符.第一个接口描述符的设备请求会一次返回配置描述表，所有的接口描述表和所有接口的端节点的描述表。第一个接口描述符紧跟着配置描述符号，第一个接口的端节点的描述符号随后。如果有其它的接口与端节点，它们的描述符欲跟在第一个接口与端节点描述符之后。与类有关的描述符，和/或厂商定义的描述符跟在标准描述符之后.

所有的设备必须提供一个设备描述符并且至少一个配置描述符，如果一个设备不支持一个请求的描述符，则返回请求错误。

缺省状态：此请求合法。

地址状态：此请求合法。

配置状态：此请求合法。

www.cpit.com.cn

4.取得接口

这个请求向指定接口返回选中的备用设备。

一些USB设备有接口设置互斥的配置。这个请求允许主机确定当前选定的备用设置。如果wValue或者wLength的值与上面指定的不一致，那么设备的行为没有定义；如果指定的接口不存在，那么设备将用请求错误响应。

- **默认状态** 当设备处于默认状态时接收到这个请求，设备的行为没 有定义
- **寻址状态** 设备给出请求错误
- **配置状态** 当设备处于配置状态时，这是一个有效的请求

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10000001B	GET_INTERFACE	Zero	Interface	One	Alternate Setting

5.取得状态

这个请求返回指定接收器的状态

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10000000B 10000001B 10000010B	GET_STATUS	Zero	Zero Interface Endpoint	Two	Device, Interface, or Endpoint Status

bmRequestType场的Recipients位指明了要求的接收器. 返回的确数据指定接收器的当前状态.

如果wValue或wLength不是上表中值，或wIndex在取设备状态请求时非0则设备响应无定义。.如果指定的端点或接口不存在,那么设备将用请求错误响应.

- 默认状态 当设备处于默认状态时接收到这个请求,设备的行为没有定义
- 寻址状态 如果指定了一个接口或者端点(端点0除外),设备将用请求错误响应
- 配置状态 如果指定的接口或端点不存在,设备将用请求错误响应.

向设备发送的GetStatus()请求返回信息的格式如图9-4

D7	D6	D5	D4	D3	D2	D1	D0
Reserved (Reset to zero)						Remote Wakeup	Self Powered
D15	D14	D13	D12	D11	D10	D9	D8
Reserved (Reset to zero)							

Figure 9-4. Information Returned by a GetStatus() Request to a Device

其中Self Powered域表明设备当前是否是自给电源。如果Do为0设备是总线电源式。如果Do被设成1，设备是自给电源式的。此域不应当被SetFeature( )或ClearFeature( )请求改变.

Remote Wakeup域表明此设备当前是否支持远程唤醒，支持远程唤醒能力的设备缺省值是无效的(disabled)，如果D1被Reset成0，远程唤醒能力就被disabled。返之，设成1就是具有该功能，此域可被SetFeature( )和Clear Feature ( )使用DEVICE-REMOTE-WAKEUP特性选择符修改，设备复位时此域被设成0.

一个GetStatus ( )的接口请求返回如图9-5的信息

一个GetStatus ( )的端点请求返回如图9-6的信息

D7	D6	D5	D4	D3	D2	D1	D0
Reserved (Reset to zero)							
D15	D14	D13	D12	D11	D10	D9	D8
Reserved (Reset to zero)							

Figure 9-5. Information Returned by a GetStatus() Request to an Interface

D7	D6	D5	D4	D3	D2	D1	D0
Reserved (Reset to zero)							Halt
D15	D14	D13	D12	D11	D10	D9	D8
Reserved (Reset to zero)							

Figure 9-6. Information Returned by a GetStatus() Request to an Endpoint

Halt特性应当在所有的中断及端结点中实现。如果端节点当前被halted了，那么这个Halt特性就设成1，否则为0。Halt特性可选择性地由Set Feature(ENDPOINT-HALT)请求来设置，一旦被SetFeature( )请求设置，设备的响应就会像这个域由硬件条件设置的一样，如果导止停机(halt)的条件去除了，用Clear Feature (ENDPOINT-HALT)请求清除halt特性会导致端节点再也不会返回STALL信号。对于使用 (Date toggle)的端节点，不管一个端节点的Halt特性是否已被设置，一个Clear Feature (ENDPOINT-HALT)总会导致 (date toggle)被重新初始化成DATAO Halt特性在收到SetConfiguration ( )或Set Interface( )请求后总会被复位成0。

Halt特性不要求也不建议在缺省控制通道实现。然而，设备可设置缺省控制通道的Halt特性来反映一个功能出错的状态。如果这个特性被设了的话，设备将对除Getstatus( ),SetFeature( )，Clear Feature ( )之外的请求返回STALL信号，设备可不对类有关的及厂商定制的请求返回STALL信号.

6. 设置地址

这个请求为所有未来的设备访问设置设备地址

wValue指出所要设置成的地址值

www.cpit.com.cn

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B	SET_ADDRESS	Device Address	Zero	Zero	None

像在另处所述，请求实际可分成三个阶段。在第一阶段，Setup包被送至设备，在第二个可有无的阶段，数据在设备与主机之间传送，在第三阶段，状态信息在主机与设备之间传送。数据与状态传送的方向要看是主机发数据给设备还是设备发数据给主机。状态的传送方向总是与数据传送方向是相反的，如果没有数据传输阶段则状态由设备传向主机的.

Setup包传送以后的两个阶段的地址保持与Setup包传送阶段的一致。USB设备只有在Status阶段过后才能改变设备地址。注意，在这方面此请求不同于其它请求。其它请求总是在状态传送阶段之前完成指定操作的.

如果所指的设备地址大于127或wIndex 或wLength非0，设备响应无定义. 设备对SetAddress(0)的响应无定义 .

缺省状态： 如果地址值非0，那设备将进入地址状态，否则地址仍留在缺省态(此非出错状态)

地址状态： 如果新地址值为0，进入缺省态，否则仍留在地址状态但使用新地址

配置状态： 在此状态下设备对此请求的响应无定义

7.设置配置

这个请求设置设备的配置

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B	SET_CONFIGURATION	Configuration Value	Zero	Zero	None

wValue域的低字节指出配置，这个配置值必须为0或与配置描表中的一个配置相配

如果配置值为0，设备置地址状态。wValue的高字节保留。

如果wIndex, wLength或wValue的高字节非0，则设备对之的响应无定义。

缺省状态：设备响应无定义

地址状态：如果所指的配置为0，设备停留在地址状态。如果所指的配置与描述表中的一个值相匹配，那个配置就被选中，设备转到配置有。否则，返回请求错误

配置状态：如果配置值为0，设备进入电址状态。如果配置值非0并与描述表中的一个配置相匹配则设备仍留在配置态，但采用新的配置值，否则返回请求错误。

8.设置描述符

此请求用于更新或添加新的描述表符.

wValue域的高字节指出了描述的类型，低字节指出了描述表索引(参风表9-5)，当将几个类型相同的描述符应用到设备中时,描述符索引用于选择一个特定的描述符(只能是配置和字符串描述符).例如:设备可以应用几个配置描述符.对于其他可以通过GetDescriptor()请求设置的标准描述符来说,描述符索引必须为0.描述符索引值的范围从0开始到设备使用该类型描述符的数量减1.

wIndex域指出了字串描述表的语言标识，对于其它描述表来说它为0。wLength指出从主机传向设备的字节数 .

如果设备不支持该请求则设备返回一个请求错误

缺省状态：此状态下设备对该请求反应无定义

地址状态：如果设备支持请求，则为合法

配置状态：如果设备支持请求，则为合法

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000000B	SET_DESCRIPTOR	Descriptor Type and Descriptor Index	Language ID (refer to Section 9.6.7) or zero	Descriptor Length	Descriptor

9.设置特性

这个请求用于设置或使能一个指定的特性.

bmRequestType	bRequest	wValue	wIndex		wLength	Data
00000000B 00000001B 00000010B	SET_FEATURE	Feature Selector	Test Selector	Zero Interface Endpoint	Zero	None

wValue域中的特性选择符必须跟接收者相配 .当接收器是设备时,只能使用设备特性选择符的值;当接收器是接口时,只能使用接口特性选择符;当接收器是端点时,只能是使用端点符特性选择符.各种接收器定义的特性选择符的值参考表9.6.

TEST_MODE特性只在设备接收器定义(即bmRequest Type=0),而且wIndex的低字节必须为0.置位TEST_MODE特性会使设备的上行口进入测试模式.如果请求包含无效的测试选择符,设备将用请求错误响应.切换到测试模式必须在请求状态阶段结束后的3ms内完成。直到请求的状态阶段结束后，上行口才能切换到测试模式。设备必须重启电源才能退出设备上行口的测试模式。设备必须在默认，寻址或配置的高速状态中支持TEST_MODE特性。

SetFeature ( )请求如果指出一个不存在的特性会使得设备在交换状态阶段返回STALL信号。

www.cpit.com.cn

如果选择特性是TEST_MODE，那么Windex的最高字节用于指明待定的测试模式。SetFeature(TEST_MODE…)的接收器必须是设备，即wIndex 的低字节必须是0，而且bmRequestType也必须设置为0。设备必须重启电源以退出测试模式。有效的测试模式选择符在表9.7列出.

如果wLength非0,那么设备的行为不确定;如果指定的端点或接口不存在,则设备会以请求错误响应.

- **默认状态** 当处于默认状态时，设备必须能接收SetFeature（TEST_MODE，TEST_SELECTOR）请求.设备处于默认状态时接收到其他SetFeature请求,设备的行为不确定.
- **寻址状态** 如果指定了一个接口或端点(端点0出外),设备用请求错误响应.
- **配置状态** 当设备处于配置状态时,这是一个有效的请求.

Table 9-7. Test Mode Selectors

Value	Description
00H	Reserved
01H	Test_J
02H	Test_K
03H	Test_SE0_NAK
04H	Test_Packet
05H	Test_Force_Enable
06H-3FH	Reserved for standard test selectors
3FH-BFH	Reserved
C0H-FFH	Reserved for vendor-specific test modes.

www.cpit.com.cn

10.设置接口

这个请求允许主机为指定的接口选择另一个设置.

如有USB设备接口配置中有互斥设置。此请求让主机选择所要的设置。如果设备的接口只支持缺省设置，在状态交换阶段设备返回STALL.这个请求不能用于改变已配置的接口的集合(必须使用SetConfiguration()请求).

如果接口或者备用设置不存在,则设备必须以请求错误响应.如果 wLength 的值非0,那么设备的行为不确定.

- **默认状态** 当设备处于默认状态时接收到这个请求,设备的行为不确定
- **寻址状态** 设备必须以请求错误响应
- **配置状态** 当设备处于配置状态时,这是一个有效的请求.

bmRequestType	bRequest	wValue	wIndex	wLength	Data
00000001B	SET_INTERFACE	Alternate Setting	Interface	Zero	None

11.同步帧

这个请求用于设置和报告端点的同步帧

如果一个端节点支持同步传输，端节点可能会根据某一特点的模式来以变长方式传送每一帧。主机与端节点必须在什么时候出现重复模式的第一帧出现上达成一致。模式开始帧的序号由设备返回给主机.

如果高速设备支持同步帧请求,它必须在内部与第0号微型帧同步,并且对标准帧有时间概念.只有帧号码用于且由设备端点报告(即无微型帧号码).端点必须与第0号微型帧同步.

这个值仅用于隐式模式的同步数据传输。如果wValue非0或wLength非2，设备响应无定义.

如果所指的端节点不支持此请求，设备返回一个请求错误。

缺省状态： 设备响应无定义

寻址地址： 设备返回请求错误

配置状态： 此请求合法

bmRequestType	bRequest	wValue	wIndex	wLength	Data
10000010B	SYNCH_FRAME	Zero	Endpoint	Two	Frame Number

描述符

USB设备通过描述表来反映他们的属性。描述符号是有定义好的格式的数据结构，每一个描述符以一个字节打头表明本描述符的长度，紧跟其后是一个字节的描述符类信息。

使用描述符使得单个配置的特性存储变得简明，因为每个配置可能会重复使用其它有相同特性的配置描述表的部分或全部，用这种方法，描述符用一个关系数据库来表绘一个个的单独数据记录.

在适当的地方，描述符包括了指向字串描述符的引用。字串描述符提供了人能读懂的信息。字串描述符可有可无，但描述符中的字串指引域是不可少的。如果一个设备不支持字串描述符，该域就为0。

如果描述符值中的长度域值少于本说明的定义，此描述符非法，不能被主机接受。如果返回的描述符中的长义值大于本说明定义，则过长部分当被忽略，但下一个描述符的位置由返回长度而不是实际长度来决定。

设备可以两种方式返回类相关的或厂商定义的描述符号:

1. 如果这两种描述表的格式与标准格式相同(以长度字节打头，紧跟着类型字节)则它们可由GetDescriptor(Configuration)请求与标准描述一同返回。在这种情况下，类相关或厂商定义的描述表一般跟在被修改的或被扩展的描述之后。

2.如果这两种描述表使用的非标准格式。指定类相关的或厂商定义的描述表及索引的GetDescriptor( )请求可从设备返回这两种描述表。类或厂商说明会指出正确取出这两种描述表的途径。

标准USB描述符的定义

本说明中有关标准描述表的定义只能被本说明的校订本修改或扩展.

注意：一个对USB 1.0标准结点描述表的扩展已由《声音设备类说明修订本1.0》公布。仅此是除USB说明外被许可的，将来的USB说明的修订本会用此来对标准节点描述表进行扩展来避免与《声》的冲突.

1. 设备

设备描述符给出了USB设备的一般信息。这包括对设备及所有设备配置起全程作用的信息。一个USB设备只能有一个设备描述符.

能进行高速操作的设备与全速和低速设备的设备信息不一样,因此还要有设备限定符.

能进行高速操作的设备，其设备描述符版本号是2.0(0200H).只能进行全速和低速操作的设备按本规范设计时不应用版本号2.0(0200H).如果设备只能进行全速或者低速操作,那么版本号表示它能正确响应对设备限定描述符的请求,即它会以请求错误响应.

www.cpit.com.cn

bcdUSB场包含的是BCD版本号. bcdUSB场的值是0xJJMN,对应于版本JJ.M.N（JJ-主版本号,M-副版本号,N-副子版本号）,例如版本2.1.3用值0x0213表示,版本2.0用值0x0200表示.

bNumConfigurations场指出在当前的操作速度下配置的数量.其他操作速度的配置不包含在这个计数值中.如果在特定速度下有特殊的设备配置,bNumConfigurations只反映单个速度下配置的数量,而不是所有速度下总的配置的数量.

如果设备在高速下操作,bMaxPacketSize0场必须是64，表示有最大包且为64字节。高速操作不允许控制端点（端点0）有其他最大包值。

所有的USB设备都有一条默认的控制管道。设备默认的控制管道最大包的大小在设备描述符中说明。端点的具体配置及其接口将在配置描述符中介绍。配置及其接口不包含默认控制管道的端点描述符。除了最大包的大小值外，默认控制管道的特性都在本规范定义，而且所有USB设备都相同。

bNumConfigurations场确定了设备支持的配置数量.表9.8列出了标准的设备描述符.

www.cpit.com.cn

Table 9-8. Standard Device Descriptor

Offset	Field	Size	Value	Description
0	*bLength*	1	Number	Size of this descriptor in bytes
1	*bDescriptorType*	1	Constant	DEVICE Descriptor Type
2	*bcdUSB*	2	BCD	USB Specification Release Number in Binary-Coded Decimal (i.e., 2.10 is 210H). This field identifies the release of the USB Specification with which the device and its descriptors are compliant.
4	*bDeviceClass*	1	Class	Class code (assigned by the USB-IF). If this field is reset to zero, each interface within a configuration specifies its own class information and the various interfaces operate independently. If this field is set to a value between 1 and FEH, the device supports different class specifications on different interfaces and the interfaces may not operate independently. This value identifies the class definition used for the aggregate interfaces. If this field is set to FFH, the device class is vendor-specific.
5	*bDeviceSubClass*	1	SubClass	Subclass code (assigned by the USB-IF). These codes are qualified by the value of the *bDeviceClass* field. If the *bDeviceClass* field is reset to zero, this field must also be reset to zero. If the *bDeviceClass* field is not set to FFH, all values are reserved for assignment by the USB-IF.

Table 9-8. Standard Device Descriptor (Continued)

Offset	Field	Size	Value	Description
6	*bDeviceProtocol*	1	Protocol	Protocol code (assigned by the USB-IF). These codes are qualified by the value of the *bDeviceClass* and the *bDeviceSubClass* fields. If a device supports class-specific protocols on a device basis as opposed to an interface basis, this code identifies the protocols that the device uses as defined by the specification of the device class. If this field is reset to zero, the device does not use class-specific protocols on a device basis. However, it may use class-specific protocols on an interface basis. If this field is set to FFH, the device uses a vendor-specific protocol on a device basis.
7	*bMaxPacketSize0*	1	Number	Maximum packet size for endpoint zero (only 8, 16, 32, or 64 are valid)
8	*idVendor*	2	ID	Vendor ID (assigned by the USB-IF)
10	*idProduct*	2	ID	Product ID (assigned by the manufacturer)
12	*bcdDevice*	2	BCD	Device release number in binary-coded decimal
14	*iManufacturer*	1	Index	Index of string descriptor describing manufacturer
15	*iProduct*	1	Index	Index of string descriptor describing product
16	*iSerialNumber*	1	Index	Index of string descriptor describing the device's serial number
17	*bNumConfigurations*	1	Number	Number of possible configurations

2.设备限定符

设备限定描述符说明了呢功能进行高速操作的设备在其他速度操作时产生的变化信息.例如,如果设备当前在全速下操作,设备限定符返回它如何在高速操作的信息或者相反.表9.9列出了设备限定描述符的场.

这个描述符不包含标准设备描述符中的厂商,产品,设备,制造厂商,产品和序列号场,因为在所有支持的速度下设备的这些信息都是常数。这个描述符的版本号至少是2.0(0200H).

主机使用GetDescriptor()请求访问这个描述符。 GetDescriptor()请求中的描述符类型被设置为设备的限定符（见表9.5）.

如果只能进行全速操作的设备(设备描述符的版本号等于0200H)接收到请求设备限定符的GetDescriptor()请求,它必须用请求错误响应.主机不能请求其他速度配置(other_speed_configuration)描述符,除非它已经成功取得设备限定描述符.

Table 9-9. Device_Qualifier Descriptor

Offset	Field	Size	Value	Description
0	*bLength*	1	Number	Size of descriptor
1	*bDescriptorType*	1	Constant	Device Qualifier Type
2	*bcdUSB*	2	BCD	USB specification version number (e.g., 0200H for V2.00)
4	*bDeviceClass*	1	Class	Class Code
5	*bDeviceSubClass*	1	SubClass	SubClass Code
6	*bDeviceProtocol*	1	Protocol	Protocol Code
7	*bMaxPacketSize0*	1	Number	Maximum packet size for other speed
8	*bNumConfigurations*	1	Number	Number of Other-speed Configurations
9	*bReserved*	1	Zero	Reserved for future use, must be zero

3.配置

配置描述表给出了一设备配置的信息，描述表包括一个b Configuration Value域，在SetConfiguration( )请时被用作参数来设置所需配置 .

此描述表给出了此配置下的接口数，每个接口可能独立操作。比如，一个ISDN设备可能配置有两个接口，每个都提供64KB/S的有独立数据源与数据接收者的双向通道在另一个配置下ISDN可能表现为单个接口，将两个通道合成一个128KB/S的双向通道。

当主机发出请求要得配置描述表时，所有相关接口与端节点的描述表都被返回.

一个USB设备有一个或多个配置。每个配置只有一个或多个接口。而每个接口又有0个或多个端节点。在一个配置下，一个端节不会在接口之间共享，除非端节点被同一个接口的不同设置使用。在不同配置端节点，可无此限制。

一个配置好后，设备可支持对配置的有限调整，如果一个接口有备选设置，在配置好后可选择不同设置。表9-10是标准配置描述表。

www.cpit.com.cn

Table 9-10. Standard Configuration Descriptor

Offset	Field	Size	Value	Description
0	*bLength*	1	Number	Size of this descriptor in bytes
1	*bDescriptorType*	1	Constant	CONFIGURATION Descriptor Type
2	*wTotalLength*	2	Number	Total length of data returned for this configuration. Includes the combined length of all descriptors (configuration, interface, endpoint, and class- or vendor-specific) returned for this configuration.
4	*bNumInterfaces*	1	Number	Number of interfaces supported by this configuration
5	*bConfigurationValue*	1	Number	Value to use as an argument to the SetConfiguration() request to select this configuration
6	*iConfiguration*	1	Index	Index of string descriptor describing this configuration

Table 9-10. Standard Configuration Descriptor (Continued)

Offset	Field	Size	Value	Description
7	*bmAttributes*	1	Bitmap	Configuration characteristics D7: Reserved (set to one) D6: Self-powered D5: Remote Wakeup D4...0: Reserved (reset to zero) D7 is reserved and must be set to one for historical reasons. A device configuration that uses power from the bus and a local source reports a non-zero value in *bMaxPower* to indicate the amount of bus power required and sets D6. The actual power source at runtime may be determined using the GetStatus(DEVICE) request (see Section 9.4.5). If a device configuration supports remote wakeup, D5 is set to one.
8	*bMaxPower*	1	mA	Maximum power consumption of the USB device from the bus in this specific configuration when the device is fully operational. Expressed in 2 mA units (i.e., 50 = 100 mA). Note: A device configuration reports whether the configuration is bus-powered or self-powered. Device status reports whether the device is currently self-powered. If a device is disconnected from its external power source, it updates device status to indicate that it is no longer self-powered. A device may not increase its power draw from the bus, when it loses its external power source, beyond the amount reported by its configuration. If a device can continue to operate when disconnected from its external power source, it continues to do so. If the device cannot continue to operate, it fails operations it can no longer support. The USB System Software may determine the cause of the failure by checking the status and noting the loss of the device's power source.

4.其他速度配置

其他速度配置(other_speed_configuration)描述符如表9.11所列,它说明了能进行高速操作的设备在其他可能的速度下操作配置.其他速度配置的结构与配置描述符一样.

主机用GetDescriptor()请求访问这个描述符. GetDescriptor()请求中的描述符类型被设置为其他速度配置(表9.5)

Table 9-11. Other_Speed_Configuration Descriptor

Offset	Field	Size	Value	Description
0	*bLength*	1	Number	Size of descriptor
1	*bDescriptorType*	1	Constant	Other_speed_Configuration Type
2	*wTotalLength*	2	Number	Total length of data returned
4	*bNumInterfaces*	1	Number	Number of interfaces supported by this speed configuration
5	*bConfigurationValue*	1	Number	Value to use to select configuration
6	*iConfiguration*	1	Index	Index of string descriptor
7	*bmAttributes*	1	Bitmap	Same as Configuration descriptor
8	*bMaxPower*	1	mA	Same as Configuration descriptor

5.接口

此描述表在一个配置内给出一个接口的信息。如果一个配置支持不止一个接口，端节点的描述表会跟在接口描述表后被返回，接口描述表总是作为配置描述表的一部分被返回。接口描述不可直接用Set Description（ ）和Get Descriptor（ ）请求访问.

一个接口可能包含备选设置，以使得端节点或他们的特性在设备配置好以后能改变.接口的默认设置一般是备用设置0.一个接口的缺省设置总是可选设置。SetInterface（ ）与GetInterface（ ）用来选择与返回选择了的接口设置.

可选的接口设置使得部分的设备配置能在其它接口进行操作的情况下改变。如果一个配置对于它的一个或多个接口有备选设置，每一设置包括一个独立接口描述表和相关结点.

如果一个设备配置支持单个接口，并此接口有两个可选设置，配置描述表返回以后会紧跟着返回bInterfaceNumber与bAlternateSetting域皆为0的第一个设置的接口描述表及相关的结点描述表，而随之后是另一个设置接口描述表与结点描述表。第二个接口描述表的bInterfaceNumber域也应为0，但bAlternate Setting域应为1.

如果一个接口仅使用节点0，则接口描述表以后就不再返回节点描述表，并且此接口表示的是一个请求接口，它使用连在节点0上的缺省通道。在这种情况下bNumberEndpoints域应被设置成0。

一个接口描述表的节点个数不把结点0计在内。表8-9是标准节口描述表。

Table 9-12. Standard Interface Descriptor

Offset	Field	Size	Value	Description
0	*bLength*	1	Number	Size of this descriptor in bytes
1	*bDescriptorType*	1	Constant	INTERFACE Descriptor Type
2	*bInterfaceNumber*	1	Number	Number of this interface. Zero-based value identifying the index in the array of concurrent interfaces supported by this configuration.
3	*bAlternateSetting*	1	Number	Value used to select this alternate setting for the interface identified in the prior field
4	*bNumEndpoints*	1	Number	Number of endpoints used by this interface (excluding endpoint zero). If this value is zero, this interface only uses the Default Control Pipe.
5	*bInterfaceClass*	1	Class	Class code (assigned by the USB-IF). A value of zero is reserved for future standardization. If this field is set to FFH, the interface class is vendor-specific. All other values are reserved for assignment by the USB-IF.
6	*bInterfaceSubClass*	1	SubClass	Subclass code (assigned by the USB-IF). These codes are qualified by the value of the *bInterfaceClass* field. If the *bInterfaceClass* field is reset to zero, this field must also be reset to zero. If the *bInterfaceClass* field is not set to FFH, all values are reserved for assignment by the USB-IF.

Table 9-12. Standard Interface Descriptor (Continued)

Offset	Field	Size	Value	Description
7	*bInterfaceProtocol*	1	Protocol	Protocol code (assigned by the USB). These codes are qualified by the value of the *bInterfaceClass* and the *bInterfaceSubClass* fields. If an interface supports class-specific requests, this code identifies the protocols that the device uses as defined by the specification of the device class. If this field is reset to zero, the device does not use a class-specific protocol on this interface. If this field is set to FFH, the device uses a vendor-specific protocol for this interface.
8	*iInterface*	1	Index	Index of string descriptor describing this interface

6.接口集合

接口集合描述符用于描述与相同功能相关的两个或多个接口,集合包括两个或多个接口及它们所有备用设置接口.设备必须在要求一个或多个接口的各个设备功能中使用接口集合.接口集合描述符通常作为配置信息的一部分由GetDescriptor(Configuration)请求返回。不能使用GetDescriptor（）或SetDescriptor（）请求直接访问接口集合描述符.接口集合描述符必须在与其相关的接口和接口描述符组(包括所有备用设置前).相关接口集的所有接口编号都是连续的.表9.13列出了标准的接口集合描述符.它包括功能类型,子类型和协议场.这些场中的值可以与任何一相关接口的接口类型,子类型和协议值相同.对于已有设备类型的首选实现是使用相关接口列表中首个接口的接口类型,子类型和协议场.

偏移	场	大小	值	描述
0	bLength	1	数字	按字节计算的描述符大小
1	bDescriptorType	1	常数	INTERFACE ASSOCIATION 描述符
2	bFirstIterface	1	数字	与这个功能相关的首个接口的接口号码
3	bInterfaceCount	1	数字	与这个功能相关的连续接口数
4	bFunctionClass	1	类型	类型码（由USB-IF分配） 这个场不能为0，如果值是FFH，功能类型为厂商专用的，其他值保留，有USB-IF分配
5	bFunctionSubClass	1	子类型	子类型码（由USB-IF分配） 如果bFunctionSubClass的值是FFH，表示所有值都保留，由USB-IF分配
6	bFunctionProtocol	1	协议	协议码（由USB-IF分配）。这些代码受bFunctionClass和bFunctionSubClass场的限制
7	iFunction	1	索引	描述这个功能的字符串描述符索引

PoteVio 中国普天 通信未来 普天制造

7.端点

用于接口的每个端点都有自己的描述符.这个描述符包含主机请求确定每个端点的带宽要求的信息.端点描述符一般是作为GetDescriptor(Configuration)请求返回的配置信息的一部分返回. GetDescriptor()或者SetDescriptor()请求不能直接访问端点描述符.端点0没有端点描述符.

bmAttributes场提供了有关端点传输类型(位1~0)和同步类型(位3~2)的信息.另外,使用类型位(位5~4)指出这是用于正常数据传输的端点(位5~4=01B),还是用于从一个或多个数据端点返回显示反馈信息的端点位(位5~4=10B).位5~2只对同步端点有意义,而且在其他传输类型中必须被复位为0.

如果端点是显示反馈端点(位5~4=01B),那么传输类型必须设置为同步(位1~0=01B),而且同步类型必须设为非同步(位3~2=00B).

反馈端点(现式或隐式)需要和提供反馈服务的一个(或多个)同步数据端点关联.这个关联基于端点号的匹配.反馈端点一般与它服务的数据端点的传输方向相反.如果相同的反馈端点要服务多个数据端点,数据端点的端点号必须按升序排列但不必连续.第一个数据端点与反馈端点的端点号必须相同(但方向相反).这就确保了数据端点可以通过查找端点号等于或小于自己端点号的第一个反馈端点唯一识别它的反馈端点.

Offset	Field	Size	Value	Description
0	*bLength*	1	Number	Size of this descriptor in bytes
1	*bDescriptorType*	1	Constant	ENDPOINT Descriptor Type
2	*bEndpointAddress*	1	Endpoint	The address of the endpoint on the USB device described by this descriptor. The address is encoded as follows: Bit 3...0: The endpoint number Bit 6...4: Reserved, reset to zero Bit 7: Direction, ignored for control endpoints 0 = OUT endpoint 1 = IN endpoint

Offset	Field	Size	Value	Description
3	*bmAttributes*	1	Bitmap	This field describes the endpoint's attributes when it is configured using the *bConfigurationValue*. Bits 1..0: Transfer Type 00 = Control 01 = Isochronous 10 = Bulk 11 = Interrupt If not an isochronous endpoint, bits 5..2 are reserved and must be set to zero. If isochronous, they are defined as follows: Bits 3..2: Synchronization Type 00 = No Synchronization 01 = Asynchronous 10 = Adaptive 11 = Synchronous Bits 5..4: Usage Type 00 = Data endpoint 01 = Feedback endpoint 10 = Implicit feedback Data endpoint 11 = Reserved Refer to Chapter 5 for more information. All other bits are reserved and must be reset to zero. Reserved bits must be ignored by the host.

Offset	Field	Size	Value	Description
4	*wMaxPacketSize*	2	Number	Maximum packet size this endpoint is capable of sending or receiving when this configuration is selected. For isochronous endpoints, this value is used to reserve the bus time in the schedule, required for the per-(micro)frame data payloads. The pipe may, on an ongoing basis, actually use less bandwidth than that reserved. The device reports, if necessary, the actual bandwidth used via its normal, non-USB defined mechanisms. For all endpoints, bits 10..0 specify the maximum packet size (in bytes). For high-speed isochronous and interrupt endpoints: Bits 12..11 specify the number of additional transaction opportunities per microframe: 00 = None (1 transaction per microframe) 01 = 1 additional (2 per microframe) 10 = 2 additional (3 per microframe) 11 = Reserved Bits 15..13 are reserved and must be set to zero. Refer to Chapter 5 for more information.
6	*bInterval*	1	Number	Interval for polling endpoint for data transfers. Expressed in frames or microframes depending on the device operating speed (i.e., either 1 millisecond or 125 µs units). For full-/high-speed isochronous endpoints, this value must be in the range from 1 to 16. The *bInterval* value is used as the exponent for a $2^{(bInterval-1)}$ value; e.g., a *bInterval* of 4 means a period of 8 ($2^{(4-1)}$). For full-/low-speed interrupt endpoints, the value of this field may be from 1 to 255. For high-speed interrupt endpoints, the *bInterval* value is used as the exponent for a $2^{(bInterval-1)}$ value; e.g., a *bInterval* of 4 means a period of 8 ($2^{(4-1)}$). This value must be from 1 to 16. For high-speed bulk/control OUT endpoints, the *bInterval* must specify the maximum NAK rate of the endpoint. A value of 0 indicates the endpoint never NAKs. Other values indicate at most 1 NAK each *bInterval* number of microframes. This value must be in the range from 0 to 255.

例 考虑一种极端的情况,需要5组OUT异步同步(asynchronous isochronous)端点,同时还需要4组IN自适应同步端点.每一组都需要一个独立的反馈端点.

OUT Group	Nr of OUT Endpoints	IN Group	Nr of IN Endpoints
1	1	6	1
2	2	7	2
3	2	8	3
4	3	9	4
5	3		

Figure 9-7. Example of Feedback Endpoint Numbers

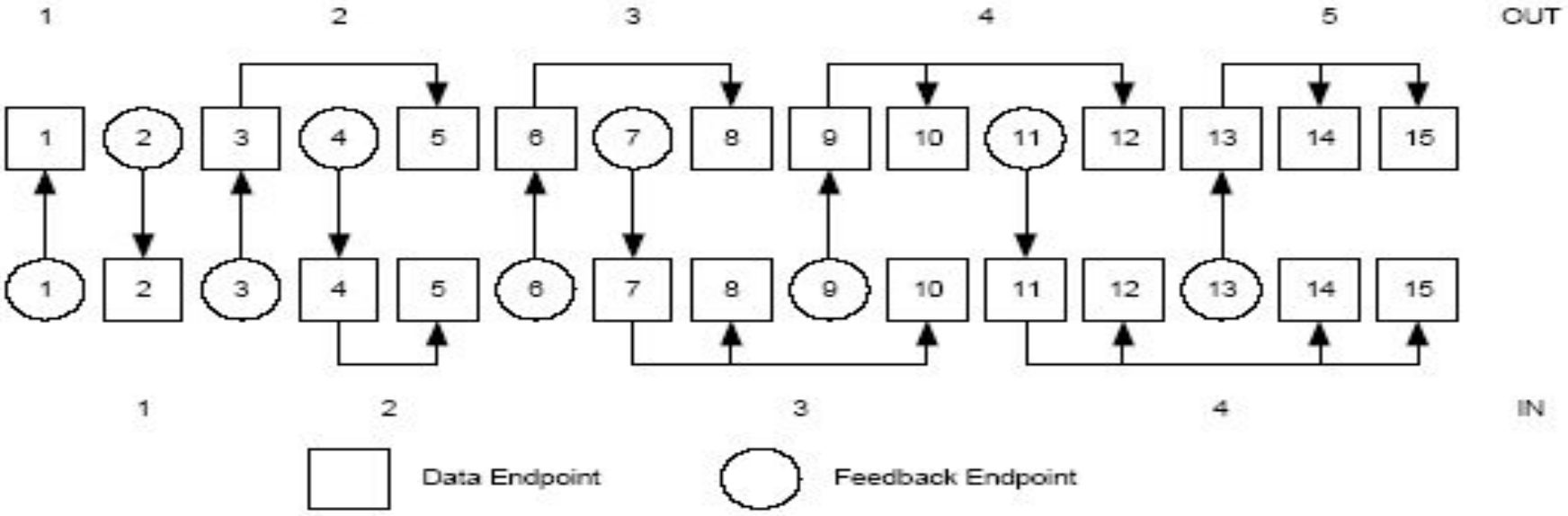


Figure 9-8. Example of Feedback Endpoint Relationships

高速同步和中断端点用wMaxPacketSize场的位是12~11指明了用bInterval指定的每个微型帧有多小个处理.如果wMaxPacketSize场的位是12~11是0,端点最大包的大小可以是任何允许值;如果wMaxPacketSize场的位是12~11不是0, wMaxPacketSize场的位10~0的允许值如下表所示:

Allowed wMaxPacketSize Values for Different Numbers of Transactions per Microframe

wMaxPacketSize bits 12..11	*wMaxPacketSize* bits 10..0 Values Allowed
00	1 – 1024
01	513 – 1024
10	683 – 1024
11	N/A; reserved

对于高速批量和控制OUT端点来说, bInterval场只用于一致性的目的;主机控制器不要求根据这个场的值而改变它的行为.

8.字符串

字串描述表是可有可无的。如前所述，如果一个设备无字串描述表，所有其它描述表中有关字串描述表的索引都必须为0.

字串描述表使用的是UNICODE编码，则《Unicode标准世界范围的字符编码》1.0版，第一和第二卷定义，字串描述表支持多语言编码。当请求字串描述表时，请求者用一个6位的语言标识指出语言，此语言ID由微软Windows定义(《开发Windows 95及Windows NT的国际化软件》，Nadine Kano，微软出版，华盛顿)。所有语言的0号字串索引返回一个字串描述表，该字串描述表为双字节的LANGID数组，表示设备支持的语言，表8-11表示了该LANGID数组。USB设备可删除所有的字串描述表。USB设备删了字串描述表后就不能返回LANGID码了。LANGID数组不是以NULL结尾的。它的大小为bLength-2 .

UNICODE字串也不是NULL结尾的(见表8-12)，字串长为bLength-2。

Table 9-15. String Descriptor Zero, Specifying Languages Supported by the Device

Offset	Field	Size	Value	Description
0	*bLength*	1	N+2	Size of this descriptor in bytes
1	*bDescriptorType*	1	Constant	STRING Descriptor Type
2	*wLANGID[0]*	2	Number	LANGID code zero
...	...	...	...	...
N	*wLANGID[x]*	2	Number	LANGID code *x*

Table 9-16. UNICODE String Descriptor

Offset	Field	Size	Value	Description
0	*bLength*	1	Number	Size of this descriptor in bytes
1	*bDescriptorType*	1	Constant	STRING Descriptor Type
2	*bString*	N	Number	UNICODE encoded string

设备类型定义

所有的设备必须支持本章所讲的请求与描述表，多数设备还有设备特有的扩展的请求与描述表。另外，设备还会支持一组设备共有的服务。为了定义一个设备类，下面信息必须被提供出来，来定义此类设备的表现与行为

1 描述表

如果此类须有对标准描述表特有的定义，则此定义必须在类说明中指出。另外，如果此类定义了标准的扩展描述表集合，它们必须在类中说明。扩展的描述表定义方法与标准描述表定义方法一致，比如说，所有描述表以表长打头 .

2 接口与结点的使用

当一个设备类标准化以后，此类设备使用的接口以及结点们如何被使用必须在类定义中说明，设备类在满足基本的类定义以后还可扩展一些类特性 .

3 请求

此类的所有请求必须被定义 .

www.cpit.com.cn